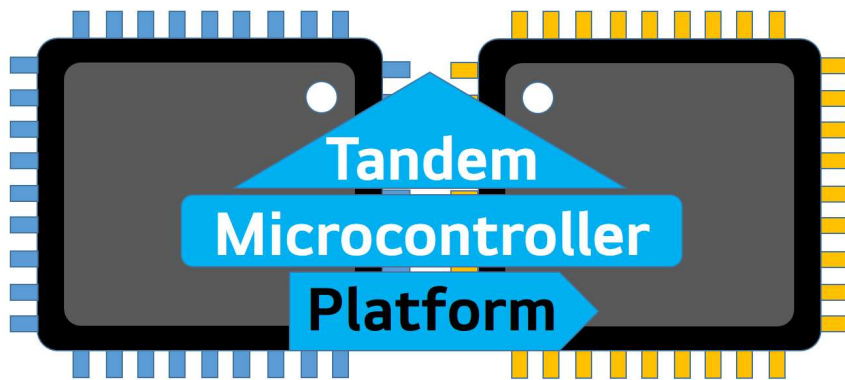


(비전공자 : 문과생을 위한 코딩가이드)

파이썬 입문

텐덤 로보틱스 시작하기

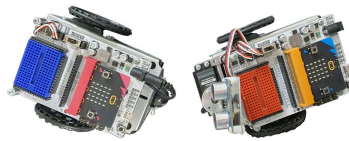
"Python for Tandem_Robotics
: An Integrated Approach"



 으로 여행!

이 책을 읽는 독자에게

"아두이노 로봇과 함께 시작한 여정이,
인공지능으로 마이크로비트를 탐구하는 길로 이어졌습니다.
독자를 꿈꾸며 비로소 전달할 수 있어 기쁩니다."



머릿말

코딩을 경험해보지 않은 컴퓨터 비전공자들에게 추천할 코딩 교재가 많지 않다고 생각하고, 이 책을 준비하였습니다. 물론 유아부터 초등학생 중고등학생까지 사용할 수 있는 초급 코딩 자료들은 많지만, 고등학교 이상의 컴퓨터 비전공자용으로 체계적인 난이도를 갖춘 교재는 많지 않은 현실입니다.

한가지 더 이 책을 쓴 동기는 인공지능 출현과 함께 코딩을 배우는 방법도 크게 바뀐다고 생각했습니다. 나의 코딩수준에 맞춰서 독자적으로 인공지능에게 질문하기 위해, 스스로 단계별 코딩으로 나아가는 중급수준까지 로봇코딩 안내서가 필요한 이유입니다.

챗GPT와 같은 인공지능은 스스로 코딩을 배워나가는데 매우 훌륭한 안내자이고, 당면한 나의 코딩문제를 해결해줄 파트너입니다. 챗GPT는 내가 관심을 가지고 깊게 파고들어 탐색하고 싶은 많은 코딩분야들 중에서, 나의 목적지를 향해 가장 지름길로 나아가는 데 도움을 줄 21세기 인간이 만든 최고의 도구입니다. 이 책은 두 개의 마이크로컨트롤러에 적용되는 C/C++ / python 코딩을 배우는 안내서입니다. 챗GPT의 도움으로 작성되었으며, 여러분도 항상 GPT에게 질문하고, 유료 사용자라면 GPTs(CodingRobot-abot)에게 질문할 수 있습니다.

코딩 플랫폼 텐덤로봇은 마이크로비트와 아두이노를 직접 연결해서 사용하는 교육용 로봇입니다. 아두이노는 C/C++언어를 사용하고 마이크로비트는 python언어를 사용하며, 난이도는 초급부터 중급까지 그리고 사물인터넷(IOT)을 구성하는 다양한 통신 응용기술 기초를 배울 수 있습니다. 다만, 본 교재의 체계적 구성은 아두이노 보다는 마이크로비트 중심으로 설명하고, 초급에서 중급 수준까지, C/C++ 언어보다는 python 언어를 기본으로 로봇 자율제어와 무선 응용 로봇기술을 중점적으로 다룹니다.

마이크로비트 사용법이 너무 기초라고 생각하거나, 파이썬 언어 지식을 어느 정도 갖춘 독자라면, 제 I 부를 생략하고 제 II 부부터 시작해도 좋습니다. 그리고 C/C++언어 사용법을 익히고 싶다면, 마이크로비트를 제거한 텐덤로봇으로 아두이노 코딩을 시작하는 방법도 추천드립니다. C/C++ 라이브러리 코딩은 'C부터 C++언어까지 아두이노 로봇 코딩실습' 공개 자료를 프라이빗 게시판에서 다운로드받을 수 있습니다. 아울러 텐덤로봇에서 사용하는 '아두이노 보드의 C++ 라이브러리: MicroAbot.h & MicroAbot.cpp' 코드가 충분한 학습재료가 되므로 심층적으로 탐구해보기를 추천드립니다.

본 교재의 목적은 초보자 수준부터 중급 수준까지 파이썬 언어를 학습자들이 경험하는 것임

니다. 교재를 따라하기 위해 사용하는 텐덤로봇은 아두이노 우노와 마이크로:비트를 기반으로 동작하기 때문에 파이썬부터 C/C++까지 코딩 교육 경험을 확장할 수 있습니다. 많은 초보자들에게 이 교재와 텐덤로봇이 코딩에 대한 다양한 시각과 경험을 스스로 체험하고 상상하는 도구가 되기를 바랍니다.

책의 내용에 대하여

1. 초보자가 '코딩을 배운다'라는 의미에 대하여

'코딩을 배운다'라는 의미는 시대의 변화에 따라 코딩문법을 배우는 것보다 코드가 동작하는 원리를 이해하는 것이 더 중요한 시대가 되었습니다. 인공지능 시대는 이미 코드 작성 자체는 단순 작업에 해당합니다. 초보자라고 하더라도 코드의 더 추상적인 개념을 이해하고 다룰 수 있어야 합니다.

태어나서 한번도 코딩을 배우지 못한 초보자가 가장 대중적인 코딩언어 C/C++ 그리고 Python 언어를 텐덤로봇 플랫폼을 활용해서 다양하게 접근하고 경험할 수 있습니다.

2. 학생에게 코드를 가르치시는 분께

코드를 가르치기 위한 많은 측면을 제공하는 플랫폼입니다. 코딩언어의 선택과 하드웨어 기반의 동작, 유선통신 기반의 컴퓨팅 체계 그리고 무선통신까지 전체적인 코드 사용능력을 키우는 용도로 사용할 수 있습니다. 메카트로닉스 로봇 시스템 엔지니어링을 배워야하는 많은 학생들에게 특별히 유익한 선택이 됩니다.

이 책의 많은 코드 예제는 GPT인공지능 도움으로 만들었습니다. 당연히 배우는 학생도 인공지능으로 자신의 목적에 부합하는 로봇 코드를 쉽게 만들고 테스트할 수 있습니다.

학생 수준에 따라 C언어 또는 C++언어로 동작하는 로봇을 경험하거나, python언어로 동작하는 로봇을 교육에 활용할 수 있습니다. C/C++코딩언어를 로봇으로 배우는 SelfAbot 라이브러리 교재는 이미 온라인으로 쉽게 이용할 수 있습니다. 본 교재는 파이썬 언어로 로봇 동작을 배우는 데 필요한 내용들을 충실히 다루고자 노력했으며, MicroAbot C++ 라이브러리 코드 교재는 추후에 배포할 예정입니다.

코딩원리 이해와 코드를 다루는 수준이 초보이상이라고 생각한다면, 텐덤로봇 동작에 사용되는 파이썬 라이브러리 또는 아두이노 라이브러리를 탐구해볼 수 있습니다. 단순히 따라하는 수준을 넘어서서 새로운 하드웨어 센서 함수를 추가하거나, 제공되는 라이브러리를 변형 수정할 수 있습니다. 모든 소스코드가 공개되어 있으므로 학생이 쉽게 접근하고 배울 수 있는 기회를 제공합니다. 제공되는 라이브러리 코드가 최선의 코드가 아닐 수 있기 때문입니다.

3. 이 책의 구성에 대하여

이 책의 내용은 전체 3부의 대분류로 나누어져 있습니다. 각 부는 다시 장으로 구분되고, 각 장은 계속 연결됩니다.

제 I 부는 파이썬 언어에 대한 기초문법을 다루고 있습니다. 동시에 마이크로비트 하드웨어 구성과 특징을 동시에 설명합니다. 깊은 내용을 다루기보다 다양한 측면의 사용법을 익히도록 준비했습니다. 이상의 두 가지 내용에 대한 사전지식이 부족하다면 상세히 읽어보기를 추천합니다.

초보자가 제 2장 마이크로파이썬 언어 문법을 이 책만으로 충분히 이해하는 것은 매우 어려울 수 있습니다. 혹시 충분히 이해되지 않더라도 제 3장부터 다루어지는 로봇 실습 내용을 통해서 코드를 계속 배워나가기를 추천합니다.

제 II 부는 이 책의 주 목적인 텐덤로봇 기본 사용법을 다룹니다. 텐덤로봇은 마이크로비트와 아두이노를 I2C로 연결해서 동작하는 방식의 교육용로봇입니다. 제 3장에서 라이브러리 세부내용을 이해하지 않더라도 I2C통신의 기본 이해도를 높일 수 있습니다. 여기서 배운 수단으로 수많은 IOT 환경에서 요구하는 단말장치 코드를 작성할 수 있습니다. 제 4장에서는 텐덤로봇에서 사용할 다양한 센서들의 코드 동작원리를 이해할 수 있습니다. 마이크로비트 내장 센서와 아두이노 브레드보드에 장착하는 외부센서 신호를 처리하는 방법에 관한 것입니다. 제 5장은 텐덤로봇 주행동작을 위한 서보모터 동작과 ‘아두이노 로봇 실습 교재’에서 배운 센서 신호를 결합해서 자율주행 또는 다양한 로봇 주행동작을 만드는 것입니다. 특히, 로봇 동작의 정확도를 높이는 표준 동작을 만들기 위한 기초지식을 쌓을 수 있습니다.

초보자에게는 텐덤로봇 동작과 관련된 아두이노 라이브러리의 수정 또는 추가 작업이 어렵겠지만, 중급 이상 학생이라면 도전할 만한 아두이노 우노의 제한된 하드웨어 환경을 가지고 있습니다. 더 고성능의 로봇을 다루기 전에 연습 단계로 유용할 수 있습니다. 메카트로닉스 엔지니어링을 꿈꾸는 분이라면 적극적으로 도전해보기를 추천합니다.

제 III 부는 텐덤로봇으로 구현할 수 있는 창의적인 로봇 사용방법에 관한 것입니다. 텐덤로봇은 마이크로:비트와 아두이노가 협업하는 구조입니다. 따라서 마이크로비트의 내장센서를 이용하거나, 신호처리에 대한 마이크로컨트롤러 역할을 서로 분담할 수 있습니다.

무엇보다 마이크로비트가 지원하는 무선기능을 적극적으로 활용해서 더 다양한 환경의 코딩을 경험할 수 있습니다. 마이크로비트의 블루투스 기능과 라디오 기능은 텐덤로봇이 더 스마트한 동작을 할 수 있는 기초가 됩니다.

여러 명의 학생들이 각자 텐덤로봇을 동작하는 환경이라면, 군집 환경에서 로봇을 제어 명령하는 경험을 쌓을 수 있습니다. 이 책에서는 충실히 다루고 있지 않지만 더 많은 고려요소를 다루어야 할 필요가 있습니다.

마지막으로 이 책은 혼자서도 자신의 기호와 수준에 맞춰 학습하는 데 유용하도록 충실히 고려했습니다. 부득이 책에서 언급하지 않은 추가설명이 필요하다면, 아래 ‘주요 자료출처’ GPTs를 사용하면 텐덤로봇과 관련된 답변을 쉽게 받을 수 있습니다. 그리고 텐덤로봇의 코드

를 추가 수정하는 목적으로도 편리하게 활용할 수 있습니다. 코딩언어 원리를 스스로 경험하고 깨우치는 도구가 되기를 바랍니다.

이 책은 파이썬 언어와 마이크로비트를 사용해서 로봇을 다루는 것에 주안점을 두었습니다. 그런 의도를 고려해서 부족한 점이 있더라도 양해해 주시기 바랍니다.

책의 모든 내용과 관련 라이브러리 코드는 GitHub 또는 Fribot.com '초보자를 위한 가이드' 게시판을 통해서 공개됩니다. 동시에 이 책의 부록에서도 전체 소스 코드를 소개합니다. 더 필요한 정보들은 다음페이지의 '아래 주요자료 출처'를 사용하기 바랍니다.

저자와 연락하기

저자는 2012년부터 프라이봇(Fribot.com)을 통해 국내에서 ‘아두이노’ 마이크로컨트롤러를 보급하고 책을 지어서 배포하는 등 국내 학생들의 코딩교육에 힘써 왔습니다. 동시에 미국 Parallax사의 제품 공식판매처로도 활동하고 있습니다. 국내 사용자들에게 아두이노로봇 제품을 출시하고 많은 교육자료를 배포하게된 배경도 미국 Parallax사와의 협력에 기인한 바가 큼니다. 본 교재 핵심인 텐덤로봇 역시 패럴렉스의 기술적 아이디어와 교육 내용을 동일하게 수용합니다.

이 책의 저자에게 연락하기 위한 수단은 프라이봇 홈페이지(fribot.com)에 공개된 주소/연락처 정보를 활용하면 됩니다. 연락수단으로는 전화/이메일/블로그 등 다양한 채널들을 사용할 수 있습니다. 국내 사용자인 경우 카카오톡(ID: fribot)으로도 많은 질문을 받고 있으므로, 프라이봇 홈페이지 카카오톡 아이콘을 활용하면 됩니다.

감사합니다. 여러분의 성원이 함께 한다면 앞으로도 더 많은 자료를 제공할 수 있다고 생각합니다. 모두에게 성공적인 미래가 펼쳐지기를 희망합니다.

2025년 4월 21일

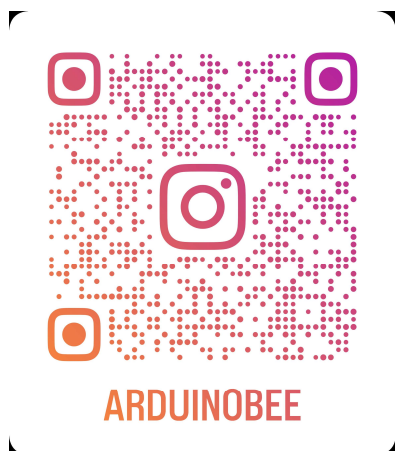
주요 자료 출처

1) GitHub 다운로드 : ‘Tandem_Robotics’ 라이브러리와 ‘파이썬을 이용한 텐덤 로보틱스’ 문서
https://github.com/wookjin-chung/Tandem_Robotics

2) Fribot 다운로드 : ‘Tandem_Robotics’ 라이브러리와 ‘파이썬을 이용한 텐덤 로보틱스’ 문서
https://www.fribot.com/board/view?id=community_guide&seq=197

3) 유튜브 채널
<https://www.youtube.com/channel/UCJw4FMNvVXiektPpQPUriyQ>

4) 인스타그램 채널 : arduinobee



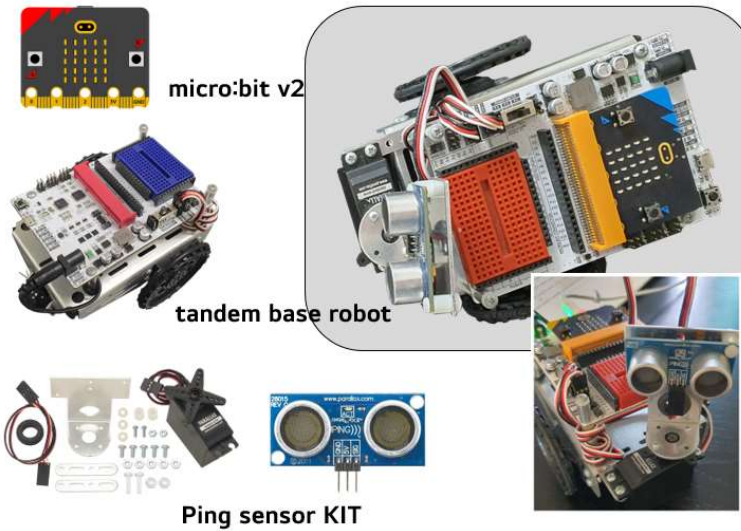
5) GPTs 연결 링크
<https://chatgpt.com/g/g-eD99s1qbU-codingrobot-abot>

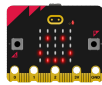
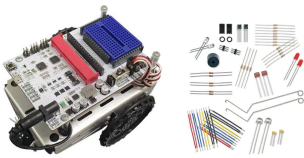
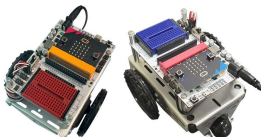


6) 프라이봇 블로그 채널 (한글 버전)
<https://blog.naver.com/fribot/>

각 장별로 필요한 텐덤로봇 하드웨어 구성품들

(텐덤로봇 키트 구성)



	하드웨어 구성품	필요 부품과 수량
chapter 1 & chapter 2		마이크로비트 1대
chapter 3		마이크로비트 ----- 텐덤 베이스로봇 1대
chapter 4	 Ping 센서	마이크로비트 텐덤 베이스로봇 1대 ----- 초음파센서 1대
chapter 5	 초음파센서 마운팅 키트	마이크로비트 텐덤 베이스로봇 1대 ----- 초음파센서 1대 초음파센서 마운팅 키트 1대
chapter 6	 듀얼쇼크 무선컨트롤러	마이크로비트 텐덤 베이스로봇 1대 ----- 마이크로비트 1대 (필요시 리포 배터리 1개) 듀얼쇼크 무선컨트롤러 1대
chapter 7		마이크로비트 텐덤 베이스로봇 2대 ----- 초음파센서 1대 초음파센서 마운팅 키트 1대

차 례

제 I 부 마이크로비트 마이크로파이썬 언어 기초

제 1 장 마이크로비트 소개

- 1.1 마이크로비트의 등장
- 1.2 마이크로파이썬 언어
- 1.3 펌웨어 업그레이드
- 1.4 마이크로비트 온라인 편집기 사용법

제 2 장 마이크로파이썬 기초

- 2.1 데이터 출력하기
- 2.2 파이썬 기초
- 2.3 코드 제어 구조
- 2.4 함수의 정의와 호출
- 2.5 클래스의 선언과 객체 생성
- 2.6 생성자, 속성, 메소드
- 2.7 상속, 다형성, 캡슐화
- 2.8 예외 처리

제 II 부 마이크로비트-아두이노 텐덤로보틱스

제 3 장 I2C통신과 텐덤로봇(tandem robot)

- 3.1 I2C 통신과 텐덤로봇
 - (1) I2C통신 소개
 - (2) 텐덤(tandem) 로봇이란?
- 3.2 텐덤로봇 I2C 데이터통신 기초
 - (1) 문자 데이터 송수신
 - (2) 숫자 데이터 송수신
 - (3) 더 복잡한 데이터의 송수신
- 3.3 텐덤로봇 라이브러리 설치
 - (1) 아두이노에 라이브러리 설치하는 방법
 - (2) 마이크로비트에 라이브러리 설치하는 방법
- 3.4 텐덤로봇 라이브러리 개요
 - (1) 마이크로비트 라이브러리 개요
 - (2) 아두이노 라이브러리 개요
 - (3) 아두이노 메인 파일 microbit_abot.ino

제 4 장 텐덤로보틱스 시작하기

- 4.1 LED 켜고 끄는 방법
 - (1) 마이크로비트 핀 P0에 연결된 LED를 ON/OFF 하는 방법
 - (2) 마이크로비트-아두이노의 P8핀에 연결된 LED를 ON/OFF 하는 방법
- 4.2 부저 멜로디 소리내기

- (1) 마이크로비트에 내장된 부저로 '작은별' 멜로디 연주하기
- (2) 마이크로비트 외부 P0핀에 부저를 연결하고 '작은별' 연주하기
- (3) 아두이노 디지털핀 P8에 연결된 부저로 '작은별' 연주하기
- 4.3 버튼회로 사용하기
 - (1) 마이크로비트 버튼 A를 사용해서 LED 켜고 끄기
 - (2) 마이크로비트가 아두이노 디지털핀 버튼회로를 사용해서 LED 켜고 끄기
- 4.4 센서 활용하기: 마이크로비트 내장센서
 - (1) 가속도센서
 - (2) 자기센서 (컴파스 센서)
 - (3) 광센서 (light sensor)
 - (4) 온도센서
 - (5) 터치센서
- 4.5 센서 활용하기: I2C 아두이노 센서
 - (1) 포토트랜지스터 광센서 신호처리
 - (2) 적외선(IR) 센서
 - (3) 접촉센서(더듬이 센서)
 - (4) 초음파센서

제 5 장 스마트한 자율주행 텐덤로봇

- 5.1 로봇 동작을 위한 준비
 - (1) 텐덤로봇의 마이크로비트-아두이노 보드
 - (2) 텐덤로봇 동작 바퀴용 서보모터
- 5.2 서보모터 중심 맞추기와 기본동작 시작하기
 - (1) 서보모터 중심 맞추기
 - (2) 로봇의 기본동작과 회전속도
- 5.3 텐덤로봇 자율주행하기
 - (1) 더듬이센서 자율주행
 - (2) 광센서(포토트랜지스터) 빛 추적하기
 - (3) 적외선 송수신센서 자율주행
 - (4) 초음파센서 PID제어
- 5.4 텐덤로봇 초음파센서 레이더 스캔하기
- 5.5 텐덤로봇 직선주행 보정하기
 - (1) 텐덤로봇 직선주행 보정하는 법
 - (2) 모듈 방식으로 전진/후진 주행 보정효과 유지하기

제 III 부 텐덤로봇 네트워크/스마트 제어

제 6 장 텐덤로봇 무선제어

- 6.1 마이크로비트 무선기능 살펴보기
 - (1) 블루투스(bluetooth) 통신
 - (2) 라디오(radio) 통신
- 6.2 라디오(Radio) 통신 시작하기
 - (1) 라디오통신으로 두 대의 마이크로비트 채팅하기
 - (2) 다양한 데이터 송수신하기
 - (3) 개별통신과 다중통신(브로드캐스팅)의 조합
 - (4) 라디오통신 신호강도와 통신거리

6.3 텐덤로봇 라디오 통신으로 원격제어하기

- (1) 버튼으로 텐덤로봇 제어하기
- (2) 가속도센서로 텐덤로봇 제어하기
- (3) 노트북 키보드 신호로 텐덤로봇 제어하기
- (4) 플레이스테이션 무선 컨트롤러로 텐덤로봇 제어하기
- (5) 플레이스테이션 무선 컨트롤러로 실감 제어하기

제 7 장 텐덤로봇 스마트제어

7.1 간단한 데이터 통합 텐덤로봇

- (1) 단일축 기반 로봇제어
- (2) 다축 통합 로봇제어

7.2 초음파센서 스캔으로 장애물 회피하기

- (1) 조건별 탐색 스캔으로 장애물 회피하기
- (2) 전범위 탐색 스캔으로 장애물 회피하기
- (3) 오픈루프 동적제어 방법으로 장애물 회피하기
- (4) PID 폐루프 동적제어 방법으로 장애물 회피하기

7.3 팔로워-로봇 리더 따라가기

- (1) 하드웨어 성능 차이를 극복하는 제자리회전 보정
- (2) 로봇에 장착된 초음파센서 하드웨어 정렬하기
- (3) 순차 제어로 따라가는 로봇 만들기
- (4) 조향과 거리유지 데이터 합산으로 전방로봇 따라가기

7.4 두 대 이상의 로봇 제어하기

- (1) 서로 다른 라디오 성능을 극복하는 방법
- (2) 마스터-슬레이브 로봇 군무 만들기
- (3) 분산제어로 로봇 군무 만들기

부록1: 아두이노 라이브러리

MicroAbot.h / MicroAbot.cpp / microbit_abot.ino

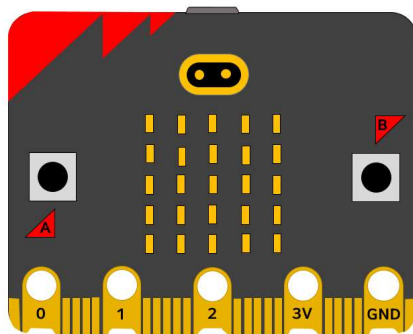
부록2: 파이썬 라이브러리

microbit_abot.py / ultrasonic.py / abot_straight_calibrate.py / abot_rotation_calibrate.py
abot_calibrated.py

제 1 부 마이크로비트 마이크로파이썬 언어 기초

제 1 장 마이크로비트 소개

- 1.1 마이크로비트의 등장
- 1.2 마이크로파이썬 언어
- 1.3 펌웨어 업그레이드
- 1.4 마이크로비트 온라인 편집기 사용법



마이크로비트는 대중적으로 가장 많이 알려진 소형 교육용 컴퓨터중 하나입니다. 많은 사람들에게 친숙한 마이크로컨트롤러는 코딩에 전혀 접근할 기회가 없었던 많은 사람들에게 파이썬 언어를 배우는 계기를 만들어 드립니다.

이 장에서는 마이크로비트에 대한 소개와 하드웨어 부품을 다루는 방법을 소개합니다. 소형 컴퓨터에 대한 기초 지식을 쌓는 데 도움이 됩니다.

마이크로파이썬 언어를 사용하는 컴퓨터를 어떻게 다루어야 하는지 배울 수 있습니다. 가령 내가 작성한 파이썬 코드를 어떤 편집기 도구를 사용해서 마이크로비트로 코드를 보내고, 마이크로비트가 내보내는 출력은 어떻게 데스크탑 컴퓨터에서 출력하는지 살펴보기 바랍니다.

1.1 마이크로비트의 등장

마이크로비트(micro:bit)는 BBC의 "Make it Digital" 캠페인의 일환으로 개발되었습니다. 이 캠페인은 디지털 기술 교육의 중요성을 강조하고, 특히 어린이와 청소년들이 코딩을 배울 수 있도록 돕기 위해 시작되었습니다.

마이크로비트의 초기 목표 중 하나는 영국의 모든 11~12세 학생들에게 마이크로비트를 무료로 배포하는 것이었습니다. 이를 통해 약 백만 명의 학생들이 무료로 마이크로비트를 받을 수 있었습니다. 마이크로비트의 개발은 BBC가 주도했지만, 삼성, 마이크로소프트, 랩탑 회사 등 다양한 기업과 협력하여 진행되었습니다. 이로 인해 하드웨어와 소프트웨어의 통합이 원활하게 이루어졌습니다.

마이크로비트에 숨겨진 기능은 '암호화 기능'과 '전원 절약 모드'입니다. 마이크로비트에는 블루투스 통신을 위한 암호화 기능이 내장되어 있어, 데이터를 안전하게 전송할 수 있습니다. 이는 교육용 도구로서의 활용뿐 아니라 보안 교육에도 사용할 수 있습니다. 그리고 전력 소비를 최소화하기 위한 다양한 기능이 포함되어 있습니다. 이를 통해 배터리 수명을 최대화할 수 있습니다.

마이크로비트는 단순한 교육 도구를 넘어, 창의력과 문제 해결 능력을 기르는 중요한 도구로 자리 잡고 있습니다.

마이크로비트 v2 최신버전의 하드웨어 기능을 살펴보겠습니다.

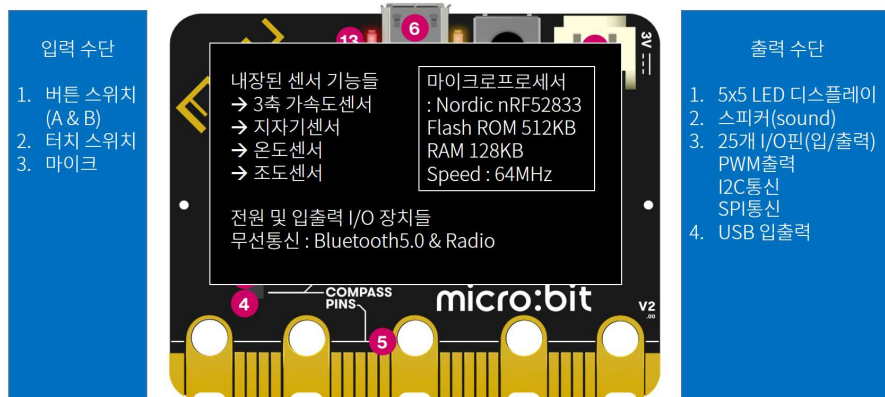


그림 1.1 마이크로비트v2 기능과 입력/출력 수단

v2.0 프로세서 및 메모리

프로세서: Nordic nRF52833 (64MHz ARM Cortex-M4 32bit)

메모리: 512KB Flash 메모리, 128KB RAM

입력 장치

버튼: 2개의 사용자 버튼 (A와 B)

터치 스위치: 로고에 터치기능 내장

마이크: MEMs 마이크 & 상태표시등

핀 헤더: 25개의 외부 연결 I/O핀, 이 중 5개는 크고 사용하기 쉽게 설계됨

출력 장치

LED 매트릭스: 5x5 개의 개별 제어 가능한 빨간색 LED (총 25개)

내장 스피커

핀 헤더: 25개의 외부 연결 I/O핀 & USB입출력

센서

가속도계: 3축 가속도계로 움직임 감지

자력계: 3축 자력계로 방향 및 자력 감지

온도 센서: 프로세서 내장 온도 센서

라이트 센서: LED 매트릭스를 이용한 간접 조도 센서 기능

통신

무선 통신: Bluetooth 통신(BLE) 5.0 & 2.4GHz 라디오 통신

USB 인터페이스: USB 2.0

전원

전원 공급: 마이크로 USB 포트 또는 3V 배터리 팩 (2x AAA 배터리 사용)

전력 관리: 전력 절약 모드 지원 (전원상태 표시 LED)

1.2 마이크로파이썬 언어

이 책에서 소개하는 파이썬 언어는 정확하게 마이크로파이썬 언어입니다. 파이썬 언어를 마이크로비트 컨트롤러에서 사용할 수 있는 버전으로 최적화한 프로그래밍 언어입니다. 파이썬 언어와 마이크로파이썬 언어의 공통점과 차이점을 비교해보겠습니다. 두 언어 모두 파이썬 기반이지만, 각각의 용도와 특징에서 차이점이 존재합니다.

표 1.1 파이썬 언어와 마이크로파이썬 언어 비교

	파이썬 언어	마이크로파이썬 언어
공통점	1. 기본 언어 구조: 파이썬과 마이크로파이썬은 모두 파이썬 언어를 기반으로 하여, 문법과 기본 구조가 거의 동일합니다. 예를 들어, 변수 선언, 제어문(if, for, while), 함수 정의 등의 문법이 동일하게 사용됩니다. 2. 간결함과 읽기 쉬운 코드: 두 언어 모두 간결하고 가독성이 높은 코드를 작성할 수 있도록 설계되었습니다. 코드의 가독성을 높이기 위한 들여쓰기와 같은 규칙이 동일하게 적용됩니다.	

	3. 인터프리터 방식: 두 언어 모두 인터프리터 방식으로 코드를 실행합니다. 즉, 소스 코드를 한 줄씩 읽어 실행하는 방식입니다. 4. 풍부한 표준 라이브러리: 파이썬의 기본 라이브러리 중 일부가 마이크로파이썬에서도 지원됩니다. 예를 들어, 문자열 처리, 리스트, 딕셔너리 등 기본 자료구조와 관련된 라이브러리는 동일하게 사용됩니다.	
차 이 점	사용목적	범용 프로그래밍 언어로, 웹 개발, 데이터 분석, 인공지능, 자동화 등 다양한 분야에서 사용됩니다.
	실행 환경	데스크톱, 서버, 클라우드 등 다양한 플랫폼에서 실행됩니다. Windows, macOS, Linux 등에서 사용 가능합니다.
	메모리 및 자원 사용	더 많은 메모리와 CPU 자원을 요구합니다. 고사양 환경에서 다양한 작업을 수행할 수 있습니다.
	표준 라이브러리	매우 방대한 표준 라이브러리를 포함하고 있어, 거의 모든 기능을 쉽게 구현할 수 있습니다.
	설치 및 사용	파이썬 인터프리터(예: CPython)를 설치하여 스크립트를 실행합니다.
		임베디드 시스템용으로 설계된 경량화된 파이썬 구현으로, 마이크로컨트롤러와 같은 자원이 제한된 장치에서 사용됩니다.
		주로 마이크로컨트롤러(예: 마이크로비트, ESP8266, ESP32 등)와 같은 소형 장치에서 실행됩니다.
		매우 적은 메모리와 CPU 자원으로 실행할 수 있도록 최적화되어 있습니다. 제한된 자원에서도 효율적으로 동작합니다.
		파이썬의 표준 라이브러리 중 핵심 기능만을 포함하고 있으며, 임베디드 시스템에 특화된 추가 모듈이 포함되어 있습니다.
		마이크로컨트롤러에 마이크로파이썬 펌웨어를 플래시하고, 이를 통해 스크립트를 실행합니다.

마이크로비트로 프로그래밍할 수 있는 편집도구는 다양하게 존재합니다. 블록형태의 코딩 도구로 Microsoft MakeCode와 Scratch 도구 이외에도 여러 가지가 존재합니다. 여기서는 파이썬 언어를 편집하는 대표적인 도구 2가지를 소개합니다. 아래 소개하는 온라인 편집기 또는 Mu 편집기 중 하나를 선택해서 사용하면 됩니다.

(1) 파이썬 기반 텍스트 코딩 온라인 편집기 (<https://python.microbit.org/v/3>)

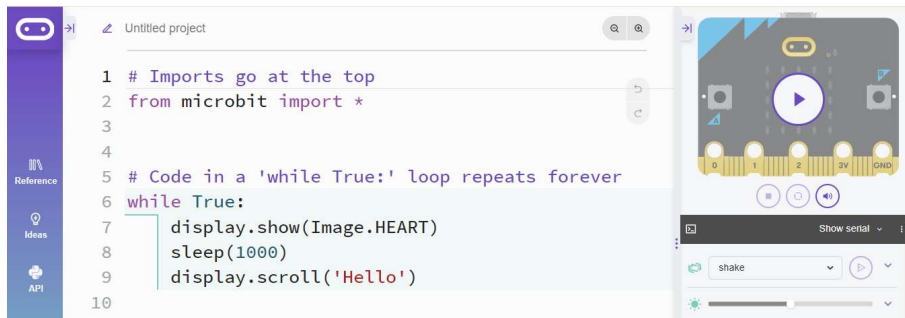


그림 1.2 마이크로파이썬 온라인 편집기

마이크로파이썬 편집기의 더 상세한 사용방법을 살펴보려면 링크를 눌러 [마이크로비트 관련 페이지](#)를 찾아보세요! 그리고 이 책의 1.4절에서 추가 설명이 이어집니다.

(2) 파이썬 기반 Mu 편집기 (<https://codewith.mu/>)



그림 1.3 마이크로파이썬 Mu 편집기

[1] 간단한 설치 및 사용

- Mu Editor는 Windows, macOS, Linux에서 설치할 수 있습니다.
- 설치 파일은 <https://codewith.mu/> 에서 다운로드할 수 있습니다.
- 설치 후, "Mode"를 선택할 때 "BBC micro" 모드를 선택하면 마이크로비트를 위한 설정이 완료됩니다.

[2] 직관적인 사용자 인터페이스

- Mu Editor는 간단하고 직관적인 인터페이스를 제공하여 초보자도 쉽게 사용할 수 있습니다. 코드 작성, 실행, 저장 등의 기본 기능을 쉽게 이용할 수 있습니다.

[3] 다양한 기능 지원

- REPL (Read-Eval-Print Loop): 파이썬 코드를 즉시 실행하고 결과를 확인할 수 있는 대화형 콘솔을 제공합니다.
- 시리얼 모니터: 마이크로비트와 연결된 시리얼 포트를 통해 실시간 데이터를 주고받을 수 있습니다.
- 파일 관리: 마이크로비트내 파일을 관리할 수 있는 간단한 인터페이스를 제공합니다.
- 에러 체크 및 디버깅: 코드 작성 중 발생하는 오류를 실시간으로 체크하고 디버깅을 도와줍니다.

[4] 코드 작성 및 업로드

- 마이크로비트를 USB로 컴퓨터에 연결한 후, Mu Editor를 열고 코드를 작성합니다. 코드를 작성한 후, "Flash" 버튼을 눌러 마이크로비트에 코드를 업로드합니다. 업로드가 완료되면, 마이크로비트가 코드를 실행하게 됩니다.

예제 1 "Hello, Micro! - 디스플레이에 메시지와 이미지 표시하기"

```
# ch01_hello_microbit.py

from microbit import *

while True:
    display.scroll('Hello, Micro:bit!')
    display.show(Image.HEART)
    sleep(1000)
    display.clear()
```

```
sleep(1000)
```

위의 코드를 Mu Editor에 작성한 후 "Flash" 버튼을 눌러 마이크로비트에 업로드하면, 마이크로비트의 LED 매트릭스에 "Hello, Micro!" 메시지가 스크롤되고 하트 모양이 표시됩니다.

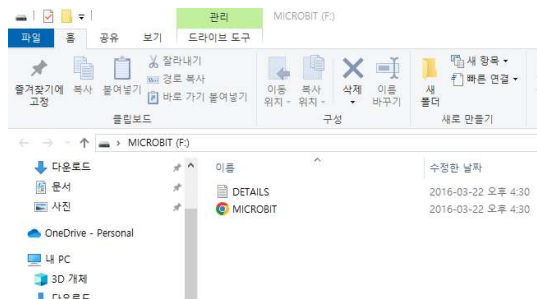
1.3 펌웨어 업그레이드

※ 펌웨어 업그레이드 출처: <https://microbit.org/get-started/user-guide/firmware/>

마이크로비트의 소프트웨어로 부르는 펌웨어는 마이크로비트에서 작성하고 실행하는 프로그램의 기초입니다. 장치가 제대로 작동하도록 만드는 특별한 종류의 소프트웨어입니다. 이는 micro:bit의 인터페이스 칩에 물리적으로 저장되며 프로그램을 작성하거나 micro:bit를 꺼도 변경되지 않지만 업데이트될 수 있습니다.

micro:bit는 함께 제공된 펌웨어와 함께 작동하므로 꼭 필요한 경우에만 업데이트해야 합니다.

(1) micro:bit에 있는 펌웨어 버전 확인



(a) 마이크로비트 펌웨어 알림 정보

```
DETAILS - Windows 메모장
파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)
# DAPLink Firmware - see https://mbed.com/daplink
Unique ID: 9904360261974e45004a00130000000b000000009796990b
HIC ID: 9796990b
Auto Reset: 1
Automation allowed: 0
Overflow detection: 0
Incompatible image detection: 1
Page erasing: 0
Daplink Mode: Interface
Interface Version: 0255
Bootloader Version: 0255
Git SHA: 1436bdcc67029dfdc0ff03b73e12045bb6a9f272
Local Mods: 0
USB Interfaces: MSD, CDC, HID, WebUSB
Bootloader CRC: 0x828c6069
Interface CRC: 0x5b5cc0f5
Remount count: 0
URL: https://microbit.org/device/?id=9904&v=0255
```

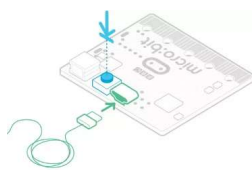
(b) DETAILS 파일정보

그림 1.4 마이크로비트 펌웨어 버전정보

- USB 케이블을 사용하여 컴퓨터에 연결하세요.
 - MICROBIT 드라이브 에서 DETAILS.TXT 파일을 엽니다.
 - '인터페이스 버전'으로 시작하는 줄에서 숫자를 찾으세요.
- 예를 들어 펌웨어 버전이 0255인 micro:bit는 다음과 같이 읽습니다.

Interface Version: 0255

(2) micro:bitdp 내장된 펌웨어 업데이트 방법



- micro:bit에서 USB 케이블과 배터리 팩을 분리합니다.
- micro:bit 뒷면에 있는 재설정(reset) 버튼을 누른 상태에서 USB 리드를 micro:bit와 컴퓨터에 연결하세요. 파일 관리자에 MAINTENANCE (MICROBIT 폴더 MICROBIT (F:) 를 표시하지 않고) 라는 드라이브가 나타나는 것을 볼 수 있습니다.
- MAINTENANCE 드라이브 MAINTENANCE (F:) 는 컴퓨터에 따라 다음과 같습니다.

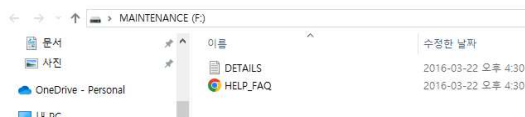


그림 1.5 마이크로비트 Maintenance 폴더 화면

- 귀하의 micro:bit 버전에 적합한 최신 .hex 파일을 PC컴퓨터로 다운로드하세요. 버전 번호는 장치 뒷면의 오른쪽 하단에 인쇄되어 있습니다.
- 펌웨어 다운로드 페이지는 <https://microbit.org/get-started/user-guide/firmware/> (마이크로비트 V1용과 V2용은 나의 하드웨어 버전에 맞춰 구분해서 사용해야 합니다.)

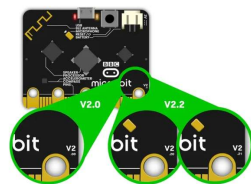


그림 1.6 마이크로비트 버전 확인 (그림출처: microbit.org)

- 다운로드한 .hex 파일을 MAINTENANCE 드라이브에 끌어다 놓고 장치 뒷면의 노란색 LED가 깜박임을 멈출 때까지 기다립니다. 업그레이드가 완료되면 micro:bit가 재설정되어 컴퓨터에서 스스로 빠져나오고 일반 MICROBIT 드라이브 모드로 다시 나타납니다.
- 마지막으로 MICROBIT 드라이브에 있는 DETAILS.TXT 파일을 확인하고 방금 다운로드 하여 인터페이스 칩에 플래시한 .hex 펌웨어와 버전 번호가 동일한지 확인하세요 (예: 0255).

(Notice) -----

마이크로비트 하드웨어 버전과 펌웨어 버전

마이크로비트 하드웨어 v1.5에 펌웨어 v2.0, v2.20, v2.21 을 직접 업데이트하는 것은 불가능합니다. 각각의 펌웨어는 해당 하드웨어 버전에 맞게 설계되어 있으므로, v1.5에는 v1.5 전용 펌웨어를, v2.x에는 v2.x 전용 펌웨어를 사용해야 합니다. 각 버전에 맞는 최신 펌웨어를 설치하여 사용하는 것을 추천합니다.

그리고 마이크로비트 v2.0 하드웨어 버전에 펌웨어 버전 v2.20이나 v2.21은 업데이트할 수 있습니다. 마이크로비트 펌웨어 v2.x는 하드웨어 버전 v2.x에 상관없이 업데이트가 가능하도록 설계되었습니다.

1.4 마이크로비트 온라인 편집기 사용법

파이썬 프로그래밍에 사용할 온라인 편집기 사용법을 설명합니다. 기본 사용법의 몇가지 팁을 소개합니다.

온라인 편집기 한국어 설정

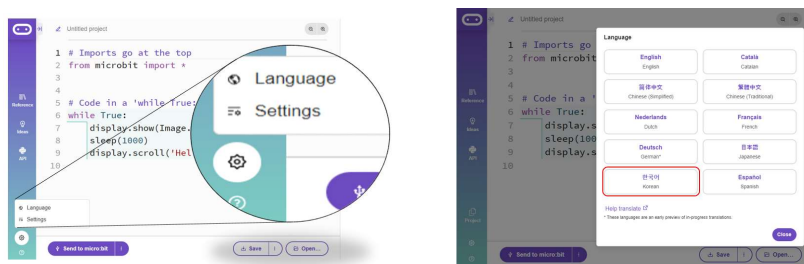
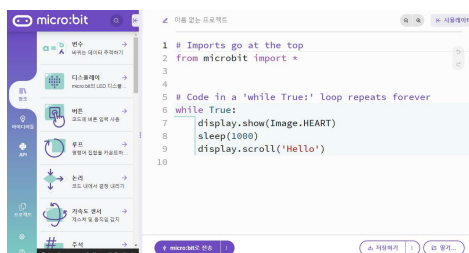


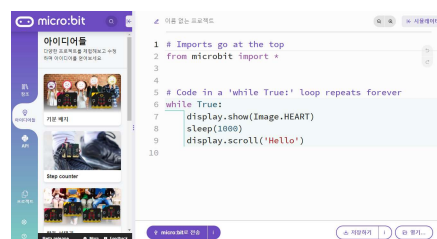
그림 1.7 마이크로비트 온라인편집기 한국어 설정방법

참조, 아이디어 그리고 API

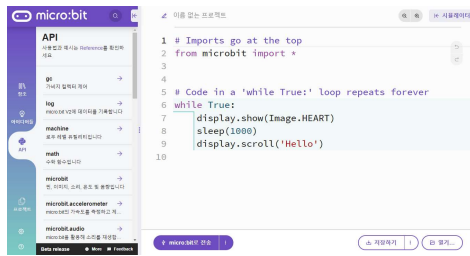
아래 화면의 좌측 기능들은 모두 마우스로 클릭해서 오른쪽 편집화면으로 옮기면 코드가 생성됩니다. 그러면 옮겨진 코드를 사용해서 사용자의 조건에 맞도록 수정해서 사용하면 됩니다.



(a) 참조



(b) 아이디어



(c) API

그림 1.8 편집기 기능 사용방법

편집기에 마이크로비트 연결하는 방법

그림 1.9 (b)의 점3개를 클릭하고 연결을 클릭한 다음, (c)단계에서 연결하면 됩니다.

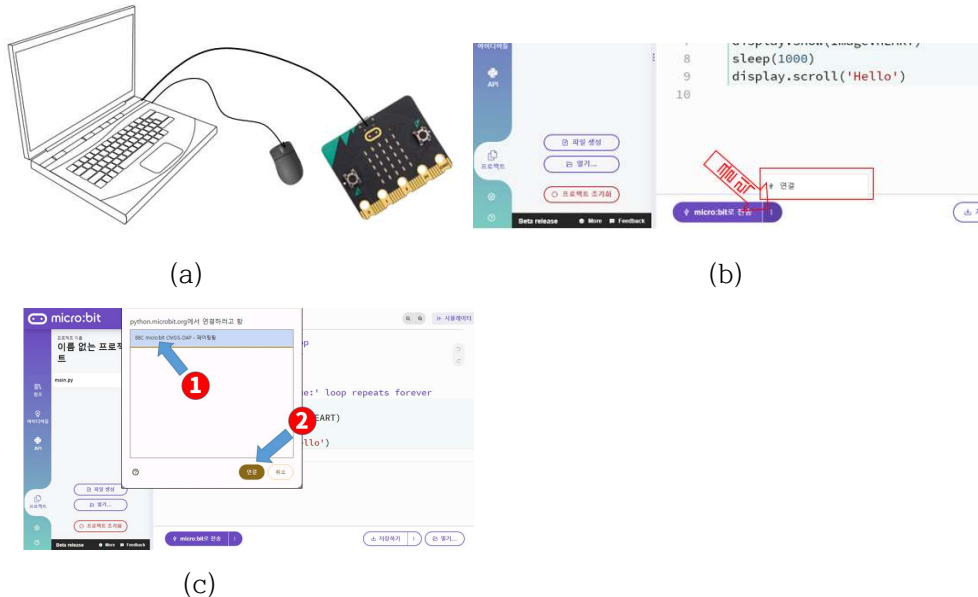


그림 1.9 마이크로비트를 온라인편집기에 연결하는 방법

마이크로비트로 코드를 전송(플래싱)하는 방법

컴퓨터에서 사용자가 작성한 파이썬 코드를 플래싱하는 방법입니다. 전송하려는 코드를 먼저 준비한 다음 'micro:bit로 전송' 버튼을 클릭하면 코드가 전송되고 실행됩니다. 동일한 코드를 재실행하는 방법은 외부 전원을 차단하고 재연결하는 방법, 리셋버튼을 사용해서 재부팅하는 방법, 온라인 편집기 코드를 다시 플래싱하는 방법 등이 있습니다.

코드 실행 결과를 화면에서 출력하려면, 마우스 클릭으로 '시리얼 표시'와 '시리얼 숨기기'를 선택하면 아래 그림1.10 화면처럼 표시할 수 있습니다.

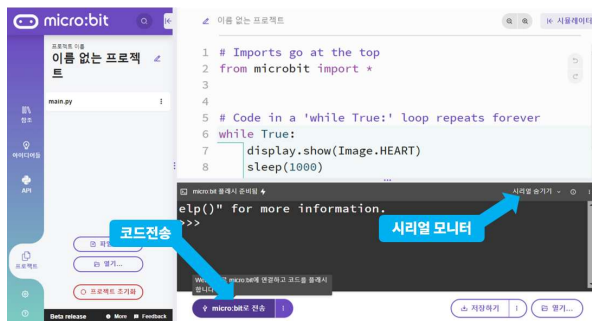
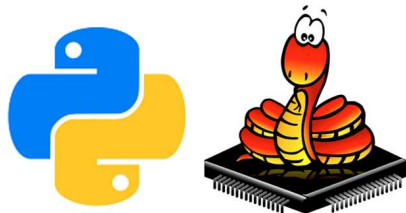


그림 1.10 마이크로비트 온라인편집기 플래싱과 시리얼 출력화면

제 1 부 마이크로비트 마이크로파이썬 언어 기초

제 2 장 마이크로파이썬 기초

- 2.1 데이터 출력하기
- 2.2 파이썬 기초
- 2.3 코드 제어 구조
- 2.4 함수의 정의와 호출
- 2.5 클래스의 선언과 객체 생성
- 2.6 생성자, 속성, 메소드
- 2.7 상속, 다형성, 캡슐화
- 2.8 예외 처리



Python MicroPython

마이크로비트에서 사용하는 파이썬 언어는 마이크로파이썬입니다. 제1장에서 두 언어 차이점을 소개한바와 같이 마이크로비트로 파이썬 언어를 실행할 수 있습니다. 파이썬 언어로 작성한 코드의 의미를 마이크로비트 출력으로 직접 살펴보고 느껴보세요!

마이크로비트의 다양한 입력/출력 수단은 더 생동감 있게 코드의 의미를 느끼게 해줍니다. 더 나아가서 마이크로비트에 이미 내장된 센서를 활용해서 코드 작성의 재미를 더 많이 생각하게 만듭니다. 이런 코드 작성 사례는 이 책 전체에서 계속 다루어집니다.

파이썬 언어의 기초적인 문법과 간단한 코드 실행결과를 경험해보기 바랍니다. 다만, 파이썬 언어를 어느 정도 이해한 분이라면, 이 책의 본론에 해당하는 텐덤로봇부터 시작하기 위해 제 3장부터 시작해도 좋습니다.

이 장에서 설명하는 파이썬 언어의 클래스 사용법은 충분하지 않을 수 있습니다. 클래스 코딩 문법은 많은 지면과 사례를 포함하기 때문에 간단히 요약해서 설명합니다. 다만, 텐덤로봇의 동작 예제를 다루면서 클래스에 대한 개념을 부분적으로 소개합니다.

2.1 데이터 출력하기

마이크로비트는 파이썬 코드를 테스트하고 디버깅하기 위해 REPL과 시리얼 모니터 기능을 제공합니다. 이 두 가지 방법을 사용하면 마이크로비트와 실시간으로 상호작용하고 데이터를 확인할 수 있습니다. REPL을 사용해서 데이터를 직접 출력하거나 또는 시리얼출력을 위해 준비된 마이크로비트 함수 `print`를 사용하고, 마이크로비트 5x5 LED 디스플레이 출력을 위해 `display.scroll`, `display.show` 함수를 사용할 수 있습니다.

(1) REPL 사용하기

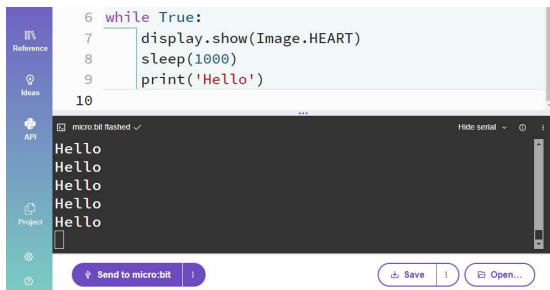
REPL(Read-Eval-Print Loop)은 파이썬 인터프리터의 대화형 모드로, 명령어를 입력하면 즉시 실행하고 결과를 출력해주는 환경입니다. 마이크로비트의 REPL을 사용하면 실시간으로 코드를 테스트하고 빠르게 피드백을 받을 수 있습니다.

온라인 편집기

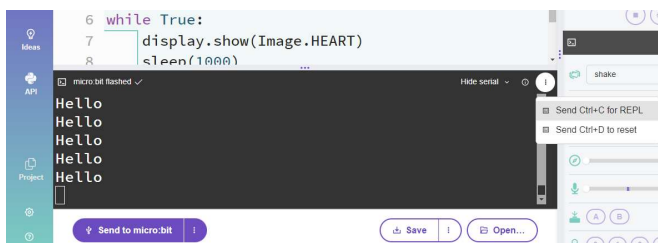
온라인 편집기를 열고 마이크로비트를 연결해서, 초기 코드를 플래시한 다음 편집기 하단에서 'Show serial'을 찾고 클릭하면 시리얼출력이 제공됩니다.



(a) 시리얼 출력 표시 전 화면



(b) 시리얼 출력 표시 후 화면



(c) REPL 화면으로 전환하기

그림 2.1 파이썬 온라인 편집기에서 데이터 출력방법: 시리얼 / REPL

Mu 에디터

- USB 케이블을 통해 마이크로비트를 컴퓨터에 연결한 다음, Mu Editor 설정 및 REPL 모드를 실행합니다.

- Mu Editor 상단 메뉴에서 'Mode' 버튼을 클릭하고 'BBC micro' 모드를 선택합니다.

- 마이크로비트를 연결한 후, Mu Editor 하단 메뉴에서 'REPL' 버튼을 클릭합니다.

- 새로운 창이 열리며, REPL 환경이 나타납니다.

- REPL 환경에서 명령어를 입력하고 Enter 키를 누르면, 명령어가 즉시 실행되고 결과가 출력됩니다.

```
>>> from microbit import *
>>> display.scroll("Hello, Micro:bit!")
>>> temperature = temperature()
>>> print("Temperature:", temperature)
```

(2) 시리얼 모니터 사용하기

시리얼 모니터는 마이크로비트와 컴퓨터 간의 시리얼 통신을 통해 데이터를 주고받을 수 있는 도구입니다. 이를 통해 마이크로비트의 센서 데이터나 디버깅 메시지를 실시간으로 모니터링할 수 있습니다.

- USB 케이블을 통해 마이크로비트를 컴퓨터에 연결한 다음, 시리얼 출력을 포함한 파이썬 코드를 작성합니다. 예를 들어, 마이크로비트의 온도 센서 값을 시리얼 모니터에 출력하는 코드를 작성해보겠습니다.

```
from microbit import *
while True:
    temperature = temperature()
    print("Temperature:", temperature)
    sleep(1000)
```

- Mu Editor에서 "Flash" 버튼을 눌러 코드를 마이크로비트에 업로드합니다.

- 코드가 업로드된 후, Mu Editor 하단 메뉴에서 'Serial' 버튼을 클릭하여 시리얼 모니터를 엽니다.

- 시리얼 모니터 창에서 마이크로비트의 시리얼 출력을 실시간으로 확인할 수 있습니다.

2.2 파이썬 기초

(1) 변수 (variable)

변수는 데이터를 저장하기 위한 공간입니다. 변수는 이름을 갖고 있으며, 데이터를 저장하고 필요할 때 불러올 수 있습니다.

예제2.1 “변수 사용하여 디스플레이에 메시지와 데이터 표시하기”

```
# ch2_01_variables_display.py
from microbit import *
```

```
# Declare and initialize variables
temperature = 25
message = "Hello, Micro:bit!"

# Use variables
display.scroll(message)
display.scroll("Temperature: " + str(temperature) + "C")
```

(2) 자료형(Data Types)

파이썬에는 여러 가지 자료형이 있습니다. 주요 자료형에는 정수(int), 실수(float), 문자열(str), 불리언(bool) 등이 있습니다. 마이크로비트에서도 이 자료형을 사용할 수 있습니다. 아래 기본 자료형 이외에도 컬렉션 자료형과 고급 자료형이 있습니다.

[1] 기본 자료형

정수형 (int):

정수 값을 저장합니다. 예: 1, -5, 42

실수형 (float):

소수점을 포함하는 숫자를 저장합니다. 예: 3.14, -0.001, 2.5

문자열형 (str):

문자열을 저장합니다. 예: "Hello", '123', "Micro"

불리언형 (bool):

참(True) 또는 거짓(False)의 값을 저장합니다. 예: True, False

예제2.2 “다양한 데이터 타입 사용하여 디스플레이에 출력하기”

```
# ch2_02_data_types.py

from microbit import *

# Use various data types
count = 10          # Integer
temperature = 23.5  # Float
greeting = "Hi!"    # String
is_raining = False  # Boolean

# Display variable values
display.scroll(greeting)
display.scroll("Count: " + str(count))
display.scroll("Temp: " + str(temperature))

if is_raining:
    display.scroll("It is raining")
else:
    display.scroll("It is not raining")
```

[2] 컬렉션 자료형

리스트 (list):

순서가 있는 변경 가능한 시퀀스를 저장합니다. 대괄호([])로 묶으며, 다양한 자료형을

포함할 수 있습니다.

```
fruits = ['apple', 'banana', 'cherry']
numbers = [1, 2, 3, 4, 5]
mixed_list = [1, 'apple', True]
```

튜플 (tuple):

순서가 있는 변경 불가능한 시퀀스를 저장합니다. 소괄호(())로 묶으며, 다양한 자료형을 포함할 수 있습니다.

```
coordinates = (10.0, 20.0)
colors = ('red', 'green', 'blue')
```

집합 (set):

순서가 없고 중복을 허용하지 않는 컬렉션을 저장합니다. 중괄호({})로 묶으며, 고유한 값을 가집니다.

```
unique_numbers = {1, 2, 3, 4, 5}
```

사전 (dictionary):

키-값 쌍을 저장합니다. 중괄호({ })로 묶으며, 키와 값은 콜론(:)으로 구분합니다.

```
person = {
    'name': 'Alice',
    'age': 25,
    'city': 'New York'
}
```

[3] 고급 자료형

배열 (array):

동일한 자료형의 값들을 저장합니다. 배열 모듈을 통해 사용되고, 리스트와의 차이점은 리스트는 다양한 자료형을 포함할 수 있지만, 배열은 동일한 자료형만 포함할 수 있습니다.

NoneType:

값이 없음을 나타내고, None 키워드를 사용합니다.

```
nothing = None
```

[4] 자료형 변환

파이썬에서는 서로 다른 자료형 간에 변환이 가능합니다. 이를 형 변환(Type Casting)이라고 합니다.

```
# 정수 -> 문자열
a = 123
a_str = str(a) # '123'

# 문자열 -> 정수
b = "456"
b_int = int(b) # 456

# 문자열 -> 실수
c = "3.14"
c_float = float(c) # 3.14
```

```
# 정수 -> 실수
d = 10
d_float = float(d) # 10.0

# 실수 -> 정수
e = 9.99
e_int = int(e) # 9
```

[5] 자료형 확인

파이썬에서는 변수의 자료형을 확인하기 위해 `type()` 함수를 사용할 수 있습니다.

```
x = 10
print(type(x)) # <class 'int'>

y = 3.14
print(type(y)) # <class 'float'>

z = "Hello"
print(type(z)) # <class 'str'>
```

(3) 연산자(Operators)

파이썬 연산자는 변수와 값에 대해 산술 연산, 비교 연산, 논리 연산 등을 수행하는 데 사용됩니다.

산술연산자

덧셈 (+), 뺄셈 (-), 곱셈 (*), 나눗셈 (/), 몫 (/), 나머지 (%), 거듭제곱 (**)

예제2.3 "기본 산술 연산으로 계산 결과 디스플레이하기"

```
# ch2_03_arithmetic_operations.py

from microbit import *

# Arithmetic operations
a = 10
b = 3

sum = a + b          # Addition
difference = a - b    # Subtraction
product = a * b       # Multiplication
quotient = a / b      # Division
quotient_int = a // b # Integer division
remainder = a % b     # Modulus
power = a ** b        # Exponentiation

# Display results
display.scroll("Sum: " + str(sum))
display.scroll("Diff: " + str(difference))
display.scroll("Prod: " + str(product))
display.scroll("Quot: " + str(quotient))
display.scroll("Int Quot: " + str(quotient_int))
display.scroll("Rem: " + str(remainder))
display.scroll("Pow: " + str(power))
```


비교연산자

값들을 비교하여 참(True) 또는 거짓(False)을 반환합니다.

== (같다), != (같지 않다), > (크다), < (작다), >= (크거나 같다), <= (작거나 같다)

예제2.4 "비교 연산자를 사용하여 관계 평가하기"

```
# ch2_04_comparison_operators.py

from microbit import *

# Comparison operations
a = 10
b = 5

is_equal = (a == b)           # Equal to
is_not_equal = (a != b)      # Not equal to
is_greater = (a > b)         # Greater than
is_lesser = (a < b)          # Less than
is_greater_or_equal = (a >= b) # Greater than or equal to
is_lesser_or_equal = (a <= b) # Less than or equal to

# Display results
display.scroll("Equal: " + str(is_equal))
display.scroll("Not Equal: " + str(is_not_equal))
display.scroll("Greater: " + str(is_greater))
display.scroll("Lesser: " + str(is_lesser))
display.scroll("Greater or Equal: " + str(is_greater_or_equal))
display.scroll("Lesser or Equal: " + str(is_lesser_or_equal))
```

논리연산자

불리언 값을 조합하여 참(True) 또는 거짓(False)을 반환합니다.

and (그리고), or (또는), not (부정)

예제2.5 "논리 연산자를 사용하여 조건 평가하기"

```
# ch2_05_logical_operations.py

from microbit import *

# Logical operations
is_raining = True
is_windy = False

is_bad_weather = is_raining and is_windy # Both are True
is_either_bad = is_raining or is_windy   # At least one is True
is_not_raining = not is_raining          # Negation

# Display results
display.scroll("Bad Weather: " + str(is_bad_weather))
display.scroll("Either Bad: " + str(is_either_bad))
display.scroll("Not Raining: " + str(is_not_raining))
```

2.3 코드 제어 구조

마이크로비트를 사용해서 파이썬 사용법의 기초 조건문과 반복문을 소개합니다. 프로그래

밍에서 조건문과 반복문은 기본적이면서도 매우 중요한 개념입니다. 이 두 가지는 프로그램의 흐름을 제어하고, 특정 작업을 여러 번 수행할 수 있게 해줍니다. 조건문은 특정 조건이 참(True)인지 거짓(False)인지에 따라 다른 코드를 실행하는 구조입니다. 조건문을 사용하면 프로그램이 상황에 따라 다르게 동작하도록 할 수 있습니다.

(1) 조건문 (if, elif, else)

조건문은 특정 조건에 따라 다른 코드를 실행하고자 할 때 사용됩니다. 마이크로비트에서는 버튼을 누르거나 센서 값을 확인하여 조건문을 사용할 수 있습니다. 아래 코드는 버튼 A가 눌리면 "Button A" 메시지를, 버튼 B가 눌리면 "Button B" 메시지를 LED 디스플레이에 스크롤합니다. 아무 버튼도 눌리지 않으면 하트 모양을 표시합니다.

예제2.6 "버튼 입력 감지하여 디스플레이에 반응하기"

```
#ch2_06_button_input.py
from microbit import *
while True:
    if button_a.is_pressed():
        display.scroll("Button A")
    elif button_b.is_pressed():
        display.scroll("Button B")
    else:
        display.show(Image.HEART)
    sleep(100)
```

(2) 반복문 (for, while)

반복문은 특정 코드 블록을 여러 번 실행하고자 할 때 사용됩니다. 마이크로비트에서는 LED 매트릭스에 패턴을 반복해서 표시하거나, 센서 값을 계속 확인하는 등의 작업에 사용할 수 있습니다. 이 코드는 숫자 0부터 4까지를 LED 매트릭스에 순차적으로 스크롤합니다. 아래 예제는 for 반복문과 while 반복문 사용법을 나타낸 것입니다.

예제2.7 "반복문을 사용하여 숫자 디스플레이하기"

```
# ch2_07_loop_display_numbers.py
from microbit import *
for i in range(5):
    display.scroll(str(i))
    sleep(500)
```

예제2.8 "온도 센서를 사용하여 실시간 온도 평가하기"

```
#ch2_08_temperature_sensor.py
```

```

from microbit import *

while True:
    temp = temperature()
    if temp > 30:
        display.scroll("Hot!")
    elif temp < 10:
        display.scroll("Cold!")
    else:
        display.scroll("Okay")

    sleep(1000)

```

이 코드는 온도 센서 값을 지속적으로 확인하여, 온도가 30도 이상이면 "Hot!", 10도 이하이면 "Cold!", 그 외에는 "Okay" 메시지를 LED 디스플레이에 스크롤합니다.

시퀀스 자료형은 파이썬에서 순서가 있는 데이터의 집합을 표현하는 자료형인데, 해당 자료형은 자료형으로는 리스트, 튜플, 문자열이 있습니다. 반복문과 함께 사용하면 각 요소를 순차적으로 처리할 수 있습니다.

[1] 리스트(List)를 사용한 반복문

리스트는 변경 가능한 시퀀스 자료형입니다. 리스트의 각 요소를 순차적으로 처리하기 위해 for 반복문을 사용할 수 있습니다.

```

fruits = ['apple', 'banana', 'cherry', 'date']

for fruit in fruits:
    print(fruit)

```

리스트의 각 숫자를 제공하여 새로운 리스트 생성

```

numbers = [1, 2, 3, 4, 5]
squared_numbers = []

for number in numbers:
    squared_numbers.append(number ** 2)

print(squared_numbers)

```

[2] 튜플(Tuple)을 사용한 반복문

튜플은 변경 불가능한 시퀀스 자료형입니다. 리스트와 마찬가지로 for 반복문을 사용하여 각 요소를 순차적으로 처리할 수 있습니다.

```

coordinates = (10.0, 20.0, 30.0, 40.0)

for coordinate in coordinates:
    print(coordinate)

```

[3] 문자열(String)을 사용한 반복문

문자열은 문자들의 시퀀스입니다. for 반복문을 사용하여 문자열의 각 문자를 순차적으로 처리할 수 있습니다.

```

message = "Hello, Micro:bit!"

for char in message:
    print(char)

```

(3) 조건문과 반복문을 함께 사용하기

이 코드는 버튼 A를 누를 때마다 숫자가 1씩 증가하고, 그 숫자를 LED 매트릭스에 표시합니다. 버튼 A는 스크롤이 지나간 다음 버튼을 누르면 카운터됩니다. 버튼 B를 누르면 숫자가 0으로 초기화되고 "Reset" 메시지가 표시됩니다.

예제2.9 "버튼 입력으로 카운터 조작하기"

```
# ch2_09_button_counter.py
from microbit import *

count = 0

while True:
    if button_a.is_pressed():
        count += 1
        display.scroll(str(count))
        sleep(200) # Add delay to prevent rapid button press detection

    elif button_b.is_pressed():
        count = 0
        display.scroll("Reset")
        sleep(200)
    sleep(100)
```

2.4 함수의 정의와 호출

함수는 프로그래밍에서 코드의 재사용성을 높이고, 코드를 구조화하여 더 읽기 쉽게 만드는 중요한 도구입니다. 파이썬에서 함수를 정의하고 호출하는 방법을 상세히 설명합니다.

(1) 함수란 무엇인가?

함수는 특정 작업을 수행하는 코드 블록입니다. 함수를 사용하면 동일한 작업을 여러 번 작성하지 않고도 재사용할 수 있습니다. 함수는 입력을 받아서 처리한 후 결과를 반환할 수 있습니다.

(2) 함수 정의하기

함수를 정의하려면 `def` 키워드를 사용합니다. 함수의 이름과 매개변수(필요한 경우)를 정의하고, 함수 본문에는 수행할 작업을 작성합니다. 아래 함수 예제는 'a'와 'b'라는 두 개의 매개변수를 받아서, 그 합을 'result' 변수에 저장하고 반환합니다.

```
def 함수이름(매개변수1, 매개변수2, ...):
    수행할 코드
    return 반환값
```

두 수를 더하는 함수를 정의하기

```
def add(a, b):
    result = a + b
```

```
return result
```

(3) 함수 호출하기

정의된 함수를 호출하려면 함수 이름과 함께 필요한 매개변수를 괄호 안에 넣어주면 됩니다.

함수이름(인수1, 인수2, ...)

'add' 함수를 호출하여 결과 출력하기

```
sum = add(3, 5)
print(sum) # 출력: 8
```

(4) 매개변수가 없는 함수

매개변수가 필요 없는 함수도 정의할 수 있습니다.

인사말을 출력하는 함수를 정의하고 호출하기

```
def greet():
    print("Hello, World!")

greet() # 출력: Hello, World!
```

(5) 매개변수가 있는 함수

매개변수를 사용하면 함수를 더 유연하게 만들 수 있습니다.

이름을 입력받아 인사말을 출력하는 함수

```
def greet(name):
    print(f"Hello, {name}!")

greet("Alice") # 출력: Hello, Alice!
greet("Bob")   # 출력: Hello, Bob!
```

(6) 기본 매개변수

함수를 정의할 때 매개변수에 기본값을 지정할 수 있습니다. 기본값이 지정된 매개변수는 함수 호출 시 해당 매개변수를 생략할 수 있습니다.

기본 매개변수를 사용하는 함수

```
def greet(name="Guest"):
    print(f"Hello, {name}!")

greet() # 출력: Hello, Guest!
greet("Alice") # 출력: Hello, Alice!
```

예제2.10 "함수를 사용하여 디스플레이 제어하기"

```
# ch2_10_function.py
from microbit import *
```

```

# Function definitions
def display_message(message):
    """Scroll the given message on the display."""
    display.scroll(message)
    sleep(500)

def display_heart():
    """Display a heart image on the screen."""
    display.show(Image.HEART)
    sleep(500)

def clear_display():
    """Clear the display."""
    display.clear()

# Main loop
while True:
    if button_a.is_pressed():
        display_message("Hello!")
    elif button_b.is_pressed():
        display_heart()
    else:
        clear_display()
    sleep(100) # Add delay to avoid processing button inputs too quickly

```

(7) 여러 값을 반환하는 함수

함수는 하나 이상의 값을 반환할 수 있습니다. 여러 값을 반환할 때는 튜플을 사용합니다.

두 수의 합과 곱을 반환하는 함수

```

def add_and_multiply(a, b):
    sum = a + b
    product = a * b
    return sum, product

result_sum, result_product = add_and_multiply(3, 5)
print("Sum: {}, Product: {}".format(result_sum, result_product))
# 출력: Sum: 8, Product: 15

```

예제2.11 "버튼 입력을 통해 함수로 간단한 연산 수행하기"

```

# ch2_11_button_input_function.py

from microbit import *

# Define a function to calculate the sum and product of two numbers
def add_and_multiply(a, b):
    total = a + b
    product = a * b
    return total, product

while True:
    if button_a.is_pressed():
        # When button A is pressed, calculate the result using 3 and 5
        result_sum, result_product = add_and_multiply(3, 5)
        display.scroll("Sum: {}".format(result_sum))
        display.scroll("Product: {}".format(result_product))
        sleep(1000) # Wait briefly for the next input

        print("Sum: {}, Product: {}".format(result_sum, result_product))

```

```
sleep(100) # Default waiting time between button input detection
```

(8) 지역 변수와 전역 변수

지역 변수: 함수 내에서 정의된 변수로, 함수 내에서만 접근할 수 있습니다.

전역 변수: 함수 외부에서 정의된 변수로, 모든 함수에서 접근할 수 있습니다.

지역변수와 전역변수 사용방법을 비교합니다.

```
x = 10 # 전역 변수
def my_function():
    x = 5 # 지역 변수
    print("지역 변수 x: {}".format(x))

my_function() # 출력: 지역 변수 x: 5
print("전역 변수 x: {}".format(x)) # 출력: 전역 변수 x: 10
```

my_function() 가 호출되면, 함수내에서만 사용되는 지역변수 x 를 사용하므로 출력 값은 5가 됩니다. 그리고 print("전역 변수 x: {}".format(x)) 문장은 전역변수 x 를 사용합니다. 그래서 출력값은 10이 됩니다.

예제2.12 "지역 변수와 전역 변수를 사용하여 카운터 조작하기"

```
# ch2_12_global_local_variables.py

from microbit import *

# Declare a global variable
counter = 0

def update_counter(increment):
    global counter # Declare usage of the global variable
    # Local variable
    temp_counter = counter
    temp_counter += increment
    counter = temp_counter
    return counter

def display_counter():
    # Global variable can be accessed inside the function
    display.scroll("Counter: {}".format(counter))

# Main loop
while True:
    if button_a.is_pressed():
        # Increase the counter by 1 when button A is pressed
        update_counter(1)
        display_counter()
        sleep(1000) # Wait briefly before the button can be pressed again

    elif button_b.is_pressed():
        # Decrease the counter by 1 when button B is pressed
        update_counter(-1)
        display_counter()
        sleep(1000)

    sleep(100) # Default delay between loops
```

- 전역 변수:

`counter = 0`: 전역 변수 `counter`는 프로그램 전체에서 접근할 수 있으며, 초기값으로 0을 설정합니다.

- 지역 변수와 전역 변수의 사용:

`update_counter(increment)`: 전역 변수 `counter`를 업데이트하는 함수로, `increment` 값을 인자로 받아 카운터를 증가 또는 감소시킵니다. 이 함수는 `global counter`를 선언하여 함수 내부에서 전역 변수를 수정할 수 있게 합니다.

함수 내부의 `temp_counter`는 지역 변수로, 함수가 실행되는 동안에만 존재합니다.

- 전역 변수 출력:

`display_counter()`: 전역 변수 `counter`의 값을 디스플레이에 출력하는 함수입니다.

- 메인 루프:

`while True`: 프로그램이 계속해서 버튼 입력을 감지하고 카운터를 업데이트할 수 있도록 무한 루프를 사용합니다.

`if button_a.is_pressed()`: 버튼 A가 눌렸을 때 `update_counter(1)`을 호출하여 카운터를 1 증가시킵니다.

`elif button_b.is_pressed()`: 버튼 B가 눌렸을 때 `update_counter(-1)`을 호출하여 카운터를 1 감소시킵니다.

2.5 클래스의 선언과 객체 생성

마이크로비트로 파이썬의 클래스와 객체 지향 프로그래밍(OOP)의 기본 개념을 이해하도록 설명합니다. 클래스는 객체를 생성하기 위한 설계도이며, 객체는 클래스의 인스턴스입니다. 여기서는 클래스의 선언과 객체 생성을 마이크로비트를 사용하여 상세히 설명하겠습니다.

(1) 클래스 선언

클래스는 `class` 키워드를 사용하여 선언합니다. 클래스 내부에는 속성(변수)과 메서드(함수)가 정의됩니다.

클래스의 기본 구조:

```
class 클래스이름:
    def __init__(self, 매개변수1, 매개변수2, ...):
        # 초기화 메서드 (생성자)
        self.속성1 = 매개변수1
        self.속성2 = 매개변수2
        ...

    def 메서드이름(self, 매개변수1, 매개변수2, ...):
        # 메서드 정의
        ...
```

(2) 객체 생성

객체는 클래스를 인스턴스화하여 생성됩니다. 클래스 이름 뒤에 괄호를 사용하여 생성자를 호출합니다.

객체생성 기본 구조:

```
객체이름 = 클래스이름(매개변수1, 매개변수2, ...)
```

예제 1 : LED 제어 클래스

마이크로비트의 LED 디스플레이를 제어하는 클래스를 정의하고, 객체를 생성하여 사용해보겠습니다.

[1] 클래스 정의

```
from microbit import *

class LEDController:
    def __init__(self, message):
        self.message = message

    def display_message(self):
        display.scroll(self.message)

    def display_heart(self):
        display.show(Image.HEART)

    def clear_display(self):
        display.clear()
```

[2] 객체 생성 및 메서드 호출

```
# 객체 생성
led = LEDController("Hello, Micro:bit!")

# 메시지 출력
led.display_message()

# 하트 이미지 출력
led.display_heart()
sleep(2000) # 2초 동안 하트 이미지를 표시

# 디스플레이 지우기
led.clear_display()
```

마이크로비트의 LED 디스플레이를 제어하기 위해 정의한 클래스와 생성된 객체를 설명하면 다음과 같습니다.

클래스 정의:

LEDController 클래스를 정의하고, `__init__` 메서드(생성자)를 통해 `message` 속성을 초기화합니다.

`display_message`, `display_heart`, `clear_display` 메서드를 정의하여 LED 디스플레이를 제어하는 기능을 추가합니다.

객체 생성 및 메서드 호출:

LEDController 클래스를 인스턴스화하여 `led` 객체를 생성합니다. 이때, "Hello, Micro!" 메시지를 초기 값으로 전달합니다.

`led.display_message()`를 호출하여 LED 디스플레이에 메시지를 스크롤합니다.

`led.display_heart()`를 호출하여 하트 이미지를 표시합니다.

`sleep(2000)`를 사용하여 2초 동안 하트 이미지를 표시한 후, `led.clear_display()`를 호출하여 디스플레이를 지웁니다.

클래스와 객체의 필요성

코드의 재사용성: 클래스를 정의하면, 동일한 코드를 여러 번 작성할 필요 없이 객체를 생성하여 재사용할 수 있습니다.

구조적 프로그래밍: 클래스와 객체를 사용하면 코드가 더 구조적이고 이해하기 쉬워집니다.

캡슐화: 클래스 내부에 데이터와 메서드를 캡슐화하여 외부에서 접근하지 못하도록 보호할 수 있습니다.

유연성: 클래스를 사용하면 프로그램을 유연하게 확장하고 수정할 수 있습니다.

예제 2 : 버튼입력 처리 클래스

마이크로비트의 버튼 입력을 처리하는 클래스를 정의하고 사용해보겠습니다.

[1] 클래스 정의

```
from microbit import *
class ButtonHandler:
    def __init__(self):
        self.button_a_pressed = False
        self.button_b_pressed = False

    def check_buttons(self):
        if button_a.is_pressed():
            self.button_a_pressed = True
        else:
            self.button_a_pressed = False

        if button_b.is_pressed():
            self.button_b_pressed = True
        else:
            self.button_b_pressed = False

    def display_button_status(self):
        if self.button_a_pressed:
            display.scroll("A Pressed")
        if self.button_b_pressed:
            display.scroll("B Pressed")
```

[2] 객체 생성 및 메서드 호출

```
# 객체 생성
button_handler = ButtonHandler()

while True:
    # 버튼 상태 확인
    button_handler.check_buttons()

    # 버튼 상태 출력
    button_handler.display_button_status()

    sleep(100)
```

마이크로비트의 버튼 입력을 처리하는 클래스를 정의하고 사용하는 코드를 설명해보겠습니다.

클래스 정의:

ButtonHandler 클래스를 정의하고, `__init__` 메서드에서 버튼 A와 B의 상태를 나타내는 속성을 초기화합니다.

`check_buttons` 메서드는 버튼 A와 B가 눌렸는지 확인하여 상태를 업데이트합니다.

`display_button_status` 메서드는 버튼이 눌린 상태를 LED 디스플레이에 출력합니다.

객체 생성 및 메서드 호출:

ButtonHandler 클래스를 인스턴스화하여 `button_handler` 객체를 생성합니다.

무한 루프를 사용하여 버튼 상태를 지속적으로 확인하고, 눌린 상태를 디스플레이에 출력합니다.

클래스와 객체는 파이썬의 강력한 기능 중 하나로, 프로그램을 더 구조적이고 유지보수하

기 쉽게 만들어줍니다. 마이크로비트를 사용하여 클래스를 정의하고 객체를 생성하는 방법을 이해하면, 더욱 복잡하고 유연한 프로그램을 작성할 수 있습니다. 이 개념들을 연습하고 다양한 예제를 시도해보세요!

예제 3 : 로봇 클래스

로봇 클래스를 만들어 보겠습니다.

```
class Robot:
    def __init__(self, name): # 생성자
        self.name = name # 속성

    def greet(self): # 메소드
        print(f"Hello, I am {self.name}")
```

Robot이라는 클래스를 선언한 후, `__init__` 메소드를 통해 객체가 생성될 때 초기화됩니다. 객체 생성은 아래와 같이 이루어집니다:

```
robot1 = Robot("MicroBot") # 객체 생성
robot1.greet() # 출력: Hello, I am MicroBot
```

생성자는 객체가 생성될 때 호출되는 메소드로, 객체의 초기 상태를 정의합니다. Python에서는 `__init__` 메소드가 생성자 역할을 합니다. 속성은 객체의 데이터나 상태를 저장하는 변수입니다. 메소드는 객체가 수행할 수 있는 동작을 정의하는 함수입니다.

2.6 생성자, 속성, 메소드

파이썬에서 객체 지향 프로그래밍(OOP)을 이해하려면 생성자, 속성, 메소드에 대한 이해가 필수적입니다.

(1) 생성자 (Constructor)

생성자는 클래스의 인스턴스(객체)가 생성될 때 자동으로 호출되는 특별한 메소드입니다. 주로 객체의 초기 상태를 설정하는 데 사용됩니다. 파이썬에서 생성자는 `__init__` 메소드로 정의됩니다.

객체의 초기화: 생성자는 객체가 생성될 때 필요한 초기 값을 설정합니다.

속성 초기화: 생성자를 통해 객체의 속성을 정의하고 초기 값을 할당할 수 있습니다.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

# 객체 생성
person1 = Person("Alice", 30)
print(person1.name) # 출력: Alice
print(person1.age) # 출력: 30
```

이 예제에서 `__init__` 메소드는 `name`과 `age` 속성을 초기화합니다. `Person` 클래스의 인스턴스를 생성할 때, 생성자가 자동으로 호출되어 `name`과 `age`를 설정합니다.

(2) 속성 (Attributes)

속성은 클래스에서 정의된 변수로, 객체의 상태나 데이터를 저장합니다. 속성은 클래스의 인스턴스 변수로 접근됩니다.

데이터 저장: 속성은 객체의 상태나 데이터를 저장합니다.

객체의 특성 표현: 객체의 특성을 나타내는 변수들입니다.

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

# 객체 생성
car1 = Car("Toyota", "Corolla")
print(car1.brand) # 출력: Toyota
print(car1.model) # 출력: Corolla
```

이 예제에서 `brand`와 `model`은 `Car` 클래스의 속성입니다. `car1` 객체를 생성할 때, 이 속성들은 `Toyota`와 `Corolla`로 초기화됩니다.

(3) 메소드 (Methods)

메소드는 클래스 내부에 정의된 함수로, 객체의 동작이나 기능을 정의합니다. 메소드는 객체의 속성을 이용해 작업을 수행하거나 객체의 상태를 변경할 수 있습니다.

동작 정의: 메소드는 객체가 수행할 수 있는 동작이나 기능을 정의합니다.

속성 접근 및 수정: 메소드를 통해 객체의 속성을 읽거나 변경할 수 있습니다.

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def bark(self):
        print(f"{self.name} says Woof!")

    def birthday(self):
        self.age += 1

# 객체 생성
dog1 = Dog("Buddy", 5)
dog1.bark() # 출력: Buddy says Woof!
print(dog1.age) # 출력: 5
dog1.birthday()
print(dog1.age) # 출력: 6
```

이 예제에서 `bark`와 `birthday`는 `Dog` 클래스의 메소드입니다. `bark` 메소드는 개의 이름과 함께 "Woof!"를 출력하고, `birthday` 메소드는 개의 나이를 한 살 증가시킵니다.

(4) 종합 예제

다음은 생성자, 속성, 메소드를 모두 포함하는 종합적인 예제입니다.

```
class BankAccount:
    def __init__(self, account_holder, balance=0):
        self.account_holder = account_holder
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        print(f'{amount} deposited. New balance is {self.balance}.')

    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient funds!")
        else:
            self.balance -= amount
            print(f'{amount} withdrawn. New balance is {self.balance}.')

# 객체 생성
account = BankAccount("Alice", 1000)
account.deposit(500)    # 출력: 500 deposited. New balance is 1500.
account.withdraw(200)   # 출력: 200 withdrawn. New balance is 1300.
account.withdraw(1500)  # 출력: Insufficient funds!
```

종합예제를 상세히 설명하면 다음과 같습니다.

생성자 (`__init__`): `__init__` 메소드는 계좌 소유자와 초기 잔액을 설정합니다. 객체가 생성될 때 호출되며, 초기 상태를 설정합니다.

속성 (Attributes): `account_holder`와 `balance`는 계좌 소유자와 잔액을 저장하는 속성입니다. 객체의 상태나 데이터를 저장하는 변수입니다.

메소드 (Methods): `deposit`과 `withdraw` 메소드는 각각 입금과 출금을 처리합니다. 객체의 동작이나 기능을 정의하는 함수입니다.

2.7 상속, 다형성, 캡슐화

객체 지향 프로그래밍(OOP)의 중요한 개념인 상속, 다형성, 캡슐화에 대해 설명하겠습니다. 이 개념들은 코드의 재사용성을 높이고, 유지보수를 용이하게 하며, 프로그램의 구조를 체계적으로 만드는 데 중요한 역할을 합니다.

(1) 상속 (Inheritance)

상속은 기존 클래스(부모 클래스 또는 슈퍼 클래스)의 속성과 메소드를 새로운 클래스(자식 클래스 또는 서브 클래스)가 물려받아 사용하는 개념입니다. 상속을 통해 코드의 재사용성을 높이고, 클래스 간의 계층 구조를 만들 수 있습니다.

코드 재사용: 기존 클래스 기능을 재사용하여 새로운 클래스를 쉽게 생성할 수 있습니다.

계층 구조: 클래스 간의 계층 구조를 통해 코드의 구조를 명확하게 할 수 있습니다.

확장성: 기존 클래스의 기능을 확장하거나 수정하여 새로운 기능을 추가할 수 있습니다.

[1] 부모로부터 상속받은 자녀 클래스의 메소드 사용방법

```
# 부모 클래스
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        pass # 자식 클래스에서 구현될 메소드

# 자식 클래스
class Dog(Animal):
    def speak(self):
        return f"{self.name} says Woof!"

class Cat(Animal):
    def speak(self):
        return f"{self.name} says Meow!"

# 객체 생성
dog = Dog("Buddy")
cat = Cat("Whiskers")

print(dog.speak()) # 출력: Buddy says Woof!
print(cat.speak()) # 출력: Whiskers says Meow!
```

위 코드의 클래스 상속에서 Animal 클래스는 부모 클래스이며, Dog와 Cat 클래스는 Animal 클래스를 상속받습니다. Dog와 Cat 클래스는 Animal 클래스의 속성(name)과 메소드(speak)를 물려받습니다. 각 자식 클래스는 speak 메소드를 자신만의 방식으로 구현합니다.

[2] 부모 클래스의 메소드 명시적 호출

부모 클래스의 메소드를 명시적으로 호출하려면 super() 함수를 사용합니다. super() 함수는 부모 클래스의 메소드를 호출할 때 사용되며, 이를 통해 자녀 클래스가 부모 클래스의 메소드를 확장하거나 변경할 수 있습니다. 다음은 super() 함수를 사용하여 부모 클래스의 메소드를 호출하는 예제입니다.

```
class Animal:
    def speak(self):
        return "Generic animal sound"

class Dog(Animal):
    def speak(self):
        parent_message = super().speak() # 부모 클래스의 메소드 호출
        return parent_message + " Woof!"

class Cat(Animal):
    def speak(self):
        parent_message = super().speak() # 부모 클래스의 메소드 호출
        return parent_message + " Meow!"

# 객체 생성
dog = Dog()
cat = Cat()

print(dog.speak()) # 출력: Generic animal sound Woof!
print(cat.speak()) # 출력: Generic animal sound Meow!
```

위 코드에서 `super()` 함수는 부모 클래스의 메소드를 명시적으로 호출할 때 사용됩니다. 부모 클래스 메소드 호출할 때 자녀 클래스에서 `super().메소드명()`을 사용하여 부모 클래스의 메소드를 호출할 수 있습니다. 이런 방법으로 자녀 클래스에서 부모 클래스의 메소드를 오버라이딩하면서, 부모 클래스의 메소드를 확장하여 새로운 기능을 추가할 수 있습니다.

[3] 부모 클래스에만 존재하는 메소드를 자녀 클래스에서 호출하기

자녀 클래스는 부모 클래스를 상속받기 때문에 부모 클래스의 모든 메소드와 속성을 상속받습니다. 따라서 자녀 클래스는 부모 클래스의 메소드를 직접 호출할 수 있습니다. 다음은 부모 클래스에만 존재하는 메소드를 자녀 클래스에서 호출하는 예제입니다.

```
class Animal:
    def speak(self):
        return "Generic animal sound"

    def move(self):
        return "The animal moves"

class Dog(Animal):
    def speak(self):
        return "Woof!"

# 객체 생성
dog = Dog()

# 자녀 클래스에서 부모 클래스의 메소드 호출
print(dog.speak()) # 출력: Woof!
print(dog.move())  # 출력: The animal moves
```

위 코드에서 `Dog` 클래스의 인스턴스를 생성합니다. 그리고 `dog.speak()`는 `Dog` 클래스에서 오버라이딩된 `speak` 메소드를 호출하여 "Woof!"를 반환합니다. 다음으로 `dog.move()`는 `Dog` 클래스에 정의되어 있지 않지만, `Animal` 클래스에서 상속받은 `move` 메소드를 호출하여 "The animal moves"를 반환합니다.

상속의 개념을 요약하면,

상속: 자녀 클래스는 부모 클래스의 모든 메소드와 속성을 상속받습니다.

부모 클래스 메소드 호출: 자녀 클래스에서 부모 클래스에만 존재하는 메소드를 호출할 수 있습니다.

메소드 오버라이딩: 자녀 클래스에서 부모 클래스의 메소드를 오버라이딩하여 자신만의 기능을 구현할 수 있습니다.

상속받은 메소드 사용: 자녀 클래스에 정의되지 않은 메소드는 부모 클래스에서 상속받아 사용할 수 있습니다.

(2) 다형성 (Polymorphism)

다형성은 동일한 인터페이스를 통해 서로 다른 데이터 타입을 다룰 수 있게 하는 개념입니다. 이는 주로 메소드 오버라이딩과 관련이 있으며, 같은 메소드가 클래스에 따라 다르게 동작할 수 있습니다.

유연성: 같은 메소드 호출이 클래스에 따라 다르게 동작하여 코드의 유연성을 높입니다.

인터페이스의 일관성: 다양한 객체들이 동일한 인터페이스를 구현하여 코드의 일관성을 유지할 수 있습니다.

```
class Animal:
    def speak(self):
        pass

class Dog(Animal):
    def speak(self):
        return "Woof!"

class Cat(Animal):
    def speak(self):
        return "Meow!"

def make_animal_speak(animal):
    print(animal.speak())

# 객체 생성
dog = Dog()
cat = Cat()

# 다형성 예제
make_animal_speak(dog) # 출력: Woof!
make_animal_speak(cat) # 출력: Meow!
```

- 기본 클래스 (Base Class) 정의

Animal 클래스는 기본 클래스입니다. speak 메소드는 아무런 동작도 하지 않으며, 자식 클래스에서 오버라이딩(재정의)될 것입니다. speak 메소드는 Animal 클래스의 인터페이스 역할을 합니다.

- 자식 클래스 (Derived Class) 정의

Dog와 Cat 클래스는 Animal 클래스를 상속받습니다. 두 클래스는 모두 speak 메소드를 오버라이딩하여 자신만의 방식으로 구현합니다. Dog 클래스는 "Woof!"를, Cat 클래스는 "Meow!"를 반환합니다.

- 다형성을 이용한 함수 정의

make_animal_speak 함수는 Animal 클래스 또는 이를 상속받은 클래스의 객체를 인수로 받습니다. 이 함수는 animal 객체의 speak 메소드를 호출하고 그 결과를 출력합니다.

make_animal_speak 함수에서 인수로 전달된 객체가 Animal 클래스 또는 이를 상속받은 클래스의 객체임을 확인하는 근거는 함수 호출 시 객체가 Animal 클래스에서 정의한 speak 메서드를 가지고 있으며, 이 메서드를 호출할 수 있다는 사실입니다. 이는 주로 파이썬의 동적 타이핑(dynamic typing)과 덕 타이핑(duck typing) 개념에 의해 가능합니다.

파이썬의 덕 타이핑 개념에서는 객체가 특정 인터페이스(여기서는 speak 메서드)를 구

현하고 있으면, 그 객체를 해당 인터페이스의 객체로 간주합니다. 즉, Animal 클래스를 상속받은 객체는 speak 메서드를 가지고 있으며, make_animal_speak 함수는 이를 신뢰하고 speak 메서드를 호출합니다. 이외에 isinstance 함수를 통한 타입 검사를 통해 확인할 수 있지만, 소개를 생략합니다.

- 객체 생성 및 함수 호출

Dog 클래스와 Cat 클래스의 인스턴스를 각각 생성합니다. make_animal_speak 함수를 호출하여 각 인스턴스를 인수로 전달합니다. 함수는 전달된 객체의 타입에 따라 적절한 speak 메소드를 호출하고, 결과를 출력합니다.

다형성 개념이 적용된 함수들에 대하여 요약하면,

make_animal_speak 함수는 Animal 클래스의 인스턴스를 인수로 받아 speak 메소드를 호출합니다.

Dog와 Cat 클래스는 Animal 클래스의 자식 클래스를 사용해서 speak 메소드를 각각 자신만의 방식으로 구현합니다.

make_animal_speak 함수는 인수로 전달된 객체의 타입에 따라 다른 결과를 출력합니다.

(3) 캡슐화 (Encapsulation)

캡슐화는 객체의 속성과 메소드를 하나로 묶고, 일부를 외부에 감추어 데이터의 무결성을 보호하는 개념입니다. 이를 통해 객체 내부의 구현을 외부로부터 숨기고, 객체 간의 상호작용을 명확히 정의할 수 있습니다.

정보 은닉: 객체 내부의 상태를 외부에서 직접 접근하지 못하도록 보호합니다.

데이터 보호: 잘못된 데이터 접근이나 수정으로부터 객체의 상태를 보호합니다.

유지보수성: 객체의 내부 구현을 숨겨 변경에 유연하게 대응할 수 있습니다.

```
class Account:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self._balance = balance # 비공개 속성

    def deposit(self, amount):
        if amount > 0:
            self._balance += amount
        else:
            print("Deposit amount must be positive!")

    def withdraw(self, amount):
        if amount > self._balance:
            print("Insufficient funds!")
        else:
            self._balance -= amount
```

```

    def get_balance(self):
        return self.__balance

# 객체 생성
account = Account("Alice", 1000)

# 예금과 출금
account.deposit(500)
print(account.get_balance()) # 출력: 1500
account.withdraw(200)
print(account.get_balance()) # 출력: 1300

# 비공개 속성 접근 시도
print(account.__balance) # 에러 발생: AttributeError

```

위 코드는 캡슐화 개념을 포함하고 있습니다. 아래에서 다시 설명합니다.

Account 클래스는 __balance라는 비공개 속성을 가지고 있습니다.

deposit, withdraw, get_balance 메소드는 이 비공개 속성에 접근할 수 있는 공개 메소드입니다.

직접 __balance 속성에 접근하려고 하면 에러가 발생하여 속성이 보호됩니다.

(Notice) -----

__ (더블 언더스코어)

파이썬에서 __ (더블 언더스코어)로 시작하는 변수나 메소드는 클래스 내부에서만 접근할 수 있는 비공개 속성 또는 메소드로 간주됩니다. 이는 클래스 외부에서 직접 접근하지 못하도록 하는 메커니즘입니다.

비공개 속성(Private Attribute): __ (더블 언더스코어)로 시작하는 속성은 비공개 속성으로, 클래스 외부에서 직접 접근할 수 없습니다.

이름 장식(Name Mangling): __로 시작하는 속성의 이름이 _클래스명__속성명 형태로 변경되어 클래스 외부에서 접근할 수 없도록 합니다.

데이터 캡슐화: 비공개 속성을 사용하여 클래스 내부의 데이터와 메소드를 보호하고, 외부로부터의 직접 접근을 방지합니다.

상속, 다형성, 캡슐화 개념을 요약하면 다음과 같습니다.

상속: 기존 클래스의 속성과 메소드를 새로운 클래스가 물려받아 재사용하고 확장할 수 있는 개념입니다.

다형성: 동일한 인터페이스를 통해 다양한 객체가 서로 다르게 동작할 수 있게 하는 개념입니다.

캡슐화: 객체의 내부 상태와 동작을 외부에 감추고, 필요한 인터페이스만을 제공하여 데이터의 무결성을 보호하는 개념입니다.

2.8 예외 처리

예외처리는 프로그램의 오류를 처리하고, 예기치 않은 상황에서 프로그램이 안전하게 종료되거나 복구될 수 있도록 도와줍니다.

파이썬에서 예외처리가 강조되는 이유는 프로그램의 안정성을 높이고, 코드의 가독성과 유지보수성을 향상시키며, 다양한 오류 상황에 대처하고 자원 관리를 확실하게 할 수 있기 때문입니다. 이를 통해 개발자는 보다 견고하고 신뢰성 있는 프로그램을 작성할 수 있습니다.

마이크로파이썬에서도 파이썬 언어와 마찬가지로 예외 처리가 특별히 강조됩니다. 이는 주로 임베디드 시스템의 특성과 제약, 그리고 안정성과 신뢰성을 유지해야 하는 요구사항 때문에 그렇습니다.

(1) 프로그램의 안정성 향상

예외 발생 시 프로그램 종료 방지:

예외 처리를 통해 프로그램이 예외 상황에서 안전하게 종료되거나 계속 실행될 수 있습니다. 예외를 처리하지 않으면 프로그램이 강제로 종료될 수 있습니다.

예기치 않은 오류 처리:

프로그램 실행 중에 발생할 수 있는 다양한 예외 상황을 사전에 처리하여 예기치 않은 오류로 인한 충돌을 방지합니다.

(2) 코드의 가독성 및 유지보수성 향상

명확한 오류 처리 로직:

예외 처리를 통해 오류가 발생할 수 있는 지점을 명확히 하고, 그에 대한 처리 방법을 코드에 명시할 수 있습니다. 이는 코드의 가독성을 높이고, 유지보수 시 어떤 부분에서 오류가 발생할 수 있는지 쉽게 이해할 수 있게 합니다.

비즈니스 로직과 오류 처리 로직의 분리:

예외 처리를 사용하면 비즈니스 로직과 오류 처리 로직을 분리할 수 있어 코드가 더 명확해지고, 각 부분을 독립적으로 수정하고 유지할 수 있습니다.

(3) 다양한 오류 상황에 대한 대처

다양한 예외 처리:

파이썬은 다양한 내장 예외 클래스를 제공하며, 사용자 정의 예외를 만들어 사용할 수도 있습니다. 이를 통해 다양한 오류 상황에 대해 세분화된 처리가 가능합니다.

상황별 적절한 대응:

특정 오류에 대해 적절한 대응 방법을 정의하여, 오류가 발생하더라도 프로그램이 유연하게 대처할 수 있습니다.

(4) 자원 관리

자원 해제 보장:

예외가 발생하더라도 파일, 네트워크 연결, 데이터베이스 연결 등과 같은 자원을 확실하게 해제할 수 있도록 도와줍니다. 이를 위해 try-finally 블록을 사용하여 자원을 안전하게 해제할 수 있습니다.

예외처리 구문 예제

기본 예외 처리:

```
try:
    # 예외가 발생할 수 있는 코드
    result = 10 / 0
except ZeroDivisionError as e:
    # 예외가 발생했을 때의 처리 코드
    print(f"Error: {e}")
else:
    # 예외가 발생하지 않았을 때의 코드
    print("Success:", result)
finally:
    # 예외 발생 여부와 상관없이 실행되는 코드
    print("Execution finished")
```

사용자 정의 예외:

```
class CustomError(Exception):
    pass

def risky_function():
    raise CustomError("Something went wrong!")

try:
    risky_function()
except CustomError as e:
    print(f"Caught a custom error: {e}")
```

예외처리의 구성 요소

try 블록: 예외가 발생할 가능성이 있는 코드를 포함합니다.

except 블록: 예외가 발생했을 때 실행할 코드를 포함합니다.

else 블록: 예외가 발생하지 않았을 때 실행할 코드를 포함합니다.

finally 블록: 예외 발생 여부와 상관없이 항상 실행할 코드를 포함합니다.

제 II 부 마이크로비트-아두이노 텐덤로보틱스

제 3 장 I2C통신과 텐덤로봇(tandem robot)

3.1 I2C 통신과 텐덤로봇

- (1) I2C통신 소개
- (2) 텐덤(tandem) 로봇이란?

3.2 텐덤로봇 I2C 데이터통신 기초

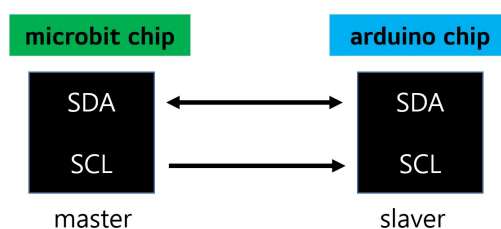
- (1) 문자 데이터 송수신
- (2) 숫자 데이터 송수신
- (3) 더 복잡한 데이터의 송수신

3.3 텐덤로봇 라이브러리 설치

- (1) 아두이노에 라이브러리 설치하는 방법
- (2) 마이크로비트에 라이브러리 설치하는 방법

3.4 텐덤로봇 라이브러리 개요

- (1) 마이크로비트 라이브러리 개요
- (2) 아두이노 라이브러리 개요
- (3) 아두이노 메인 파일 microbit_abot.ino



세상에서 가장 많이 알려진 초보자용 마이크로컨트롤러 부품인 '마이크로:비트'와 '아두이노'를 I2C통신으로 연결한 구조입니다. IOT세상을 만드는 데 두 개의 마이크로컨트롤러는 가성비가 가장 우수한 부품이라고 생각합니다.

당신이 이 장을 배우기 시작하면 IOT 세상에서 거의 모든 단말장치 동작을 스스로 해결할 수 있는 기초를 갖추게 됩니다. 마이크로컨트롤러 동작을 내 목적에 맞게 최적화하는 코딩 기술에 관한 것입니다.

이 장에서는 마이크로비트와 아두이노를 I2C통신으로 연결한 "텐덤로봇(Tandem_Robot)"을 소개하고, 마이크로비트와 아두이노 사이에서 데이터가 서로 어떻게 통신하는지 배울 기회를 제공합니다. 아두이노에서 동작하는 C++언어까지 포함하면 꽤 복잡한 코드가 적용됩니다. 먼저 마이크로비트에서만 사용하는 파이썬 언어를 먼저 익히기 바랍니다.

3.1 I2C 통신과 텐덤로봇

I2C(Inter-Integrated Circuit)는 여러 장치 간에 데이터를 직렬로 전송하기 위해 널리 사용되는 동기식, 멀티마스터, 멀티슬레이브 통신 프로토콜입니다. 이 통신 방식은 1982년 필립스 반도체(현재 NXP 반도체)가 개발했습니다. I2C는 마이크로컨트롤러, 센서, 메모리 장치들 간의 간단하고 효율적인 통신을 가능하게 합니다. 통신 데이터의 최소 단위는 **바이트(8비트)**입니다. I2C 프로토콜에서는 데이터가 바이트 단위로 전송되며, 각 바이트는 8비트로 구성됩니다. 이를 통해 데이터 전송의 표준화와 효율적인 통신이 가능합니다.

(1) I2C통신 소개

마이크로비트-아두이노 사이를 I2C통신으로 연결하는 방법은 두 개의 주요 라인, SDA(데이터 라인)와 SCL(클럭 라인)을 사용합니다. 마이크로비트 SDA 핀을 아두이노 SDA 핀에 연결하고, 마이크로비트 SCL 핀을 아두이노 SCL 핀에 연결합니다. 그 다음 마이크로비트와 아두이노의 GND 핀을 연결하여 공통 그라운드를 설정하고, 마이크로비트와 아두이노에 각각 전원을 공급합니다. (** 참고: 풀업 저항은 내부 풀업저항 또는 외부 풀업저항을 사용할 수 있고, 매우 근거리에서는 풀업저항 없이도 통신이 가능합니다.*)

- * SDA (Data Line): 데이터를 전송하는 라인
- * SCL (Clock Line): 데이터를 동기화하는 클럭 신호를 전송하는 라인

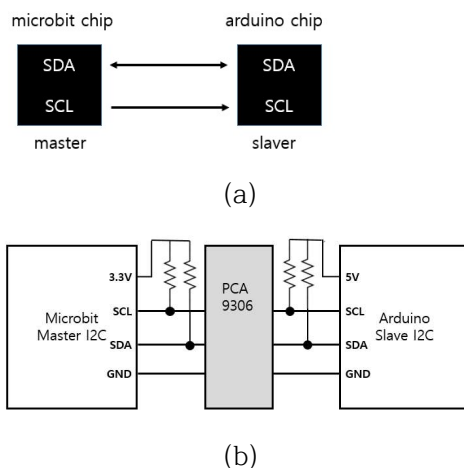


그림 3.1 마이크로비트-아두이노 텐덤로봇 I2C연결 (a) 개념도와 (b) 회로도

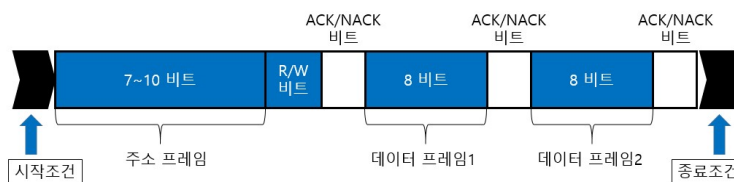


그림 3.2 텐덤로봇 I2C통신 바이트 구조 예시

[1] I2C통신 바이트 구조를 기초로, I2C통신 시퀀스

마스터: Start
마스터: 슬레이브 주소 + 쓰기 비트 (0)
슬레이브: ACK
마스터: 데이터 바이트
슬레이브: ACK
마스터: 데이터 바이트
슬레이브: ACK
마스터: Stop

- 시작과 정지 조건

시작 조건 (Start Condition): SCL이 높은 상태에서 SDA가 높음에서 낮음으로 전환될 때 발생합니다.

정지 조건 (Stop Condition): SCL이 높은 상태에서 SDA가 낮음에서 높음으로 전환될 때 발생합니다.

- 주소 전송

마스터는 시작 조건을 생성한 후 슬레이브 주소와 읽기/쓰기 비트를 전송합니다.

슬레이브 주소는 7비트 또는 10비트로 구성됩니다. 일반적으로 7비트를 사용합니다.

읽기/쓰기 비트: 1 비트로, 마스터가 데이터를 읽을지(1) 쓸지(0) 결정합니다.

- ACK/NACK 비트

ACK (Acknowledge): 슬레이브가 주소를 인식했음을 나타내기 위해 SDA 라인을 낮게 유지합니다.

NACK (Not Acknowledge): 슬레이브가 주소를 인식하지 못했음을 나타내기 위해 SDA 라인을 높게 유지합니다.

- 데이터 전송

마스터와 슬레이브 간의 데이터는 8비트 단위로 전송됩니다.

각 바이트 전송 후에는 ACK/NACK 비트가 따라옵니다.

- 반복 시작

마스터는 통신 도중에 반복 시작 조건을 생성하여 새로운 통신을 시작할 수 있습니다.

[2] I2C 데이터 통신속도

- 표준 모드(Standard Mode): 100 kbps
- 패스트 모드(Fast Mode): 400 kbps
- 패스트 모드 플러스(Fast Mode Plus): 1 Mbps
- 고속 모드(High-Speed Mode): 3.4 Mbps
- 울트라 패스트 모드(Ultra-Fast Mode): 5 Mbps

[3] I2C통신 전기적 특성

- 전압 레벨: 일반적으로 3.3V 또는 5V (다른 전압도 사용 가능)
- 풀업 저항: 각 라인에 풀업 저항 필요 (1kΩ ~ 10kΩ 범위에서 선택)

[4] I2C통신 주소

- 7비트 주소는 최대 127개 슬레이브 장치 연결 가능 (주소 0은 일반적으로 사용되지 않음)
- 7비트 주소: 일반적으로 사용되는 주소 길이
- 10비트 주소: 고유 주소가 더 많이 필요한 경우 사용

[5] I2C통신 멀티마스터 및 충돌방지

- 멀티마스터: 여러 마스터가 버스에 존재할 수 있으며, 충돌 발생 시 충돌 감지 및 회피 메커니즘 사용
- 충돌 감지: 두 마스터가 동시에 데이터를 전송하려 할 때 충돌 감지 (SDA 라인의 상태를 모니터링하여 충돌 확인)

[6] I2C통신 슬레이브 장치

- 응답 대기: 슬레이브는 주소를 수신하고 일치할 경우 ACK를 전송하여 응답
- 데이터 준비: 요청된 데이터 또는 명령에 따라 데이터를 준비하고 전송

(Notice) -----

아두이노와 마이크로비트의 I2C통신에 대하여

아두이노는 I2C 통신에서 마스터와 슬레이브 기능을 모두 지원합니다. 반면, 마이크로비트는 I2C 통신에서 마스터 기능만 지원합니다. 이는 마이크로비트가 교육용으로 단순성과 사용 편의성을 강조한 설계개념을 적용한 결과입니다.

(2) 텐덤(tandem) 로봇이란?

텐덤(tandem) 용어의 대중적 의미는 텐덤 자전거처럼 두 명 이상의 사람이 함께 탈 수 있도록 설계된 자전거를 의미하고, 항공기에서 두 명의 조종사가 앞뒤로 나란히 앉는 배치를 의미하기도 하며, 자동차 레이싱에서 텐덤 드라이빙이 두 대의 자동차가 서로 밀착하여 앞뒤로 주행하는 전략을 말하기도 합니다. 언어 학습에서 서로 다른 언어를 알고 있는 두 사람이 서로 다른 언어를 교차로 배우는 학습 방식을 텐덤 학습이라고 말하며, 비즈니스에서는 두 사람이 나란히 협력하여 작업하는 것을 의미하기도 합니다.

지금까지 I2C (Inter-Integrated Circuit) 통신으로 두 개의 마이크로컨트롤러를 연결하는 구조에 대해 "텐덤"이라는 용어가 사용된 사례는 없습니다. "텐덤"이라는 용어는 보통

두 개체가 동등하게 협력하는 상황을 나타내지만, I2C 통신에서는 마스터와 슬레이브 간의 명확한 역할 분담이 있기 때문에 이 용어는 적합하지 않습니다.

마이크로:비트와 아두이노가 서로 협력하는 의미를 "협력 로봇 (Cooperative Robots)" 또는 "멀티-로봇 시스템 (Multi-Robot Systems)" 용어로 표현하거나, "멀티 마이크로컨트롤러 시스템"이라고 말할 수 있지만, 교육 목적의 독창성을 반영하기 어려운 것 같습니다.

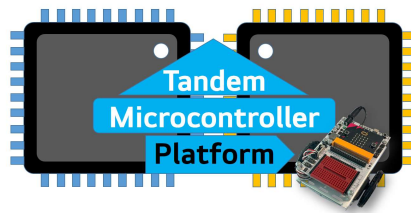


그림 3.3 텐덤로봇 마이크로컨트롤러 플랫폼

두 개의 마이크로컨트롤러(마이크로비트-아두이노)가 I2C 통신으로 데이터를 주고받으며 협력하는 방식의 코딩로봇에 대하여 "텐덤(tandem)"을 사용하는 이유입니다.

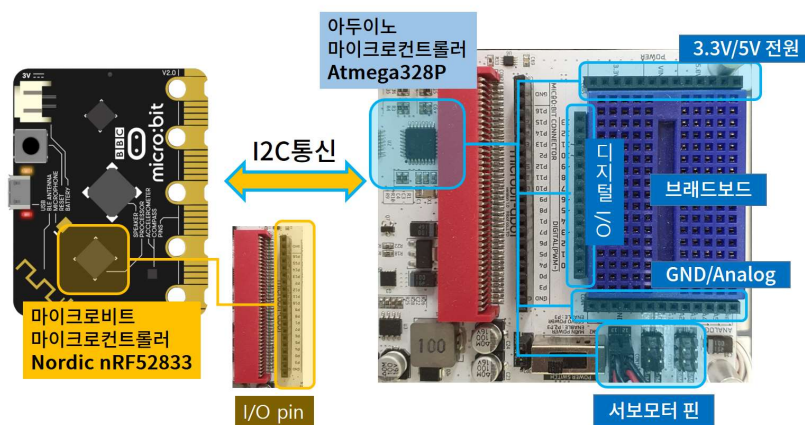


그림 3.4 마이크로비트-아두이노 텐덤로봇 개요도

마이크로비트의 마이크로컨트롤러 기능과 아두이노 우노의 마이크로컨트롤러 기능으로 아두이노로봇이 동작합니다. 텐덤로봇은 마이크로비트가 파이썬 언어를 사용하고 아두이노는 C/C++ 언어를 사용합니다. 이 책은 파이썬 언어로 로봇을 제어하는 내용을 담고 있습니다.

마이크로:비트 보드와 I2C 연결되는 'microbit-abot' PCB보드는 '아두이노 우노' 기능에 마이크로비트 슬롯과 서보모터를 구동하는 기능이 포함된 보드입니다. 'abot' 의미는 '아두이노_로봇'의 줄임말로 사용되며 '아두이노 보드'라고 부를 수도 있습니다.

[1] 텐덤로봇으로 어떻게 코딩교육에 활용할까?



그림 3.5 텐덤로봇으로 할 수 있는 것들

[2] 코딩언어의 선택이 넓어졌다.

C/C++ 언어: 마이크로:비트 보드를 연결하지 않고, 'microbit-abot' 보드만으로 C/C++ 언어를 실습할 수 있습니다. 'microbit-abot' 보드는 아두이노 우노 호환 보드이며, 아두이노 우노의 모든 공개 코드를 그대로 사용할 수 있습니다. 특히 아두이노_로봇(abot)을 위한 C++ SelfAbot 라이브러리와 학습교재 pdf 파일을 사용할 수 있습니다.

(※ 다운로드 출처: [프라이빗 게시판 '초보자를 위한 가이드'](#))

파이썬 언어: 마이크로비트 보드를 'microbit-abot' 보드와 결합해서 마이크로파이썬 언어로 다양한 예제의 로봇 동작을 실습할 수 있습니다. 교육 자료는 이 책과 추후에 배포될 라이브러리 해설 자료입니다.

[3] 유선 IOT 네트워킹을 위한 코딩습득

I2C통신의 특성상 마이크로비트-아두이노 통신데이터 처리 경험은 더 많은 주변기기 장치와 I2C통신할 수 있는 자신감과 실전경험으로 연결될 것입니다. 내가 머릿속에서 꿈꾸는 다양한 사물인터넷(IOT: internet of things)의 기초 코딩능력을 쌓을 수 있습니다.

무료로 배포되는 마이크로비트-아두이노 라이브러리를 사용하지 말고, 자신이 직접 라이브러리를 만들어보세요! 당신은 비로소 I2C통신 데이터교환과 로봇제어에 사용되는 라이브러리 코드를 더 깊게 이해할 수 있습니다.

[4] 무선 네트워킹으로 응용범위 확장

두 대 이상의 텐덤로봇들을 무선으로 상호 연결하는 군집제어를 실현할 수 있습니다. 로봇 상호간의 센서 자율제어와 사용자의 무선제어, 텐덤로봇들 사이의 상호제어를 직접 구현하고 경험할 수 있습니다. 무선기능은 마이크로비트의 라디오 통신을 사용합니다.

[5] 텐덤로봇을 조립하는 방법은?

기존의 '[neo-아두이노 로봇카V2](#)' 제품에서 PCB보드만 교체하면 텐덤로봇으로 바꿀 수 있습니다. 아래 그림 3.6은 microbit-abot PCB기판에 마이크로비트 컴퓨터를 소켓에 끼우면, 텐덤로봇이 완성됩니다.

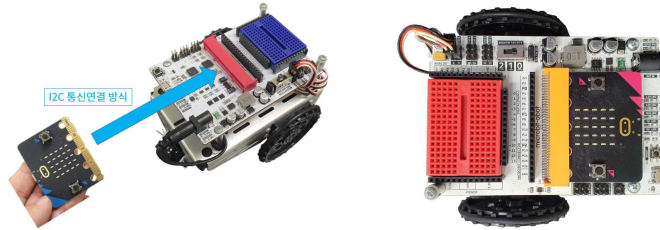


그림 3.6 마이크로비트를 소켓에 끼워서 텐덤로봇 완성하기

표3.1 마이크로비트와 아두이노 우노 마이크로컨트롤러 성능 규격

	마이크로비트 V2.0	아두이노 우노
마이크로 프로세서	Nordic nRF52833 / 32bit 클럭 Speed : 64MHz	ATmega328P / 8bit 클럭 Speed : 16MHz
I/O 핀	P0, P1, P2 외	P0 ~ P13
입출력 장치	버튼스위치 A & B 터치스위치 마이크 / 스피커 5X5 LED 디스플레이	없음
내장센서	3축 가속도센서 지자기센서 온도센서 조도센서	없음
내장 무선통신	bluetooth 5.0 radio	없음

마이크로비트는 총 25개의 입출력 핀을 제공하며, 일부 핀은 특정 기능을 사용하기 위해 예약되어 있습니다. 예약된 마이크로비트 핀은 자유롭게 사용할 수 없습니다. 여기서는 각 핀의 사용 가능 여부와 예약된 기능을 그림3.7에서 표시합니다.

P0, P1, P2 와 같은 핀들은 사용자가 자유롭게 아날로그 및 디지털 입력/출력으로 사용할 수 있습니다. 또한, SPI 통신을 사용하지 않는 경우 P13, P14, P15, P16 핀들을 일반 디지털 입력/출력 핀으로 사용할 수 있습니다.

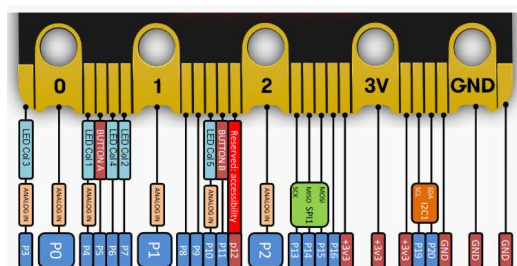


그림 3.7 마이크로비트 v2.0 핀아웃 배치도 (그림출처: microbit.org)

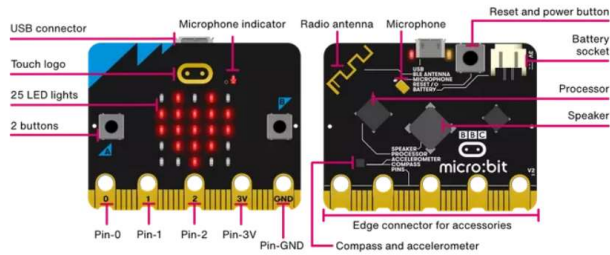


그림 3.8 마이크로비트 v2.0 기능 개요도 (그림출처: microbit.org)

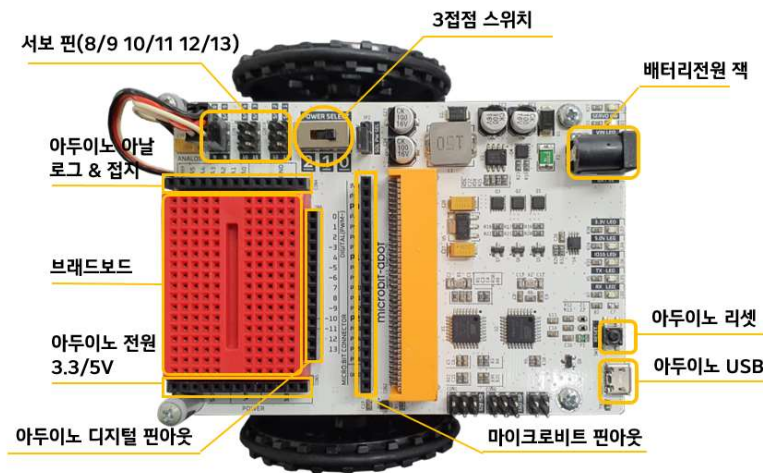


그림 3.9 텐덤로봇 주요 구성부 개요도

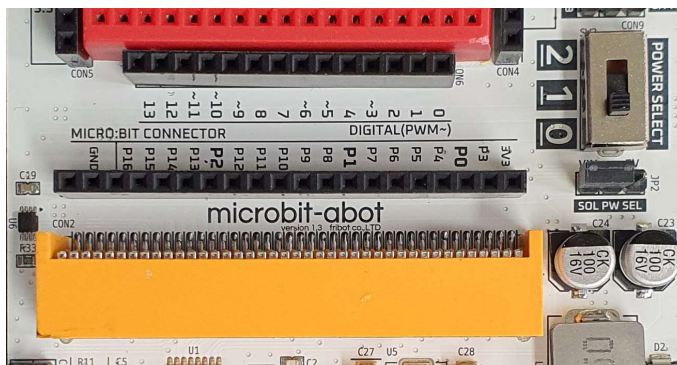


그림 3.10 마이크로비트-아봇 보드의 핀아웃 배치 화면

(주의사항) 텐덤로봇을 물리적으로 결합할 때 마이크로비트-아두이노 보드의 전원을 차단한 상태에서 마이크로비트를 소켓에 플러그-인하기 바랍니다. 아두이노 보드의 배터리 전원이나 USB전원을 공급한 상태에서 마이크로비트 보드를 소켓에 끼우면 전기적 손상을 입을 수도 있습니다.

(Notice) -----

텐덤 방식의 교육용 로봇 최초는?

미국 Parallax사가 2021년도부터 online 판매하기 시작한 사이버:봇(Cyber:bot) 제품

텐덤로봇 효시 모델입니다. 사이버봇은 마이크로:비트(micro:bit)와 프로펠러(propeller)칩을 I2C 통신으로 연결한 제품입니다.



(Notice) -----

텐덤보드 아두이노 COM포트 인식

Arduino UNO 호환 보드를 USB 포트에 연결할 때 Windows에서 “COM50” 이상의 높은 번호가 할당되는 경우가 있습니다. 이런 현상은 장치나 드라이버의 문제가 아니라, Windows의 ‘COM 포트 관리 방식’ 때문에 발생하는 정상 현상입니다.

윈도우는 여러 개의 USB-시리얼, 웹캠, 외장하드 등 다양한 장치를 연결하면, Windows가 매번 새 장치로 인식하여 기존에 쓰던 번호를 건너뛰고 새로운(높은) 번호를 할당해서 과거에 연결했던 장치 정보 누적하기 때문입니다.

사용자 컴퓨터의 숨은(비활성) 포트를 삭제하면 낮은 숫자의 COM 포트를 사용하거나 수동 설정하는 방법이 있지만 상세한 설명은 생략합니다.

3.2 텐덤로봇 I2C 데이터통신 기초

마이크로비트-아두이노 텐덤로봇은 I2C통신으로 서로 다른 마이크로컨트롤러 데이터를 송수신하고, 로봇의 동작을 만드는 플랫폼입니다. 한 장치에서 센서 데이터를 수집하고 다른 장치에서 데이터를 처리하거나 출력할 수 있습니다. 예를 들어, 마이크로비트의 가속도계 데이터를 아두이노로 전송하여 모터를 제어하는 로봇을 만들 수 있습니다.

마이크로비트를 마스터로 설정하고, I2C 통신을 통해 다양한 장치를 추가할 수 있어 시스템을 확장하는 데 유리합니다. 예를 들어, 여러 센서를 하나의 버스에 연결하여 데이터를 중앙에서 관리할 수 있습니다. 특히, I2C는 두 개의 와이어만 사용하여 여러 디바이스를 연결할 수 있기 때문에, 배선이 단순하고 효율적입니다.

텐덤로봇 I2C통신을 위한 하드웨어 연결 및 코드실행 방법

그림 3.6처럼 텐덤로봇에 마이크로비트 부품을 물리적으로 결합하면 됩니다. 마이크로비트와 아두이노에 모두 USB케이블을 컴퓨터와 연결하는 방법이 가능하지만, 여기서는 하나의 USB케이블만 사용해서 실습하는 방법을 소개합니다. 아래 순서로 I2C통신 실습을 시작해보세요.

[1] 텐덤로봇 전원공급

로봇 아래쪽 AA배터리홀더에 건전지를 5개 채우고 3접점 스위치를 '1'의 위치에 놓으면 로봇에 전원이 공급됩니다. 3접점 스위치 사용법은 Notice 내용을 참조하세요.

[2] 텐덤로봇 데이터 출력 케이블 연결하기

두 개의 마이크로컨트롤러에서 출력되는 통신 데이터를 모두 출력하려면, 두 개의 USB케이블을 모두 연결할 수 있습니다. 다만, 여기서는 아두이노 측의 데이터를 화면에 출력하려면 아두이노 USB 포트에 케이블을 연결하고, 마이크로비트 측의 데이터를 화면에 출력하려면 마이크로비트 USB 포트에 케이블을 연결하십시오.

[3] 텐덤로봇 I2C데이터 송수신하기

두 개의 마이크로컨트롤러가 I2C통신으로 데이터를 송수신하려면, 마이크로비트에서 데이터를 처리할 코드를 플래싱하고, 아두이노에서 데이터를 처리할 코드를 업로드합니다.

마이크로비트 코드는 1.2절 내용을 참고해서 컴퓨터와 '온라인편집기 연결 작업'을 완료하고 다운로드 받은 main.py 예제를 플래싱합니다.

또 다른 마이크로컨트롤러인 아두이노는 'IDE통합프로그램의 컴포트와 아두이노 종류를 'arduino uno'로 설정'하고 I2C통신 예제 스케치를 업로드하면 텐덤로봇의 I2C통신을 위한 준비작업이 완료됩니다.

I2C통신 실습에 사용할 상세한 코드 예제들은 아래에서 설명합니다.

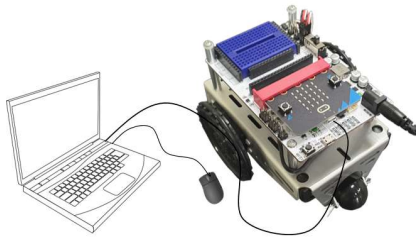
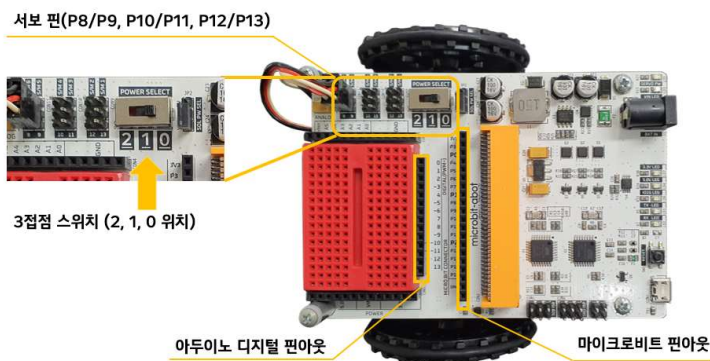


그림 3.11 마이크로비트에 USB케이블 연결하기

(Notice) -----

텐덤로봇의 3점점 스위치 선택방법은?

로봇의 아두이노 보드에 전원을 공급하는 방법은, 아두이노 보드에 USB케이블을 연결하는 것과 AA배터리를 사용하는 것입니다. 아래 그림은 '3점점 스위치' 위치별로 각각 다른 아두이노보드 전원공급 상태를 나타냅니다.



- '0' 위치 : AA배터리 전원은 모두 차단 / USB케이블 전원은 보드만 공급
- '1' 위치 : AA배터리 전원은 보드만 공급 / USB케이블 전원은 보드만 공급
- '2' 위치 : AA배터리 전원은 보드와 서보 모두 공급 / USB케이블도 모두 공급

아래 내용에서는 I2C통신 데이터 종류별 코드를 소개합니다. 다음 내용'3.3 텐덤로봇 라이브러리 설치'를 먼저 실행하면, 아두이노 IDE프로그램 메뉴의 파일->예제->MicroAbot 폴더에서 예제3.1 예제3.2 예제3.3의 아두이노 코드를 이용해서 쉽게 업로드할 수 있습니다. 그렇지 않다면, 아래에서 소개하는 실습 코드를 아두이노에 복사 붙여넣기하고 업로드 하세요.

(1) 문자 데이터 송수신

I2C통신 데이터중 문자 데이터를 송수신하는 간단한 코드 유형입니다. 기본적인 I2C 통신 프로토콜을 이해하는 데 도움이 됩니다. 마이크로비트는 I2C 마스터 장치로 동작하며,

아두이노는 I2C 슬레이브 장치로 동작합니다.

마이크로비트가 이진 데이터 'A'를 아두이노로 전송하고, 아두이노가 이진 데이터를 수신하는 간단한 사례입니다.

아래 코드 예제는 실습을 위해 직접 각각의 마이크로컨트롤러에 쓰기하면 됩니다.

마이크로비트 코드 (마스터):

```
from microbit import *

while True:
    data = b'A' # 송신할 문자 데이터
    i2c.write(0x08, data)
    sleep(1000)
```

아두이노 코드 (슬레이브):

```
#include <Wire.h>

void setup() {
    Wire.begin(8);
    Wire.onReceive(receiveEvent);
    Serial.begin(9600);
}

void loop() {
    delay(100);
}

void receiveEvent(int howMany) {
    while (Wire.available()) {
        char c = Wire.read();
        Serial.print(c);
    }
    Serial.println();
}
```

마이크로비트 코드를 먼저 설명하면, 라이브러리를 먼저 가져옵니다.

`data = b'A'` 는 보낼 데이터 바이트 문자 'A'를 이진 데이터로 전송할 때 사용됩니다.

`i2c.write(0x08, data)` 는 I2C 주소 0x08을 가진 슬레이브 장치(Arduino)에 `data`를 보냅니다. 여기서 0x08은 장치 주소입니다.

아두이노 코드를 설명하면, 먼저 Arduino에서 I2C를 설정하고 관리하는 데 사용되는 I2C 통신을 위한 Wire 라이브러리를 설정합니다.

`Wire.begin(8)`은 슬레이브 모드에서 I2C를 초기화하고, 슬레이브 주소를 8로 설정합니다. 이 주소는 마이크로비트의 `i2c.write(0x08, data)`와 일치해야 합니다.

`Wire.onReceive(receiveEvent)` 함수는 데이터 수신 시 호출될 이벤트 핸들러 함수이고, `receiveEvent` 함수가 호출됩니다.

`Serial.begin(9600)`은 시리얼 통신을 초기화하고, 보드레이트를 9600bps로 설정합니다. 이는 수신된 데이터를 시리얼 모니터로 출력하기 위함입니다.

`receiveEvent(int howMany)` 함수는 I2C 데이터를 수신할 때 호출되는 함수입니다.

- `while (Wire.available())`: 수신 버퍼에 데이터가 있는 동안 반복합니다.
- `char c = Wire.read()`: I2C 버퍼에서 한 바이트를 읽어와 문자로 저장합니다.

- Serial.print(c): 읽어온 문자를 시리얼 모니터에 출력합니다.
- Serial.println(): 각 메시지 끝에 줄바꿈을 추가하여 출력이 더 깔끔하게 보이도록 합니다.

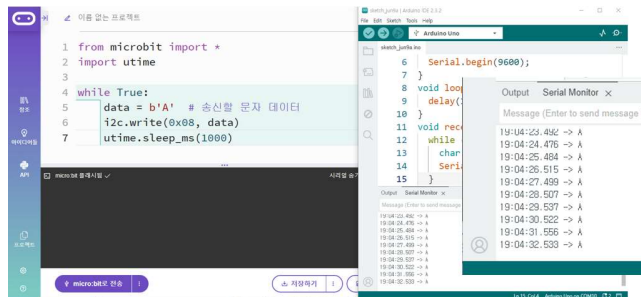


그림 3.12 마이크로비트가 전송하는 'A'데이터 아두이노 수신화면

마이크로비트가 'A'문자를 아두이노로 전송하면, 아두이노가 마이크로비트로 'B'문자를 수신하는 코드예제를 다시 소개합니다. 다운로드 받은 Tandem_Robotics 폴더내에서 마이크로비트 파일은 마이크로비트 예제 폴더, 아두이노 파일은 아두이노 MicroAbot 폴더의 예제 폴더에 저장되어 있습니다.

아두이노 예제는 MicroAbot.zip 라이브러리를 먼저 설치하면, 아두이노 IDE프로그램의 파일->예제->MicroAbot 폴더에서 아래 아두이노 예제 파일을 선택하고 사용할 수 있습니다. MicroAbot.zip 라이브러리 설치방법은 3.3절에서 설명합니다.

예제3.1 "I2C를 이용한 Micro와 아두이노 문자데이터 송수신"

<i>microbit (master)</i>
<pre># ch3_01_i2c_text_communication.py from microbit import * while True: try: # Send text data message = 'A' i2c.write(0x08, message.encode()) print('Sent:', message) display.show(message) sleep(1000) # Receive text data from Arduino received_data = i2c.read(0x08, 1) # Read 1 byte received_message = received_data.decode() print('Received:', received_message) display.show(received_message) sleep(1000) display.clear() sleep(2000) # Wait for 2 seconds except Exception as e: print('Error: ' + str(e))</pre>
<i>arduino (slave)</i>

```

#include <Wire.h>
char received_char;
bool new_data = false;

void setup() {
  Wire.begin(0x08); // set I2C address
  Wire.onReceive(receiveEvent);
  Wire.onRequest(requestEvent);
  Serial.begin(9600);
}

void loop() {
  delay(100);
  if (new_data) {
    Serial.print("Received: ");
    Serial.println(received_char);
    new_data = false;
  }
}

void receiveEvent(int howMany) {
  while (Wire.available()) {
    received_char = Wire.read();
    new_data = true;
  }
}

void requestEvent() {
  if (received_char == 'A') {
    Wire.write('B'); // after 'A' received, 'B' return
  }
}

```

i2c.write 함수는 바이트 스트림을 전송하는 프로토콜입니다. 파이썬의 문자열은 기본적으로 유니코드로 인코딩되어 있으며, I2C 통신에서는 이러한 문자열을 바이트 형태로 변환해야 합니다. 따라서 문자열 데이터를 바이트 형식으로 인코딩하여 전송합니다. 파이썬의 encode() 메서드는 문자열을 바이트 형식으로 변환하는 데 사용됩니다.

예제 코드는 위 코드처럼 사용해서 'A' 'B' 문자를 화면 출력하고, 마이크로비트 디스플레이 출력으로 'A' 'B' 문자를 표시할 수 있습니다.

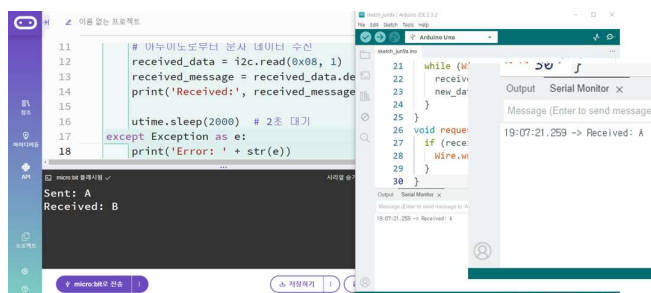


그림 3.13 마이크로비트-아두이노 문자 데이터 송수신

data.encode()는 문자열을 바이트로 변환하여 전송하는 방식이고, 동적으로 문자열을 전송할 때 유용합니다.

바이트 리터럴 (b'A')은 바이트 형식으로 표현된 데이터를 직접 전송하는 방식이며, 고정된 바이트 데이터를 전송할 때 유용합니다.

(2) 숫자 데이터 송수신

마이크로비트가 숫자 42를 전송하면, 아두이노가 수신한 숫자에 10을 더해서 다시 회신하는 코드예제를 소개합니다.

예제3.2 "I2C를 이용한 Micro와 아두이노 숫자 데이터 송수신"

microbit (master)

```
# ch3_02_i2c_numeric_communication.py
from microbit import *

while True:
    try:
        number = 42 # Transfer numeric data
        i2c.write(0x08, bytes([number]))
        print('Sent:', number)

        # Receiving numeric data from Arduino
        received_data = i2c.read(0x08, 1) # read 1 byte
        received_number = received_data[0]
        print('Received:', received_number)

        sleep(2000) # delay 2 seconds
    except Exception as e:
        print('Error: ' + str(e))
```

arduino (slave)

```
#include <Wire.h>
int received_number;
bool new_data = false;

void setup() {
    Wire.begin(0x08); // set I2C address
    Wire.onReceive(receiveEvent);
    Wire.onRequest(requestEvent);
    Serial.begin(9600);
}

void loop() {
    delay(100);
    if (new_data) {
        Serial.print("Received: ");
        Serial.println(received_number);
        new_data = false;
    }
}

void receiveEvent(int howMany) {
    while (Wire.available()) {
        received_number = Wire.read();
        new_data = true;
    }
}

void requestEvent() {
    int response_number = received_number + 10; // Add 10 to the number you received
    Wire.write(response_number);
}
```

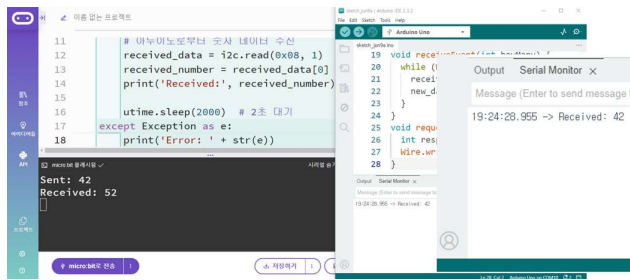


그림 3.14 마이크로비트와 아두이노의 숫자 데이터 송수신

`bytes([number])` 코드에서 숫자를 리스트에 담는 이유는 `bytes` 함수가 시퀀스(예: 리스트, 튜플, 문자열 등)를 받아서 바이트 배열을 생성하기 때문입니다.

(3) 더 복잡한 데이터의 송수신

`bytes()` 함수의 작동 방식

`bytes` 함수는 인자로 받은 시퀀스를 바이트 배열로 변환합니다. 시퀀스는 여러 값을 순서대로 저장하는 자료 구조로, 리스트, 튜플, 문자열 등이 포함됩니다. `bytes` 함수는 각 요소를 바이트로 변환하여 새로운 바이트 객체를 만듭니다.

단일 숫자를 바이트로 변환하기

만약 단일 숫자를 `bytes` 함수로 변환하려면, 그 숫자를 시퀀스(리스트 등)에 넣어야 합니다. 이는 `bytes` 함수가 시퀀스를 요구하기 때문입니다.

```
number = 42
byte_array = bytes([number])
print(byte_array) # 출력: b'*
```

여기서 `[number]`는 숫자 42를 단일 요소로 가지는 리스트입니다. `bytes([number])`는 리스트의 요소를 바이트로 변환하여 바이트 배열을 만듭니다. 42는 ASCII 코드로 *에 해당하므로 `b'*`가 됩니다.

여러 숫자를 바이트로 변환하기 : 리스트

```
numbers = [42, 65, 66]
byte_array = bytes(numbers)
print(byte_array) # 출력: b'*AB'
```

여기서 `numbers` 리스트의 각 요소가 바이트로 변환되어 바이트 배열 `b'*AB'`가 생성됩니다. 42는 *, 65는 A, 66은 B에 해당합니다.

여러 숫자를 바이트로 변환하기 : 튜플

```
numbers = (42, 65, 66)
byte_array = bytes(numbers)
print(byte_array) # 출력: b'*AB'
```

문자열을 `bytes()` 함수를 사용해서 바이트로 변환하기

```
text = "Hello"
byte_array = bytes(text, 'utf-8')
print(byte_array) # 출력: b'Hello'
```

bytes() 함수를 사용하지 않고 encode() 함수를 사용해서 문자열을 처리하기

다음은 코드예제입니다. 파이썬 코드와 C++ 코드를 소개합니다.

```
microbit -----
from microbit import *

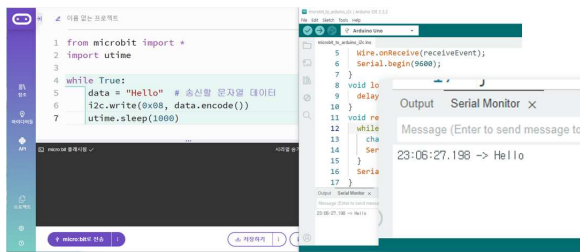
while True:
    data = "Hello" # 송신할 문자열 데이터
    i2c.write(0x08, data.encode())
    sleep(1000)
```

```
arduino -----
#include <Wire.h>

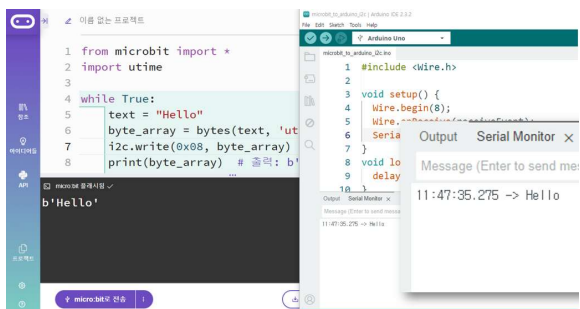
void setup() {
    Wire.begin(8);
    Wire.onReceive(receiveEvent);
    Serial.begin(9600);
}

void loop() {
    delay(100);
}

void receiveEvent(int howMany) {
    while (Wire.available()) {
        char c = Wire.read();
        Serial.print(c);
    }
    Serial.println();
}
```



(a) encode() 함수 사용해서 보내기



(b) bytes() 함수 사용해서 보내기

그림 3.15 문자열 데이터를 I2C 전송하기

마이크로비트 센서 데이터 아두이노로 전송하기

아래 코드는 마이크로비트의 가속도계 데이터를 전송하는 예제입니다. 마이크로비트 가속도센서의 x-축 데이터를 아두이노가 수신하고 시리얼모니터에 표시합니다.

예제3.3 "I2C를 이용한 Micro가속도 데이터 송수신“

<i>microbit (master)</i>
<pre># ch3_03_i2c_acceleration_data_transfer.py from microbit import * while True: acceleration = accelerometer.get_x() # Collect average X-axis acceleration data data = bytes(str(acceleration), 'ascii') # Convert data to ASCII byte format i2c.write(0x08, data) # Send data to Arduino via I2C try: response = i2c.read(0x08, 2) # Receive 2 bytes of data from Arduino print("Received data:", response) # Print the received data except Exception as e: print("Error:", e) print("X-axis acceleration:", acceleration) sleep(1000) # Wait for 1 second</pre>
<i>arduino (slave)</i>
<pre>#include <Wire.h> volatile int receivedData = 0; // Variable to store received data void setup() { Wire.begin(0x08); // Set I2C address Wire.onRequest(requestEvent); // Set data request event Wire.onReceive(receiveEvent); // Set data receive event Serial.begin(9600); } void loop() { delay(100); } void requestEvent() { // Send the received data Wire.write((byte*)&receivedData, sizeof(receivedData)); } void receiveEvent(int howMany) { if (howMany >= sizeof(receivedData)) { receivedData = 0; for (int i = 0; i < howMany; i++) { char c = Wire.read(); // Read the received data if (c >= '0' && c <= '9') { receivedData = receivedData * 10 + (c - '0'); // Convert ASCII number to integer } } Serial.print("Received data: "); Serial.println(receivedData); // Print the received data to the serial monitor } }</pre>

마이크로비트 가속도센서의 X-축 값을 I2C통신을 사용해서 아두이노로 전송하고, 다시 아두이노가 마이크로비트로 센서 값을 회신하는 코드 예제를 소개한 것입니다. 방금 소개한 예제는 가장 간단하게 센서 값을 송수신하는 코드이기 때문에 더 정교하게 코드를 수정해야 하지만, I2C통신을 처음 이해하는 용도로 소개한 것입니다.

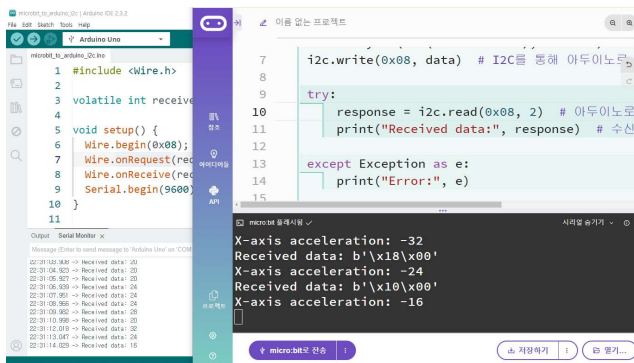


그림 3.16 마이크로비트 가속도센서 x-축 데이터 아두이노 출력하기

(Notice) -----

'ASCII 숫자 문자열'을 정수로 변환하는 코드방식에 대하여

그림 3.14 예제 코드중에서 `receivedData * 10 + (c - '0')` 코드를 사용해서 설명하겠습니다. 이 코드는 ASCII 형식으로 수신된 숫자 문자열을 정수로 변환하는 역할을 수행하는데, 예를 들어, '1', '2', '3'이라는 문자를 순서대로 읽어서 123이라는 정수를 만드는 방법에 관한 코드입니다.

[1] ASCII 값에서 숫자 값으로 변환:

ASCII 코드는 각 문자를 숫자로 표현합니다. 예를 들어, '0'의 ASCII 값은 48이고, '1'의 ASCII 값은 49입니다.

`c - '0'`는 ASCII 코드로부터 실제 숫자 값을 얻기 위한 표현입니다. 예를 들어, '3' - '0'은 51 - 48로 계산되어 3이 됩니다.

[2] 정수로의 변환 과정:

처음 수신된 문자가 '1'이라고 가정하면, `receivedData`는 0으로 초기화되어 있습니다.

`receivedData = receivedData * 10 + (c - '0');`를 수행하면, `receivedData`는 $0 * 10 + (49 - 48) = 1$ 이 됩니다.

다음 수신된 문자가 '2'라면, `receivedData`는 $1 * 10 + (50 - 48) = 12$ 가 됩니다.

마지막으로 수신된 문자가 '3'이라면, `receivedData`는 $12 * 10 + (51 - 48) = 123$ 이 됩니다.

이 과정을 통해 '1', '2', '3'이라는 연속된 문자가 123이라는 하나의 정수로 변환됩니다.

3.3 텐덤로봇 라이브러리 설치

이 절에서는 마이크로비트와 아두이노 사이의 I2C통신 데이터를 '사용자 정의 프로토콜'에 맞추어 송수신할 목적으로 별도의 라이브러리를 각각 설치해서 사용하는 방법에 관한 것입니다.

텐덤로봇으로 이 책 실습을 따라하려면, 'Tandem_Robotics'라고 이름 붙여진 파일 위치를 찾아서 아두이노와 마이크로비트에 필요한 라이브러리를 각각 설치해야 합니다. 텐덤로봇을 동작시키는데 필요한 파일은 아두이노 라이브러리 'MicroAbot.zip' 파일과 마이크로비트 라이브러리 'microbit_abot.py' 와 파이썬 예제 'examples' 폴더를 다운로드받아야 합니다.

위의 파일들이 포함된 'Tandem_Robotics' 위치는 프라이빗 '초보자를 위한 가이드' 게시판 또는 '공용 저장소 GitHub' 입니다. 자세한 웹주소는 아래를 참고하세요!

- 프라이빗 게시판: https://www.fribot.com/board/view?id=community_guide&seq=197
- 공용 저장소 GitHub: https://github.com/wookjin-chung/Tandem_Robotics

(1) 아두이노에 라이브러리 설치하는 방법

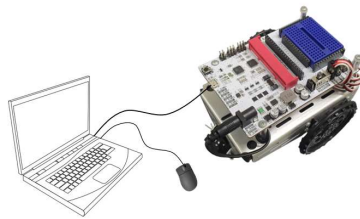


그림 3.17 아두이노 라이브러리 설치를 위한 USB케이블 연결방법

- [1] 컴퓨터에 MicroAbot.zip 파일을 다운로드받습니다.
- [2] USB케이블을 아두이노 호환 microbit-abot 보드에 연결하고, 아두이노 IDE에서 라이브러리 추가를 선택합니다.

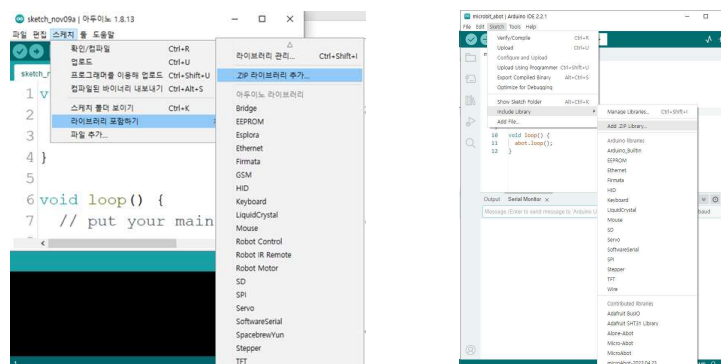


그림 3.18 (a) 아두이노 IDE프로그램 구버전 (b) 아두이노 IDE프로그램 신버전

라이브러리를 추가한 후, 당신 컴퓨터에서 조금 전 다운로드 받은 'MicroAbot.zip' 파일을 클릭합니다.

- [3] 'MicroAbot' 라이브러리를 설치하면, 파일->예제 메뉴에서 MicroAbot 폴더속의 microbit_abot.ino 파일을 열고 컴파일후 업로드하면 됩니다. 그러면 아두이노는 I2C 슬레이브로 동작할 준비가 완료됩니다.

(2) 마이크로비트에 라이브러리 설치하는 방법

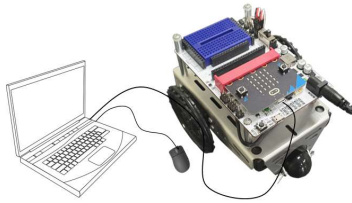
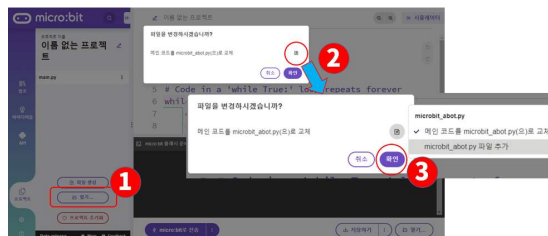


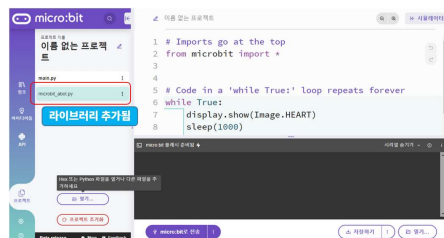
그림 3.19 마이크로비트 라이브러리 설치를 위한 USB케이블 연결방법

[1] 컴퓨터에 microbit 폴더의 파일들을 다운로드 받습니다.

[2] 컴퓨터와 마이크로비트(micro:bit)를 USB케이블로 연결하고, 마이크로비트 온라인 편집기(<https://python.microbit.org/>)를 엽니다. 그리고 마이크로비트와 여러분의 컴퓨터를 연결합니다.



(a) 라이브러리 추가 순서



(b) 라이브러리가 추가된 이후 장면

그림 3.20 마이크로비트 온라인 편집기에서 라이브러리 추가방법

①파일 열기를 선택하고, 업로드할 라이브러리 파일 'microbit_abot.py'를 윈도우 창에

서 찾아서 선택합니다. ②그 다음 팝업창에서 표시부분을 클릭하고, ③창 우측의 'microbit_abot 파일추가'를 클릭하고 확인을 누르면 라이브러리가 추가됩니다.

(Notice) -----

.py 파일과 함께 제공되는 .hex 파일을 사용하면 편리한 점

라이브러리가 포함된 hex 예제 파일을 사용하면, 사용할 때마다 라이브러리를 설치할 필요 없이 이미 준비된 사용자 환경에서 프로그래밍을 시작할 수 있습니다. 설치과정이 없어서 초보자에게 친화적이고 에러 발생 가능성을 줄입니다.

다운로드 받은 hex 파일을 마우스로 클릭한 다음 온라인 편집기로 끌어다 놓기만 하면 프로그래밍 시작환경이 완료됩니다. 다운로드 받은 hex 파일은 v2.0 이상에서 사용해야 합니다.

다만, hex 파일은 작성된 펌웨어 버전과 동일하거나 더 높은 버전의 펌웨어를 사용하는 것이 좋습니다. 펌웨어 버전이 낮으면 파일을 적용할 수 없거나, 특정 기능이 제대로 작동하지 않을 수 있으므로, 최신 펌웨어를 사용하는 것이 가장 안전한 방법입니다.

3.4 텐덤로봇 라이브러리 개요

텐덤로봇의 I2C 데이터통신을 위해 마이크로비트에는 'microbit_abot.py' 라이브러리를 설치하고, 아두이노에는 'MicroAbot.zip' 라이브러리를 각각 설치했습니다. 두 개의 마이크로컨트롤러에 각각 설치하는 라이브러리는 I2C통신으로 데이터를 통신하는 중요한 도구입니다. I2C통신을 위한 라이브러리 코드 개요는 다음과 같습니다.

(1) 마이크로비트 라이브러리 개요

마이크로비트에 설치된 라이브러리 코드 중에서 I2C통신을 위한 주요부분을 발췌하면 다음과 같습니다.

아래 코드는 앞부분이 기본적인 I2C통신 설정을 실행하는 코드와 bot 클래스로 구분해서 설명합니다. 그리고 bot클래스에서 send_c() 함수는 데이터를 전송하는 용도이고, read_r() 함수는 데이터를 수신하는 용도입니다.

```
handshake_successful = False
while not handshake_successful:
    try:
        i2c.write(93, b'WxAA')
        sleep(100)
        response = i2c.read(93, 1)
        if response == b'Wx55':
            print("Handshake successful")
            handshake_successful = True
    except OSError:
        pass

class bot:
    def __init__(self, p=27, q=33):
```

```

super().__init__()
self.pA = p
self.pB = q

def botdisable(self):
    pin8.set_pull(pin8.NO_PULL)
    sleep(200)
    reset()

def send_c(self, c, s=0, d=None, f=None):
    a = bytes([1, self.pA, self.pB, s])
    if d is not None:
        a += (round(d)).to_bytes(4, 'little')
    if f is not None:
        a += (round(f)).to_bytes(4, 'little')
    try:
        i2c.write(93, a)
        i2c.write(93, bytes([0, c]))
        c = b'\x01'
        while c != b'\x00':
            i2c.write(93, b'\x00')
            c = i2c.read(93, 1)
    except:
        self.botdisable()

def read_r(self):
    try:
        request_command = bytes([0x18, self.pA])
        i2c.write(93, request_command)
        r = i2c.read(93, 4)
        converted = int.from_bytes(r, 'little')
        return converted
    except:
        self.botdisable()

```

기본적인 I2C통신설정

I2C 통신을 연결하는 코드는 "핸드셰이크(handshake)" 과정을 수행합니다.

handshake_successful = False로 설정하고, while not handshake_successful: 루프에서 핸드셰이크가 성공할 때까지 계속 시도합니다.

i2c.write(93, b'\xAA') 는 마이크로비트가 아두이노에 주소 93으로 데이터 0xAA를 전송하고, response = i2c.read(93, 1) 실행으로 마이크로비트가 아두이노로부터 1바이트 데이터를 읽습니다. 아두이노가 적절히 응답하면 특정 데이터(b'\x55')가 반환됩니다.

if response == b'\x55': 코드로 아두이노의 응답이 0x55 (핸드셰이크 성공 신호)인지 확인하고, 성공하면 "Handshake successful"을 출력하고 handshake_successful = True로 설정하여 루프를 종료합니다.

bot클래스 함수

__init__ 함수:

이 함수는 bot 클래스의 생성자로, 객체를 초기화할 때 호출됩니다.

self.pA와 self.pB는 각각 27과 33으로 설정되며, I2C 통신에 사용할 핀 번호로 사용됩니다.

send_c 함수:

이 함수는 명령을 아두이노로 보내기 위해 사용됩니다.

c: 명령 코드

s: 선택적 매개변수, 기본값은 0

d와 f: 선택적 데이터 값으로, 값이 있을 경우 바이트로 변환하여 전송 데이터에 추가됩니다.

먼저 데이터 패킷 a를 생성하여 I2C를 통해 아두이노에 보냅니다.

그런 다음 명령 코드 c를 보내고, 전송이 완료될 때까지 반복하여 확인합니다.

만약 오류가 발생하면 self.botdisable()을 호출하여 bot을 비활성화합니다.

read_r 함수:

이 함수는 아두이노로부터 데이터를 읽어오기 위해 사용됩니다.

아두이노에 b'\x18'과 핀 데이터(self.pA)로 요청을 하고, 해당 핀의 4바이트 데이터를 읽어와서 정수로 변환합니다.

변환된 데이터를 반환합니다.

만약 오류가 발생하면 self.botdisable()을 호출하여 bot을 비활성화합니다.

(2) 아두이노 라이브러리 개요

아두이노 우노에서 동작하는 I2C통신용 라이브러리입니다. 공개된 'MicroAbot.zip' 라이브러리를 다운로드받아서 설치하고, I2C데이터 송수신을 위한 microbit_abot.ino 파일을 컴파일하고 업로드하면 됩니다.

한 번만 스케치를 업로드하면 이후의 모든 텐덤로봇 실습에서는 아두이노 보드의 새로운 스케치 업로드 작업이 필요 없습니다.

아두이노 라이브러리 역시 I2C통신과 직접적으로 관련된 코드부분만 발췌해서 간단히 설명합니다. 아두이노 I2C통신을 위해 사용하는 Wire.h 라이브러리의 슬레이브 동작에 사용되는 receiveEvent() 함수와 requestEvent() 함수를 설명합니다. 이 두 개의 데이터 송수신 함수는 아두이노 setup() 함수에서 호출됩니다.

```
void MicroAbot::receiveEvent(int howMany) {
    if (instance == nullptr || this->currentI2CState != I2C_IDLE) {
        return; // Exit if instance is null or state is not idle
    }
    this->currentI2CState = RECEIVING;

    if (howMany <= 0) {
        handleI2CError();
        this->currentI2CState = I2C_IDLE;
        return;
    }

    if (howMany == 1) {
        _ackByte = Wire.read();
        handleOneByteCommand(_ackByte);
    } else if (howMany == 2) {
        _firstByte = Wire.read();
        _secondByte = Wire.read();
        if (_firstByte == 0x18) {
            handleReadCommand(_secondByte);
        } else if (_firstByte == 0x00) {
            handleTwoByteCommand(_secondByte);
        }
    }
}
```

```

    } else {
        // Do nothing or add additional handling if required
    }

    } else if (howMany >= 4 && howMany <= sizeof(_receivedData)) {
        memset(_receivedData, 0, sizeof(_receivedData));
        for (int i = 0; i < howMany; i++) {
            _receivedData[i] = Wire.read();
        }
        handleMultiByteData(_receivedData, howMany);
        _howmanyByte = howMany;
        _receivedDataUpdated = true;
    } else {
        handleI2CError(); // Handle cases where howMany is 3 or > sizeof(_receivedData)
    }
    this->currentI2CState = I2C_IDLE;
}

void MicroAbot::requestEvent() {
    if (instance == nullptr || this->currentI2CState != I2C_IDLE) {
        return; // Exit if instance is null or state is not idle
    }
    this->currentI2CState = REQUESTED;

    if (_handshakeReceived) {
        uint8_t handshakeResponse = 0x55;
        Wire.write(&handshakeResponse, 1);
        _handshakeReceived = false;
    } else if (_sendAcknowledged) {
        byte ack = 0x00;
        Wire.write(&ack, sizeof(ack));
        _sendAcknowledged = false;
    } else if (_firstByte == 0x18) {
        Wire.write(_sendData, 4);
        _firstByte = 0x99; // initialize to the default value 0x99
    } else {
        // Default response or error handling
        byte defaultResponse = 0xFF; // Choose an appropriate default response
        Wire.write(&defaultResponse, sizeof(defaultResponse));
    }

    this->currentI2CState = I2C_IDLE;
}

```

receiveEvent() 함수 호출과 실행

이 함수는 마이크로비트에서 데이터를 수신할 때 호출됩니다. 데이터를 수신할 때 데이터를 처리하는 방식은 다음과 같습니다.

1바이트: handleOneByteCommand 함수 호출

2바이트: handleTwoByteCommand 함수 호출 (아두이노함수 호출 데이터 포함)

4바이트~12바이트: 데이터를 receivedData 배열에 저장하고 업데이트 플래그 설정

requestEvent() 함수 호출과 실행

이 함수는 아두이노가 마이크로비트로부터 데이터를 요청받았을 때 호출됩니다. 아두이노가 데이터를 송신하는 방식은 다음과 같습니다.

_handshakeReceived 플래그가 설정된 경우: 마이크로비트가 전송한 '0xAA' 데이터를 receiveEvent에서 수신하면 해당 플래그를 True로 설정하고, 아두이노에서 '0x55' 데이터를

마이크로비트로 송신합니다.

_sendAcknowledged 플래그가 설정된 경우: ACK 신호(0x00)를 전송

_firstByte가 0x18인 경우: sendData 배열 데이터 4바이트를 전송

(3) 아두이노 메인 파일 microbit_abot.ino

```
#include <MicroAbot.h>
```

```
MicroAbot abot;
```

```
void setup() {  
  Serial.begin(9600);  
  abot.setup();  
}
```

```
void loop() {  
  abot.loop();  
}
```

제 II 부 마이크로비트-아두이노 텐덤로보틱스

제 4 장 텐덤로보틱스 시작하기

4.1 LED 켜고 끄는 방법

- (1) 마이크로비트 핀 P0에 연결된 LED를 ON/OFF 하는 방법
- (2) 마이크로비트-아두이노의 P8핀에 연결된 LED를 ON/OFF 하는 방법

4.2 부저 멜로디 소리내기

- (1) 마이크로비트에 내장된 부저로 '작은별' 멜로디 연주하기
- (2) 마이크로비트 외부 P0핀에 부저를 연결하고 '작은별' 연주하기
- (3) 아두이노 디지털핀 P8에 연결된 부저로 '작은별' 연주하기

4.3 버튼회로 사용하기

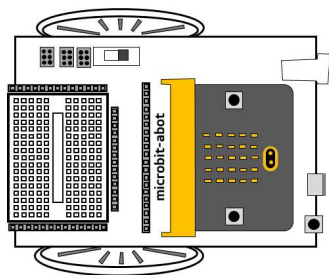
- (1) 마이크로비트 버튼 A를 사용해서 LED 켜고 끄기
- (2) 마이크로비트가 아두이노 디지털핀 버튼회로를 사용해서 LED 켜고 끄기

4.4 센서 활용하기: 마이크로비트 내장센서

- (1) 가속도센서
- (2) 자기센서 (컴파스 센서)
- (3) 광센서 (light sensor)
- (4) 온도센서
- (5) 터치센서

4.5 센서 활용하기: I2C 아두이노 센서

- (1) 포토트랜지스터 광센서 신호처리
- (2) 적외선(IR) 센서
- (3) 접촉센서(터덤이 센서)
- (4) 초음파센서



텐덤로봇으로 파이썬 코딩을 다양하게 경험해보세요! 코딩은 문법을 익히는 것보다 코드가 어떤 원리로 동작하는지를 이해하는 것이 컴퓨팅사고를 쌓는데 더 도움이 됩니다. 코드는 인공지능을 사용해서 얼마든지 쉽게 작성할 수 있습니다.

코딩능력이란 '코드를 어떻게 문법적으로 완성하는가?'라는 결과보다, '코드를 어떤 목적

으로 작성하는지, 어떤 구체적 논리를 적용하는지를 판단하고 선택하는 능력'이라고 생각합니다.

텐덤로봇은 파이썬 언어부터 시작해서, 아두이노에서 실행되는 C/C++코딩언어를 직접 체험할 수 있고 더 나아가서 I2C통신으로 데이터가 어떻게 다루어지는지에 대한 이해를 높여 줍니다.

(주의사항) 텐덤로봇을 물리적으로 결합할 때 마이크로비트-아두이노 보드의 전원을 차단한 상태에서 마이크로비트를 소켓에 플러그인하기 바랍니다. 아두이노 보드의 배터리 전원이나 USB전원을 공급한 상태에서 마이크로비트 보드를 소켓에 끼우면 전기적 손상을 입을 수도 있습니다.

4.1 LED 켜고 끄는 방법

이 장에서는 I2C통신 코드가 포함된 라이브러리 파일을 마이크로비트와 아두이노에 각각 설치하고, 파이썬 코드로 아두이노 I/O핀에 연결된 LED 불빛을 켜고 끄는 실습입니다.

마이크로비트와 아두이노에서 각각 시리얼 출력되는 데이터를 화면에 표시하려면, 아래 그림 4.1처럼 USB케이블을 사용자 컴퓨터와 각각 연결해야 합니다. 만약 시리얼출력 데이터를 표시할 필요가 없다면 제거해도 됩니다. USB케이블 연결은 '시리얼데이터 통신'과 '전원공급' 두가지 목적으로 사용합니다.

또한, 3.2절에서 소개한 '텐덤로봇의 3접점 스위치 사용법'을 참고해서, 아두이노 보드에 전원을 공급하려면 3접점 스위치를 적절한 위치에 놓는 것을 잊지마세요!

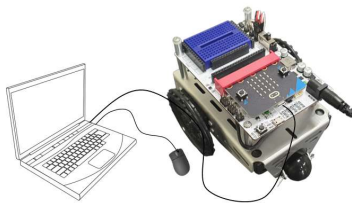
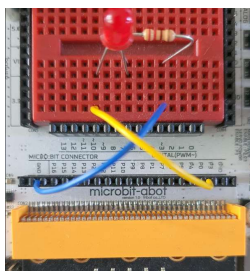


그림 4.1 마이크로비트-아두이노 텐덤로봇 시리얼 출력

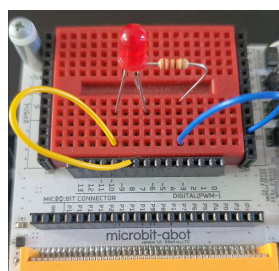
아래에서 소개하는 마이크로비트 파이썬 코드를 실행하려면, 먼저 아두이노 라이브러리를 설치해야 합니다. 앞서 소개한 3.3절의 아두이노 라이브러리 설치를 참고하세요.

다음 단계로 마이크로비트-아두이노 텐덤로봇에서, 아두이노 I/O핀에 연결된 LED를 켜고 끄기 위해, 파이썬 코드를 마이크로비트에서 실행할 순서입니다.

마이크로비트에서 파이썬 코드를 실행하는 경우에도, 마이크로비트의 I2C통신을 사용해서 아두이노 핀에 연결된 LED를 동작하는 경우 별도의 파이썬 라이브러리를 설치해야 합니다. 텐덤로봇을 위한 파이썬 라이브러리 설치 역시 3.3절을 참고해서 'microbit_abot.py'를 설치하십시오.



(a)



(b)

그림 4.2 (a)마이크로비트 P0에 연결된 LED회로 (b)아두이노 핀 P8에 연결된 LED회로

그림 4.2 (a)는 I2C통신과 관계없이 마이크로비트 외부핀(P0)에 LED를 직접 연결하고 LED를 켜고 끄는 실습입니다. (b)는 I2C통신으로 아두이노 I/O핀 P8에 LED를 연결하는 실습입니다.

(1) 마이크로비트 핀 P0에 연결된 LED를 ON/OFF 하는 방법

그림 4.2(a) 마이크로비트 P0에 LED와 220Ω 저항을 직렬로 연결하고, 아래 코드를 업로드하면 디지털 출력으로 LED를 ON/OFF 할 수 있습니다.

예제4.1 “마이크로비트 외부핀 P0로 LED 켜고 끄기”

```
# ch4_01_external_micro_pin_LED_control.py
from microbit import *

# Turn on the LED connected to pin P0
pin0.write_digital(1)
sleep(1000)

# Turn off the LED
pin0.write_digital(0)
```

(2) 마이크로비트-아두이노의 P8핀에 연결된 LED를 ON/OFF 하는 방법

그림 4.2(b) 아두이노 디지털I/O 핀 P8에 LED와 220Ω 저항을 직렬로 연결하고, 아래 코드를 업로드하면 디지털 출력으로 LED를 ON/OFF 할 수 있습니다.

예제4.2 “마이크로비트-아두이노 디지털핀 P8로 LED 켜고 끄기”

```
# ch4_02_I2C_arduino_pin_LED_control.py
from microbit_abot import *

abot = bot(8)

while True:
    abot.write_digital(1) # Turn on the LED connected to pin 8
    sleep(1000)          # Wait for 1 second
    abot.write_digital(0) # Turn off the LED connected to pin 8
    sleep(1000)          # Wait for 1 second
```

아래 그림 4.4는 마이크로비트에서 파이썬코드로 아두이노 P8에 연결된 LED를 켜고/끄는 때 마이크로비트와 아두이노의 시리얼 데이터를 나타냅니다. 그림4.4의 시리얼출력을 표시하려면, 먼저 마이크로비트 microbit_abot.py 라이브러리의 send_c 메서드에서 아래 그림4.3의 print문 주석을 해제한 코드를 실행하면, 마이크로비트에서 아두이노로 전송되는 바이트를 print 출력할 수 있습니다.

```
print(" Sending data: ", ' '.join(str(byte) for byte in a))
print(" Sending data over I2C:", [0, c])
```

```
def send_c(self, c, s=0, d=None, f=None):
    a = bytes([1, self.pA, self.pB, s])
    if d is not None:
        a += (round(d)).to_bytes(4, 'little')
    if f is not None:
        a += (round(f)).to_bytes(4, 'little')
    try:
        i2c.write(93, a)
        # print(" Sending data: ", ' '.join(str(byte) for byte in a))
        i2c.write(93, bytes([0, c]))
        # print(" Sending data over I2C:", [0, c])
        c = b'\x01'
        while c != b'\x0':
            i2c.write(93, b'\x0')
            c = i2c.read(93, 1)
```

그림 4.3 라이브러리내의 메서드에 출력문 추가하기

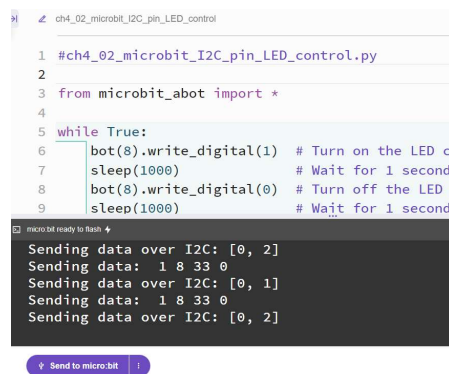
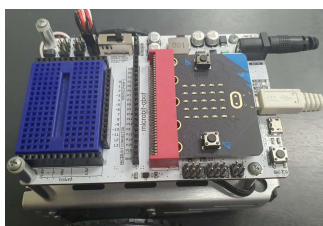


그림 4.4 마이크로비트-아두이노 LED 켜기/끄기 데이터 시리얼출력

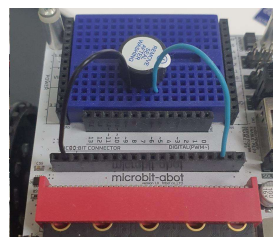
Sending data over I2C: [0, 1] 또는 [0, 2] 의 표현에서, 2 바이트 값 중에서 두 번째 바이트 값이 1 이면 아두이노 ‘MicroAbot.h 라이브러리’의 digitalWriteCommand 함수에서 digitalWrite를 HIGH 상태로 실행하고, 두 번째 바이트 값이 2 이면 digitalWrite LOW 상태를 실행합니다. (참고: 실습에서 그림의 시리얼 출력을 표시하려면, 라이브러리 send_c 함수의 #주석을 풀어야 합니다.)

4.2 부저 멜로디 소리내기

마이크로비트에 내장된 부저를 연주하거나 또는 외부 P0핀에 부저를 연결해서 ‘작은별’ 멜로디를 연주하는 파이썬 코드를 소개합니다. 두 가지 파이썬 코드는 아두이노와 무관하게 마이크로비트에서만 독립적으로 실행됩니다.



(a)



(b)

그림 4.5 마이크로비트에 (a) 내장된 부저로 작은별 연주하기 (b) P0에 연결된 부저회로

마지막으로 세 번째 파이썬 코드는 I2C통신을 사용해서 아두이노에 연결된 부저로 '작은 별' 멜로디를 연주하는 코드입니다.

(1) 마이크로비트에 내장된 부저로 '작은 별' 멜로디 연주하기

아래 예제는 그림 4.5 (a)의 마이크로비트에 내장된 부저를 사용해서 작은별을 연주하는 실습입니다.

예제4.3 "마이크로비트를 사용한 멜로디 재생"

```
# ch4_03_micro_melody_playback.py

from microbit import *
import music

# Define note frequencies
notes = {
    'C': 1047,
    'D': 1175,
    'E': 1319,
    'F': 1397,
    'G': 1568,
    'a': 1760,
    'b': 1976,
    'c': 2093,
    ' ': 0 # Rest
}

# Melody to play
score = "CCGGaaGFFEEDDC GGFFEEDGGFFEED CCGGaaGFFEEDDC"

# Duration of each note (in milliseconds)
duration = 333

# Main loop
while True:
    for note in score:
        if note in notes:
            frequency = notes[note]
            # If frequency > 0, play the note; otherwise, it's a rest
            if frequency > 0:
                music.pitch(frequency, duration)
            else:
                sleep(duration)
        # Wait 5 seconds before repeating the melody
        sleep(5000)
```

(2) 마이크로비트 외부 P0핀에 부저를 연결하고 '작은 별' 연주하기

아래 파이썬 코드는 그림 4.5 (b)의 마이크로비트 외부 입력핀 P0에 부저를 별도로 연결해서 멜로디를 연주하는 예제입니다.

예제4.4 "마이크로비트 핀으로 작은별 연주하기"

```
# ch4_04_micro_external_pin_twinkle_twinkle.py

from microbit import *
import utime
```

```

# Define note frequencies
notes = {
    'C': 1047,
    'D': 1175,
    'E': 1319,
    'F': 1397,
    'G': 1568,
    'a': 1760,
    'b': 1976,
    'c': 2093,
    ' ': 0 # Rest
}

# Melody to play
score = "CCGGaaGFFEEDDC GGFFEEDGGFFEED CCGGaaGFFEEDDC"

# Duration of each note (in milliseconds)
duration = 333

def play_note(frequency, duration):
    if frequency > 0:
        # Set pin0 to mid-point voltage (analog write)
        pin0.write_analog(512)
        # Set the analog period based on the note's frequency
        pin0.set_analog_period_microseconds(int(1000000 / frequency))
        # Play the note for the specified duration
        utime.sleep_ms(duration)
        # Turn off the pin after the duration
        pin0.write_digital(0)
    else:
        # Rest for the duration if the frequency is 0
        sleep(duration)

# Main loop
while True:
    for note in score:
        if note in notes:
            frequency = notes[note]
            play_note(frequency, duration)
        # Wait 5 seconds before repeating the melody
        sleep(5000)

```

(3) 아두이노 디지털핀 P8에 연결된 부저로 '작은별' 연주하기

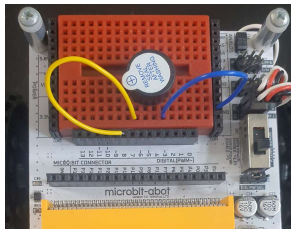


그림 4.6 아두이노 P8에 연결된 부저회로

그림 4.6의 부저회로를 사용해서 작은별을 연주하는 예제는 마이크로비트 파이썬 코드 명령을 아두이노로 전달해서, 아두이노의 `tone` 함수로 작은별을 연주합니다.

마이크로비트 파이썬 라이브러리 `microbit_abot.py` 의 `tone` 함수를 사용해서, 아두이노

'MicroAbot 라이브러리'의 `toneCommand` 함수가 I2C로 호출되고 실행됩니다. 아래 코드를 실행해서 I2C통신으로 송수신되는 데이터를 직접 출력해보세요.

(아래 코드를 실행하려면, 먼저 마이크로비트와 아두이노에 해당 라이브러리를 먼저 설치해야 하는 점을 잊지마세요. 그리고 출력데이터를 표시하려면 라이브러리의 `print`문 주석을 풀어야 하는 점도 기억하기 바랍니다.)

예제4.5 "마이크로비트와 아두이노로 작은별 연주하기"

```
# ch4_05_arduino_pin_twinkle_twinkle.py

from microbit import *
from microbit_abot import bot # Include the custom library

# Define the notes and their corresponding frequencies
notes = {
    'C': 262,
    'D': 294,
    'E': 330,
    'F': 349,
    'G': 392,
    'a': 440,
    'b': 494,
    'c': 523,
    '.': 0 # Rest
}

# Define the melody for "Twinkle Twinkle Little Star"
melody = "CCGGaaGFFEEDDC GGFFEEDGGFFEED CCGGaaGFFEEDDC"

# Define the duration of each note in milliseconds
duration = 333

# Function to play a note using the Arduino buzzer
def play_note_on_arduino(note, duration):
    if note in notes:
        frequency = notes[note]
        if frequency > 0:
            bot(8).tone(frequency, duration)
            sleep(duration)

# Main loop to play the melody
while True:
    for note in melody:
        play_note_on_arduino(note, duration)
        sleep(5000)
```

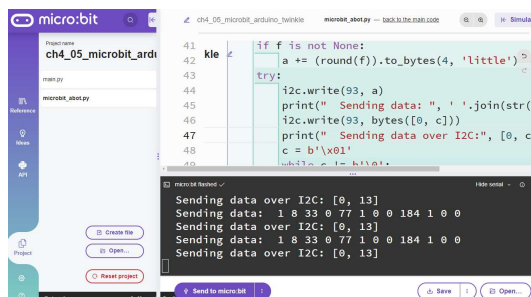


그림 4.7 마이크로비트-아두이노 작은별 멜로디 부저 소리내기

마이크로비트에서 출력된 데이터는 `send_c(self,c,s=0,d=None,f=None)` 함수의 `d` 값과 `f` 값을 사용해서 `tone()` 함수의 주파수와 지연시간 데이터를 아두이노로 전송합니다. 여기서 `d`와 `f`는 각각 4바이트로 표현됩니다.

4.3 버튼회로 사용하기

버튼을 사용한 LED제어 사례 두가지를 소개합니다. 첫 번째 버튼 실습은 마이크로비트의 버튼A를 사용해서 아두이노보드의 LED를 제어하는 예제입니다. 두 번째 버튼 실습은 마이크로비트에서 I2C통신으로 아두이노 버튼 누름 신호를 읽은 다음 I2C통신으로 아두이노에 연결된 LED를 켜고 끄는 예제입니다.

(1) 마이크로비트 버튼 A를 사용해서 LED 켜고 끄기

먼저 소개할 예제는 마이크로비트 버튼A 동작으로 아두이노 LED를 켜고 끄는 실습입니다. 아래 실습은 마이크로비트에 내장된 버튼A를 눌러서, 입력신호를 생성하고 아두이노로 전송해서 아두이노의 LED를 동작합니다. 아두이노 LED회로는 디지털핀 9번에 연결합니다.

예제4.6 "마이크로비트 버튼으로 아두이노 LED핀 제어하기"

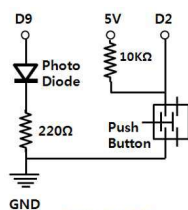
```
# ch4_06_micro_arduino_button_LED_cowork.py

from microbit import *
from microbit_abot import * # Include the custom library

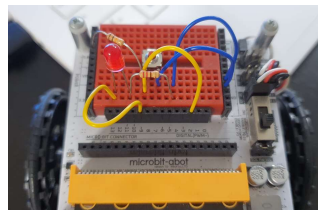
abot = bot(9)

while True:
    # If button A is pressed, show "0" and turn the LED off
    if button_a.is_pressed():
        display.show("0")
        abot.write_digital(0)
    else:
        # Otherwise, show "1" and turn the LED on
        display.show("1")
        abot.write_digital(1)
    sleep(1000)
```

(2) 마이크로비트가 아두이노 디지털핀 버튼회로를 사용해서 LED 켜고 끄기



(a)



(b)

그림 4.8 아두이노 디지털핀에 연결된 버튼회로와 LED회로 (a) 개요도 (b) 실제사진

두 번째 소개할 예제는 마이크로비트가 아두이노 보드에 연결된 버튼회로의 입력신호를 읽고, 파이썬 조건문에 따라 아두이노 보드에 연결된 LED를 켜고 끄는 실습입니다. 아두이노 디지털 핀2와 핀9에 풀업방식의 버튼회로와 LED회로를 각각 연결하는 방법을 그림 4.7에 나타냅니다.

예제4.7은 마이크로비트와 아두이노가 I2C통신으로 연결된 상태에서, 데이터를 읽고 쓰는 실습을 위한 것입니다.

파이썬 코드 `read_digital`과 `write_digital` 라이브러리 함수가 사용되고, I2C통신으로 아두이노 라이브러리의 `digitalReadCommand`와 `digitalWriteCommand` 함수를 호출해서 아두이노가 정의하고 사용하는 함수 `digitalRead`, `digitalWrite`를 실행합니다.

예제4.7 "마이크로비트와 아두이노를 통한 디지털 입력 및 출력 제어"

```
# ch4_07_micro_arduino_digital_read_write.py
from microbit import *
from microbit_abot import * # Include the custom library

abot = bot(9)

while True:
    # If the digital reading from the Arduino pin is 1,
    # display "0" on the micro:bit and write a digital 0 (LED off, for example).
    if abot.read_digital() == 1:
        display.show("0")
        abot.write_digital(0)

    # If the digital reading from the Arduino pin is 0,
    # display "1" on the micro:bit and write a digital 1 (LED on, for example).
    elif abot.read_digital() == 0:
        display.show("1")
        abot.write_digital(1)

    sleep(1000)
```

아래 그림 4.9에서 마이크로비트 시리얼 출력화면의 Read_r data: 값이 1 이면 아두이노 버튼신호 풀업상태 값이 전달되고 마이크로비트 디스플레이는 '0'을 표시합니다. 이어서 아두이노 LED는 꺼진 상태입니다.

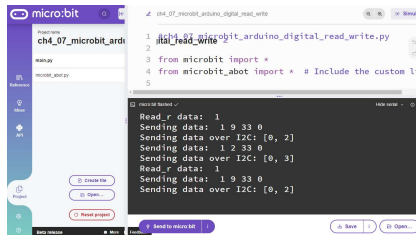
아두이노의 버튼이 눌러지면 시리얼 출력화면의 Read_r data: 값이 0 으로 바뀌고 아두이노버튼 신호에 따라 마이크로비트는 '1'을 표시하고 아두이노 LED는 켜집니다.

마이크로비트 라이브러리의 `send_c` 메서드에서 아래 그림4.3의 `print`문을 추가해서 편집기에서 시리얼 데이터를 출력했던 것처럼, 라이브러리의 `read_r` 메서드에서 아래 `print` 문을 추가하면 됩니다. (아래 그림에서 주석을 제거하고 적용하면 됩니다.)

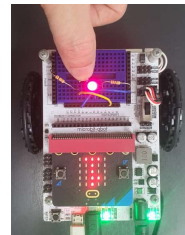
```
print(" Read_r data: ", converted)
```

```
def read_r(self):
    try:
        request_command = bytes([0x18, self.pA])
        i2c.write(93, request_command)
        r = i2c.read(93, 4)
        converted = int.from_bytes(r, 'little')
        # print(" Read_r data: ", converted)
        return converted
    except:
        self.botdisable()
```

그림4.9 read_r 메서드 데이터 출력 프린트문



(a) 편집기 시리얼 출력화면



(b) 실습장면

그림 4.10 마이크로비트의 아두이노 버튼신호 수신과 LED 켜고/끄기 신호 송신

참고로 앞서 설명한 버튼회로 원리는 텐덤로봇과 함께 제공되는 더듬이 센서 신호처리 용도로 코드가 재활용될 수 있습니다. 좌측 더듬이 또는 우측 더듬이가 생성하는 디지털신호는 버튼회로의 신호와 동일하게 처리됩니다.

4.4 센서 활용하기: 마이크로비트 내장센서

마이크로비트에 내장된 센서 신호처리 방법을 소개합니다. 마이크로비트(micro)는 여러 가지 내장 센서를 가지고 있는데, 다음과 같습니다.

- 가속도계 (Accelerometer): 마이크로비트를 움직일 때의 가속도를 측정할 수 있습니다. 이 센서를 사용하여 기울기, 충격, 자유 낙하 등을 감지할 수 있습니다.
- 자기 센서 (Magnetometer): 지구의 자기장을 감지하여 나침반으로 사용할 수 있습니다. 방향을 결정하는 데 사용됩니다.
- 광 센서 (Light Sensor): 마이크로비트의 LED 화면을 사용하여 주변 광의 세기를 감지할 수 있습니다. 이 센서를 통해 빛의 밝기를 측정하고, 이를 바탕으로 다양한 프로젝트에 활용할 수 있습니다.
- 온도 센서 (Temperature Sensor): 마이크로비트 자체의 실리콘 다이온 특성을 이용하여 주변 온도를 측정할 수 있습니다.

(1) 가속도센서

마이크로비트에 내장된 가속도 센서는 기본적으로 3축 (x, y, z) 방향으로의 가속도 값을 측정할 수 있습니다. 이 가속도 값은 주로 선형 운동을 측정하는 데 사용됩니다. 가속도 센서는 물체가 이동할 때 그 방향과 속도의 변화를 감지하는 데 적합합니다.

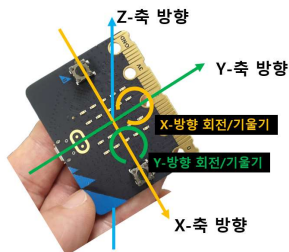


그림 4.11 마이크로비트 가속도센서의 x,y,z 3축 방향

마이크로비트를 3축 방향 선형으로 움직이면서 기울기 값을 표시하는 코드 예제는 다음과 같습니다.

```
from microbit import *
while True:
    x = accelerometer.get_x() # x축 방향의 가속도 값 읽기
    y = accelerometer.get_y() # y축 방향의 가속도 값 읽기
    z = accelerometer.get_z() # z축 방향의 가속도 값 읽기

    # 읽어온 값을 스크롤로 표시
    display.scroll("X: " + str(x) + " Y: " + str(y) + " Z: " + str(z))
    sleep(1000) # 1초 동안 대기
```

하지만 기기를 기울일 때 중력 가속도의 변화를 측정함으로써 기기의 회전 또는 기울기를 간접적으로 파악할 수 있습니다.

마이크로비트를 x축 방향으로 기울이면 accelerometer.get_x()의 값이 변하게 됩니다. 비슷하게 y축, z축 방향으로 기울이면 각각 accelerometer.get_y(), accelerometer.get_z() 값이 변합니다. 다만, 회전 운동(특히 각속도)은 자이로스코프가 측정하는 것이 일반적입니다.

예를 들어 y-축 기울기 값을 변화시키기 위해서는 그림에서처럼 x-축을 기준으로 마이크로비트를 시계/반시계방향으로 회전시킬 수 있습니다. x-축 기울기 값을 변화시키기 위해서는 그림에서처럼 y-축을 기준으로 마이크로비트를 시계/반시계방향으로 회전시킬 수 있습니다. z-축 기울기 변화는 z-축을 중심으로 마이크로비트를 회전시키면 됩니다.

요약하면 가속도 센서는 기울기 변화를 중력 가속도의 변화로 감지합니다. x-축 기울기를 롤(roll), y-축 기울기를 피치(pitch), z-축 기울기를 요(yo)라고 부르기도 합니다. 아래 코드는 x-축 방향과 y-축 방향의 기울기 변화를 출력하는 실습 예제입니다.

예제4.8 "마이크로비트를 사용한 가속도 기반 각도 추정"

```
# ch4_08_accelerometer_angle_estimation.py
from microbit import *

def estimate_angle(accel_value):
    # The accel_value ranges from -1024 to 1024
    # The angle is estimated between -90 and 90 degrees
    # Assume the angle is  $\pm 90^\circ$  at maximum or minimum acceleration values
    return (accel_value / 1024) * 90

while True:
    # Read X and Y acceleration data from the sensor
    x = accelerometer.get_x()
    y = accelerometer.get_y()

    # Estimate angles for each axis
    angle_x = estimate_angle(x)
    angle_y = estimate_angle(y)

    # Print the angle values to the serial port
    print("X angle:", angle_x, "degrees")
    print("Y angle:", angle_y, "degrees")

    # Simple indication of tilt along the X or Y axis
    if abs(x) > abs(y):
        if x > 200:
            display.show("R") # Tilted to the right
        elif x < -200:
            display.show("L") # Tilted to the left
    else:
        if y > 200:
            display.show("U") # Tilted upward
        elif y < -200:
            display.show("D") # Tilted downward

    # Wait for 500 ms
    sleep(500)
```

이 코드는 X와 Y 축의 가속도 값을 읽어들이며, 기울기 정도에 따라 'R', 'L', 'U', 'D' 문자를 디스플레이에 표시합니다. 이 문자들은 각각 오른쪽, 왼쪽, 위, 아래로 기울었다는 것

을 나타냅니다. 감도 조절을 위해 가속도 값의 임계값을 200으로 설정했습니다만, 이 값을 조정하여 더 민감하거나 덜 민감하게 만들 수 있습니다.

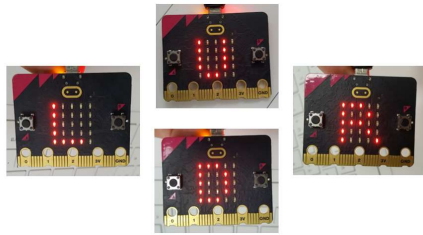


그림 4.12 마이크로비트 가속도센서 기반 각도 표시 화면

(2) 자기센서 (컴파스 센서)

마이크로비트(micro)의 자기센서(Magnetometer)는 주변의 자기장을 감지하여 방향이나 근처의 자기장을 측정할 수 있는 센서입니다. 가령 전자 나침반기능으로 자기장을 이용해 북쪽 방향을 감지할 수 있습니다. 주변의 자기장 강도를 측정하여 특정 물체(예: 자석)의 존재를 감지할 수 있습니다.

자기센서는 X, Y, Z 세 축을 기준으로 자기장을 측정하며, 이를 통해 3차원 공간에서의 방향과 자기장 강도를 계산할 수 있습니다.

아래 코드는 자기장 센서 값을 읽는 기본적인 방식입니다.

```
from microbit import *
# 자기 센서 교정 시작
compass.calibrate()

while True:
    # 나침반 헤딩 읽기
    bearing = compass.heading()
    # 시리얼 출력을 통해 헤딩 값 출력
    print("Compass Heading:", bearing)
    # 디스플레이에 간략화된 헤딩 표시 (0~359도를 0~7로 변환)
    display.show(str(bearing // 45))
    # 데이터 업데이트 간 500ms 대기
    sleep(500)
```

자기 센서 교정: `compass.calibrate` 함수를 호출하여 마이크로비트 자기 센서를 교정합니다. 이 과정은 사용자에게 마이크로비트를 움직이라는 지시를 통해 이루어집니다.

헤딩 값 읽기: `compass.heading` 함수를 사용하여 현재 마이크로비트의 방향을 0에서 359도 사이의 각도로 읽습니다.

시리얼 출력: `print` 함수를 사용하여 읽은 헤딩 값을 시리얼 콘솔로 전송합니다. 이를 통해 컴퓨터의 시리얼 모니터에서 실시간으로 값 확인이 가능합니다.

디스플레이 표시: 헤딩 값을 간략화하여 디스플레이에 보여줍니다 (예: `bearing // 45`을 사용하여 0~7 사이의 값(8방위 숫자)을 표시).

대기 시간: `sleep(500)`을 사용하여 각 데이터 업데이트 사이에 0.5초의 대기 시간을 설정합니다.

X, Y, Z 세 축을 기준으로 자기장을 측정해서, 3차원 공간에서의 방향과 자기장 강도를 계산하는 방식의 코드를 소개합니다.

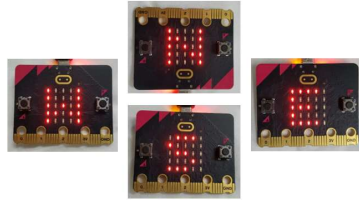


그림4.13 마이크로비트 컴파스 센서로 방향 측정화면

예제4.9 "마이크로비트 컴파스 보정 및 방향 측정"

```
# ch4_09_compass_calibration_and_direction.py

from microbit import *

# Calibration function
def calibrate_compass():
    display.scroll("CAL")
    compass.calibrate()
    display.clear()

# Perform calibration at the start
calibrate_compass()

while True:
    # Recalibrate if button A is pressed
    if button_a.is_pressed():
        calibrate_compass()

    # Measure magnetic field values on X, Y, Z axes
    mag_x = compass.get_x()
    mag_y = compass.get_y()
    mag_z = compass.get_z()

    # Get the current heading and initialize a string for direction
    direction = compass.heading()
    direction_str = ""

    # Display direction on the micro:bit LED display
    if 0 <= direction < 45 or 315 <= direction < 360:
        display.show("S") # South
        direction_str = "S"
    elif 45 <= direction < 135:
        display.show("W") # West
        direction_str = "W"
    elif 135 <= direction < 225:
        display.show("N") # North
        direction_str = "N"
    elif 225 <= direction < 315:
        display.show("E") # East
        direction_str = "E"

    # Print the measured values to the serial console
    print("X: {} Y: {} Z: {} Direction: {}".format(mag_x, mag_y, mag_z, direction_str))

    # Wait for 0.2 seconds
    sleep(200)
```

이 코드는 필요할 때마다 버튼A를 눌러서 자기센서를 다시 캘리브레이션할 수 있습니다. 다만, 지면에 마이크로비트를 붙여서 회전하면 방위를 비교적 정확하게 표시하지만, 손으로 들고 회전하면 상하 움직임 때문에 오류가 포함되어 표시될 수 있습니다.

마이크로비트를 손으로 들고 방위를 측정할 때, 컴퍼스 센서의 방위 표시가 불안정한 문제가 생기는 이유는 상하 움직임에 의한 간섭 때문입니다.

이를 개선하기 위해서 (1) 필터링으로 갑작스러운 변화나 노이즈를 줄이기 위해 이동 평균 필터나 저역 필터를 사용하여 데이터를 평활화할 수 있습니다. (2) 가속도계를 사용하여 마이크로비트의 기울기를 측정하고, 이를 컴퍼스 데이터와 결합하여 보정할 수 있습니다. 이런 보정 효과를 반영한 코드를 소개합니다.

예제4.10 "마이크로비트 컴퍼스 이동 평균 필터 적용"

```
# ch4_10_compass_moving_average.py
from microbit import *

# Moving average filter class
class MovingAverageFilter:
    def __init__(self, size):
        self.size = size
        self.buffer = [0] * size
        self.index = 0

    def add_value(self, value):
        self.buffer[self.index] = value
        self.index = (self.index + 1) % self.size
        return sum(self.buffer) / self.size

# Compass calibration function
def calibrate_compass():
    display.scroll("CAL")
    compass.calibrate()
    display.clear()

# Perform calibration at the start
calibrate_compass()

# Create a moving average filter (size = 5)
direction_filter = MovingAverageFilter(5)

while True:
    if button_a.is_pressed():
        calibrate_compass()

    # Measure magnetic field values on X, Y, Z axes
    mag_x = compass.get_x()
    mag_y = compass.get_y()
    mag_z = compass.get_z()

    # Display the direction on the micro:bit LED
    raw_direction = compass.heading()
    direction = direction_filter.add_value(raw_direction)
    direction_str = ""

    if 0 <= direction < 45 or 315 <= direction < 360:
        display.show("S") # South
        direction_str = "S"
    elif 45 <= direction < 135:
```

```

display.show("W") # West
direction_str = "W"
elif 135 <= direction < 225:
display.show("N") # North
direction_str = "N"
elif 225 <= direction < 315:
display.show("E") # East
direction_str = "E"

# Print the measured values to the serial console
print("X: {} Y: {} Z: {} Direction: {}".format(mag_x, mag_y, mag_z,
direction_str))

# Wait 0.2 seconds
sleep(200)

```

자기센서의 응용 분야:

전자 나침반: 스마트폰 마이크로비트 같은 장치에서 방향을 알기 위해 사용됩니다.

AR/VR 기기: 사용자의 머리 움직임을 추적하여 방향을 감지합니다.

로봇 공학: 로봇의 방향 감지와 내비게이션에 사용됩니다.

보안 시스템: 금속 탐지거나 자석을 이용한 문 열림 감지기 등에서 활용됩니다.

(Notice) -----

"tilt to fill screen" 메시지

마이크로비트의 디스플레이에 스크롤되는 것은 일반적으로 자기 센서(나침반)를 사용하기 전에 필요한 교정 과정에서 나타나는 표시입니다. 이 메시지는 사용자가 마이크로비트를 다양한 방향으로 기울여서 자기 센서를 정확하게 교정하도록 요구하는 지시사항입니다.

- ① 교정 시작: 코드에서 compass.calibrate() 함수를 호출하면 교정 과정이 시작됩니다.
- ② 기기 회전: 마이크로비트를 서로 다른 방향으로 천천히 움직이라는 지시를 따릅니다. 보통은 스크린에 나타나는 화살표를 따라 마이크로비트를 돌려야 합니다.
- ③ 교정 완료: 모든 방향으로 충분히 움직였다면, 교정 과정이 자동으로 완료되고, 메시지가 사라집니다.

(3) 광센서 (light sensor)

마이크로비트의 광센서는 특별히 독립된 센서가 있는 것이 아니라, LED 디스플레이를 구성하는 LED를 통해 빛의 세기를 감지합니다. 마이크로비트 보드의 앞면에 있는 5x5 LED 매트릭스가 광센서 역할을 수행합니다.

아래 코드는 광센서의 위치를 감안하여 LED 매트릭스가 빛의 세기를 감지하고 이를 시각적으로 표시하는 것입니다. 광센서 값에 따라 5x5 매트릭스의 모든 LED를 사용할 수 있도록

룩 값을 25로 나누고, 이를 기반으로 빛의 세기에 따라 LED를 켤 수 있습니다.

마이크로비트의 LED 매트릭스는 개별적으로 빛의 방향을 감지할 수 없기 때문에, 정지된 상태에서 특정 방향에서의 빛의 방향을 정확히 탐지하는 데는 한계가 있습니다. 그러나, 마이크로비트를 회전시키면서 측정하거나, 전방 빛의 존재 유무를 기반으로 스캔하는 방식을 사용하면 빛의 방향을 인식할 수 있습니다.

아래 코드는 단순히 외부 빛의 세기를 LED화면에 표시하는 예제입니다.

예제4.11 "마이크로비트로 광센서 값 그래프로 표시하기"

```
# ch4_11_light_sensor_graph.py
from microbit import *

while True:
    # Read the light sensor value (range: 0 to 255)
    light_level = display.read_light_level()

    # Clear the display before plotting the graph
    display.clear()

    # Convert the light_level to a number of LEDs to light up in a 5x5 matrix
    num_leds = light_level * 25 // 256 # Maximum of 25 LEDs

    # Light up LEDs according to the converted value
    for i in range(num_leds):
        # Set brightness to 9
        display.set_pixel(i % 5, i // 5, 9)

    # Print the light level value to the serial monitor
    print("Light level:", light_level)

    # Wait for 1 second
    sleep(1000)
```

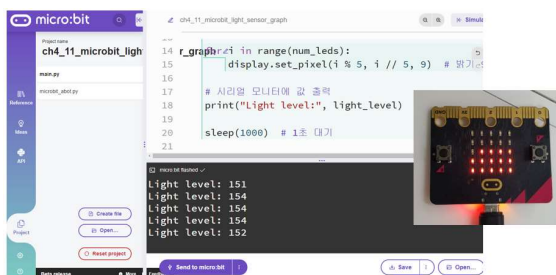


그림4.14 마이크로비트 광센서 실행장면

광센서 값 읽기: `display.read_light_level()` 함수를 사용하여 광센서 값을 읽습니다.

LED 디스플레이에 그래프로 표시:

`light_level` 값을 256으로 나눈 후 25를 곱하여 `num_leds`를 계산합니다. 이 값은 0에서 25 사이의 값을 가집니다.

`num_leds` 값을 사용하여 5x5 매트릭스의 LED를 순서대로 켵니다. 최대 25개의 LED가 켜질 수 있습니다.

(4) 온도센서

마이크로비트의 온도센서는 별도의 독립된 센서가 아니라 마이크로비트의 메인 마이크로컨트롤러(MCU)에 내장되어 있습니다. 이 온도센서는 마이크로컨트롤러 칩 자체의 온도를 측정하며, 이를 통해 주변 온도를 간접적으로 추정합니다. 마이크로비트에서 온도 값을 읽고 이를 LED 디스플레이와 시리얼 모니터에 출력하는 코드 예제는 다음과 같습니다.

예제4.12 "마이크로비트를 사용한 실시간 온도 측정 및 표시"

```
# ch4_12_temperature_display.py

from microbit import *

while True:
    # Read the temperature in Celsius
    temp = temperature()

    # Display the temperature on the LED
    display.scroll(str(temp) + "C")

    # Print the temperature value to the serial monitor
    print("Temperature:", temp, "C")

    # Wait for 1 second
    sleep(1000)
```

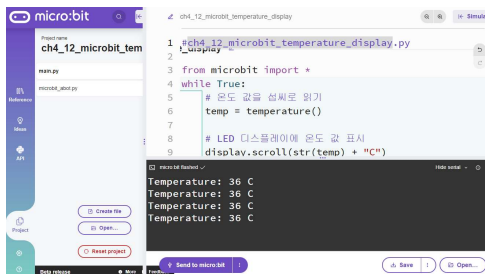


그림4.15 마이크로비트 온도센서로 분위기 온도 측정하기

(5) 터치센서

마이크로비트에는 내장된 터치센서가 있습니다. 이 터치센서는 보드의 A, B 버튼과 핀 P0, P1, P2를 통해 작동하며, 정전용량을 측정하여 터치를 감지할 수 있습니다.

로고 터치패드(microbit ver.2 이상)

- 마이크로비트 v2 보드의 전면 중앙에 로고 모양의 터치 패드가 추가되었습니다. 이 패드는 터치 입력을 감지할 수 있습니다.

핀 터치기능(P0, P1, P2)

- v1 보드와 마찬가지로 핀 0, 1, 2는 터치 입력을 감지할 수 있습니다.

예제4.13 "마이크로비트 터치 입력 감지 및 표시"

```
# ch4_13_micro_touch_detection.py

from microbit import *

while True:
    # Detect logo touch
    if pin_logo.is_touched():
        display.show(Image.HAPPY)
        print("Logo touched")
        sleep(1000) # Wait for 1 second before clearing
        display.clear()

    # Detect touch on pin 0
    elif pin0.is_touched():
        display.show("0")
        print("Pin 0 touched")
        sleep(1000) # Wait for 1 second before clearing
        display.clear()

    # Detect touch on pin 1
    elif pin1.is_touched():
        display.show("1")
        print("Pin 1 touched")
        sleep(1000) # Wait for 1 second before clearing
        display.clear()

    # Detect touch on pin 2
    elif pin2.is_touched():
        display.show("2")
        print("Pin 2 touched")
        sleep(1000) # Wait for 1 second before clearing
        display.clear()

    else:
        display.clear()
```

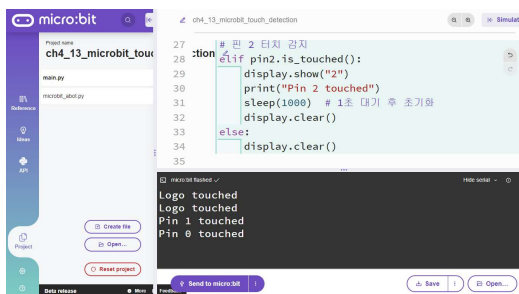


그림4.16 마이크로비트 터치센서 동작 시리얼 출력화면

4.5 센서 활용하기: I2C 아두이노 센서

I2C통신으로 연결된 아두이노 센서 신호처리 방법을 소개합니다. 브레드보드에 설치될 4가지 종류의 센서 신호(그림4.17 참조)를 아두이노 핀에 연결하고 신호처리하기 위해 마이크로비트가 I2C통신 송수신하는 실습입니다. 센서 종류는 포토트랜지스터 광센서, 적외선 센서, 더듬이 센서, 초음파 센서.



그림4.17 텐덤로봇 아두이노에 연결된 센서신호 처리 개요도

센서 신호처리 원리는 마이크로비트가 센서신호 호출 요구 코드를 실행하면, I2C통신으로 아두이노 특정 함수를 실행해서 센서 데이터를 수집하고, 수집된 센서 데이터를 I2C통신으로 마이크로비트에 전달합니다. 이 개념을 실현하는 상세한 논리 설명은 생략합니다. 지금부터 센서부품 연결방법과 동작원리를 설명합니다.

(1) 포토트랜지스터 광센서 신호처리

포토트랜지스터 광센서가 로봇동작 자율제어에 많이 사용되는 이유는 정확한 감지능력, 빠른 반응속도, 소형화와 경량화에 유리, 에너지 효율성 측면에서 장점이 있고 다양한 빛의 파장에 반응하는 특성과 간단한 회로 설계로도 신뢰할 수 있는 결과를 얻을 수 있기 때문입니다.

포토트랜지스터가 사용되는 경우는 라인 트레이서, 장애물 회피, 로봇 팔의 위치제어, 산업용로봇 생산라인 제어, 자동차 주차보조 시스템, 로봇 청소기 등입니다. 실습에서 사용할 포토트랜지스터 부품의 사용방법과 원리에 대하여 설명합니다.

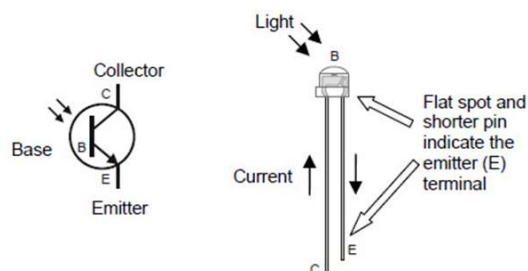
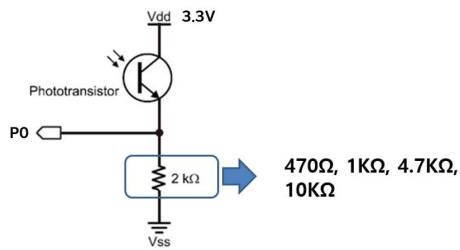


그림4.18 포토트랜지스터 심볼과 핀 개념도 (출처: parallax.com)

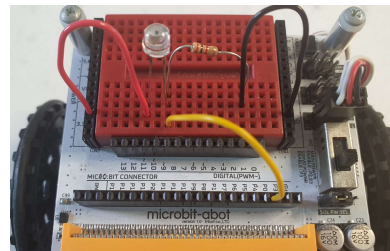
아날로그 신호처리 방식

포토트랜지스터 아날로그 신호처리는 아두이노 보드를 이용하지 않고 마이크로비트만 사용합니다. 아래 그림4.19 회로에서 3.3V 전원과 접지는 아두이노에 연결하고, 광센서 감지 전압 신호는 마이크로비트의 외부핀 P0에 연결합니다.

(참고: 아날로그 입력전압을 I2C통신으로 읽고 전송하는 기능은 microbit-abot 보드가 제공하지 않습니다.)



(a) 회로도



(b) 실습장면

그림4.19 포토트랜지스터 광센서의 아날로그 신호처리 회로

포토트랜지스터가 빛을 감지하는 원리는 포토트랜지스터의 베이스 영역이 외부 빛에 민감한 물질로 만들어져서, 빛의 세기에 따라 컬렉터와 에미터 사이의 전류가 다르게 증폭되는 동작을 이용하는 것입니다. 예를 들면 빛의 강도가 강할수록 베이스 전류가 증가하고, 그 결과 컬렉터와 에미터 사이의 전류가 증폭되어 2KΩ 저항양단의 전압이 증가해서 최대 3.3V에 다다를 수 있습니다.

아래 코드는 광센서의 아날로그 신호처리 사례를 보여주는 것입니다. 마이크로비트가 `pin0.read_analog()` 함수로 읽는 빛의 세기 범위는 0~1023 사이의 `analog_value` 아날로그 입력입니다.

예제4.14 "마이크로비트를 사용한 아날로그 전압 측정 및 출력"

```
# ch4_14_micro_analog_voltage_reader.py
from microbit import *

# Infinite loop
while True:
    # Read the analog input from pin P0
    analog_value = pin0.read_analog()

    # Convert the analog value to an actual voltage
    voltage = (analog_value / (1024 - 1)) * 3.3

    # Display the measured voltage on the micro:bit LED
    # display.scroll("{:.2f}V".format(voltage))

    # Print the measured voltage to the serial monitor
    print("{:.2f}V".format(voltage))
```

```
# Wait briefly
sleep(100) # Wait for 0.1 second
```

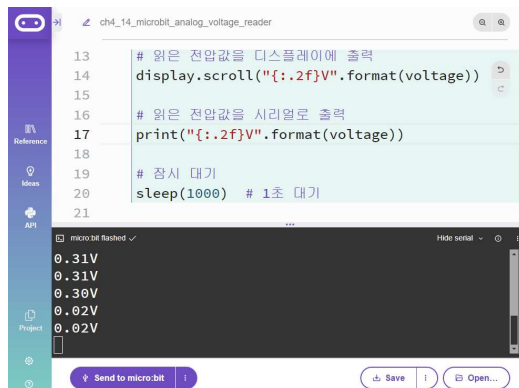


그림4.20 마이크로비트로 포토트랜지스트 광신호 감지하는 화면

디지털 신호처리방식 : rcTime()

포토트랜지스터 디지털 신호처리는 마이크로비트에서도 처리할 수 있지만, 아두이노 보드를 사용해서 처리하는 방식에 대하여 설명합니다. 사용할 수 있는 디지털 핀 갯수가 마이크로비트보다 아두이노가 더 많기 때문에 아두이노 핀을 사용하는 것이 유리합니다.

광센서 신호를 디지털 방식으로 처리하면 유리한 점이 있습니다. 특히, 로봇 작업의 특성과 정밀도, 단순성 및 소음에 대한 견고성 측면에서 디지털 신호의 장점이 결합되어 디지털 제어를 선호하는 경우가 많습니다. 포토트랜지스터 광센서를 사용해서 디지털 신호 처리방식을 설명합니다.

그림4.21은 광센서 감지신호를 RC회로와 결합해서, 디지털신호로 변환할 수 있습니다. 회로에서 디지털 출력 핀모드 HIGH 상태에서 캐패시터는 충전상태 전압(아두이노의 경우 5V)이 유지되고, 이어서 디지털 입력 핀모드 상태로 전환하면 캐패시터 전압은 저항을 통해 방전을 시작합니다. 이때부터 커패시터의 전압이 디지털 핀의 LOW 임계값(약 1.5V) 이하로 떨어질 때까지의 시간을 측정합니다. 저항 값이 클수록 커패시터는 더 천천히 방전되고, 저항 값이 작을수록 더 빨리 방전됩니다.



그림4.21 디지털신호 측정 회로와 캐패시터 심볼 (출처: parallax.com)

아두이노 라이브러리의 rcTime() 함수 기본 형태는 다음과 같습니다. 아래 코드를 사용해서 포토트랜지스터 빛의 세기를 측정할 수 있습니다. 1ms 동안 핀을 HIGH 상태로 유지해서 캐패시터를 충전한 다음, 방전 시간을 측정함으로써 빛의 세기에 대한 정보를 얻을 수 있습니다.

```
uint32_t MicroAbot::rcTime(uint8_t pin) {
    pinMode(pin, OUTPUT);
    digitalWrite(pin, HIGH);
    delay(1);

    pinMode(pin, INPUT);

    uint32_t startTime = micros();
    uint32_t elapsedTime = 0;
    while (digitalRead(pin) == HIGH) {
        elapsedTime = micros() - startTime;
    }

    return elapsedTime;
}
```

아두이노 보드만으로 로봇을 동작하는 경우 위 코드를 변형없이 사용할 수 있습니다. 다만, 텐덤로봇의 경우 센서 데이터를 효율적으로 처리하기 위해 유한상태기계(FSM)가 적용된 형태로 위 코드를 변형해서 아두이노 라이브러리로 처리합니다. 상세한 rcTime 함수의 아두이노 라이브러리 설명은 이 책의 주제와 다르기 때문에 생략합니다.

아두이노에서 실행되는 rcTime 함수를 마이크로비트가 호출하기 위해 아두이노 디지털 8번 핀에 그림4.19 회로를 구성하고, 마이크로비트에서 파이썬 코드 예제4.15를 실행합니다.

예제4.15 "마이크로비트를 사용한 센서 응답 시간 측정"

```
#ch4_15_micro_abot_rcTime.py

from microbit_abot import *
abot = bot(8)

while True:
    abot.write_digital(1)
    qt_left = abot.rc_time(1)

    #display.scroll(str(qt_left))
    print("Left_sensor:", qt_left)
    sleep(50)
```

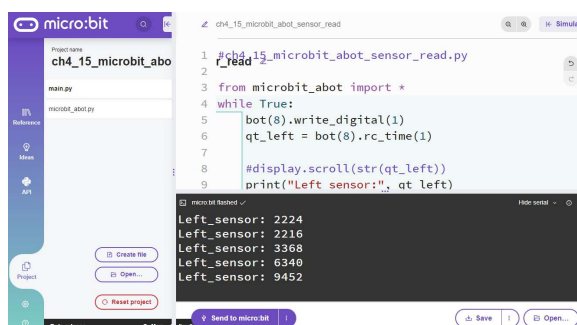


그림4.22 마이크로비트 포토트랜지스터 광신호 시리얼 출력화면

코드에서 `abot.rc_time(1)` 메서드는 `abot` 인스턴스로 센서 핀번호를 전달하고, `rc_time()` 메서드의 매개변수를 '1'로 설정하면 HIGH 상태로 캐패시터가 충전되고 LOW 까지 감소하는 방전시간을 측정해서 반환하는 결과를 얻습니다.

포토트랜지스터의 `qt_left` 는 `rcTime` 함수의 시간을 측정한 결과입니다. 아두이노 라이브러리는 `rcTime` 함수 최대 시간을 100ms 까지 제한하였습니다. 외부 빛이 차단된 경우 센서가 최대로 감지할 수 있는 시간제한에 의해 시리얼 출력 최대 값은 100000 입니다.

(Notice) -----

rcTime 시정수(time constant) 어떤 의미인가요?

RC 회로의 시정수(RCtime)는 충전시 커패시터 양단의 전압이 최대 값의 약 63.2%까지 충전되거나, 방전시 약 36.8%까지 감소하는데 걸리는 시간을 측정한 상대적 개념의 시간을 의미하고 1 타우(τ)로 부릅니다. 시정수 τ 는 다음 공식을 사용하여 계산됩니다.

$$\tau = R \times C$$

τ (tau)는 초 단위의 시간 상수입니다.

R은 저항(Ω)입니다. C는 패럿(F) 단위의 정전용량입니다.

텐덤로봇의 포토트랜지스터 디지털 신호처리방법

텐덤로봇 전면부의 오른 편과 왼 편에 `rcTime()` 함수 센서회로를 설치합니다. 왼 쪽 회로는 아두이노 디지털핀 8번에 연결하고, 오른 쪽 회로는 디지털핀 6번에 연결해서, 왼 쪽 방향의 불빛과 오른 쪽 방향의 불빛을 인식하고 자율동작을 유도할 수 있습니다.

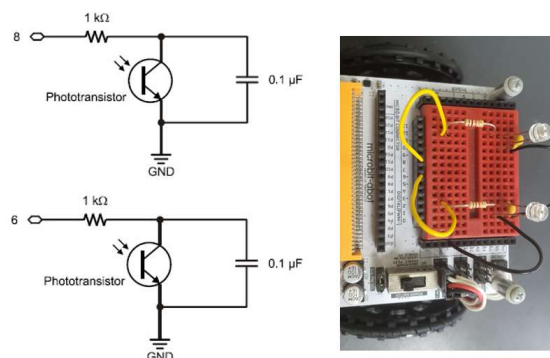


그림 4.23 포토트랜지스터 광신호 rcTime 시정수 활용 회로도

불빛을 감지할 수 있는 포토트랜지스터 회로에서 포토트랜지스터가 바라보는 방향은 지면 또는 수평으로 향하게 하거나 하늘 방향을 향하게 할 수 있습니다. 빛을 감지하는 주요 방

향이 어디인지를 감안해서 결정하면 됩니다.

회로 구성은 아두이노 디지털 핀에 연결된 포토트랜지스터 감지신호가 I2C통신으로 마이크로비트에 전달되어 마이크로비트가 로봇의 동작을 제어할 목적으로 신호를 처리하는 것이 목적입니다.

잊지마세요! 아래 코드의 실행 결과를 얻으려면, 아두이노(microbit-abot) 보드의 3접점 스위치를 '1'의 위치(중간 위치)에 놓아야 합니다.

예제4.16 "마이크로비트를 사용한 좌측 우측 센서 응답 시간 측정"

```
# ch4_16_micro_abot_dual_rcTime.py

from microbit_abot import *
left_sensor = bot(8)
right_sensor = bot(6)

while True:
    left_sensor.write_digital(1)
    qt_left = left_sensor.rc_time(1)
    right_sensor.write_digital(1)
    qt_right = right_sensor.rc_time(1)

    #display.scroll(str(qt_left))
    print("Left_sensor: ",qt_left)
    #display.scroll(str(qt_right))
    print("Right_sensor: ",qt_right)

    sleep(100)
```

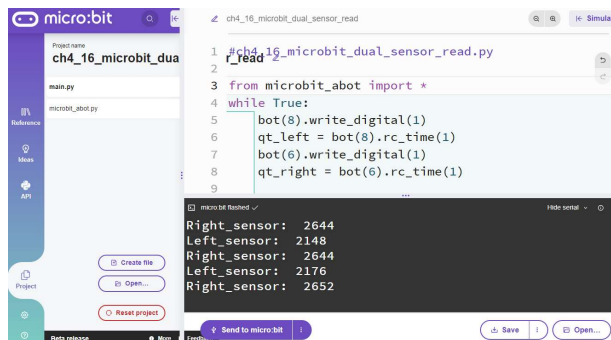


그림 4.24 빛의 세기에 반응하는 포토트랜지스터 rcTime() 함수 출력

아래 소개하는 코드는 두 개의 포토트랜지스터 광센서 출력값 크기에 따라 마이크로비트의 x열 0과 1열 그리고 3열과 4열에서 Right_sensor 값과 Left_sensor 값이 대응되는 LED 출력을 표시하는 예제입니다.

예제4.17 "마이크로비트를 사용한 듀얼 센서 데이터 시각화"

```
# ch4_17_micro_abot_dual_rcTime_display.py

from microbit_abot import *
```

```

left_sensor = bot(8)
right_sensor = bot(6)

while True:
    # Write a digital high (1) before measuring rc_time
    left_sensor.write_digital(1)
    qt_left = left_sensor.rc_time(1)
    right_sensor.write_digital(1)
    qt_right = right_sensor.rc_time(1)

    # Print the raw sensor readings
    print("Left_sensor:", qt_left)
    print("Right_sensor:", qt_right)

    # Handle None values returned by rc_time()
    if qt_left is None:
        print("Error: rc_time() for left sensor returned None")
        qt_left = 0 # Set a default value or handle appropriately
    else:
        print("qt_left:", qt_left) # Verify correct return value

    if qt_right is None:
        print("Error: rc_time() for right sensor returned None")
        qt_right = 0 # Set a default value or handle appropriately
    else:
        print("qt_right:", qt_right) # Verify correct return value

    # Set the maximum value to 25,000
    max_value = 25000
    if qt_left > max_value:
        qt_left = max_value
    if qt_right > max_value:
        qt_right = max_value

    # Convert the sensor reading to a value from 0 to 25
    # The closer the reading is to 0, the more LEDs will turn on
    left_display_value = 25 - int(qt_left / (max_value / 25))
    right_display_value = 25 - int(qt_right / (max_value / 25))

    display.clear()

    # For the left sensor: fill columns x=3 and x=4
    for x in [3, 4]:
        for y in range(5):
            if left_display_value > (y * 5 + (x - 3) * 5):
                display.set_pixel(x, y, 9)

    # For the right sensor: fill columns x=0 and x=1
    for x in [0, 1]:
        for y in range(5):
            if right_display_value > (y * 5 + x * 5):
                display.set_pixel(x, y, 9)

    sleep(50)

```

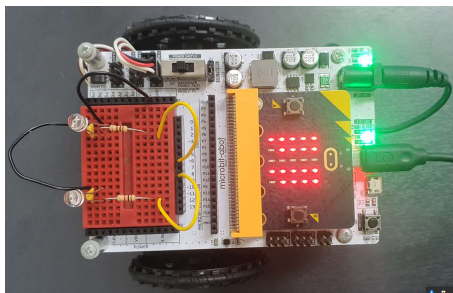


그림 4.25 외부 빛의 세기를 마이크로비트 디스플레이에 표시하기

포토트랜지스터 광센서로 빛의 세기를 감지하고 텐덤로봇이 외부 빛에 반응하는 자율동작은 제 5장에서 추가로 다루어질 예정입니다. 다른 종류의 센서를 더 살펴보기 전에 포토트랜지스터 로봇 자율제어 동작으로 계속 탐구하려면, 다음 내용들을 건너서 제 5장으로 넘어가도 좋습니다.

(2) 적외선(IR) 센서

적외선 센서는 근적외선(NIR 파장: 700nm ~ 2500nm)과 원적외선(FIR 파장: 15μm ~ 1000μm)으로 나누어집니다. 그리고 근/원 적외선 센서를 점센서(point sensor)로 사용하는 방식과 면센서(area sensor)로 사용하는 방식에서 다양한 차이를 만들어냅니다.

아래 실습에서는 근적외선 센서를 점센서 방식으로 사용하는 예제입니다. 이 센서는 특정 지점의 빛 반사율이나 투과율을 측정하여 물질의 특성을 분석하고, 특정 지점에서의 거리 측정에 사용될 수 있습니다. 센서를 사용해서 식물 잎의 수분 상태를 특정 지점에서 측정하거나, 로봇 전방의 물체를 감지하고, 물체와의 거리를 측정할 수 있습니다.

실습용 적외선센서는 송신용 LED와 수신용 receiver 부품으로 구성됩니다. 앞에서 사용했던 포토트랜지스터 부품과는 다른 형상의 적외선 LED에 하우징을 사용해서 적외선 빛의 직진성을 더 좋게 만듭니다. 적외선 센서 송수신 부품을 아두이노 핀에 연결하는 방법은 아래 회로도를 사용해서 각각 연결하기 바랍니다. 실습에 사용된 적외선 LED의 파장은 980nm 이고, 적외선 수신기의 파장도 동일합니다.

표4.1 적외선 센서의 주요 특징

동작원리	- 적외선 방출: 적외선 송신기(LED)는 특정 파장의 적외선을 방출합니다. - 적외선 감지: 적외선 수신기(포토 다이오드 또는 포토 트랜지스터)는 반사되거나 방출된 적외선을 감지합니다. - 신호 처리: 수신된 적외선의 강도나 패턴을 분석하여 감지된 물체의 거리, 존재 여부 등을 판단합니다.
감지범위	- 단거리 감지: 적외선 센서는 일반적으로 짧은 거리(몇 센티미터에서 수십 센티미터) 내의 물체를 감지하는 데 적합합니다. - 범위 조절 가능성: 일부 센서는 감지 범위를 조절할 수 있어, 필요에 따라 근거리 또는 중거리 감지로 조정할 수 있습니다.
감지방식	- 반사형 감지: 방출된 적외선이 물체에 반사되어 수신기로 돌아오는 것을 감지합니다. 주로 장애물 감지 및 거리 측정에 사용됩니다. - 차단형 감지: 적외선이 물체에 의해 차단되는 것을 감지합니다. 주로 출입구 모니터링 등에 사용됩니다. - 수동형(PIR) 감지: 환경의 열 복사(적외선)를 감지하여 움직임을 탐지합니다. 주로 보안 시스템에 사용됩니다.
장 점	- 빠른 반응 속도: 적외선 센서는 빛의 속도로 작동하기 때문에 매우 빠르게 반응할 수 있습니다. - 저비용: 생산 비용이 낮아 경제적으로 사용할 수 있습니다. - 다양한 응용: 장애물 회피, 거리 측정, 물체 감지, 라인 트레이싱 등 다양한 분야에 활

	용됩니다.
한계점	• 환경 조건에 민감: 적외선은 빛이기 때문에 주변 조명 조건에 영향을 받을 수 있으며, 특히 강한 태양광에서는 감지 성능이 저하될 수 있습니다. • 감지 거리 제한: 감지 거리가 제한적이므로 장거리 감지에는 적합하지 않습니다. • 물체 특성에 따른 감지 변화: 물체의 색상, 표면 특성(반사율)에 따라 감지 성능이 달라질 수 있습니다.
응용분야	• 로봇 공학: 로봇의 장애물 감지 및 회피, 라인 트레이싱 등 • 보안 시스템: 움직임 감지를 통한 경보 시스템 • 산업 자동화: 물체 유무 감지, 포장 및 조립 라인에서의 사용 • 가전 제품: TV 리모컨, 에어컨 제어 등 적외선 통신

외부 빛과 색상에 대한 적외선 센서의 감지 민감도를 이해하고 활용하는 것은 중요합니다. 적외선 센서의 정확한 동작을 위해 몇가지 특성을 이해하고 조정할 수 있기를 바랍니다. 특히, 적외선 센서의 민감도를 조절하기 위해 하드웨어와 소프트웨어를 모두 활용하여 최적화하는 것이 중요합니다.

빛의 영향

주변광(환경광) 영향:

- 적외선 센서는 주변광의 영향을 받을 수 있습니다. 특히 태양광은 적외선 파장을 포함하고 있어 센서의 수신기(포토 다이오드/포토 트랜지스터)가 혼동할 수 있습니다.
- 강한 조명이나 직사광선이 비추는 환경에서는 적외선 센서가 오작동할 수 있으므로, 센서를 보호하거나 외부 광원을 차단할 수 있는 방법을 고려해야 합니다.

IR LED 강도 조절:

- 송신기의 IR LED 강도를 조절하여 감지 범위를 변경할 수 있습니다. 예를 들어, 더 강한 신호는 먼 거리의 물체를 감지하는 데 유리하지만, 가까운 거리에서는 신호가 포화될 수 있습니다.
- 가변 저항을 통해 LED의 전류를 조절하여 송신 강도를 조절할 수 있습니다.

광학 필터 사용:

- IR 센서에 적외선만 통과시키는 광학 필터를 사용하여 특정 파장의 빛만 감지하도록 할 수 있습니다. 이 필터는 다른 파장의 빛을 차단하여 주변광의 영향을 줄입니다.

색상의 영향

물체의 반사율:

- 적외선은 물체의 표면 특성에 따라 다르게 반사됩니다. 밝은 색상이나 매끄러운 표면은 적외선을 더 많이 반사하고, 어두운 색상이나 거친 표면은 적외선을 더 적게 반사합니다.
- 검은색 물체는 적외선을 거의 흡수하므로, 센서의 반응이 감소할 수 있습니다.

물체와 배경의 대비:

- 물체의 색상이 배경과 큰 대조를 이룰 경우, 센서는 물체를 더 잘 감지할 수 있습니다.

다. 이 점을 이용하여 센서 배치를 최적화할 수 있습니다.

보정 및 캘리브레이션:

- 적외선 센서를 사용할 때 색상에 따른 감도 차이를 보정하기 위해 캘리브레이션을 수행할 수 있습니다.
- 이를 위해 다양한 색상과 표면에서 센서의 출력을 측정하고, 소프트웨어적으로 보정하는 방법이 있습니다.

※ 실습할 때 유의사항

적외선 센서를 실습에 적용하는 경우 적외선 센서는 주변의 빛 환경에 많은 영향을 받습니다. 센서를 점검하는 공간과 실제 실습을 진행하는 공간이 주변의 태양 빛 또는 형광등 조명 조건에서 동일하거나 유사한지 점검하십시오. 형광등 빛의 적외선 파장이 텐덤로봇의 적외선 센서 신호에 영향을 미치는지 점검하려면, 텐덤로봇에 적외선 센서를 설치하고 관찰하고 싶은 방향으로 센서를 향하게 해서 신호가 감지되는지 살펴볼 수 있습니다.

적외선 센서로 장애물 유무 탐지하기: ir_detect() 함수

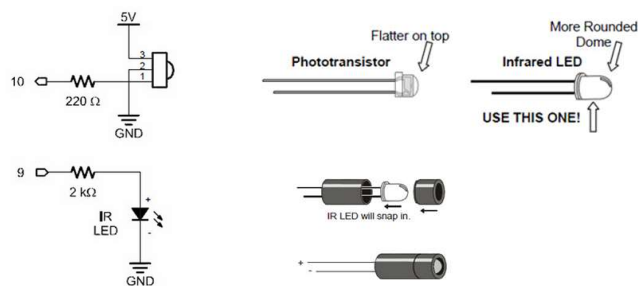


그림 4.26 적외선 송수신센서 회로도와 부품 개략도 (출처: parallax.com)

아두이노 디지털 핀에 연결된 적외선 센서를 마이크로비트에서 처리하는 예제4.18을 소개합니다.

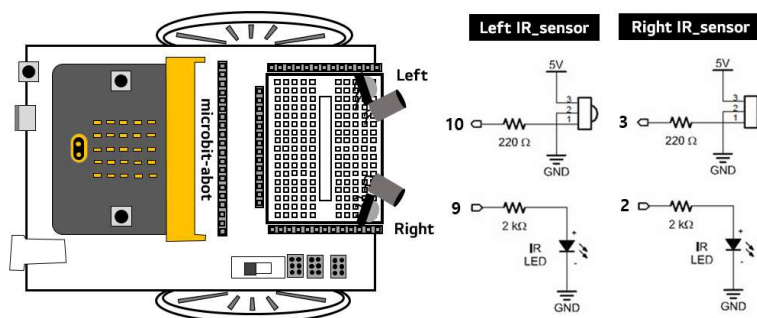


그림 4.27 텐덤로봇의 왼쪽과 오른쪽 적외선 센서 설치방법 개요 및 회로도

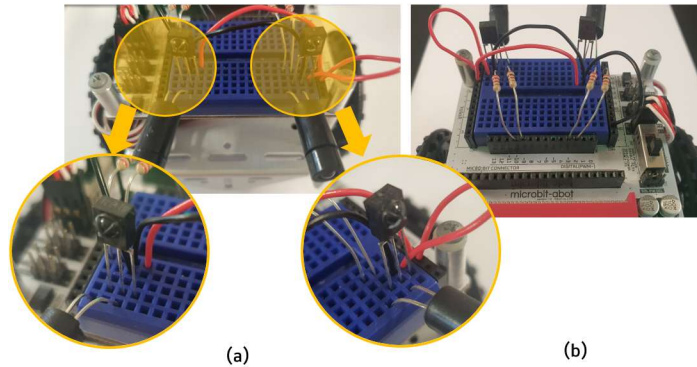


그림4.28 텐덤로봇 전면에 적외선 센서를 장착한 모습 (a) 전면 (b) 후면

로봇의 왼 쪽에 설치하는 적외선 센서는 적외선 LED를 디지털 9번핀, 적외선 수신기(receiver)는 10번핀에 연결합니다. 그리고 적외선 주파수는 38000 Hz로 설정합니다. 그리고 오른 쪽에 설치하는 적외선 센서는 적외선 LED를 2번핀, 적외선 수신기(receiver)는 3번핀에 연결합니다.

그림4.27과 그림4.28 그림을 참조해서 두 개의 센서를 각각 설치하고, 예제4.18 코드를 플래싱하세요. 아래 코드의 실행 결과를 얻으려면, 아두이노(microbit-abot) 보드의 3점점 스위치를 '1'의 위치에 놓아야 합니다.

예제4.18 "마이크로비트를 사용한 적외선 감지 및 화면 표시하기"

```
# ch4_18_micro_abot_ir_detect_display.py

from microbit_abot import *

left_sensor = bot(9, 10)
right_sensor = bot(2, 3)

while True:
    # Detect IR signals at 38 kHz
    irL = left_sensor.ir_detect(38000)
    irR = right_sensor.ir_detect(38000)

    # Toggle micro:bit display LED (x, y) on/off based on IR detection
    if irL:
        display.set_pixel(4, 2, 0) # If IR is detected on the left sensor, turn off pixel
    else:
        display.set_pixel(4, 2, 9) # If not detected, turn on pixel

    if irR:
        display.set_pixel(0, 2, 0) # If IR is detected on the right sensor, turn off pixel
    else:
        display.set_pixel(0, 2, 9) # If not detected, turn on pixel

    # Print IR detection status
    print("Left_IR:", irL)
    print("Right_IR:", irR)

    sleep(100)
```

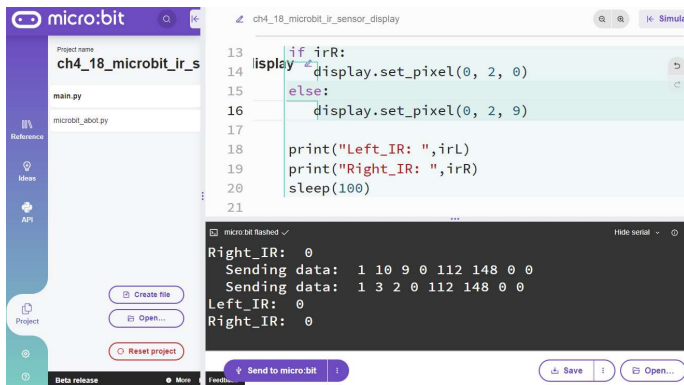


그림4.29 로봇 전방 물체를 감지한 경우의 시리얼 출력 '0'표시

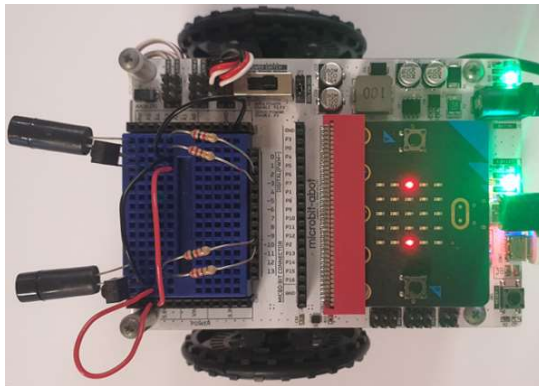


그림4.30 로봇 전방 물체를 감지한 경우의 LED 디스플레이 표시

그림4.30은 로봇의 전방에 장애물이 있는 경우이고, 그래서 로봇의 왼 쪽과 오른 쪽 센서가 물체를 감지하고 마이크로비트 LED 디스플레이에서 두 개의 LED불빛을 표시합니다.

적외선 센서로 장애물 거리 측정하기: ir_distance() 함수

아래 코드 예제는 로봇에 장착된 적외선 센서로 물체가 존재하는지('0') 존재하지 않는지('1')를 감지하는 ir_detect 함수를 사용하는 것이 아니라, ir_detect 함수를 주파수 스캔해서 물체와의 상대적 거리를 감지하는 함수 ir_distance 함수를 사용한 실습입니다.

ir_distance 함수는 아두이노의 irDistance 함수를 호출합니다. irDistance 함수의 동작원리는 그림4.31 적외선 센서의 주파수별 특성 그래프를 사용해서 설명합니다.

주파수에 대한 감도 특성을 살펴보면, 38000 ~ 42000 Hz 범위에서 선형적 특성이 나타나고 이런 선형성을 이용해서 물체와의 거리를 측정하는 용도로 쉽게 변환할 수 있습니다.

주파수 스캔 방법으로 물체와의 거리를 측정하는 원리는 물체와의 거리가 멀수록 38000 Hz 조건에서만 물체가 감지되고, 물체와의 거리가 가까울수록 42000 Hz 주파수까지 물체를 감지할 수 있습니다. 아래 소개할 아두이노 irDistance 메소드는 주파수를 스캔해서 각

주파수별 감지신호를 합산하는 방법으로 물체와의 상대적 거리를 차별화할 수 있습니다.

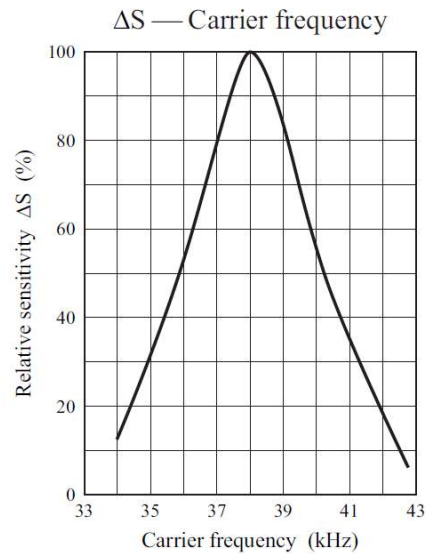


그림4.31 적외선 센서의 주파수별 특성 그래프 (파나소닉 PNA4602M)

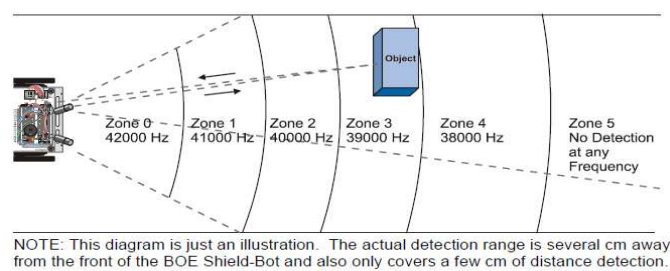


그림4.32 주파수별 거리측정을 위한 irDistance 원리 (출처: parallax.com)

마이크로비트 `ir_distance` 거리 측정값은 0~9 사이의 값으로 표현합니다. 물체와 거리가 감지거리 이내이면 0을 표시하고 물체를 감지하지 못하는 먼 거리는 9를 표시합니다. 그리고 마이크로비트 디스플레이 표시는 가까운 감지거리 0 값에서 5개의 LED가 켜지고, 먼 감지거리 9 값에서 1개의 LED만 켜집니다.

본 실습은 `ir_detect` 함수보다 더 많은 연산을 처리해야 하므로 아두이노 라이브러리 코드 구현에서 더 복잡하게 코드가 실행됩니다. 따라서 아래 코드에서 반복주기를 아래 코드에서 제시한 `sleep(50)` 보다 너무 짧게 줄이지 않아야 합니다.

예제4.19 "마이크로비트를 사용한 적외선 감지거리 LED표시하기"

```
# ch4_19_micro_abot_ir_distance_display.py

from microbit_abot import *
from microbit import display, sleep

left_sensor = bot(9, 10)
right_sensor = bot(2, 3)
```



```

def display_distance(sensor_value, column):
    """
    Displays the distance reading on a single column of the micro:bit LED display.

    :param sensor_value: The distance value returned by the IR distance sensor.
    :param column: The x-coordinate (column) on the micro:bit display to use.
    """
    # Calculate the number of LEDs to light based on the sensor value
    num_leds_to_light = 5 - (sensor_value // 2)

    for y in range(5): # y goes from 0 to 4
        if y < num_leds_to_light:
            display.set_pixel(column, y, 9) # Turn on the LED at full brightness
        else:
            display.set_pixel(column, y, 0) # Turn off the LED

while True:
    # Get the left and right sensor values
    left_distance = left_sensor.ir_distance()
    right_distance = right_sensor.ir_distance()

    # Adjust the brightness of the LEDs in column 0 according to the left sensor
    display_distance(left_distance, 0)

    # Adjust the brightness of the LEDs in column 4 according to the right sensor
    display_distance(right_distance, 4)

    # Print the sensor values to the console
    print("Left_IR:", left_distance)
    print("Right_IR:", right_distance)

    sleep(50)

```

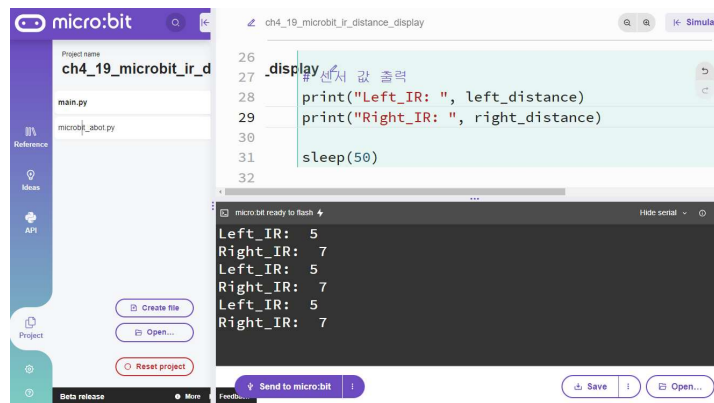


그림4.33 적외선 센서로 물체와의 거리측정 값을 출력하는 화면

예제는 마이크로비트와 적외선(IR) 센서를 사용하여 물체와의 거리를 측정하고, 그 값을 마이크로비트의 LED 디스플레이를 통해 시각적으로 표현하는 코드입니다.

[1] 주요 동작:

- `display_distance(sensor_value, column)` 함수:
 센서 값(`sensor_value`)을 입력받아 지정된 열(`column`)에서 LED를 제어합니다.
 센서 값이 작을수록 더 많은 LED가 켜지고, 값이 클수록 적은 LED가 켜집니다.
- 메인 루프:
 왼쪽 적외선 센서로부터 거리를 읽어 x축 0열의 LED를 제어합니다.

오른쪽 적외선 센서로부터 거리를 읽어 x축 4열의 LED를 제어합니다.

센서 값은 콘솔에 출력되며, 50밀리초마다 갱신됩니다.

[2] 디스플레이 동작:

- `display.set_pixel(x, y, brightness)` 함수로 마이크로비트의 LED를 제어합니다.
- 센서 값이 0일 때는 5개의 LED가 모두 켜지고, 값이 9일 때는 1개의 LED만 켜집니다.

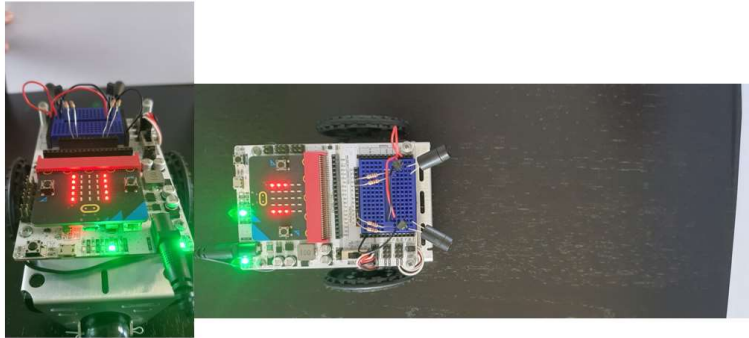


그림4.34 두 개의 적외선 거리센서신호를 마이크로비트 LED로 표시하기

(3) 접촉센서(더듬이 센서)

로봇의 자율제어를 만들기 위해 로봇 외부와의 물리적 접촉을 감지하는 센서가 사용되는 이유는 다음과 같습니다. 접촉센서가 필요한 중요한 이유는 로봇이 환경과의 상호작용에서 더 정밀하고 안전하게 동작하기를 희망하기 때문입니다.

특히, 비접촉 센서(예: 초음파, 적외선)가 감지하지 못하는 작은 물체나 투명한 물체를 감지할 수 있습니다. 터치센서는 로봇이 예기치 않은 접촉을 감지하고 이를 바탕으로 동작을 멈추거나 수정할 수 있습니다.

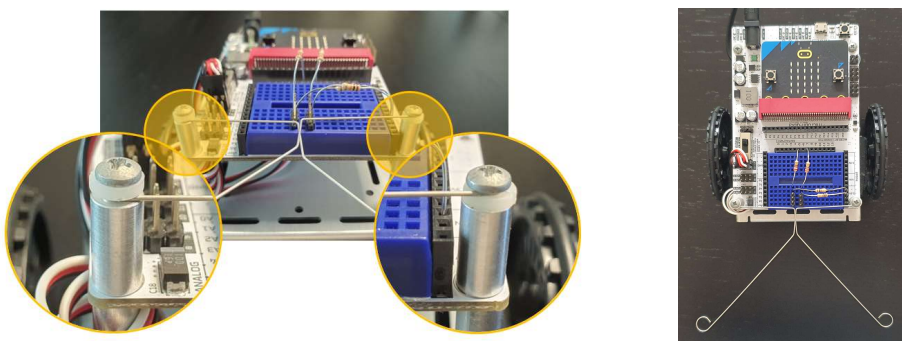


그림 4.35 더듬이 센서를 텐덤로봇 왼쪽과 오른쪽에 설치하는 방법

이런 목적의 터치센서 실습에 대한 사례를 소개합니다. 더듬이(whisker) 센서는 스탠드오프에 장착된 와셔를 기준으로 위쪽과 아래쪽으로 그림4.35를 참고해서 서로 엇갈리게 더듬

이센서를 결합하는 것이 요령입니다. 그렇게 함으로써 두 개의 더듬이 센서가 서로 겹치지 않고 독립적인 신호를 전달하게 됩니다.

양쪽 더듬이 센서의 접촉신호를 아두이노 디지털핀 5번과 7번에 연결하는 방법은 풀업방식의 버튼회로와 유사합니다.

더듬이 센서가 로봇 새시와 분리된 상태이면, 아두이노 디지털핀(5번/7번)은 5V 전압 HIGH 상태를 유지합니다. 그리고 더듬이 센서가 새시와 접촉하면, 새시를 접지로 하는 회로가 추가되어 5V 전원은 새시로 전류가 흐르고, 아두이노 디지털핀은 0V 전압 LOW 상태가 됩니다. 아래 코드의 실행 결과를 얻으려면, 아두이노(microbit-abot) 보드의 3접점 스위치를 1의 위치에 놓아야 합니다.

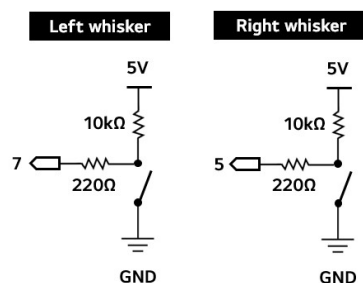


그림 4.36 텐덤로봇 왼쪽 오른쪽 더듬이 센서 회로도

예제4.20 "마이크로비트를 사용한 더듬이 센서 감지 및 화면 표시하기"

```
# ch4_20_micro_abot_whisker_sensor_display.py

from microbit_abot import *
left_sensor = bot(5)
right_sensor = bot(7)

while True:
    whisker_left = left_sensor.read_digital()
    whisker_right = right_sensor.read_digital()

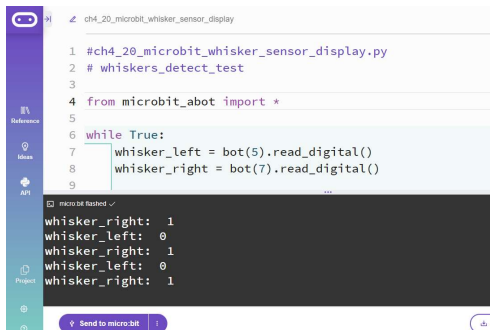
    if whisker_left == 0:
        display.set_pixel(4, 2, 9) #LED on if left whisker is pressed
    else:
        display.set_pixel(4, 2, 0)
    if whisker_right == 0:
        display.set_pixel(0, 2, 9) #LED on if right whisker is pressed
    else:
        display.set_pixel(0, 2, 0)

    print("whisker_left: ",whisker_left)
    print("whisker_right: ",whisker_right)
```

디지털핀 5번과 7번의 값을 read_digital() 함수로 읽으면, 더듬이 센서가 새시와 접촉한 상태('0')인지 또는 새시와 분리된 상태('1')인지를 구분하는 신호로 감지할 수 있습니다.

더듬이 센서가 읽은 값이 '0'일 때, 더듬이 센서가 외부 물체와 접촉해서 접지와 연결되

고 디지털 값은 LOW 상태가 됩니다. 마이크로비트 LED는 왼쪽 과 오른쪽의 더듬이 센서가 접촉하는지를 불빛으로 표시합니다.



(a)



(b)

그림4.37 왼쪽 더듬이센서가 접촉한 상태의 (a) 시리얼출력 (b) 실습장면

더듬이 센서의 기본 동작을 텐덤로봇의 바퀴 동작과 연결해서 자율주행하는 실습은 제 5장에서 계속됩니다.

(4) 초음파센서

로봇동작에서 초음파센서가 사용되는 이유는 로봇의 자율성과 효율성을 높이는데 중요한 역할을 하기 때문입니다. 대표적인 이유는 장거리 감지, 넓은 범위 감지, 다양한 환경에 대한 신뢰성, 정확한 거리 측정, 비접촉성과 비용 효율성, 안전성과 빠른 응답특성 등입니다.

아래 실습에서는 초음파센서를 이용해서 전방물체와의 거리를 측정하는 예제를 몇가지 방법으로 소개합니다. 텐덤로봇에서 초음파센서 신호 처리는 '마이크로비트에서 처리하는 방법'과 '아두이노에 연결해서 I2C통신으로 수신해서 처리하는 방법'이 있습니다.

마이크로비트에서 초음파센서 신호 처리하기

4핀을 사용하는 HR-04 또는 3핀을 사용하는 패럴랙스 ping 센서를 사용해서 실습할 수 있습니다. 아래 코드는 3핀 초음파센서인 ping 센서를 사용하는 파이썬 코드입니다.

예제4.21 "마이크로비트를 사용한 초음파 거리 측정"

```

# ch4_21_micro_ultrasonic_distance.py

from microbit import *
import utime

# Set up the Parallax 3-pin ultrasonic sensor on pin0
signal_pin = pin0

def get_distance():
    # Configure the signal pin as output to send the Trigger signal
    signal_pin.write_digital(0)
  
```

```

utime.sleep_us(2)
signal_pin.write_digital(1)
utime.sleep_us(5)
signal_pin.write_digital(0)

# Configure the signal pin as input to receive the Echo signal
signal_pin.set_pull(signal_pin.NO_PULL)
# Initialize signal_off and signal_on
signal_off = 0
signal_on = 0

# Wait for the Echo signal to go HIGH
while signal_pin.read_digital() == 0:
    signal_off = utime.ticks_us()

# Wait for the Echo signal to go LOW
while signal_pin.read_digital() == 1:
    signal_on = utime.ticks_us()

# Convert the time difference to distance (speed of sound is 343 m/s)
time_passed = signal_on - signal_off

# Convert time difference to centimeters
distance_cm = (time_passed * 34300) / 2 / 1000000
return distance_cm

while True:
    distance = get_distance()
    # display.scroll(" {:.2f} cm".format(distance))
    print("distance is {:.1f} cm".format(distance))
    sleep(50) # Measure distance every 50 ms

```

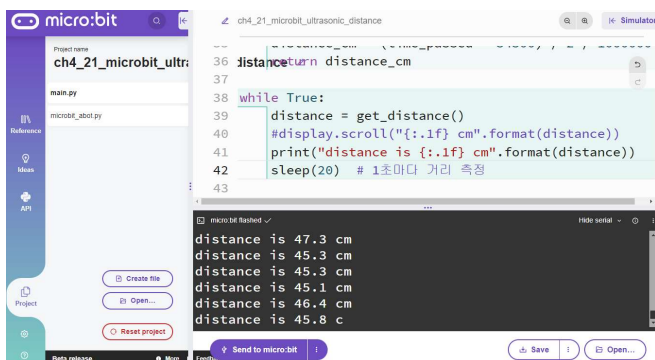


그림4.38 마이크로비트 P0핀으로 3핀 초음파센서 거리측정 출력화면

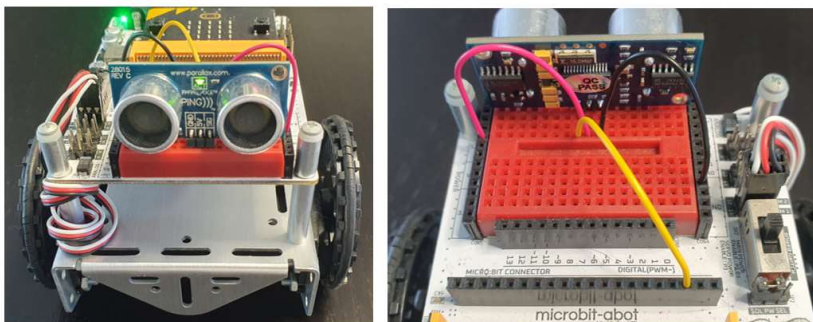


그림4.39 마이크로비트 P0핀으로 초음파센서 거리측정하기

아두이노에 연결해서 I2C로 초음파센서 신호 처리하기

초음파 센서를 아두이노 디지털핀에 연결하고, 마이크로비트에서 센서 신호를 수신하는 방법에 관한 코드입니다. 4핀을 사용하는 HR-04 또는 3핀을 사용하는 패럴렉스 ping 센서를 사용하는 경우 모두 I2C통신으로 데이터를 처리할 수 있습니다.

잊지마세요! 아두이노 보드를 사용하는 경우, 아두이노(microbit-abot) 보드의 3점점 스 위치를 1의 위치에 놓아야 합니다.

초음파센서로 거리를 측정하려면, 특정 신호를 먼저 보내고 되돌아오는 신호를 감지해야 합니다. 마이크로비트-아두이노 I2C통신으로 센서의 송수신 신호를 각각 처리하는 것은 한계가 있습니다. 그래서 마이크로비트에서는 초음파센서로 거리를 측정하기 위한 함수만 호출하고, 실제로 초음파센서에 신호를 발생시키는 트리거 신호와 되돌아오는 신호를 감지하는 에코 신호는 아두이노에서 모두 처리합니다. 이런 목적의 코드를 위해 별도의 모듈 코드 Ultrasonic.py 를 소개합니다.

Ultrasonic.py 모듈은 패럴렉스 사의 3핀 ping 센서와 HR-04로 알려진 4핀 규격 초음파센서 중에서 선택적으로 텐덤로봇에 적용해서 사용하는 코드입니다.

```
# microbot_module ultrasonic.py
from microbit_abot import *

class Ultrasonic:
    def __init__(self, p=33, q=None):
        self.pinA = p
        self.pinB = q

    def distance(self, u=None):
        if self.pinB is None:
            # 3핀 초음파 센서의 경우
            bot(self.pinA).send_c(35)
            d = bot(self.pinA).read_r()
        else:
            # 4핀 초음파 센서의 경우
            bot(self.pinA, self.pinB).send_c(36)
            d = bot(self.pinA).read_r()

        if d is None:
            print("Error: No signal received or invalid reading.")
            return None
        if u == 'us': return d
        elif u == 'in': return d / 148
        else: return d / 58
```

위 코드에서 p는 트리거 핀이고, q는 에코 핀으로 사용됩니다. 다만, 3핀 초음파센서의 경우 p가 트리거 핀과 에코 핀의 역할로 모두 사용됩니다. q 에코 핀의 기본값은 None 이므로, 아래 예제 코드에서처럼 3핀을 사용하는 경우 Ultrasonic(9).distance('cm') 코드로 매개변수를 한 개만 사용하고, 4핀 초음파센서인 경우 Ultrasonic(9,10).distance('cm') 코드로 두 개의 매개변수를 사용합니다. Ultrasonic(9,10).distance('cm') 코드에서 9는 트리거 핀이고 10은 에코 핀을 의미합니다.

def distance(self, u=None) 메서드는 초음파 센서를 사용하여 거리를 측정합니다. 3핀

초음파센서를 사용하는 경우와 4핀 초음파센서를 사용하는 경우 각각 다른 아두이노 함수를 호출하고 처리한 후 측정거리를 수신합니다.

u는 단위를 지정하는 인자입니다. us, in, 또는 아무 값도 전달하지 않을 수 있습니다.

us: 마이크로초(us) 단위로 거리를 반환합니다.

in: 인치(inch) 단위로 거리를 반환합니다.

기본값(아무 값도 전달하지 않으면): 센티미터(cm) 단위로 거리를 반환합니다.

아래 예제는 3핀 ping 초음파센서를 사용해서 거리를 측정하는 코드입니다. 신호선은 아두이노 디지털 9번핀에 연결되어 있습니다.

예제4.22 “아두이노 핀에 연결된 3핀 ping센서로 거리 측정하기”

```
# ch4_22_micro_abot_ultrasonic_distance.py

from microbit_abot import *
from ultrasonic import *

ultrasonic_sensor = Ultrasonic(9)

while True:
    distance = ultrasonic_sensor.distance('cm')
    print("distance is {:.1f} cm".format(distance))

    sleep(20)
```

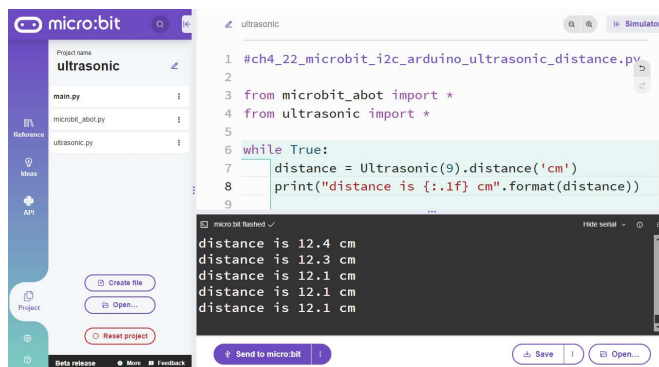


그림4.40 아두이노 P9핀으로 3핀 초음파센서 거리측정 출력화면

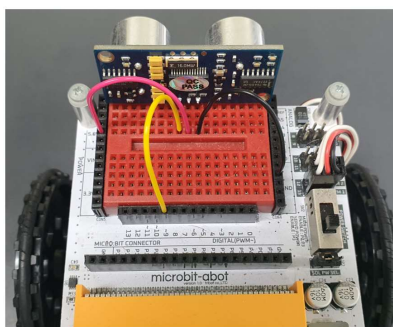


그림4.41 아두이노 P9핀으로 3핀 초음파센서 거리측정하기

4핀 초음파센서를 사용해서 거리를 측정하는 코드를 소개합니다. 트리거 핀은 9번 에코 핀은 10번의 아두이노 디지털핀에 연결합니다. HR04 초음파센서의 경우 센서 감지동작에서 ping센서보다 안정적으로 신호를 처리하지 못하는 것으로 생각됩니다.

예제4.23 “4핀 초음파센서 HR04로 거리 측정하기”

```
# ch4_23_micro_abot_4pin_ultrasonic_distance.py

from microbit_abot import *
from ultrasonic import *

ultrasonic_sensor = Ultrasonic(9, 10)

while True:
    distance = ultrasonic_sensor.distance('cm')
    print("distance is {:.1f} cm".format(distance))

    sleep(30)
```

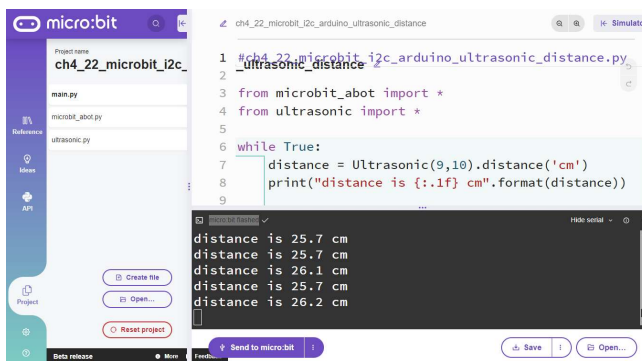


그림4.42 아두이노 P9, 10 핀으로 4핀 HR04 초음파센서 거리측정 화면

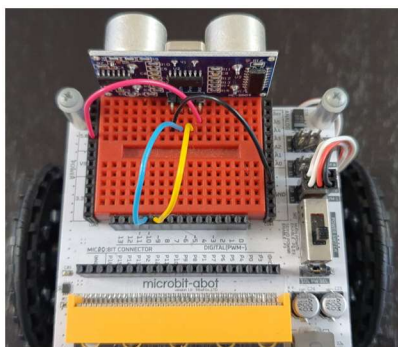


그림4.43 아두이노 4핀 초음파센서 거리측정하기

(Notice) -----

파이썬 언어의 모듈, 패키지 그리고 라이브러리에 대하여

파이썬 언어에서 모듈, 패키지 그리고 라이브러리 개념은 코드의 구조화와 재사용성을 높

이는 중요한 개념입니다.

모듈: 파이썬 코드 파일을 의미합니다. .py 확장자를 가진 하나의 파일을 모듈이라고 부르고, 모듈에는 변수, 함수, 클래스 등이 포함될 수 있습니다. 모듈을 사용하면 코드를 여러 파일로 분리해서 관리할 수 있으며, 다른 파이썬 파일에서 이를 import 해서 사용할 수 있습니다.

패키지: 여러 모듈을 모아 놓은 디렉토리(폴더)입니다. 모듈들이 모인 폴더를 패키지라고 부르고, 디렉토리 구조를 이용해 여러 모듈을 계층적으로 정리할 수 있습니다. 패키지 안에는 `__init__.py` 파일이 있어야 하며, 이 파일이 있으면 해당 디렉토리가 패키지로 인식됩니다.

라이브러리: 모듈과 패키지를 모아 놓은 코드의 모음을 의미합니다. 라이브러리는 특정 기능을 쉽게 구현할 수 있도록 여러 모듈과 패키지가 모인 큰 집합입니다. 라이브러리는 일반적으로 외부에서 pip로 설치하여 사용합니다.

참고로 `microbit_abot.py` 와 추가되는 모듈 .py 코드들은 패키지라고 부르는 것이 더 적절하지만, 텐덤로봇의 특정 기능(예: I2C 통신)을 수행하는 점에서 라이브러리라고 부를 수도 있습니다.

제 5 장 스마트한 자율주행 텐덤로봇

5.1 로봇 동작을 위한 준비

- (1) 텐덤로봇의 마이크로비트-아두이노 보드
- (2) 텐덤로봇 동작 바퀴용 서보모터

5.2 서보모터 중심 맞추기와 기본동작 시작하기

- (1) 서보모터 중심 맞추기
- (2) 로봇의 기본동작과 회전속도

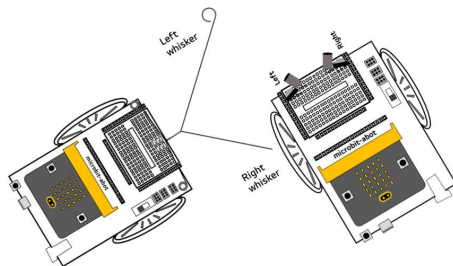
5.3 텐덤로봇 자율주행하기

- (1) 더듬이센서 자율주행
- (2) 광센서(포토틀랜지스터) 빛 추적하기
- (3) 적외선 송수신센서 자율주행
- (4) 초음파센서 PID제어

5.4 텐덤로봇 초음파센서 레이더 스캔하기

5.5 텐덤로봇 직선주행 보정하기

- (1) 텐덤로봇 직선주행 보정하는 법
- (2) 모듈 방식으로 전진/후진 주행 보정효과 유지하기



이 장은 센서를 장착한 로봇이 지능적으로 주행하는 실습을 소개합니다. 텐덤로봇 자율주행은 주어진 환경에서 독립적으로 움직이며, 간단한 작업을 수행하는 것입니다. 자율주행을 위해 더듬이 센서, 광센서, 적외선 센서를 활용합니다. 그리고 초음파센서로 일정한 거리를 유지하는 실습을 설명합니다.

'neo-아두이노 로봇카'와 유사한 방식의 자율주행입니다. 왼 쪽과 오른 쪽 두 개의 광센서 신호를 정규화하고 빛의 방향을 감지하는 경험을 시작해보세요. 적외선 센서의 경우 디지털 방식으로 신호를 처리하는 방법과 아날로그 거리 데이터를 처리하는 방법을 사용할 수 있습니다. 그리고 PID제어를 사용해서 물체와의 거리를 일정하게 유지하는 실습을 초음파센서로 경험할 수 있습니다.

이 장은 텐덤로봇의 기본동작을 제어하는 내용부터 시작하므로 상세히 살펴보기를 추천합니다. 로봇을 다루기 위한 초보적인 공학개념이 사용되고, 공학개념에 대한 설명이 있으므로 초보자도 충분히 이해하고 활용할 수 있습니다.

5.1 로봇 동작을 위한 준비

(1) 텐덤로봇의 마이크로비트-아두이노 보드

텐덤로봇을 동작하려면 조립된 로봇의 하드웨어 준비와 함께 마이크로비트와 아두이노에 필요한 라이브러리를 다운로드받고 모두 설치해야 합니다. 로봇동작에 필요한 라이브러리 설치 방법은 3.3절에서 소개하였습니다. 아래 하드웨어 준비작업은 순서대로 따라하기 바랍니다.

서보모터 3핀 커넥터 연결 및 3점점 스위치 조작 방법

로봇을 동작하려면 아래 그림 5.1에 위치한 서보핀에 서보모터의 3핀 연결선을 결합해야 합니다. 3핀 커넥터의 연결방법은 흰색선이 브레드보드 방향으로 향하도록 연결하면 됩니다.

두 개의 바퀴를 동작하기 위한 서보 커넥터는 P8/P9번 또는 P10/P11번 또는 P12/P13번 핀으로 짝을 지어서 연결해야 합니다. 표준형 서보모터를 연결하는 경우 바퀴로 사용하는 핀 2개를 제외한 나머지 4개의 핀들 중에서 한 개를 선택해서 전원 방향(흰색선이 브레드보드 방향)에 맞게 연결하면 됩니다.

3점점 스위치의 '0' 위치는 보드에서 모든 전원을 차단하고, '1' 위치(중간 위치)는 아두이노 보드와 마이크로비트 보드에만 전원이 공급되고, '2' 위치에서 로봇의 서보모터 핀에도 전원을 공급합니다.

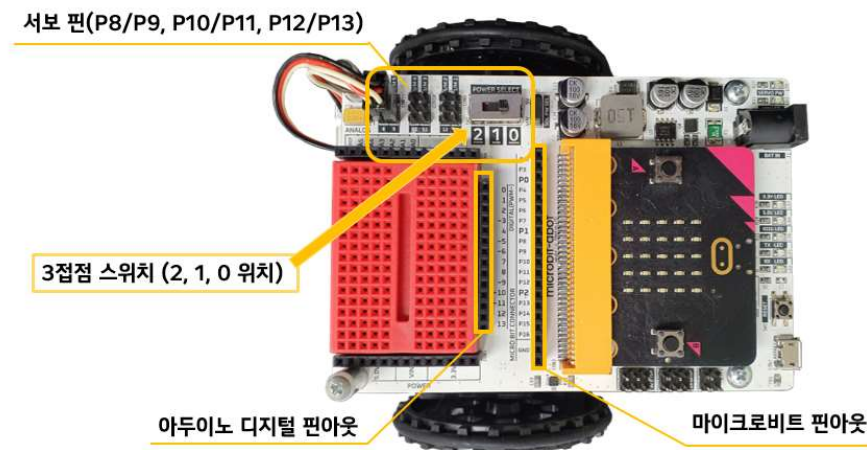


그림 5.1 텐덤로봇 동작을 위한 서보핀 & 핀아웃

(주의사항) 텐덤로봇을 물리적으로 결합할 때 마이크로비트-아두이노 보드의 전원을 차단한 상태에서 마이크로비트를 소켓에 플러그인하기 바랍니다. 아두이노 보드의 배터리 전원이나 USB전원을 공급한 상태에서 마이크로비트 보드를 소켓에 끼우면 전기적 손상을 입을 수도 있습니다.

텐덤로봇 전원공급에 대하여

텐덤로봇을 동작하려면 반드시 AA배터리 5개를 모두 채운다음 3접점 스위치를 사용해야 합니다. 마이크로비트 USB케이블 연결만으로는 서보모터 또는 아두이노 보드를 구동할 충분한 전력이 공급되지 않습니다. 만약 AA배터리가 준비되지 않은 상태에서 아두이노 보드에 충분한 전력을 공급하려면, 아두이노 보드에도 USB케이블을 추가 연결하면 됩니다.

(Notice) -----

텐덤로봇의 AA배터리 용량관리

텐덤로봇은 알카라인 AA건전지 5개를 직렬구조로 사용합니다. 배터리 전원으로 로봇을 동작하는 경우, 배터리 전압이 6.5V 이하로 떨어지면 내부 회로가 필요한 전류를 안정적으로 공급하지 못해 로봇이 켜졌다 꺼졌다를 반복(전원 리셋)할 수 있습니다.

이는 로봇이나 부품 고장이 아닌, 배터리 전압이 충분치 않을 때 나타나는 정상적인 동작 특성입니다. 정상적인 로봇 동작을 위해 배터리 교체 주기를 준수하기 바랍니다.

서보모터를 동작하기 위한 파이썬 라이브러리 함수

`servo_attachpins()` : 로봇의 바퀴를 동작하는 연속회전형 서보모터의 `attach()` 함수를 호출합니다. 서보모터의 양쪽 바퀴 핀번호 2개를 동시에 매개변수로 설정해야 합니다. 왼쪽 바퀴와 오른쪽 바퀴의 서보허용 핀번호는 (13, 12) 또는 (11, 10), (9, 8)을 선택할 수 있습니다.

`servo_angle()` : 표준형 서보모터를 동작하기 위한 호출 함수입니다. 매개변수의 값 범위는 0도에서 180도 사이 값입니다. 표준형 서보의 `attach` 기능이 포함되어 `servo_angle` 함수가 실행됩니다. 따라서 별도의 서보 `attach` 함수 호출이 필요하지 않습니다.

`servo_speed()` : 연속 회전형 서보모터를 동작하기 위한 호출 함수입니다. 매개변수의 값 범위는 -200 ~ +200 사이의 값을 사용합니다. 매개변수는 `servo_attachpins()`핀에서 설정한 바퀴의 서보 핀번호를 사용할 수 있습니다. 하나의 바퀴만 동작하도록 하나의 매개변수를 사용할 수 있고, 두 개의 바퀴를 동시에 동작하기 위해 두 개의 매개변수를 사용할 수 있습니다.

`detach()` : 서보모터의 준비상태를 해제하려는 호출 함수입니다.

(※ 서보모터 관련함수 사용에 관한 기본설정:

`servo_attachpins()` 함수를 실행하면, 이전의 6개 서보핀 중에서 바퀴로 설정하는 2개 핀을 제외한 나머지 4개 핀을 모두 `detach` 하는 기능이 자동 실행됩니다. 초보 텐덤로봇 사용자를 위한 초기 기본 설정입니다. 따라서 표준형서보 `servo_angle()` 함수 실행은 항상 `servo_attachpins()` 다음 순서로 실행해야 합니다.)

(Notice) -----

아두이노 디지털 13번 핀의 부트스트랩 동작에 대하여

텐덤로봇은 아두이노로봇의 기능이 포함된 로봇입니다. 아두이노 디지털 13번 핀은 내장된 LED와 연결되어 있으며, 이를 통해 간단한 시각적 피드백을 제공하도록 설계되어 있습니다. 부트스트랩(부트로더) 동작 시, 13번 핀 LED는 부트로더가 실행되고 있음을 알리기 위해 깜박입니다. 새로운 스케치가 업로드되는 동안에도 깜박입니다. 업로드가 완료되면, LED는 깜박임을 멈추고 새로운 코드명령을 기다립니다.

텐덤로봇 3점점 스위치 '2'위치에서 아두이노 보드 전원이 처음 공급되면, 부트스트랩 때문에 P13번에 연결된 바퀴는 미세한 움직임이 발생합니다. 이런 문제를 피하려면, P(13,12) 이외에 사용 가능한 P(11,10) 또는 P(9,8) 핀을 사용할 수 있습니다.

(2) 텐덤로봇 동작 바퀴용 서보모터

마이크로비트-아두이노 텐덤로봇의 바퀴동작에 대한 설명입니다. 실습에서 사용할 텐덤로봇은 두 개의 바퀴를 사용해서 주행합니다. 로봇의 양쪽 각 한 개씩 서보모터가 연결되어 바퀴동작을 만들고, 정지상태 로봇의 안정적인 자세 유지를 위해 1인치 테일 볼을 사용합니다. 꼬리 볼 바퀴의 경우 서보에 의한 속도 제어 또는 방향 제어에 기여하지 않습니다.

로봇의 바퀴에 연결된 서보모터는 Parallax사의 연속회전형 서보모터입니다. 그리고 센서를 장착할 목적으로 표준형 서보모터가 서보핀에 연결되어 사용됩니다. 표준형 서보모터의 경우 0~180도 사이의 각도에서 반복 회전합니다.

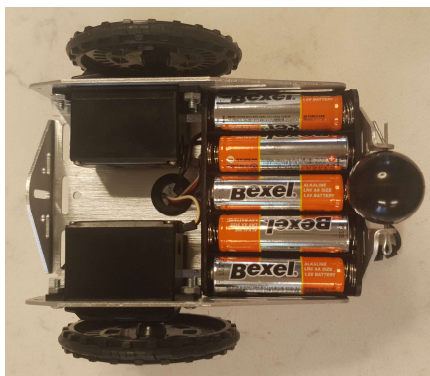


그림 5.2 텐덤 로봇 아랫방향 사진

서보모터는 고정 주기내에서 펄스폭 변조(PWM) 제어를 사용합니다. 서보모터의 시계방향 회전 또는 반시계 방향 회전은 PWM 제어에서 펄스폭의 중앙 값인 '0'에서 변화합니다. 즉 서보 모터의 회전 방향과 크기는 PWM 신호의 활성 펄스 폭을 변경하여 제어합니다. 표준 범위 내의 펄스가 더 짧거나 길면 모터가 중립 위치에서 시계 방향 또는 시계 반대 방향으로 회전하는 양을 정밀하게 제어할 수 있습니다.

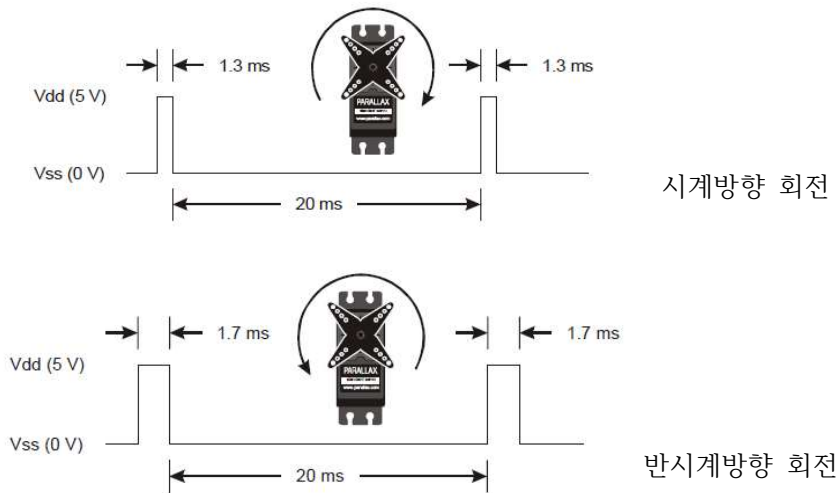


그림 5.3 패럴랙스 연속회전형 서보모터의 시계방향 & 반시계방향 동작원리

파이썬 라이브러리에서 바퀴로 연결된 서보모터 동작 파이썬 명령은 `servo_speed()` 함수입니다. 그리고 표준형 서보모터 동작명령은 `servo_angle()` 함수입니다. `servo_speed()` 함수가 실행되려면, 먼저 `servo_attachpins()` 함수가 실행되어야 합니다. 마지막으로 서보를 분리하는 `detach()` 함수를 사용할 수 있습니다. 파이썬 서보 함수들은 I2C통신으로 아두이노 라이브러리의 서보동작을 위한 함수 실행 명령을 전달합니다.

5.2 서보 중심 맞추기와 기본동작 시작하기

(1) 서보모터 중심 맞추기

텐덤로봇의 서보모터는 마이크로비트의 파이썬 함수 명령으로부터 I2C통신으로 아두이노에 전달되어 아두이노 라이브러리의 특정함수가 실행되도록 설계됩니다.

로봇에 사용하는 서보모터는 공장에서 출하되어 사용될 때, 특히 정밀 제어가 필요한 로봇공학에서는 서보의 움직임을 위해 물리적으로 보정하는 것이 필요합니다. 서보모터는 이런 목적의 물리적 보정을 위해 가변저항을 내장합니다. 사용자는 스크루를 조정하는 방법으로 물리적 중간점을 찾아서 정렬할 수 있습니다.

아래 코드는 마이크로비트에 플래싱할 '서보모터 중심 맞추기' 코드입니다. 서보모터의 물리적 중심이 잘 맞추어져 있다면, 아래 예제5.1을 플래싱하면 'HAPPY' 이미지가 디스플레이되는 동안 로봇 바퀴는 움직이지 않아야 합니다.

예제5.1 "마이크로비트 로고 터치 리셋과 서보 모터 중심맞추기"

```
# ch5_01_abot_servo_centering.py
```

```

from microbit_abot import *

def check_logo_touch():
    if pin_logo.is_touched():
        reset()

display.show(Image.HAPPY)
bot(13, 12).servo_attachpins()
bot(13, 12).servo_speed(0, 0)
sleep(5000)
display.clear()

bot(13).detach()
bot(12).detach()

while True:
    check_logo_touch()
    sleep(100)

```



그림 5.4 서보모터 중심맞추기 가변저항 위치 및 조정 방법

위의 코드를 플래싱했을 때 5초 이내에 바퀴가 미세하게 움직이고 있다면, 스크류를 움직여서 미세한 움직임이 없도록 조정해야 합니다. 이런 작업을 '서보모터 중심 맞추기'라고 부릅니다.

실습 코드는 서보모터 핀이 13번과 12번에 연결되어 5초동안 모터에 전원을 공급한 다음 detach() 함수로 정지하게 되어 있습니다.

서보모터 중심 맞추기 이외에 추가된 코드는 로봇에 장착된 마이크로비트의 하드웨어 리셋 버튼 조작이 불편하기 때문에, '마이크로비트의 로고 터치센서를 사용해서 소프트웨어 리셋을 시도하려는 코드'입니다. 추가 코드에 의해 마이크로비트를 다시 플래싱하지 않고 5초간의 코드 실행이 종료된 이후에도 로고 터치를 사용해서 '중심 맞추기'코드를 재시작할 수 있습니다.

```

def check_logo_touch():
    if pin_logo.is_touched():
        reset()
while True:
    check_logo_touch()
    sleep(100)

```

(2) 로봇의 기본동작과 회전속도

패럴렉스 연속회전형 서보모터 속도변화 그래프를 아래 그림 5.5에서 소개합니다.

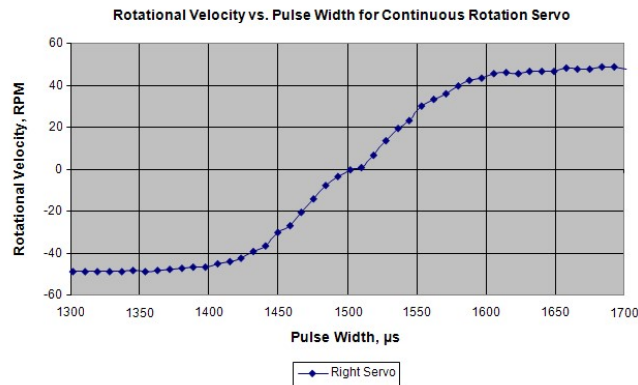


그림 5.5 패럴렉스 연속회전형 서보모터의 속도에 대한 펄스폭 변화량
(출처: parallax.com)

속도값의 범위는 1500을 기준으로 -200부터 +200까지 변화시킬 수 있습니다. 아래 그래프가 나타내는 실효적인 속도는 -100 us부터 +100 us까지 구간에서 거의 모든 변화가 이루어집니다. 그리고 -40 us 또는 +40 us 에서 거의 중간속도를 나타냅니다.

텐덤로봇 기본동작은 전진, 후진, 좌회전, 우회전 그리고 정지동작으로 구분됩니다. 로봇은 두 개의 좌측과 우측 바퀴동작으로 다양한 로봇 동작을 만들 수 있습니다.

두 개의 매개변수 함수 `bot(left_pin, right_pin).servo_speed(left_value, right_value)`를 사용하는 경우, 함수의 매개변수 규칙을 준수해야 합니다. 사용자 편의에 따라 왼쪽과 오른쪽 구분은 바뀌서 사용할 수 있습니다.

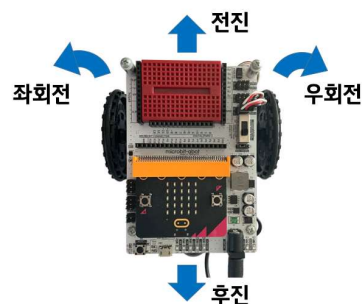


그림 5.6 : 로봇의 기본동작 방향

로봇의 전진동작은 왼쪽 서보모터(P13)는 반시계방향으로 회전하고 오른쪽 서보모터(P12)는 시계방향으로 회전해야 합니다. 양쪽 바퀴의 속도 값이 동일한 값으로 부호만 반대로 사용하면 됩니다. 후진 동작은 왼쪽 서보모터는 시계방향으로 회전하고 오른쪽 서보모터는 반시계방향으로 회전해야 합니다.

```
bot(13,12).servo_speed(100, -100) # 전진
bot(13,12).servo_speed(-100, 100) # 후진
```

좌회전 우회전 동작은 왼쪽 바퀴와 오른쪽 바퀴의 속도와 회전방향을 변화시키면 회전동

작이 만들어집니다. 회전은 커브로 완만하게 회전하거나, 피봇동작으로 회전할 수 있습니다.

```
bot(13,12).servo_speed(0, -100) # 피봇 좌회전
bot(13,12).servo_speed(100, 0)  # 피봇 우회전
```

시계방향 제자리 회전과 반시계방향으로 제자리 회전하는 코드는 다음과 같습니다.

```
bot(13,12).servo_speed(100, 100) # 시계방향 제자리회전
bot(13,12).servo_speed(-100, -100) # 반시계방향 제자리회전
```

앞에서 설명한 로봇 바퀴동작 코드는 2개의 매개변수를 사용하는 방법입니다. 이 방법 이외에 또 다른 서보모터 동작방법으로 1개의 매개변수를 사용하는 것입니다. 이 방법은 로봇 동작을 위한 매개변수 사용이 명확합니다.

```
# 전진
bot(13).servo_speed(100)
bot(12).servo_speed(-100)
# 후진
bot(13).servo_speed(-100)
bot(12).servo_speed(100)
```

로봇이 전진 3초, 좌회전 1초, 후회전 1초 그리고 후진 3초 동작을 만드는 코드를 소개합니다. 아래 코드는 매개변수가 1개인 서보모터 동작 함수로 수정해서 사용하는 것이 가능합니다.

예제5.2 "마이크로비트를 사용한 로봇 기본동작 제어"

```
# ch5_02_abot_basic_drive.py
from microbit_abot import *

def check_logo_touch():
    if pin_logo.is_touched():
        reset()

bot(13, 12).servo_attachpins()

# forward drive
bot(13, 12).servo_speed(100, -100)
sleep(2000)

# pivot left
bot(13, 12).servo_speed(0, -100)
sleep(1000)

# pivot right
bot(13, 12).servo_speed(100, 0)
sleep(1000)

# backward drive
bot(13, 12).servo_speed(-100, 100)
sleep(2000)

# detach servo
bot(13).detach()
bot(12).detach()

while True:
```

```
check_logo_touch()
sleep(100)
```

5.3 텐덤로봇 자율주행하기

로봇에 장착된 센서를 이용해서 자율주행하는 실습입니다. 앞에서 설명한 제4장 5절의 내용을 기초로 센서를 장착하고, 왼쪽 센서와 오른쪽 센서 신호를 사용해서 로봇 바퀴를 제어하는 실습입니다. 여기에 간단한 공학 알고리즘을 추가해서 스마트한 로봇 동작을 만들 수 있습니다.

(1) 더듬이센서 자율주행

앞장 제4장 5절 ‘아두이노 센서 활용하기’에서 설명한 더듬이 센서를 기억하십시오. 더듬이 센서 신호를 로봇의 주행동작과 연결하는 것이 지금 실습 목표입니다.

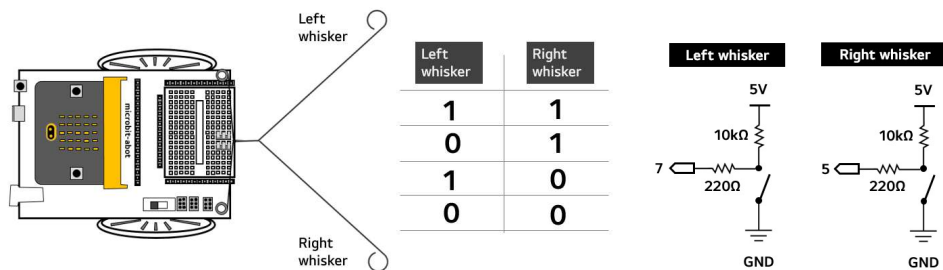


그림5.7 더듬이센서를 장착한 텐덤로봇의 4가지 센서 상태표와 회로도

더듬이 센서는 아날로그 신호가 아니라 디지털 신호를 생성한다는 점을 기억하세요. 로봇 전면에 설치하는 두 개의 더듬이 센서가 만들 수 있는 디지털신호 상태는 모두 4가지가 됩니다. 우리는 4가지 디지털신호 상태에 따라 로봇동작을 각각 다르게 지정하는 방법으로 센서 신호와 로봇 동작의 연관성을 부여할 것입니다.

기억하세요! 더듬이 센서가 눌러진 상태의 센서 값은 ‘0’이 되고 눌러지지 않는 상태의 센서 값은 ‘1’이며, 외부 물체와 접촉해서 ‘0’의 센서 값일 때 마이크로비트 디스플레이 LED 불빛으로 표시되는 예제4.20 코드를 재사용하거나 아래의 영어 알파벳 표시방법을 사용할 수 있습니다. 센서를 구성하는 방법은 4.5절의 회로도를 참고하기 바랍니다.

두 개의 더듬이 센서가 모두 눌러지지 않는 상태는 장애물이 로봇과 접촉하지 않은 상태입니다. 따라서 로봇은 전진할 수 있습니다.

왼 쪽 더듬이가 눌러지고 오른 쪽 더듬이는 눌러지지 않은 상태이면, 왼 쪽에 장애물이 접촉되었다는 의미이므로 로봇은 오른 쪽으로 회전할 수 있습니다. 이때 한 가지 생각할 것

은 접촉한 시점부터는 물체와 계속 접촉하고 있는 상태일 수 있으므로, 뒤로 잠깐 후진한 다음 오른 쪽으로 회전하는 것이 유연한 로봇동작일 수 있습니다.

왼 쪽 더듬이는 눌러지지 않고, 오른 쪽 더듬이가 눌러진 상태라면, 조금 전 로봇동작과 반대 방향으로 회전하면 됩니다.

마지막으로 왼 쪽 더듬이와 오른 쪽 더듬이가 모두 눌러진 상태이면, 로봇은 더 이상 전진하기 어려운 상태일 수 있습니다. 물론 더 지능적인 감지능력이 로봇에 부여된다면 미로를 탐색할 수 있지만, 지금처럼 원시적인 센서 동작으로는 후진이 가장 간편한 해결책일 수 있습니다. 충분히 후진해서 오른 쪽이나 왼 쪽으로 충분히 회전하고 다시 전진하는 로봇동작을 계속합니다.

지금까지 요약한 더듬이 센서에 반응하는 로봇동작을 아래 예제에서 소개합니다.

예제5.3 “더듬이 센서로 자율주행하는 텐덤로봇”

```
# ch5_03_whisker_navigation_drive.py
from microbit_abot import *

abot = bot(13, 12)
left_sensor = bot(5)
right_sensor = bot(7)
abot.servo_attachpins()

def forward():
    abot.servo_speed(100, -100)
    display.show("F")
    sleep(20)

def backward():
    abot.servo_speed(-100, 100)
    display.show("B")
    sleep(1000)
    abot.servo_speed(100, 100)
    sleep(1200)

def turnright():
    abot.servo_speed(-100, 100)
    display.show("R")
    sleep(1000)
    abot.servo_speed(100, 0)
    sleep(800)

def turnleft():
    abot.servo_speed(-100, 100)
    display.show("L")
    sleep(1000)
    abot.servo_speed(0, -100)
    sleep(800)

while True:
    whisker_left = left_sensor.read_digital()
    whisker_right = right_sensor.read_digital()

    if whisker_left == 0 and whisker_right == 0:
        backward()
    elif whisker_left == 1 and whisker_right == 0:
        turnleft()
    elif whisker_left == 0 and whisker_right == 1:
        turnright()
```

```
else:  
    forward()
```

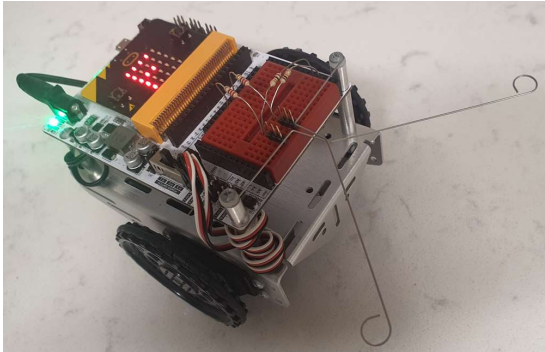


그림5.8 더듬이센서로 자율주행하는 텐덤로봇

아래 그림5.9는 로봇이 공간내의 모서리에 위치한 상태를 묘사한 그림입니다. 두 개의 더듬이 센서가 번갈아가면서 접촉하는 모서리에서 로봇이 탈출하려면, 또 다른 모서리 탈출 알고리즘이 필요합니다.

모서리 탈출 논리가 필요한 이유는 전방 장애물을 감지하는 두 개의 센서가 전기적으로 동시에 접촉하지 않는 상태에 대응하기 위한 것입니다. 아래 모서리 탈출 논리를 요약하면 다음과 같습니다.

먼저 두 개의 센서가 교차로 감지하는 조건의 횟수를 저장합니다. `last_sensor` 변수를 사용해 이전에 감지된 센서가 무엇인지 추적합니다. 이 변수는 교차 동작을 확인하는 데 사용됩니다.

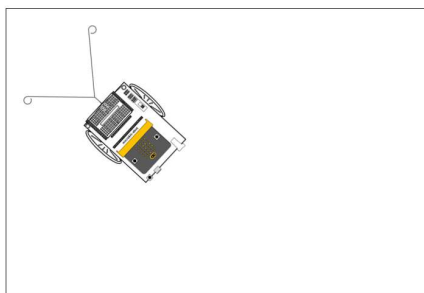


그림5.9 주행중 모서리에 위치한 텐덤로봇의 개념도

왼 쪽 센서가 감지된 후에 오른 쪽 센서가 감지되거나, 오른 쪽 센서가 감지된 후에 왼쪽 센서가 감지되면 교차가 발생했다고 간주하고 카운트를 증가시킵니다. 만약 동일한 센서가 연속적으로 감지되면 `turn_count`는 리셋됩니다. 즉, 교차가 일어나지 않은 경우에는 카운트가 증가하지 않고 다시 0으로 초기화됩니다. 그리고 두 센서 모두 장애물을 감지하지 않고 전진하는 경우, 카운트는 그대로 유지되며 리셋되지 않습니다.

교차 동작이 설정된 횟수 이상 발생하면 `escape_corner()` 함수가 호출되고 카운트가 초기화됩니다. 아래 코드의 설정 횟수는 4회입니다.

예제5.4 “모서리 탈출 알고리즘 자율주행 텐덤로봇”

```
# ch5_04_whisker_navigation_edge_escape_drive.py

from microbit_abot import *

abot = bot(13, 12)
left_sensor = bot(5)
right_sensor = bot(7)

abot.servo_attachpins()

def forward():
    abot.servo_speed(100, -100)
    display.show("F")
    sleep(20)

def backward():
    abot.servo_speed(-100, 100)
    display.show("B")
    sleep(1000)
    abot.servo_speed(100, 100)
    sleep(1200)

def turnright():
    abot.servo_speed(-100, 100)
    display.show("R")
    sleep(1000)
    abot.servo_speed(100, 0)
    sleep(800)

def turnleft():
    abot.servo_speed(-100, 100)
    display.show("L")
    sleep(1000)
    abot.servo_speed(0, -100)
    sleep(800)

def escape_corner():
    # Specific pattern to escape from a corner
    abot.servo_speed(-100, 100)
    display.show(Image.SQUARE)
    sleep(1000)
    abot.servo_speed(100, 100)
    sleep(1200)

# Count of intersection actions and previous state
turn_count = 0
last_sensor = None

while True:
    whisker_left = left_sensor.read_digital()
    whisker_right = right_sensor.read_digital()

    if whisker_left == 0 and whisker_right == 0:
        escape_corner()
        turn_count = 0
        last_sensor = None

    elif whisker_left == 1 and whisker_right == 1:
        forward()

    else:
        if whisker_left == 1 and whisker_right == 0:
            turnleft()
            if last_sensor == "right":
```

```

        turn_count += 1
        last_sensor = "left"
    else:
        turn_count = 0
        last_sensor = "left"

    elif whisker_left == 0 and whisker_right == 1:
        turnright()
        if last_sensor == "left":
            turn_count += 1
            last_sensor = "right"
        else:
            turn_count = 0
            last_sensor = "right"

    # Call escape_corner if intersection actions occur a certain number of times
    if turn_count >= 4: # If the number of intersections is limited to 4
        escape_corner()
        turn_count = 0
        last_sensor = None

```

(2) 광센서(포토티랜지스터: phototransistor) 빛 추적하기

두 개의 포토트랜지스터를 로봇의 왼 쪽과 오른 쪽 앞부분에 설치하고, 외부 빛의 방향에 반응하는 로봇동작을 만들 수 있습니다. 광센서에 대한 개요와 기본적인 센서 사용법은 4.5 절에서 설명했으므로, 참고하기 바랍니다.

지금은 광센서 신호를 로봇의 서보모터와 연결해서, 로봇 외부 빛의 방향에 반응하는 동작을 만드는 것입니다. 아래 두 가지 종류의 예제코드는 모두 광센서를 사용한 로봇동작을 만드는 것입니다.

한 가지 방법은 조금 전 더듬이센서 제어동작과 유사하게 왼 쪽 광센서 감지신호와 오른 쪽 광센서 감지신호를 '0' 과 '1'처럼 디지털 정보로 판단하여 동작하는 것이고, 다른 한 가지 방법은 두 개의 광센서 신호 크기 차이를 로봇의 양쪽 바퀴에 실시간 비례 변화로 연결 지어 로봇동작을 만드는 것입니다. 두 가지 접근방식에 대하여 직접 경험해보세요!

외부 빛에 반응하는 간단한 로봇동작

로봇이 외부 빛에 반응하는 가장 쉬운 방법은 상태논리를 만드는 것입니다. 가령, 로봇의 동작을 전진, 좌회전, 우회전, 정지 동작으로 나누고, 두 개의 광센서 신호 크기가 거의 동일하면 전진, 두 개의 광센서 신호 크기가 크게 차이난다면 좌회전 또는 우회전, 그리고 한계 크기 이하의 광센서 신호이면 정지 상태로 구분해서 코드를 작성할 수 있습니다. 이런 동작 방식은 더듬이 센서를 사용해서 로봇 동작을 만드는 것과 매우 유사합니다.

아래 코드의 실행 방법은 단순히 빛을 비추는 방향으로 로봇이 회전하거나, 외부에서 빛을 추가로 비추지 않으면 정지하는 방식입니다. 아래 코드에서 다음 두가지 변수의 값을 결정하면 됩니다. (*※ 아래 변수 설정 값은 사용자 환경에 따라 다르게 선택해야 합니다.*)

```

light_threshold = 1000
difference_threshold = 200

```

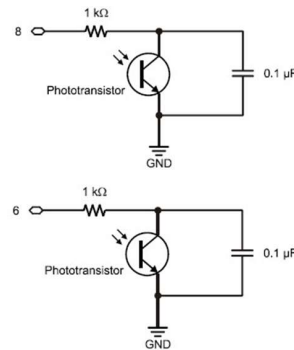
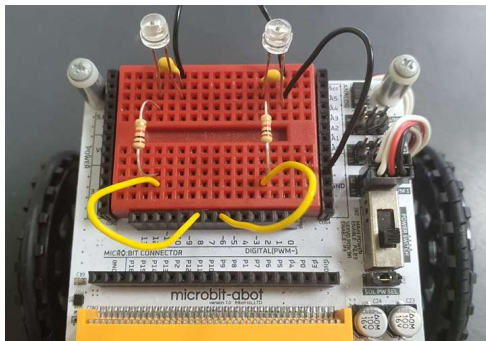


그림5.9 포토트랜지스터를 장착한 텐덤로봇

위 두 개의 변수 설정은 로봇에 추가되는 외부 빛을 비추지 않은 분위기 조명이 아래에서 실행하십시오. 이때 서보모터의 전원은 필요하지 않기 때문에 3점점 스위치는 '1'의 위치에 두면 됩니다.

주변의 빛이 어두운 환경이면 qt_left와 qt_right 값은 큰 값을 출력합니다. 대낮이면 낮은 값을 나타내고 어두운 밤이면 큰 값을 나타냅니다. 외부 빛을 비추면 qt_left와 qt_right 값은 더 낮은 값으로 출력됩니다. 따라서 현재 사용자의 light_threshold 값은 외부 빛을 비추지 않은 상태에서 출력되는 값보다 충분히 낮은 값을 설정해야 정지상태가 됩니다.

또 다른 유의사항은 두 개의 광센서 값 차이를 이용하기 때문에, qt_left와 qt_right 값이 비교적 유사한 값을 표현하는 조건으로 광센서 위치와 조건을 만드는 것이 실습에 유리합니다. 두 개 광센서 값 차이가 작을수록 매우 민감한 외부 빛 차이에 반응할 수 있습니다. 광센서 값 차이가 큰 경우 센서 회로의 1kΩ저항 크기를 변경해서 작게 만들 수 있습니다.

difference_threshold 변수의 값 크기가 작을수록 외부 빛 변화에도 민감해집니다.

두 개의 설정 값이 모두 정해졌다면, 서보모터에 전원을 공급하고 외부 불빛을 로봇의 광센서에 비춰보세요. 빛을 전방 멀리서 비추면 로봇이 전진하기 시작하고 로봇의 왼쪽 또는 오른쪽으로 불빛을 이동하면 로봇이 불빛 방향으로 회전하면서 불빛을 따라오기 시작합니다.

예제5.5 “포토트랜지스터 광센서로 자율주행하는 텐덤로봇”

```
# ch5_05_phototransistor_navigation_drive.py

from microbit_abot import *

abot = bot(13, 12)
left_sensor = bot(8)
right_sensor = bot(6)

abot.servo_attachpins()
```

```

# Define movement functions
def forward():
    abot.servo_speed(100, -100) # Move forward
    display.show("F")
    sleep(20)

def stop():
    abot.servo_speed(0, 0) # Stop
    display.show(Image.SQUARE)
    sleep(100)

def turnright():
    abot.servo_speed(100, 0) # Turn right
    display.show("R")
    sleep(100)

def turnleft():
    abot.servo_speed(0, -100) # Turn left
    display.show("L")
    sleep(100)

# Main loop
while True:
    left_sensor.write_digital(1)
    qt_left = left_sensor.rc_time(1) # Read left sensor
    right_sensor.write_digital(1)
    qt_right = right_sensor.rc_time(1) # Read right sensor
    if qt_left is None:
        qt_left = 0
    if qt_right is None:
        qt_right = 0

    # Set a threshold to determine if light is "low"
    light_threshold = 1000 # Adjust this value based on sensor characteristics
    difference_threshold = 200 # Adjust this value to determine how much
    difference causes turning

    # If both sensor signals are above the light threshold, stop
    if qt_left > light_threshold and qt_right > light_threshold:
        stop()
    else:
        # Calculate the difference between left and right sensor values
        light_difference = qt_left - qt_right

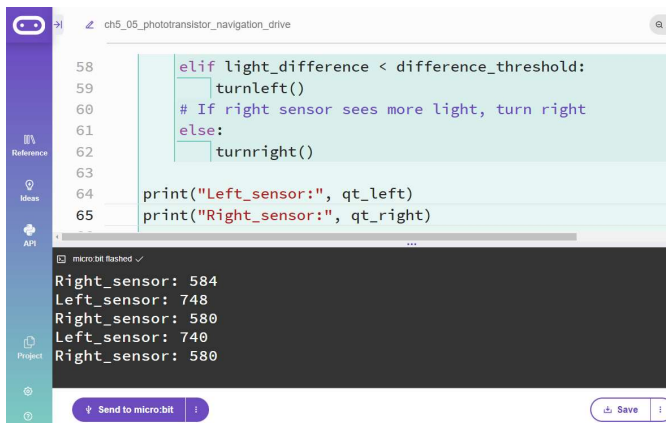
        # If the light difference is small, move forward
        if abs(light_difference) < difference_threshold:
            forward()
        # If left sensor sees more light, turn left
        elif light_difference < difference_threshold:
            turnleft()
        # If right sensor sees more light, turn right
        else:
            turnright()

    print("Left_sensor:", qt_left)
    print("Right_sensor:", qt_right)

    sleep(50) # Small delay between iterations

```

외부의 추가 빛을 비추지 않는 조건에서 아래 두 개의 광센서 출력 값이 예제5.5의 light_threshold 값 1000 이하 조건을 만족합니다. 사용자의 코드 사용조건에서 1000 보다 큰 값을 나타낸다면, light_threshold 값을 더 큰 값으로 변경해야 합니다.



```
58 elif light_difference < difference_threshold:
59     turnleft()
60     # If right sensor sees more light, turn right
61 else:
62     turnright()
63
64 print("Left_sensor:", qt_left)
65 print("Right_sensor:", qt_right)
```

micro:bit flashed ✓

Right_sensor: 584
Left_sensor: 748
Right_sensor: 580
Left_sensor: 740
Right_sensor: 580

Send to micro:bit Save

그림5.10 분위기 조명조건에서 qt_left 와 qt_right의 시리얼 프린터 화면

두 개의 광센서 신호차이에 비례 반응하는 로봇동작

외부 빛의 세기를 정확하게 측정하는 방법보다 외부 빛의 세기 차이 변화를 감지하는 방법으로 로봇이 민감하게 동작할 수 있습니다. 로봇 제어방법은 외부 빛의 세기를 단순히 구분하지 않고, 두 개의 광센서로 입력되는 외부 빛의 세기 기울기를 미세하게 로봇 동작에 반영하는 방법입니다.

코드에 적용하는 방법은 로봇 전면 왼 쪽과 오른 쪽에 설치된 광센서의 세기를 상대적으로 비교하는 방식입니다.

'qt_left / (qt_left + qt_right)' 수학적식은 qt_left 빛의 세기가 0~1 사이의 정규화 값을 표현합니다. qt_right 빛의 세기 역시 동일한 논리가 적용됩니다. 이 방식은 qt_left 와 qt_right 가 상대적 빛의 세기를 0~1 사이의 값으로 표현합니다. 수학적식 'qt_left / (qt_left + qt_right)' 에서 0.5를 뺀 'qt_left / (qt_left + qt_right) - 0.5' 수학적식은 qt_left 와 qt_right 가 -0.5 ~ +0.5 사이의 상대적 빛의 세기로 표현됩니다.

여기에 추가된 로봇동작 논리는 외부에서 추가로 불빛을 비추지 않는 분위기 조명밝기에서는 움직이지 않도록 설정하고, 분위기 빛의 양보다 추가 불빛이 광센서에 비춰질 때만 외부 빛의 변화에 반응하도록 하는 것입니다. 이를 위해 기본 조도를 로봇동작 시작시점에 스스로 측정하고 적용하는 논리를 사용합니다. 아래 코드논리는 예제5.5의 light_threshold, difference_threshold 값을 직접 수정하는 방법을 자동화한 것입니다.

아래 코드를 사용하는 로봇은 분위기 조도에는 반응하지 않다가 로봇에게 추가의 불빛이 비춰지면 두 개의 광센서 밝기변화에 따라 왼 쪽 또는 오른 쪽으로 민감하게 반응하기 시작합니다.

코드가 처음 실행될 때 분위기 조도 ambient_light_threshold 와 빛의 차이 임계값 reaction_threshold 값이 자동으로 설정됩니다. 이 값들을 다시 갱신하려면 마이크로비트의 버튼A를 누르면 됩니다.

참고로 ‘컴퓨터에서 로봇으로 코드를 플래싱하는 장소의 분위기 조명’과 ‘실제로 로봇을 움직이려는 장소의 분위기 조명’은 미세하게 다를 수 있습니다. 이런 조명변화 조건에서는 버튼A를 누르는 동작이 분위기 조도 기준 값을 재설정하는 수단으로 사용됩니다.

예제5.6 “외부 빛을 따라서 자율주행하는 텐덤로봇”

```
# ch5_06_phototransistor_smart_light_following_drive.py

from microbit_abot import *
from microbit import button_a

abot = bot(13, 12)
left_sensor = bot(8)
right_sensor = bot(6)

abot.servo_attachpins()

# Measure ambient light to use as a baseline
def measure_ambient_light():
    left_sensor.write_digital(1)
    qt_left = left_sensor.rc_time(1)
    right_sensor.write_digital(1)
    qt_right = right_sensor.rc_time(1)

    if qt_left is None or qt_right is None:
        qt_left, qt_right = 0, 0
    return (qt_left + qt_right) / 2 # Use the average of left and right
sensors as the baseline

# Measure the initial ambient light
ambient_light_threshold = measure_ambient_light()

# Set reaction threshold to respond when there's a 10% difference in light
reaction_threshold = ambient_light_threshold * 0.1

while True:
    # Update ambient_light_threshold if Button A is pressed
    if button_a.is_pressed():
        ambient_light_threshold = measure_ambient_light()
        print("New Ambient light threshold:", ambient_light_threshold)
        reaction_threshold = ambient_light_threshold * 0.1 # Recalculate the
new threshold

    # Measure the light levels from left and right sensors
    left_sensor.write_digital(1)
    qt_left = left_sensor.rc_time(1)
    right_sensor.write_digital(1)
    qt_right = right_sensor.rc_time(1)

    if qt_left is None or qt_right is None:
        qt_left, qt_right = 0, 0

    light_diff_norm = int((qt_left * 200 / (qt_left + qt_right + 1))) - 100
    current_light_level = int((qt_left + qt_right) / 2)

    if current_light_level < ambient_light_threshold - reaction_threshold:
        if light_diff_norm > 0: # Turn right
            left_speed = 100
            right_speed = -100 + light_diff_norm * 2
            display.show("R")
        else: # Turn left
            left_speed = 100 + light_diff_norm * 2
            right_speed = -100
            display.show("L")
        abot.servo_speed(left_speed, right_speed)
    else:
        abot.servo_speed(0, 0)
        display.show(Image.SQUARE)
```

```
sleep(200) # Short delay between iterations
```

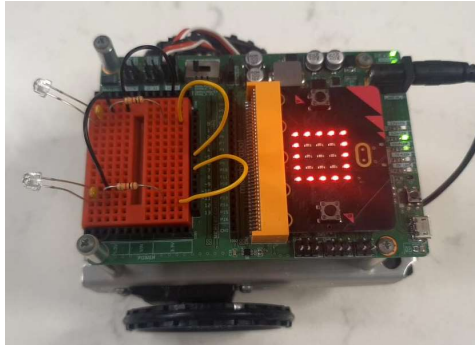


그림5.11 마이크로비트 버튼A를 눌러서 분위기 조명 기준을 설정한 장면

(3) 적외선 송수신센서 자율주행

코딩로봇이 적외선 센서를 사용하는 목적은 장애물감지, 거리측정, 라인트레이싱, 자동정지 및 회피, 그리고 적외선 통신 수단으로 상호작용할 수 있습니다. 적외선 통신은 리모콘 동작에 많이 사용됩니다.

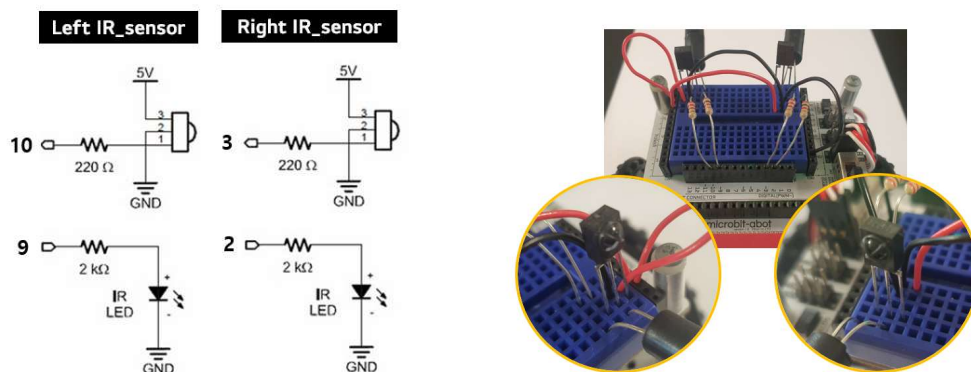


그림5.12 텐덤로봇의 적외선 센서 회로도와 실제화면

본 실습은 로봇의 왼 쪽과 오른 쪽 전면에 설치한 적외선 센서가 전방물체 감지를 위해 `ir_detcet` 함수를 사용하는 것입니다. 더듬이 센서가 물체와 직접 접촉하는 감지 방식이었다면, 적외선 센서는 물체와 직접 접촉하지 않고 가까이 접근한 상태에서 물체를 감지하는 비접촉 방식입니다. 따라서 더듬이 센서를 사용한 로봇 동작 방식과 동일한 방식으로 코드를 구현할 수 있습니다.

또한 로봇의 아래쪽 바닥으로 적외선 센서를 향하게 설치하면, 라인트레이싱 동작을 만들 수도 있습니다. 이 경우 적외선 불빛이 흰색은 반사율이 높고 검은색은 흡수율이 높은 빛의 특성을 이용하는 것입니다.

그림5.12는 텐덤로봇 전면의 왼 쪽과 오른 쪽에 적외선 센서를 설치할 때의 회로도와 센서 장착 화면입니다. 적외선 LED회로의 2K Ω 저항 크기를 변경하는 방법으로 장애물과의 감지거리를 조절할 수 있습니다. IR LED 저항 크기가 2K Ω 보다 작을수록 IR LED를 흐르는 전류의 세기가 커지고, 동시에 LED 불빛의 세기도 커지기 때문에 먼 거리를 탐지합니다. 반대로 IR LED 저항 크기가 2K Ω 보다 클수록 짧은 거리를 탐지합니다. 송신기 저항의 크기를 다양하게 조절해서 탐지거리 영역을 선택하세요.

두 개의 적외선 센서 함수를 적용해서 지능적으로 주행하는 로봇 예제를 아래에서 소개합니다. 아래 코드는 두 개의 센서가 모두 장애물을 감지하는지, 또는 한 개 센서만 장애물을 감지하는지에 따라 각각 다르게 반응합니다. 앞선 예제5.4의 모서리 탈출 코드를 상기해보세요. 아래 코드 역시 모서리 탈출 개념을 적용해서 다시 시도해보기 바랍니다.

예제5.7 “적외선 송수신 센서로 자율주행하는 텐덤로봇”

```
# ch5_07_irDetect_navigation_drive.py

from microbit_abot import *

abot = bot(13, 12)
left_sensor = bot(9, 10)
right_sensor = bot(2, 3)

abot.servo_attachpins()

def forward():
    abot.servo_speed(100, -100)
    display.show("F")
    sleep(20)

def backward():
    abot.servo_speed(-100, 100)
    display.show("B")
    sleep(1000)
    abot.servo_speed(100, 100)
    sleep(1200)

def turnright():
    abot.servo_speed(-100, 100)
    display.show("R")
    sleep(1000)
    abot.servo_speed(100, 0)
    sleep(800)

def turnleft():
    abot.servo_speed(-100, 100)
    display.show("L")
    sleep(1000)
    abot.servo_speed(0, -100)
    sleep(800)

while True:
    irL = left_sensor.ir_detect(38000)
    irR = right_sensor.ir_detect(38000)
    print("left_sensor:", irL)
    print("right_sensor:", irR)

    if irL == 0 and irR == 0:
        backward()
```

```

elif irL == 1 and irR == 0:
    turnleft()
elif irL == 0 and irR == 1:
    turnright()
else:
    forward()

```

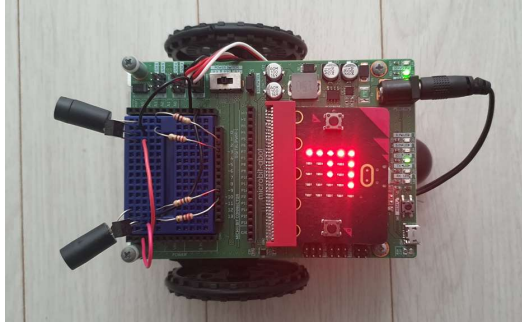


그림5.13 적외선 감지 `ir_detect` 함수로 주행하는 로봇

당신은 적외선 센서가 단순히 장애물 유/무만을 감지하는 신호처리 `ir_detect` 방식 이외에 장애물과의 거리를 측정하는 `ir_distance` 사용법을 4.5절에서 배운 것을 기억합니다.

아래 코드는 장애물과의 거리를 탐지하는 `ir_distance` 함수를 사용해서 로봇이 지능적으로 주행하는 방법입니다. 거리측정 `ir_distance` 함수는 예제4.19에서 설명하였으며, 그림4.32로 기본 개념을 소개했습니다. `ir_distance` 함수를 사용하는 예제5.8은 `ir_detect` 함수를 사용하는 예제5.7의 로봇동작보다 부드럽게 방향 전환합니다. 그 이유는 두 개의 적외선센서로 측정한 거리 차이를 로봇의 바퀴 속도에 반영하기 때문입니다.

예제5.8 “적외선 거리측정으로 자율주행하는 텐덤로봇”

```

# ch5_08_irDistance_navigation_drive.py

from microbit_abot import *
from microbit import display, sleep

abot = bot(13, 12)
left_sensor = bot(9, 10)
right_sensor = bot(2, 3)

# Connect servo motor pins
abot.servo_attachpins()

# Set maximum speed and distance
max_speed = 100
base_speed = 80 # Base forward speed
min_distance = 2 # Safe minimum distance (based on sensor value)
adjust_factor = 20 # Coefficient for adjusting rotation sensitivity
max_distance = 9 # Maximum distance value of sensors

while True:
    # Measure distance from left and right infrared sensors
    left_distance = left_sensor.ir_distance()
    right_distance = right_sensor.ir_distance()

```

```

# Handle None values
if left_distance is None:
    print("Left sensor error")
    left_distance = max_distance # Or set to 0
if right_distance is None:
    print("Right sensor error")
    right_distance = max_distance # Or set to 0

# Check if an obstacle is too close ahead
if left_distance < min_distance and right_distance < min_distance:
    # Reverse
    abot.servo_speed(-max_speed, max_speed)
    display.show("B")
    sleep(1000)
    display.clear()
    continue # Move to the next loop

# Calculate distance difference to determine rotation direction
distance_diff = right_distance - left_distance

# Calculate speed adjustment based on distance difference
speed_adjust = int(adjust_factor * distance_diff)

# Calculate speed for left and right wheels
left_speed = base_speed + speed_adjust
right_speed = base_speed - speed_adjust

# Limit speed to maximum value
left_speed = max(-max_speed, min(max_speed, left_speed))
right_speed = max(-max_speed, min(max_speed, right_speed))

# Adjust speed for servo motors
left_wheel_speed = left_speed
right_wheel_speed = -right_speed # Reverse the sign for the right wheel

# Set servo motor speed
abot.servo_speed(left_wheel_speed, right_wheel_speed)

# Print values for debugging
print("Left distance:", left_distance, "Right distance:", right_distance)
print("Left speed:", left_speed, "Right speed:", right_speed)

# Use short delay
sleep(50)

```

예제5.8의 동작원리는 서모보터 초기화를 먼저 실행하고, `ir_distance` 함수를 사용해서 거리를 측정합니다. 만약 양쪽 센서 값이 모두 `min_distance`보다 작으면 장애물이 바로 앞에 있다고 판단하고 후진합니다. `min_distance`는 사용자가 설정하는 안전거리에 해당합니다.

감시거리(센서 측정 값이 2~9 사이 또는 `min_distance`와 `max_distance` 사이)에서 로봇바퀴의 속도조절은 좌우 센서로 부터 얻은 거리차이를 `adjust_factor` 보정 계수와 곱한 다음, 좌우 바퀴 속도에 각각 반영하도록 계산합니다. `adjust_factor` 크기는 로봇의 회전 민감도에 맞도록 조절할 수 있습니다. 이외에도 로봇의 회전 민감도를 고려할 때 `base_speed` 값을 조정하는 것도 가능합니다.

각 바퀴의 서보모터 제어는 계산된 속도 값으로 `servo_speed` 함수에 적용합니다. 이때 우

측 바퀴는 부호를 반전하여 로봇이 직진하는 방향으로 회전하도록 합니다.

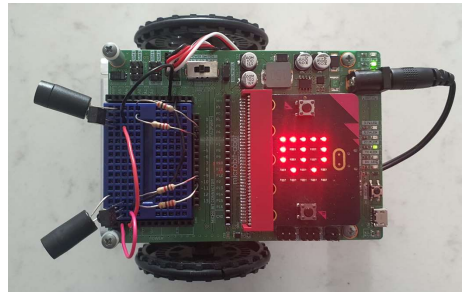
```

ch5_08_irDistance_navigation_drive

6 abot = Bot(13, 12)
7 left_sensor = bot(9, 10)
8 right_sensor = bot(2, 3)
9
10 # Connect servo motor pins
11 abot.servo_attachpins()
12
13 # Set maximum speed and distance
14 max_speed = 100

micro:bit ready to flash
Left speed: 100 Right speed: -20
Left distance: 4 Right distance: 9
Left speed: 100 Right speed: -20
Left distance: 4 Right distance: 9
Left speed: 100 Right speed: -20

```



(a)우회전 조건 출력화면

(b)후진 조건에서의 로봇표시

그림5.14 ir_distance 함수로 자율주행하는 로봇

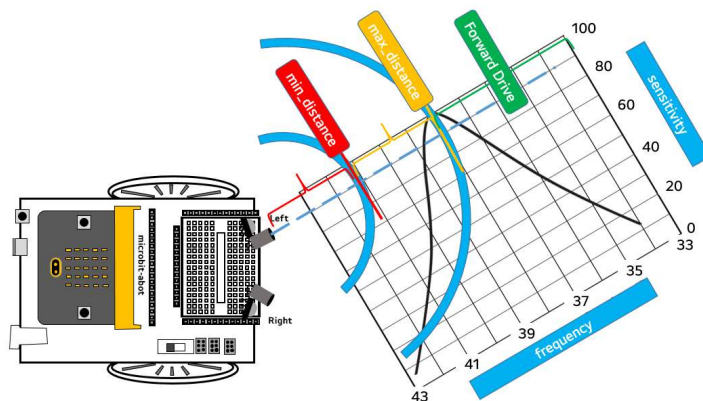


그림5.15 ir_distance 함수 거리측정에 기초한 로봇 속도제어 개념도

예제5.8의 실습에서 두 개 센서의 거리 측정 감도가 매우 다른 경우 의도하지 않은 동작을 만들 수 있으므로, 적외선 센서의 저항 값을 변경하는 방법으로 최대한 비슷하게 맞추는 작업이 필요할 수 있습니다. 또한, 로봇이 직진주행 조건에서 조금씩 왼 쪽 또는 오른 쪽으로 치우치는 경우 이후에 설명하는 5.5절의 '직선주행 보정하기'를 실습하고, 보정된 주행함수를 적용하면 로봇의 주행동작을 어느 정도 개선할 수 있습니다.

(4) 초음파센서 PID제어

초음파센서는 초음파를 전방으로 송수신하면서 전방의 물체에서 반사되는 신호를 감지해서 물체와의 거리를 측정하는 전자부품입니다.

먼저 사용할 초음파센서는 패럴랙스사의 ping 센서입니다. 물론 3핀이 아닌 4핀의 초음

파센서를 사용해서 동일한 효과를 만들 수 있습니다. 5V 전원과 접지(GND)를 보드에 연결하고, 트리거핀과 에코핀을 함께 사용하는 신호핀(SIG)을 디지털핀에 연결합니다.



그림5.16 패럴렉스 3핀 초음파센서

로봇에 장착된 초음파센서는 전방 물체와의 거리를 측정하고, 전방물체와의 거리를 일정하게 유지하는데 필요한 신호를 제공합니다. 초음파 센서를 이용하는 알려진 방식은 로봇에 고정해서 사용하는 방법과 표준형 서보모터와 결합하여 초음파센서가 0~180도 사이로 스캔하는 방법을 선택할 수 있습니다.

아래 실습은 전방물체와의 거리를 일정하게 유지하기 위해서 초음파센서를 로봇 전면 고정식으로 장착합니다. 센서 신호처리방법은 PID제어 방법에서 비례이득(P) 기반의 거리를 유지하는 수단만 적용합니다.

PID 제어에서 비례 제어는 가장 기본적인 형태로, 시스템의 응답 속도를 빠르게 하는 데 유용합니다. 비례 이득은 현재 오차에 비례하여 제어 신호를 생성하므로, 로봇이 물체와의 거리 차이에 따라 즉각적으로 반응합니다. 그리고 비례 제어는 구현이 간단하므로, 초보자에게 적합한 제어 실습이 됩니다. 추후에 적분이득과 미분이득을 더 추가해서 더 정밀한 거리 제어를 시도해보기 바랍니다.

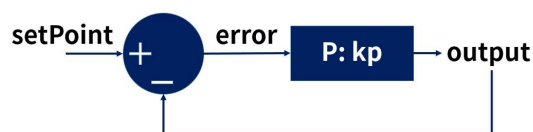


그림5.17 초음파센서로 일정 거리를 유지하는 PID 제어

아래에서 소개할 예제는 마이크로비트 P0핀에서 직접 센서신호를 입력받고 처리하는 방법과 아두이노 디지털핀 센서신호를 마이크로비트가 I2C로 수신받고 처리하는 방법 두가지로 구분할 수 있습니다. 두가지 방법을 순서대로 소개합니다.

마이크로비트 P0핀으로 초음파센서 PID제어하기

이 예제는 앞에서 소개했던 예제4.21 "마이크로비트를 사용한 초음파 거리 측정"코드를 그대로 사용하면서, 측정된 거리값을 사용해서 로봇동작 PID제어를 적용한 것입니다.

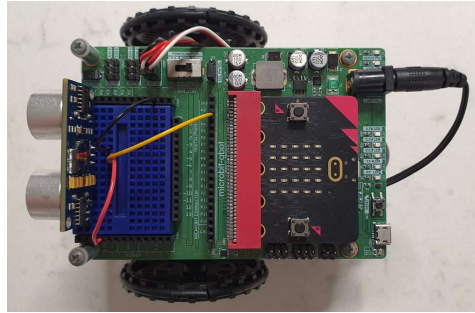
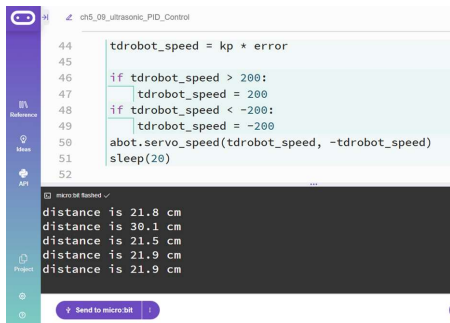


그림5.18 마이크로비트 P0핀에 연결된 초음파센서 PID제어

예제5.9 “초음파센서로 물체와 로봇간의 거리 PID 제어하기”

```

# ch5_09_ultrasonic_PID_Control.py

from microbit_abot import *
import utime

abot = bot(13, 12)

setPoint = 20
kp = -4
signal_pin = pin0
abot.servo_attachpins()

def get_distance():
    signal_pin.write_digital(0)
    utime.sleep_us(2)
    signal_pin.write_digital(1)
    utime.sleep_us(5)
    signal_pin.write_digital(0)

    signal_pin.set_pull(signal_pin.NO_PULL)
    signal_off = 0
    signal_on = 0

    # Wait for echo signal reception
    while signal_pin.read_digital() == 0:
        signal_off = utime.ticks_us()

    while signal_pin.read_digital() == 1:
        signal_on = utime.ticks_us()

    # Convert time difference to distance (speed of sound is 343m/s)
    time_passed = signal_on - signal_off

    # Convert time difference to cm
    distance_cm = (time_passed * 34300) / 2 / 1000000
    return distance_cm

while True:
    distance = get_distance()
    # display.scroll("{:.2f} cm".format(distance))
    print("distance is {:.1f} cm".format(distance))

    error = setPoint - distance
    tdrobot_speed = kp * error

    if tdrobot_speed > 200:
        tdrobot_speed = 200
    if tdrobot_speed < -200:
        tdrobot_speed = -200
    abot.servo_speed(tdrobot_speed, -tdrobot_speed)

```

```
sleep(20)
```

I2C 아두이노 디지털핀에 연결된 초음파센서 PID제어하기

아두이노 디지털 핀으로 측정한 초음파센서 거리 결과를 PID제어에 사용하는 방법입니다. 마이크로비트와는 다르게 아두이노는 사용자가 사용할 수 있는 I/O핀 숫자가 많기 때문에 아두이노 핀을 사용한 PID제어가 유용하게 사용됩니다. PID제어 논리는 예제5.9와 동일합니다.

로봇이 일정하게 유지하려는 거리와 반응속도는 코드의 전역변수 `setPoint`와 `kp`를 사용자 목적에 부합하도록 설정하면 됩니다. 초음파센서 동작에 대한 선행코드 예제4.22 “아두이노 핀에 연결된 3핀 ping센서로 거리 측정하기”를 참조하세요. 센서회로 연결은 초음파센서 신호핀(SIG)을 아두이노 디지털핀 9번에 연결하고, 5V 전원과 접지를 연결합니다.

예제5.10 “I2C 3핀 초음파센서로 물체와 로봇간의 거리 PID 제어하기”

```
# ch5_10_3pin_ultrasonic_PID_distance.py

from microbit_abot import *
from ultrasonic import *

abot = bot(13, 12)
ultrasonic_sensor = Ultrasonic(9)

setPoint = 20
kp = -10

abot.servo_attachpins()

while True:
    # Convert to float because the Ultrasonic function can return a string
    distance = ultrasonic_sensor.distance('cm')

    if distance is not None:
        # Convert distance to float
        distance = float(distance)
        print("distance is {:.1f} cm".format(distance))

        error = setPoint - distance
        tdrobot_speed = kp * error

        # Limit the range of speed values
        if tdrobot_speed > 200:
            tdrobot_speed = 200
        if tdrobot_speed < -200:
            tdrobot_speed = -200

        abot.servo_speed(int(tdrobot_speed), int(-tdrobot_speed))
    else:
        print("Invalid distance value: {}".format(distance))

    sleep(30)
```

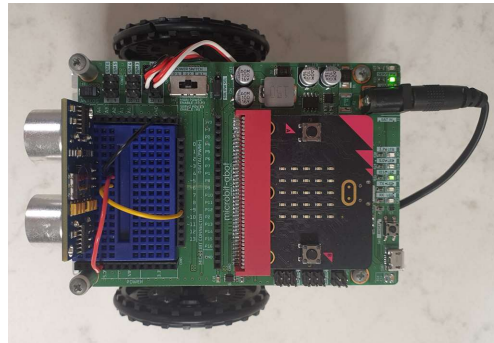
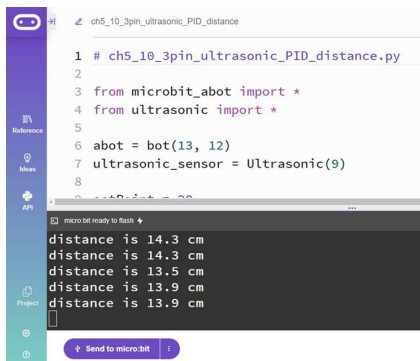


그림5.19 3핀 초음파센서로 전방물체와의 거리 PID 제어하기

4핀 초음파센서를 사용해서 PID제어를 실습하려는 경우 예제4.23을 참조할 수 있습니다. 예제4.23의 하드웨어 핀연결과 거리측정에 사용하는 함수 `Ultrasonic(9,10).distance('cm')` 로 적용하면 동일한 제어를 실습할 수 있습니다. HR04센서의 경우 ping센서보다 센서 신호를 안정적으로 처리하지 못할 수 있다는 점을 참고하기 바랍니다.

초음파센서 핀연결은 전원과 접지핀을 연결하고, 트리거핀은 디지털 핀 9번 에코핀은 10번 핀에 각각 연결하면 됩니다.

예제5.11 “I2C 4핀 초음파센서로 물체와 로봇간의 거리 PID 제어하기”

```
# ch5_11_4pin_ultrasonic_PID_distance.py

from microbit_abot import *
from ultrasonic import *

abot = bot(13, 12)
ultrasonic_sensor = Ultrasonic(9, 10)

setPoint = 20
kp = -10
abot.servo_attachpins()

while True:
    # Convert to float because the Ultrasonic function can return a string
    distance = ultrasonic_sensor.distance('cm')

    if distance is not None:
        # Convert distance to float
        distance = float(distance)
        print("distance is {:.1f} cm".format(distance))

        error = setPoint - distance
        tdrobot_speed = kp * error

        # Limit the range of speed values
        if tdrobot_speed > 200:
            tdrobot_speed = 200
        if tdrobot_speed < -200:
            tdrobot_speed = -200
        abot.servo_speed(int(tdrobot_speed), int(-tdrobot_speed))
    else:
        print("Invalid distance value: {}".format(distance))

    sleep(30)
```

(Notice)-----

PID제어란 무엇인가요?

프로세스 제어 시스템에서 가장 널리 사용되는 제어 알고리즘 중 하나로, 비례(Proportional), 적분(Integral), 미분(Derivative) 제어의 세 가지 구성 요소를 결합하여 시스템의 응답을 조절하는 제어 원리입니다.

$$u(t) = K_p \cdot e(t) + K_i \cdot \int_0^t e(\tau) d\tau + K_d \cdot \frac{d}{dt}e(t)$$

- 비례(Proportional) 제어 (P 제어) : 시스템의 현재 오차(Error)에 비례하는 제어 신호를 생성합니다. 오차는 설정값(Setpoint)과 실제값(Process Variable) 간의 차이입니다.
- 적분(Integral) 제어 (I 제어) : 시간에 따른 오차의 누적합에 비례하는 제어 신호를 생성하여 정상 상태 오차를 제거합니다.
- 미분(Derivative) 제어 (D 제어) : 오차의 변화율에 비례하는 제어 신호를 생성하여 시스템의 응답 속도를 개선하고 오버슈트를 감소시킵니다.

5.4 텐덤로봇 초음파센서 레이더 스캔하기

초음파센서로 물체와의 거리를 측정하고 스캔하는 방법은 4.5절 “센서활용하기: 초음파센서” 편에서 소개한 바 있습니다. 초음파센서를 표준형 서보모터에 장착해서 전방 0~180도 사이를 스캔하고, 컴퓨터의 COM포트로 데이터를 전송합니다. USB시리얼 통신으로 초음파센서 스캔 데이터를 송수신하는 방법을 소개합니다.

아래 사진은 텐덤로봇에 패럴렉스 사의 초음파센서 키트를 설치한 장면입니다.

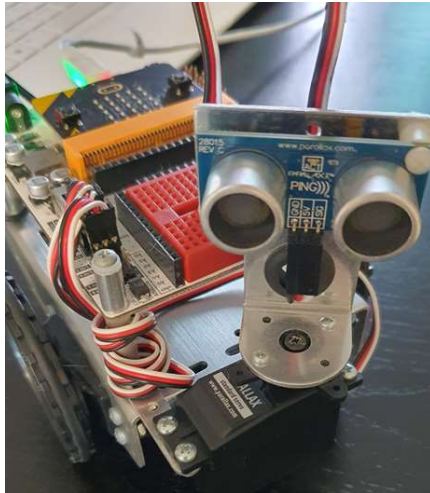


그림5.20 초음파 센서키트를 장착한 텐덤로봇

초음파센서가 0~180도 각도 사이를 스캔하려면, 표준형 서보모터에 초음파센서를 장착하면 됩니다. 표준형 서보모터의 연결선은 서보모터 연결을 위해 준비된 아두이노의 6개 디지털핀들 중에서 바퀴용으로 이미 연결한 핀을 제외한 나머지 핀을 사용하면 됩니다.

서보모터 하드웨어 연결선은 접지(검은색)방향을 잘 살펴서 연결하세요. 초음파센서는 아두이노 디지털핀 9번에 연결하고, 표준형 서보모터는 디지털핀 10번의 서보 핀에 연결합니다. 표준형 서보모터의 동작을 위해서 마이크로비트가 정의한 메서드는 `servo_angle`입니다. 이 메서드는 아두이노 라이브러리와 함께 연결되어 동작하는데, `servo_angle` 실행과 동시에 아두이노에서 `servo.attach` 메서드가 함께 실행되기 때문에 추가 코드실행이 필요하지 않습니다.

아래 예제5.12은 텐덤로봇에 장착된 마이크로비트에서 플래싱할 코드입니다. 아래 코드를 플래싱하면 USB시리얼 통신으로 텐덤로봇에 장착된 초음파센서 거리측정 데이터가 컴퓨터로 전송됩니다. 아래 코드는 3핀 초음파 센서를 위한 코드입니다.

예제5.12 “초음파센서 거리측정 데이터 USB 시리얼통신으로 송신하기”

```
# ch5_12_ultrasonic_distance_usb_serial_transfer.py
```

```

from microbit import *
from microbit_abot import *
from ultrasonic import *

ultrasonic_sensor = Ultrasonic(9)
abot = bot(10) # Create an instance of the bot class

while True:
    for angle in range(0, 181):
        abot.servo_angle(angle)
        sleep(15)
        distance = ultrasonic_sensor.distance('cm') or 0

        # Data format: "angle,distance."
        data = "%d,%d." % (angle, int(distance))
        print(data, end="")

    for angle in range(180, -1, -1):
        abot.servo_angle(angle)
        sleep(15)
        distance = ultrasonic_sensor.distance('cm') or 0

        # Data format: "angle,distance."
        data = "%d,%d." % (angle, int(distance))
        print(data, end="")

```

ultrasonic_sensor = Ultrasonic(9) 코드로 초음파 센서를 제어하고 거리 데이터를 측정할 수 있습니다. Ultrasonic 클래스의 인스턴스로 핀 번호 9를 전달하여, 초음파 센서가 연결된 핀을 지정합니다.

for angle in range(0, 181) 코드는 angle 변수를 0부터 180까지 1씩 증가시키며 반복하고, range(0, 181)은 0부터 180까지의 숫자를 생성합니다. 서보 모터를 0도부터 180도까지 회전시키기 위해 사용합니다.

servo_angle(angle) 메서드는 서보 모터를 angle 변수에 해당하는 각도로 회전시킵니다. servo_angle 함수는 microbit_abot 모듈에 정의되어 있고, 서보 모터가 지정된 각도로 이동합니다.

distance = ultrasonic_sensor.distance('cm') 코드는 초음파 센서를 사용하여 거리를 측정합니다. distance('cm') 함수는 거리를 센티미터 단위로 반환하고, 측정된 거리 값이 distance 변수에 저장됩니다.

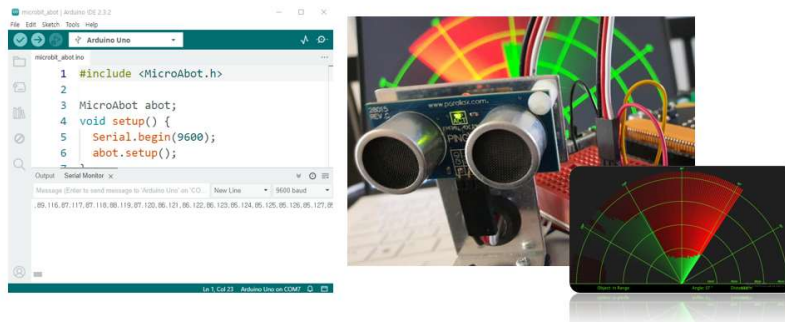
data = "%d,%d." % (angle, int(distance)) 코드에서 % 연산자는 형식 문자열에 실제 값을 넣어주는 역할을 하고, 측정된 각도와 거리를 문자열로 포맷팅합니다. 예를 들어, 각도가 90도이고 거리가 50cm라면 "90,50."과 같은 문자열이 생성되고, 데이터는 콤마로 구분하여 각도와 거리를 포함합니다.

print(data, end="") 코드는 개행 문자 없이 데이터를 전송하기 위한 출력방식입니다.

for angle in range(180, -1, -1): 코드는 angle 변수를 180부터 0까지 1씩 감소시키며 반복합니다. range(180, -1, -1)은 180부터 0까지의 숫자를 생성하고, 서보 모터를 180도부터 0도까지 역으로 회전시키기 위해 사용합니다.

컴퓨터에서 수신하는 방법은 시리얼 모니터 프로그램(예: PuTTY, Tera Term, Arduino

IDE의 시리얼 모니터 등)을 사용하여 마이크로비트가 연결된 COM 포트에 접속하면 됩니다.



(a)

(b)

그림5.21 초음파센서 시리얼출력 (a) 아두이노 (b) 프로세싱 레이더 화면

그림5.21의 출력화면을 만들려고 하면, 아래 내용을 참조하세요. 상세한 내용은 공개된 [“C 부터 C++ 언어까지 아두이노로봇 코딩실습”](#) 자료의 7.2절 내용을 참고해서 프로세싱 도구로 레이더 화면을 표시할 수 있습니다. 프로세싱으로 레이더 화면을 출력할 때 프로세싱의 컴포트 수정과 보드레이트를 115200으로 수정하기 바랍니다. (※ 마이크로비트 시리얼통신 기본 보드레이트는 115200 입니다.)

지금부터 6.3절에서 소개할 pySerial 라이브러리를 사용해서 데이터를 수신하는 방법을 소개합니다. 라이브러리 설치방법은 앞 절의 설명을 참조하세요! 마이크로비트가 연결된 시리얼 포트의 확인은 장치관리자를 살펴볼 수 있지만, 아래 코드를 실행해서 찾을 수도 있습니다. 또는 아두이노 IDE프로그램의 도구->포트에서 컴포트를 찾아볼 수도 있습니다.

```
import serial.tools.list_ports

ports = serial.tools.list_ports.comports()
for port in ports:
    print(f"디바이스: {port.device}, 설명: {port.description}")
```

컴퓨터로 수신되는 시리얼 데이터는 간단히 아두이노 IDE프로그램을 사용해서 데이터를 확인할 수 있습니다. 아래 예제6.19는 프로그램을 사용해서 수신되는 데이터를 표시하거나, 파일로 저장할 수 있음을 나타냅니다. 시리얼 통신을 통해 각도와 거리를 수신하는 프로그램으로 받은 데이터를 쉼표로 구분하여 각도와 거리로 나눈 후, 이를 정수 및 실수로 변환하여 출력합니다. 데이터 형식이 잘못된 경우 예외 처리를 통해 오류를 표시하며, 사용자가 프로그램을 종료할 때 시리얼 포트를 닫고 안전하게 종료됩니다.

예제5.13 “컴퓨터로 전송된 USB시리얼 컴포트 센서데이터 수신하기”

```
# ch5_13_receive_sensor_data_via_usb_serial.py

import serial
import serial.tools.list_ports
```

```

# Print available ports
ports = serial.tools.list_ports.comports()
print("Available Ports:")
for port in ports:
    print(f"Device: {port.device}, Description: {port.description}")

serial_port = 'COM7' # Change to the actual port
baud_rate = 115200    # Set baud rate to 115200

try:
    ser = serial.Serial(serial_port, baud_rate, timeout=1)
    print(serial_port + " is connected.")
except serial.SerialException as e:
    print(f"Cannot open serial port {serial_port}: {e}")
    ser = None

if ser:
    try:
        while True:
            line = ser.readline().decode('utf-8').strip()
            if line:
                print("received data:", line)
                try:
                    angle_str, distance_str = line.split(',')
                    angle = int(angle_str)
                    distance = float(distance_str.strip('.'))
                    print(f"angle: {angle}°, distance: {distance} cm")
                except ValueError:
                    print("wrong data format")
            except KeyboardInterrupt:
                print("closing the program.")
    finally:
        ser.close()

```

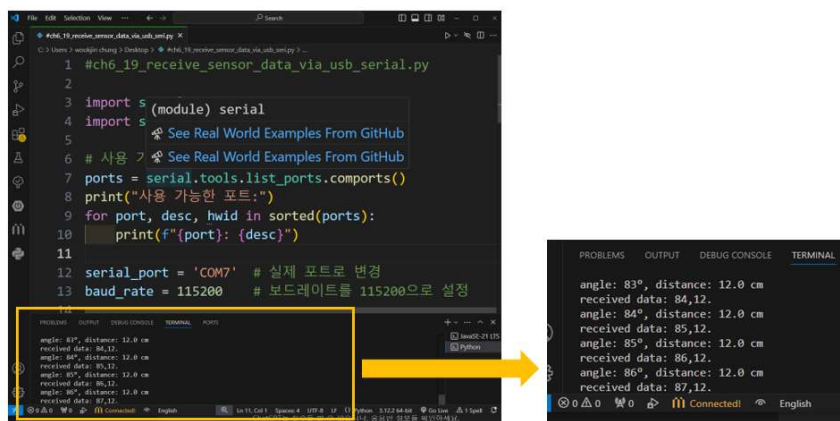


그림5.22 파이썬 비주얼스튜디오 도구로 시리얼데이터 출력하기

위 코드에서 except KeyboardInterrupt 기능은 사용자가 Ctrl + C를 눌러 프로그램을 강제로 중단하려할 때 예외를 처리합니다. 프로그램을 안전하게 종료하기 위해 이 부분에서 메시지를 출력합니다.

USB시리얼 수신 데이터를 컴퓨터에서 시각화하는 방법

컴퓨터에서 초음파센서 데이터를 2D 그래프로 표시하는 단계별 접근방법입니다.

[1] 라이브러리 설치

matplotlib 설치

numpy 설치 (3D 그래프 필요 시)

[2] 데이터의 이해 및 처리

마이크로비트로부터 수신한 데이터는 각도와 거리로 구성되어 있습니다. 이 데이터를 분리하는 작업을 ‘데이터 파싱’이라고 합니다.

각도(angle): 0도 ~ 180도

거리(distance): 센티미터(cm) 단위의 실수 값

[3] 각도와 거리 데이터를 원통좌표계로 표시

원통좌표계는 **각도(θ)**와 **반경(r)**으로 위치를 표현하는 좌표계입니다. 이는 초음파 센서나 레이더 같은 장치에서 각도와 거리를 측정할 때 사용합니다. matplotlib은 원통좌표계를 지원하며, 이를 사용하여 데이터를 시각화할 수 있습니다. subplot을 생성할 때 `projection='polar'`을 설정하면 원통좌표계를 표현합니다.

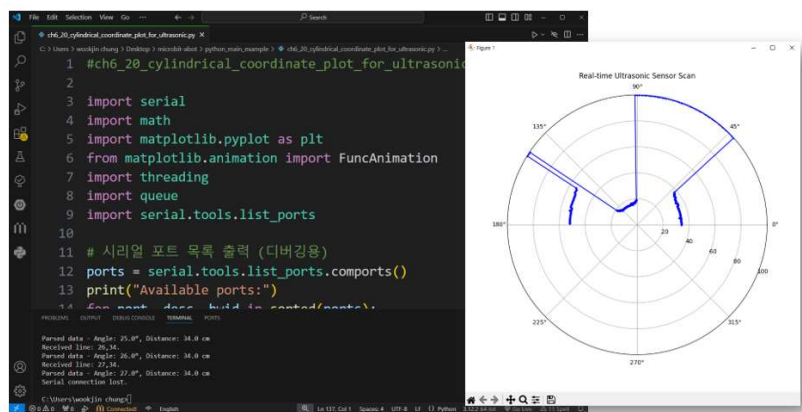


그림5.23 초음파 거리측정 데이터 원통좌표계 시각화

그림5.23은 초음파센서가 장착된 텐더로봇으로부터 전송된 시리얼 데이터를 컴퓨터에서 시각화하는 예제5.14를 실행한 결과입니다. 아래 코드를 사용해서 여러분도 직접 시도해보기 바랍니다.

예제5.14 “초음파센서 거리측정 데이터 실시간 2D 그래프로 시각화하기”

```
# ch5_14_cylindrical_coordinate_plot_for_ultrasonic_distance.py

import serial
import math
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
import threading
import queue
```

```

import serial.tools.list_ports

# Print available serial ports (for debugging)
ports = serial.tools.list_ports.comports()
print("Available ports:")
for port, desc, hwid in sorted(ports):
    print(f'{port}: {desc}')

# Set serial port
serial_port = 'COM7' # Change to your actual port
baud_rate = 115200

# Initialize data queue
data_queue = queue.Queue()

# Function to read serial data in a separate thread
def read_serial_data(ser, q):
    while True:
        try:
            # Read data until '.' character
            line_data = ser.read_until(b'.').decode('utf-8').strip()
            if line_data:
                q.put(line_data)
        except serial.SerialException:
            print("Serial connection lost.")
            break
        except UnicodeDecodeError:
            print("Received non-UTF-8 data.")
            continue

# Open serial port
try:
    ser = serial.Serial(serial_port, baud_rate, timeout=1)
    print(f'{serial_port} is connected.')
except serial.SerialException as e:
    print(f'Could not open serial port {serial_port}: {e}')
    exit()

# Start serial data reading thread
serial_thread = threading.Thread(target=read_serial_data, args=(ser, data_queue),
daemon=True)
serial_thread.start()

# Initialize lists to store data
angles = [] # Angles in radians
distances = [] # Distances in cm

# Set up the polar plot
fig, ax = plt.subplots(figsize=(8, 8), subplot_kw={'projection': 'polar'})
line, = ax.plot([], [], 'bo-', markersize=3) # Blue circles with lines

# Set plot title and labels
ax.set_title('Real-time Ultrasonic Sensor Scan', va='bottom')

# Set maximum visualization distance (adjustable by user)
max_rmax = 100 # Set to desired maximum distance in cm
ax.set_rmax(max_rmax) # Set maximum radius
ax.set_rticks([20, 40, 60, 80, 100]) # Set radial ticks at 20cm intervals
ax.set_rlabel_position(-22.5) # Adjust radial label position

ax.grid(True)

# Ensure minus signs are displayed correctly
plt.rcParams['axes.unicode_minus'] = False

# Set maximum number of data points (for memory management)
max_points = 1000

def update(frame):
    global angles, distances
    updated = False
    while not data_queue.empty():

```

```

line_data = data_queue.get()
# Print received data for debugging
print(f"Received line: {line_data}")
# Remove trailing '.' from data
line_data = line_data.strip('.')
try:
    angle_str, distance_str = line_data.split(',')
    angle_deg = float(angle_str)
    distance = float(distance_str)

    # Convert angle to radians
    angle_rad = math.radians(angle_deg)

    # Print parsed data for debugging
    print(f"Parsed data - Angle: {angle_deg}°, Distance: {distance} cm")

    # Clear data if angle is 0° or 180°
    if angle_deg == 0 or angle_deg == 180:
        angles.clear()
        distances.clear()
        print("Data cleared due to angle reset.")

    # Filter distance values: add data only if distance is less than or equal
    to max_rmax
    if distance <= max_rmax:
        angles.append(angle_rad)
        distances.append(distance)
        updated = True
    else:
        # Replace distance exceeding max_rmax with max_rmax
        angles.append(angle_rad)
        distances.append(max_rmax)
        updated = True
        print(f"Distance {distance} cm exceeds max_rmax. Replaced with
{max_rmax} cm.")

    # Remove oldest data points if exceeding max_points
    if len(angles) > max_points:
        angles.pop(0)
        distances.pop(0)

except ValueError:
    print(f"Invalid data format: {line_data}")

if updated:
    # Update the plot with new data
    line.set_data(angles, distances)

return line,

# Set up the animation
ani = FuncAnimation(fig, update, interval=100, blit=True)

try:
    plt.show()
except KeyboardInterrupt:
    print("Program terminated.")
finally:
    ser.close()

```

코드에서 사용자는 필요에 따라 그래프에서 표시할 적절한 거리를 제한하여 설정할 수 있습니다. 'max_rmax = 100' 코드에서 100은 그래프에서 표시할 최대거리를 100cm로 제한합니다. 그리고 측정거리가 100cm를 초과하는 모든 값은 최대거리만 표시합니다.

또 다른 USB시리얼 통신 방법

마이크로비트V2를 사용하는 경우, 예제7.3 “초음파센서 거리측정 데이터 USB 시리얼통신으로 송신하기”의 `print(data, end="")` 코드를 사용하지 않고, `uart.write()` 코드를 사용할 수 있습니다.

```
from microbit import uart
from microbit_abot import *
from ultrasonic import *
from time import sleep

# UART1 초기화
# tx=pin0, rx=pin1은 기본 설정이며 필요 시 변경 가능
uart.init(baudrate=115200, tx=pin0, rx=pin1)
# 초음파 센서를 핀 9에 초기화합니다.
ultrasonic_sensor = Ultrasonic(9)
abot = bot(10) # bot 클래스의 인스턴스 생성

while True:
    # 0도부터 180도까지 서보 모터를 회전합니다.
    for angle in range(0, 181):
        abot.servo_angle(angle) # 현재 각도로 서보 설정
        sleep(0.02)
        distance = ultrasonic_sensor.distance('cm') or 0

        # 각도와 거리를 문자열로 포맷팅
        data = "{};{}".format(angle, int(distance))
        uart.write(data)

    # 180도부터 0도까지 서보 모터를 역으로 회전합니다.
    for angle in range(180, -1, -1):
        abot.servo_angle(angle)
        sleep(0.02)
        distance = ultrasonic_sensor.distance('cm') or 0

        data = "{};{}".format(angle, int(distance))
        uart.write(data)
```

이 코드는 마이크로비트 UART1 시리얼통신으로 데이터를 송수신하는 방법입니다.

[1] UART1 초기화:

`uart.init(baudrate=115200, tx=pin0, rx=pin1)`을 통해 UART1을 초기화하고, UART 서비스에 사용할 핀을 설정합니다. 보드레이트는 115200으로 설정하여 Python 스크립트와 일치시킵니다. 마이크로비트 pin0, pin1을 통신용으로 할당하면, 사용자가 또 다른 목적으로 사용할 수 없습니다.

[2] `uart.write(data)` 사용:

`print(data, end="")` 대신 `uart.write(data)`를 사용하여 UART 서비스로 데이터를 전송합니다.

[3] 데이터 전송 속도 조정:

`sleep(0.02)`로 데이터 전송 속도를 20ms로 조정하였습니다. 필요에 따라 이 값을 조정하여 전송 속도를 최적화할 수 있습니다.

(Notice) -----

마이크로비트V2 UART1 용도는 무엇인가요?

마이크로비트 V2는 V1이 지원하지 않는 UART0과 UART1 두 개의 인터페이스를 지원함

니다. UART0은 마이크로비트의 기본 UART 인터페이스이고, 주로 USB를 통한 시리얼 통신에 사용하며, UART1은 마이크로비트에서 추가적인 시리얼 통신이 필요할 때 사용됩니다.

UART0을 센서 신호처리로 사용하면 USB 시리얼 통신과 센서 데이터통신이 충돌할 수 있습니다. 이를 방지하기 위해 UART1을 별도 통신 채널로 사용하는 것이 효율적입니다.

5.5 텐덤로봇 직선주행 보정하기

사용중인 텐덤로봇은 두 개의 서보모터를 바퀴에 연결해서 사용합니다. 공장 출시된 서보모터는 제조공차나 사용 환경 차이로 인해 성능차이가 발생합니다. 로봇의 좌우 서보모터 성능 차이는 속도 차이를 만들고 직진 주행하는 동안 좌우 방향으로 로봇이 치우치게 만듭니다. 이런 미세한 하드웨어 성능차이를 소프트웨어로 해결하기 위한 보정(calibration) 수단을 설명합니다.

보정은 ‘초음파센서 PID제어’처럼 closed-loop feedback 제어 방식을 사용하거나, open-loop 제어 방식을 사용할 수 있습니다. PID 제어방식으로 로봇이 직선주행하려면 엔코더나 자이로스코프 센서가 추가로 필요할 수 있습니다. 따라서 간단한 방식의 직선주행 보정을 시도하기 위해 오픈루프 제어방식을 소개합니다.

로봇이 주행할 때 좌우 방향으로 치우치는 현상은 바닥면의 미끄럼 정도, 바닥면의 기울어짐 정도 그리고 배터리 잔량 수준에 따라 꽤 다르게 나타납니다. 이런 이유로 사용자의 실습환경에 최적화된 서보모터 보정작업이 필요할 수 있습니다. 아래 소개하는 예제는 다양한 종류의 보정 방법 중에서 한 가지를 소개하는 것입니다.

(1) 텐덤로봇 직선주행 보정하는 법

아래 코드에서 사용하는 주행함수 `robot.servo_speed(int(left_speed), int(right_speed))` 에 간단한 보정 원리를 적용합니다. 주행 함수의 보정전 좌우 속도 변수를 'left_base_speed'와 'right_base_speed' 라고 할 때, 보정을 목적으로 'left_speed_offset' 'right_speed_offset' 변수를 지정하고, 사용자가 마이크로비트 버튼 입력으로 오프셋 값을 '1' 씩 조정해서 직선주행에 가깝도록 보정합니다.

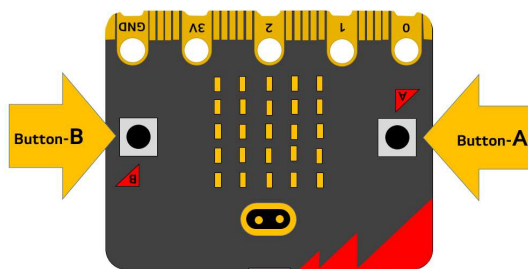


그림5.24 직선주행 오프셋 입력을 위한 버튼_A와 버튼_B

아래 코드는 보정 모드와 직선주행 모드를 포함하고 있습니다. 마이크로비트의 로고를 한 번 터치하면 보정 모드에 진입하고, 다시 한 번 더 로고를 터치하면 직선주행 모드로 복귀합니다. 전원을 리셋하지 않으면, 보정 과정을 반복 실행하는 동안 보정 효과가 누적됩니다.

로봇의 보정효과를 초기화하는 방법은 (1) 전원을 끄고 다시 켜는 방법과 (2) 버튼_A와 버튼_B를 동시에 누르는 방법입니다. 버튼_A와 버튼_B를 동시에 누르면, 두 개의 offset 변수는 '0'으로 초기화됩니다.

예를 들어 로봇이 직선 주행할 때 좌측 또는 우측으로 치우치는 주행을 한다면, 마이크로비트의 버튼_B 또는 버튼_A 중에서 하나의 버튼만을 원하는 횟수만큼 누르는 방법으로 보정합니다. 두 개의 버튼을 번갈아 누르면서 오프셋 값을 수정하는 방식은 복잡한 논리가 필요하므로 허용되지 않습니다.

상세한 직선주행 보정 방법은 다음과 같습니다.

마이크로비트에 처음 전원을 공급하면 "Ready"가 스크롤되고, 로고를 한번 터치하면 "Cal Start" 문자가 출력되면서 캘리브레이션 "X" 애니메이션이 표시되고 보정 모드가 시작됩니다.

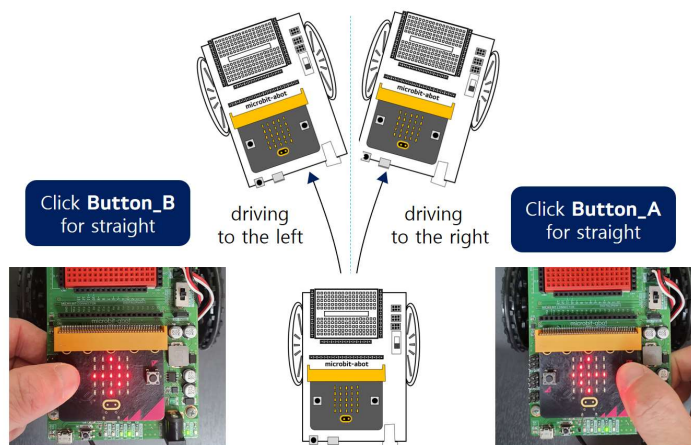


그림5.25 텐덤로봇 직진주행 보정을 위한 버튼 선택방법

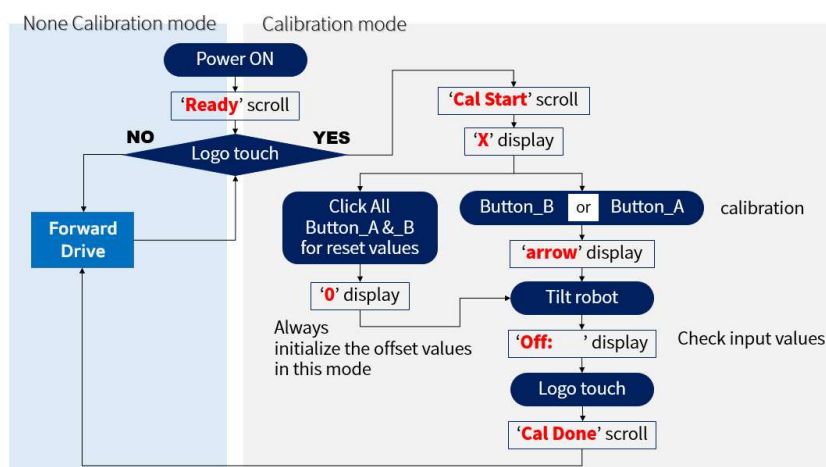


그림5.26 텐덤로봇 직선주행 보정 순서도

이때 버튼_A 또는 버튼_B를 한번 또는 여러 번 사용자의 필요 횟수만큼 누릅니다. 버튼_A를 한번 누를 때마다 해당방향으로 화살표가 표시되고 왼쪽 바퀴 오프셋 변수에 '1'이 빼지며, 버튼_B는 오른쪽 바퀴 오프셋 변수에 '1'이 더해집니다.

이어서 보정을 위해 선택한 버튼 방향으로 로봇을 기울이면 "Off. 오프셋 값" 으로 마이크로비트 디스플레이에서 확인할 수 있습니다. 예를 들어 로봇이 오른쪽으로 기울어져서 회전하면, 버튼_A를 선택해서 오프셋을 입력하고, 버튼_A의 방향(시계방향)으로 기울이면 음수의 오프셋 입력 값을 확인할 수 있습니다.

그 다음 로고를 터치하면 "Cal Done" 문자가 출력되고 직선주행 모드로 전환합니다. 이어서 로봇을 바닥면에 놓아서 직선주행을 다시 살펴본 다음 만족할 수준의 직선주행이 만들어질 때까지 보정과정을 반복해보세요.

(※ 버튼의 종류(A 또는 B)와 총 횟수를 기억해두면 다음 실습에서 간단하게 최적의 직선주행 보정 횟수를 입력하고 사용할 수 있습니다. 그리고 보정을 위해 두 개의 버튼 중에서 하나의 버튼만 사용해야 합니다. 버튼으로 입력한 오프셋은 누른 횟수보다 더 많이 입력될 수 있으므로 반드시 확인해야 합니다.)

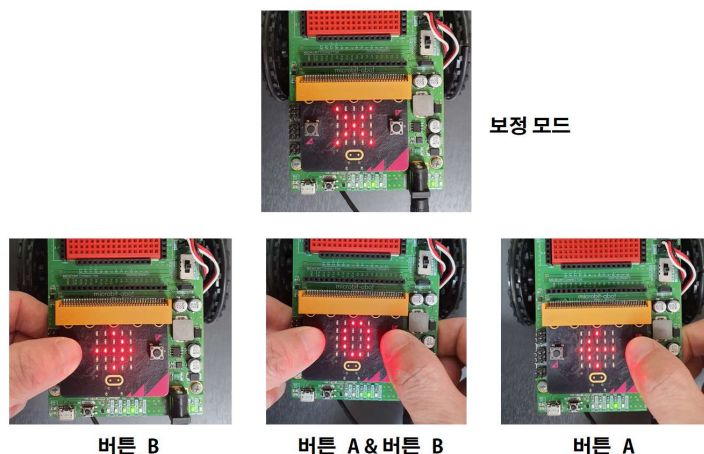


그림5.27 보정모드에서 마이크로비트 버튼을 사용하는 방법

버튼 사용법은 로봇이 오른 쪽으로 치우치는 주행을 하면 로봇을 왼쪽으로 보정하기 위해 버튼_A를 누르고, 반대로 로봇이 왼 쪽으로 치우치는 주행을 하면 버튼_B를 눌러 보정합니다.

현재의 로봇 보정단계에서, 추가 보정을 위해 버튼 누름 횟수를 더 증가하려면, 다시 보정 모드로 들어가서 추가로 해당 버튼을 누르면 오프셋 값이 누적 계산됩니다. 누적 보정횟수를 더 증가하지 않고 줄이려면, 버튼_A와 버튼_B를 동시에 눌러서('0'으로 표시됨) 오프셋 값을 초기화한 다음, 다시 버튼을 눌러서 새로운 오프셋 값으로 입력할 수 있습니다.

오프셋 보정 값을 확인하는 방법은 '보정 모드'에서 로봇을 시계방향(버튼_A 방향)이나 반시계방향(버튼_B 방향)으로 기울이면 디스플레이 화면에서 스크롤되는 값으로 확인할 수 있습니다.

버튼을 누른 총 횟수는 글로벌 변수 left_speed_offset, right_speed_offset 에 저장되어 전원을 끄기 전까지 재사용할 수 있는 상태가 됩니다.

예제5.15 “텐덤로봇 직진주행을 위한 보정하기”

```
# ch5_15_straight_driving_calibration.py
from microbit import *
from microbit_abot import *

left_servo_pin = 13
right_servo_pin = 12

robot = bot(left_servo_pin, right_servo_pin)
robot.servo_attachpins()

left_base_speed = 40 # servo_speed parameter value: 30-120
right_base_speed = -40
left_speed_offset = 0
right_speed_offset = 0
max_offset = 20
min_offset = -20

calibration_animation = [
    Image("90009:"
          "09090:"
          "00900:"
          "09090:"
          "90009:"),
    Image("00000:"
          "09090:"
          "00900:"
          "09090:"
          "00000:"),
]

def calibrate():
    global left_speed_offset, right_speed_offset
    display.scroll("Cal Start")
    animation_index = 0
    prev_button_a_state = False
    prev_button_b_state = False
    while True:
        left_speed = left_base_speed + left_speed_offset
        right_speed = right_base_speed + right_speed_offset
        left_speed = max(min(left_speed, 100), -100)
        right_speed = max(min(right_speed, 100), -100)
        robot.servo_speed(int(left_speed), int(right_speed))
        current_button_a_state = button_a.is_pressed()
        current_button_b_state = button_b.is_pressed()
        if current_button_a_state and current_button_b_state and not
        (prev_button_a_state and prev_button_b_state):
            left_speed_offset = 0
            right_speed_offset = 0
            display.show("0")
            sleep(500)
        elif current_button_a_state and not prev_button_a_state:
            left_speed_offset -= 1
            left_speed_offset = min(max(left_speed_offset, min_offset), max_offset)
            display.show(Image.ARROW_E)
            sleep(200)
        elif current_button_b_state and not prev_button_b_state:
```

```

        right_speed_offset += 1
        right_speed_offset = min(max(right_speed_offset, min_offset), max_offset)
        display.show(Image.ARROW_W)
        sleep(200)
    elif accelerometer.was_gesture('right'):
        display.scroll("Off: {}".format(right_speed_offset))
        sleep(500)
    elif accelerometer.was_gesture('left'):
        display.scroll("Off: {}".format(left_speed_offset))
        sleep(500)
    elif pin_logo.is_touched():
        robot.servo_speed(0, 0)
        display.scroll("Cal Done")
        sleep(500)
        break
    else:
        display.show(calibration_animation[animation_index])
        animation_index = (animation_index + 1) % len(calibration_animation)
        sleep(200)
    sleep(50)
    prev_button_a_state = current_button_a_state
    prev_button_b_state = current_button_b_state

def drive_straight(duration_ms):
    global left_speed_offset, right_speed_offset

    left_speed = left_base_speed + left_speed_offset
    right_speed = right_base_speed + right_speed_offset
    left_speed = max(min(left_speed, 100), -100)
    right_speed = max(min(right_speed, 100), -100)
    robot.servo_speed(int(left_speed), int(right_speed))

    start_time = running_time()
    while running_time() - start_time < duration_ms:
        if pin_logo.is_touched():
            robot.servo_speed(0, 0)
            calibrate()
            left_speed = left_base_speed + left_speed_offset
            right_speed = right_base_speed + right_speed_offset
            left_speed = max(min(left_speed, 100), -100)
            right_speed = max(min(right_speed, 100), -100)
            robot.servo_speed(int(left_speed), int(right_speed))
            start_time = running_time()
        sleep(50)
    robot.servo_speed(0, 0)

def main():
    robot.servo_speed(0, 0)
    sleep(500)
    display.scroll("Ready")
    while True:
        drive_straight(5000)
        sleep(1000)

main()

```

예제 코드의 주요 함수기능을 설명합니다.

calibrate() 메서드: 보정기능을 수행하는 메서드로 사용자가 각 바퀴의 속도를 직접 보정할 수 있습니다. 버튼_A를 누르면 왼 쪽 오프셋 값이 '1'씩 감소하고, 그래서 양수 값을 갖는 왼 쪽 바퀴 매개변수 실효 값이 감소합니다. 버튼_B를 누르면 오른 쪽 오프셋 값이 '1'씩 증가하고, 음수 값을 갖는 오른 쪽 바퀴의 매개변수 실효 값이 감소합니다.

버튼_A와 버튼_B를 동시에 누르면 오프셋 값은 '0'으로 초기화됩니다. 보정 모드에서 로

봇을 왼 쪽(버튼_B 방향)이나 오른 쪽(버튼_A 방향)으로 기울이면 버튼 클릭으로 입력한 오프셋 크기를 디스플레이화면에서 확인할 수 있습니다.

`drive_straight()` 메서드: 직선주행 보정을 위한 메서드입니다. 설정한 시간 동안 주어진 속도대로 직선으로 움직이는 명령을 실행합니다. 움직이는 동안 마이크로비트 로고를 터치하면 `calibrate()` 함수를 호출하고 실행합니다.

예제의 텐덤로봇 서보모터 속도는 `robot.servo_speed(40, -40)` 에 대한 전진 실습입니다. 아래 표5.1과 같이 30부터 120까지 다양한 전진 속도 조건으로 반복해보세요. 여러분의 실습환경(미끄럼 정도 또는 배터리 전압수준)에 따라 표와 다르게 나타날 수 있습니다.

앞에서 소개했던 그림 5.5 “패럴렉스 연속회전형 서보모터의 속도에 대한 펄스폭 변화량” 그래프를 참고하면, 서보모터의 실효값 변화가 가장 급격한 구간은 1500us부터 1600us까지 그리고 1400us부터 1500us까지입니다. 1500us를 기준으로 양쪽의 변화 구간이 동일한 속도변화를 표현한다고 가정하고 아래 표5.1의 값들에 대한 **전진**실습을 진행합니다.

표5.1 로봇 직선주행을 위한 서보모터 보정 실시 사례

펄스폭	left_speed_offset	right_speed_offset	펄스폭	left_speed_offset	right_speed_offset
30		4 (예시)	80		7 (예시)
40		4 (예시)	90		7 (예시)
50		5 (예시)	100		7 (예시)
60		6 (예시)	110		10 (예시)
70		7 (예시)	120		15 (예시)

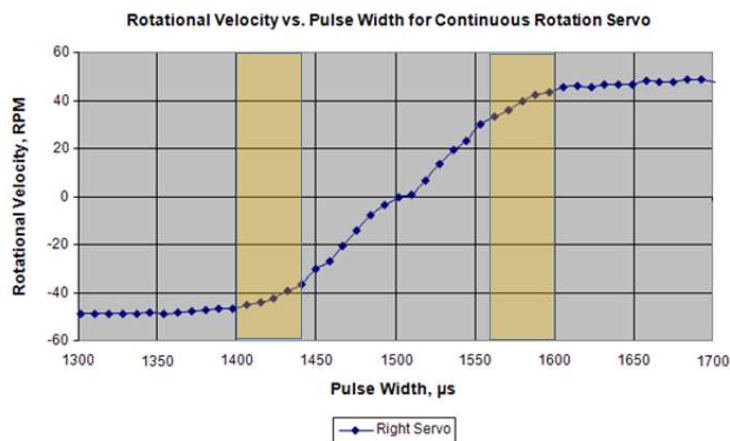


그림5.28 서보모터 펄스폭이 60~100 구간일 때의 속도변화

각자의 사용자 로봇실습을 통하여 `left_speed_offset` (버튼_A사용) 또는 `right_speed_offset`

(버튼_B사용) 중 하나의 값으로 표5.1을 완성하면 됩니다. 소개하는 실습 방법은 아주 정교한 보정방법이 아니므로 사용자가 '로봇의 직선주행에 대한 엄격한 기준'을 적용하지 않도록 추천합니다.

표5.1은 펄스폭 속도에 따라 로봇이 직선 주행하는데 필요한 오프셋 값 사례입니다. 로봇 속도를 사용하는데 유리한 구간(그림5.18의 노란색 표시구간)은 60부터 100사이 펄스폭을 사용할 때입니다. 이 구간에서는 오프셋 크기가 거의 변화하지 않는 구간입니다. 여러분도 로봇 속도 제어에서 유용한 사용구간을 파악하기 바랍니다.

바꾸어 설명하면 속도 70 또는 속도 80 조건에서 한번 보정 작업하면, 로봇 속도가 60부터 100사이 조건에서 동일한 오프셋 값을 사용해도 거의 직선 주행할 수 있다는 의미입니다.

(2) 모듈 방식으로 전진/후진 주행 보정효과 유지하기

이전까지 로봇 직선주행을 위한 보정은 main 코드의 예제5.15로 실습했습니다. 모듈 코드는 전진과 후진 로봇동작 모두 적용되도록 수정되지만, 보정을 실시하는 방법은 전진 동작에 대해서만 실행하면 됩니다. 모듈 코드는 예제5.15를 기초로 몇가지 기능이 추가되었습니다. 모듈방식의 전체 코드 "abot_straight_calibrate.py"는 부록에 추가될 것입니다. 이 코드가 정의한 몇 가지 기능을 상세히 설명합니다.

첫 번째 기능은 한번 보정한 효과를 마이크로비트 전원을 끄고 켜는 동안에도 계속 유효하게 사용하는 것입니다. 만약 전원을 껐다 켜더라도 보정된 오프셋 값을 유지하고 싶다면, 마이크로비트 내부 저장소에 오프셋 값을 저장하고 불러오는 기능을 추가할 수 있습니다. 그리고 인스턴스 변수 오프셋 값을 함수로 변경하는 기능을 추가합니다.

두 번째 기능은 직선주행으로 찾은 오프셋 값을 적용할 `servo_speed_calibrated()` 메서드를 새로이 정의하고, 로봇의 모든 매개변수 조건에서 적용하는 방법입니다. 새로운 메서드는 기존에 사용하던 `servo_speed()` 메서드에 보정 기능을 추가한 주행함수 용도입니다.

세 번째 기능은 전진할 때의 보정 오프셋과 후진할 때의 보정 오프셋은 반대 부호로 적용되어야 합니다. 그리고 로봇의 속도 크기가 '0'에 가까운 경우 보정 오프셋에 의해 바퀴가 반대로 움직일 수 있습니다. 그래서 오프셋이 적용되지 않는 속도구간(코드에서는 -20 ~ 20 구간조건)을 설정합니다.

아래 예제5.16은 모듈방식으로 로봇의 직선주행을 통하여 보정 오프셋 값을 찾는 방법입니다. 모듈 `abot_straight_calibrate.py` 를 코드에 추가해서, 앞에서 실시했던 방법과 동일하게 로봇의 직진 보정을 시도할 수 있습니다.

예제5.16 “모듈 방식으로 텐덤로봇 직선주행 보정하기”

```
# ch5_16_calibrated_bot_drive.py

from microbit_abot import *
from abot_straight_calibrate import *

def main():
    abot = calibratedbot(13, 12)
    abot.servo_attachpins()
    abot.servo_speed(0, 0)
    sleep(500)
    display.scroll("Ready")

    while True:
        abot.drive_straight(80, -80, 5000)
        sleep(1000)

main()
```

예제 코드는 main()함수를 사용해서, 코드를 구조화하고 캡슐화해서 main()함수를 호출하지 않으면 자동 실행되지 않고 의도치 않은 실행을 방지하는 코드 형식입니다. main() 함수를 정의하면 프로그램 논리를 더 쉽게 재사용하거나 수정할 수 있습니다. 또한 프로그램의 시작 시점을 관리해야 하는 경우 명령을 정확히 제어하는데 도움이 됩니다.

실습에서 한번 보정된 오프셋 효과가 전원을 끄기 전까지 유지되도록 하려면, 인스턴스 'calibratedbot(13, 12)'를 한번만 생성하고 사용하기 위해, 예제처럼 while 반복문 외부에서 인스턴스를 생성하고 초기화하는 것입니다.

만약 main() 함수 내의 'abot.drive_straight(80, -80, 5000)' 메소드를 사용하지 않고 'calibratedbot(13, 12).drive_straight(80, -80, 5000)' 메소드를 while 반복문내에서 실행하면, 'calibratedbot(13, 12)' 인스턴스가 반복적으로 생성되어 한번의 메소드 실행시간 5000ms 이후에는 다시 오프셋 값이 '0'으로 자동 초기화됩니다.

실습자가 버튼 클릭으로 입력한 오프셋 값은 인스턴스 변수에 저장되고, 인스턴스가 갱신되면 새로운 인스턴스 변수로 전달되지 않습니다. 새로운 인스턴스는 매번 초기화되고 따라서 오프셋 값은 자동으로 초기화되어서 보정효과가 사라집니다. 이런 문제를 방지하기 위해 한 번만 인스턴스를 생성하고 이를 반복적으로 사용하는 방식의 코드를 사용합니다.

[첫 번째] 마이크로비트 내부 저장소에 오프셋 값을 저장하고 불러오는 기능 추가

이를 위해 microbit 모듈의 storage 기능을 활용할 수 있습니다. 이를 위해서 다음 기능들을 코드에 추가할 수 있습니다.

[1] 오프셋 값 저장 및 불러오기 기능 추가:

save_offsets()와 load_offsets() 메서드를 추가하여 오프셋 값을 파일에 저장하고 불러옵니다. 함수 명령에 따라 'offsets.txt'라는 파일에 저장하거나 로드합니다. 파일이 없거나 기타 I/O 오류가 발생하는 상황을 관리하기 위해 예외 처리를 포함합니다.

```
def save_offsets(self):
    # 오프셋을 파일에 저장
```

```

try:
    with open('offsets.txt', 'w') as f:
        f.write('{}\n'.format(self.left_speed_offset))
        f.write('{}\n'.format(self.right_speed_offset))
    display.scroll("Offsets Saved")
except OSError:
    display.scroll("Save Error")
    pass

def load_offsets(self):
    # 파일에서 오프셋 로드
    try:
        with open('offsets.txt', 'r') as f:
            lines = f.readlines()
            if len(lines) >= 2:
                self.left_speed_offset = int(lines[0].strip())
                self.right_speed_offset = int(lines[1].strip())
                display.scroll("Offsets Loaded")
            else:
                display.scroll("Offsets Invalid")
    except OSError:
        display.scroll("No Offsets Found")
        # 오프셋은 기본값(0)으로 유지
        pass

```

[2] 초기화 시 오프셋 값 로드:

`__init__` 메서드에서 `load_offsets()`를 호출하여 기존에 저장된 오프셋 값을 로드합니다. 초기화할 때마다 `self.load_offsets()`을 호출합니다.

```

def __init__(self, pA, pB):
    self.robot = bot(pA, pB)

    self.left_base_speed = 0
    self.right_base_speed = 0
    self.left_speed_offset = 0
    self.right_speed_offset = 0
    self.max_offset = 20
    self.min_offset = -20
    self.calibration_animation = [
        Image("90009:"
              "09090:"
              "00900:"
              "09090:"
              "90009"),
        Image("00000:"
              "09090:"
              "00900:"
              "09090:"
              "00000"),
    ]
    self.load_offsets() # 초기화 시 오프셋 로드

```

[3] 보정 후 오프셋 값 저장:

`calibrate()` 메서드에서 보정이 완료되면 `save_offsets()`를 호출하여 새로운 오프셋 값을 저장합니다. 보정 완료 후(`pin_logo.is_touched()` : 로고를 터치하면) `self.save_offsets()`를 호출하여 새로운 오프셋 값을 저장합니다.

```

def calibrate(self):
    display.scroll("Cal Start")
    animation_index = 0
    prev_button_a_state = False
    prev_button_b_state = False
    while True:
        left_speed = self.left_base_speed + self.left_speed_offset
        right_speed = self.right_base_speed + self.right_speed_offset
        left_speed = max(min(left_speed, 100), -100)

```

```

right_speed = max(min(right_speed, 100), -100)
self.servo_speed(int(left_speed), int(right_speed))

current_button_a_state = button_a.is_pressed()
current_button_b_state = button_b.is_pressed()

if current_button_a_state and current_button_b_state and not (prev_button_a_state
and prev_button_b_state):
    self.left_speed_offset = 0
    self.right_speed_offset = 0
    display.show("0")
    sleep(500)

----- 생략 -----

elif pin_logo.is_touched():
    self.servo_speed(0, 0)
    display.scroll("Cal Done")
    self.save_offsets() # 보정 후 오프셋 저장
    sleep(500)
    break
else:
    display.show(self.calibration_animation[animation_index])
    animation_index = (animation_index + 1) % len(self.calibration_animation)
    sleep(200)
sleep(50)

prev_button_a_state = current_button_a_state
prev_button_b_state = current_button_b_state

```

[4] 외부에서 오프셋 값을 변경하는 메서드:

마이크로비트에 플래싱한 코드로 직선주행 보정을 완료한 다음, 로봇의 전원을 계속 켜고 끄는 동작을 반복해도 오프셋 값이 파일에 저장되어 로봇의 주행동작에서 계속 반영됩니다. 그렇지만, 새로운 코드를 마이크로비트에 플래싱하면 이전의 보정작업 기록('offsets.txt' 파일 포함)이 모두 사라집니다. 새로운 보정 작업을 반복하지 않기 위해, 이미 알고 있는 오프셋 값을 새로운 코드에 직접 반영해서 동일한 보정효과를 유지할 수 있는 방법에 관한 것입니다. `set_left_speed_offset(value)`와 `set_right_speed_offset(value)` 메서드를 추가하여 외부에서 오프셋 값으로 변경할 수 있도록 합니다.

```

def set_left_speed_offset(self, value):
    self.left_speed_offset = min(max(value, self.min_offset), self.max_offset)
    self.save_offsets()

def set_right_speed_offset(self, value):
    self.right_speed_offset = min(max(value, self.min_offset), self.max_offset)
    self.save_offsets()

```

한 가지 유의할 사항은 왼 쪽 바퀴 오프셋 값 또는 오른 쪽 바퀴 오프셋 값중 하나만 설정하는 코드를 사용해야 합니다. 양쪽 바퀴 오프셋 값 모두를 변경하기 위해, 좌우 오프셋 변경 메서드를 모두 사용하면 맨 마지막 실행된 메서드의 보정 값만 전달됩니다.

앞서 직진주행 보정에서 한쪽 방향의 버튼만 사용해서 보정했던 사실을 기억하고, 하나의 바퀴 오프셋 설정 값만 변경하도록 사용하기 바랍니다.

예제5.17 “오프셋 값 외부설정으로 로봇 속도 보정하기”

```
# ch5_17_external_offset_control.py
```

```

from microbit_abot import *
from abot_straight_calibrate import *

def main():
    abot = calibratedbot(13, 12)
    abot.servo_attachpins()
    abot.servo_speed(0, 0)
    sleep(500)

    #abot.set_right_speed_offset(7) # use the positive value
    abot.set_left_speed_offset(-7) # use the negative value

    display.scroll("Ready")

    while True:
        abot.drive_straight(80, -80, 5000)
        sleep(1000)

main()

```

위 코드는 오프셋 값 변경을 위해 보정 작업순서로 실행해서 입력하지 않고, 이미 알고 있는 오프셋 값을 외부 함수로 변경하는 예제입니다.

예제5.17의 경우 로봇이 우측으로 치우치는 주행을 하는 경우이고, 보정 작업과정을 매번 반복하지 않고, 버튼_A를 7번 클릭('off: -7')해서 직선주행하는 효과를 반영하는 방법입니다. 이때 'abot.set_left_speed_offset(-7)' 코드를 사용합니다.

반대로 로봇이 좌측으로 치우치는 주행을 하는 경우에는, 보정을 위해 버튼_B를 클릭해서 직선주행을 만들 수 있고, 'abot.set_right_speed_offset()' 코드 주석을 해제하고 사용하면 됩니다. 이때 반대쪽 바퀴 보정 오프셋 'abot.set_left_speed_offset(-7)' 코드는 반드시 주석처리해야 합니다. 두 개의 외부 함수를 동시에 사용할 수 없습니다.

'drive_straight' 메서드는 모듈 코드에서 실행되므로, 사용자가 메인 코드에서 속도 매개변수를 직접 사용할 수 있도록 3개의 매개변수를 사용하도록 변경되었습니다. 직선 주행을 위해 양쪽 바퀴의 매개변수 크기를 동일하게 입력하기 바랍니다.

위 실습의 직진주행 보정 순서를 요약하면 다음과 같습니다.

- ① 로봇에 AA건전지를 준비하고, 예제5.16을 마이크로비트에 플래싱합니다.
- ② 로봇의 3접점 스위치로 서보모터에 전원을 공급('2'의 위치)하면, 로봇이 전진합니다.
- ③ 로봇이 전진하는 동안 좌우로 치우치는 정도에 따라 버튼_A또는 버튼_B를 누르고, 주행모드에서 로봇이 전진할 때 직선주행하는지 살펴봅니다.
- ④ 로봇이 직선주행 한다고 판단되면, 방금 사용했던 버튼 방향으로 기울여서 현재의 오프셋 값을 디스플레이에서 읽고 별도로 기록해둡니다. 그리고 로고를 눌러서 보정을 완료하면 현재의 보정 값이 파일로 저장됩니다.
- ⑤ 로봇에 새로운 main 코드를 플래싱해서 시작할 때, 직선주행에 사용된 오프셋 값은 예제5.17의 오프셋입력 외부함수를 사용해서 즉시 적용할 수 있습니다.

(Notice)-----

마이크로비트에서 파일로 쓰고 읽는 기능을 아시나요?

마이크로비트 V2의 경우 간단한 파일 시스템을 지원하여 데이터를 파일로 읽고 쓸 수 있습니다. open 함수를 사용하여 파일을 열고 데이터를 읽거나 쓸 수 있으며, 저장 가능한 데이터 크기는 매우 제한적입니다. 플래시 메모리를 사용하므로 작은 크기의 데이터만 저장하는 것이 좋습니다. 마이크로비트 V1의 경우 파일 시스템을 지원하지 않습니다.

(주의: 플래시 메모리에 과도한 쓰기를 피하세요. 마이크로비트 V2의 플래시 메모리는 일반적으로 10,000회에서 100,000회 정도의 쓰기 사이클을 견딜 수 있습니다.)

[두 번째] 오프셋 값이 적용된 servo_speed_calibrated 메서드 사용하기

이전의 로봇동작 함수 servo_speed 메서드를 사용하면 사용자가 오프셋을 항상 적용하기 불편합니다. 그래서 사용자가 한번 오프셋 값을 찾으면, 이후에는 자동으로 오프셋 효과가 적용되는 새로운 메서드를 정의할 필요가 있습니다.

새롭게 만든 servo_speed_calibrated 메서드를 사용해서 다양한 로봇 동작을 만들면, 직선 주행에 필요한 오프셋 값을 자동으로 적용하므로 불편하지 않게 더 정확한 로봇동작을 만들 수 있습니다. 이것은 더 똑똑해진 servo_speed 메서드를 사용하는 것과 유사합니다.

servo_speed_calibrated 메서드: 이 함수는 servo_speed 함수를 대체해서 사용할 수 있습니다. 보정된 값을 사용하여 좌우 바퀴 속도를 맞추고, 이를 통해 직선 주행과 좌우 회전 동작에 오프셋을 모두 적용합니다. 클래스의 인스턴스 변수에 저장된 값을 이 함수에 적용합니다.

마이크로비트에 전원을 끄고 켜는 동작에는 'offsets.txt' 파일에 저장된 오프셋 값을 재사용할 수 있지만, 새로운 메인코드를 플래싱하면 이전에 저장해둔 'offsets.txt' 파일도 함께 지워집니다. 그래서 보정된 서보속도 제어를 사용하려면 외부함수로 오프셋 값을 메인코드에서 입력할 필요가 있습니다.

예제5.18 “오프셋이 적용된 로봇 동작 메서드 사용하기”

```
# ch5_18_offset_robot_drive.py

from microbit_abot import *
from abot_straight_calibrate import *

def main():
    abot = calibratedbot(13, 12)
    abot.servo_attachpins()
    abot.servo_speed(0, 0)
    sleep(500)
```

```

# abot.set_right_speed_offset(7)
abot.set_left_speed_offset(-7)

display.scroll("Ready")
abot.servo_speed_calibrated(100, -100)
sleep(2000)
abot.servo_speed_calibrated(-100, 100)
sleep(2000)

abot.servo_speed_calibrated(0, 0)

main()

```

위의 예제는 로봇이 직선으로 전진과 후진할 때 오프셋이 적용된 메서드를 구현한 것입니다. 실습을 통하여 보정을 적용하지 않은 로봇동작과 지금의 로봇동작이 어떤 차이를 보이는지 살펴보기 바랍니다.

또한, `servo_speed_calibrated()` 메서드를 사용할 때, 마이크로비트 화면에서 출력되는 스크롤 정보 때문에 로봇동작이 지연된다면, 주석처리 하십시오.

메인 코드에서 "Ready" 스크롤 출력 코드를 주석처리하고, `abot_straight_calibrate.py` 모듈에서 "No Offsets Found" "Offsets Loaded" "Offsets Saved" 스크롤 출력 코드를 주석처리하면 됩니다.

[세 번째] 전진과 후진 동작에 오프셋 값 적용하는 기능

예제5.15의 로봇 직선주행 보정은 전진동작에서만 유효합니다. 후진동작에서는 오히려 보정효과가 반대로 적용됩니다. 이런 문제를 개선하는 `servo_speed_calibrated()` 함수를 제시합니다. 동일한 코드 논리가 `drive_straight()` 함수에도 적용됩니다.

```

def servo_speed_calibrated(self, left_base_speed, right_base_speed):
    if left_base_speed == 0 and right_base_speed == 0:
        self.left_speed_offset = 0
        self.right_speed_offset = 0
        self.servo_speed(0, 0)
        return
    if self.left_speed_offset == 0 and self.right_speed_offset == 0:
        self.left_speed_offset = self._default_left_offset
        self.right_speed_offset = self._default_right_offset

    self.left_base_speed = left_base_speed
    self.right_base_speed = right_base_speed

    if abs(self.left_base_speed) < 20 and abs(self.right_base_speed) < 20:
        left_speed = self.left_base_speed
        right_speed = self.right_base_speed
    elif self.left_base_speed < -20 and self.right_base_speed > 20:
        left_speed = self.left_base_speed - self.left_speed_offset
        right_speed = self.right_base_speed - self.right_speed_offset
    else:
        left_speed = self.left_base_speed + self.left_speed_offset
        right_speed = self.right_base_speed + self.right_speed_offset

    left_speed = max(min(left_speed, 100), -100)
    right_speed = max(min(right_speed, 100), -100)
    self.robot.servo_speed(int(left_speed), int(right_speed))

```

위 코드는 오프셋이 비적용되는 구간을 설정하고, 전진과 후진을 구분해서 오프셋이 적용됩니다. 따라서 속도 값이 작을 때 불필요한 오프셋 적용으로 인한 오동작을 방지하면서, 전진과 후진 시 적절한 보정 효과를 유지할 수 있습니다.

(Notice)-----

클래스 상속(Inheritance) vs. 구성(Composition)에 대하여

특징	상속(Inheritance)	구성(Composition)
관계	"is-a" 관계	"has-a" 관계
코드재사용성	부모 클래스의 모든 속성과 메서드를 상속받아 재사용	필요한 기능만 구성 요소로 포함하여 재사용
유연성	부모 클래스에 강하게 결합되어 유연성이 낮음	구성 요소를 교체하거나 추가하기 쉬워 유연성이 높음
복잡성	다중 상속 시 복잡성이 증가할 수 있음	클래스 구조가 다소 복잡해질 수 있음
유지보수성	부모 클래스 변경 시 자식 클래스에도 영향 미침	구성 요소 독립적이어서 유지보수가 용이

super()와 super().__init__()의 역할

super()는 자식 클래스에서 부모 클래스의 메서드나 속성에 접근할 수 있도록 도와주는 내장 함수입니다. 특히 초기화 메서드(__init__)에서 초기화를 호출하거나, 부모 클래스의 메서드를 호출할 때 주로 사용됩니다.

Python은 메서드 해석 순서(Method Resolution Order, MRO)를 사용하여 어떤 부모 클래스의 메서드를 호출할지 결정합니다. super()를 사용하면 MRO에 따라 올바른 부모 클래스의 메서드를 호출할 수 있습니다.

MRO(Method Resolution Order)란?

MRO는 다중 상속 시 어떤 부모 클래스의 메서드를 우선적으로 호출할지를 결정하는 순서입니다. Python은 C3 선형화 알고리즘을 사용하여 MRO를 결정하며, 클래스 정의 순서와 상속 관계에 따라 결정됩니다.

텐덤로봇의 마이크로:비트 라이브러리의 상속과 구성에 대하여

bot 클래스 (microbit_abot.py) : 조부모 클래스

calibratedbot 클래스 (abot_calibrate_save.py): bot 클래스를 상속하는 부모 클래스

rotationCalibratedBot 클래스 (abot_rotation_calibrate.py): calibratedbot 클래스를 상속받아 조부모클래스 및 부모 클래스의 메서드들을 직접 사용.

Ultrasonic 클래스 (ultrasonic.py): bot 클래스의 인스턴스를 내부적으로 생성하여 (bot(self.pinA) 또는 bot(self.pinA, self.pinB)), send_c와 read_r 메서드를 호출합니다. 이는 Ultrasonic 클래스가 bot 클래스의 기능을 ****구성(Composition)****을 통해 사용하고 있음을 의미합니다.

제 III 부 텐덤로봇 네트워크/스마트 제어

제 6 장 텐덤로봇 무선제어

6.1 마이크로비트 무선기능 살펴보기

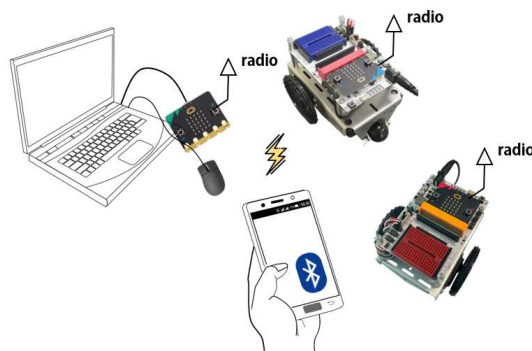
- (1) 블루투스(bluetooth) 통신
- (2) 라디오(radio) 통신

6.2 라디오(Radio) 통신 시작하기

- (1) 라디오통신으로 두 대의 마이크로비트 채팅하기
- (2) 다양한 데이터 송수신하기
- (3) 개별통신과 다중통신(브로드캐스팅)의 조합
- (4) 라디오통신 신호강도와 통신거리

6.3 텐덤로봇 라디오 통신으로 원격제어하기

- (1) 버튼으로 텐덤로봇 제어하기
- (2) 가속도센서로 텐덤로봇 제어하기
- (3) 노트북 키보드 신호로 텐덤로봇 제어하기
- (4) 플레이스테이션 무선 컨트롤러로 텐덤로봇 제어하기
- (5) 플레이스테이션 무선 컨트롤러로 실감 제어하기



이 장은 텐덤로봇을 무선으로 다루는 코딩 방법에 대한 내용입니다. 마이크로비트는 기본적으로 BLE 통신과 라디오 통신을 제공하지만, 마이크로파이썬은 라디오 통신으로 무선통신할 수 있습니다. BLE 통신은 마이크로파이썬으로는 가능하지 않지만, MakeCode를 사용하면 가능합니다.

이 장에서는 라디오통신을 사용해서 여러 대의 텐덤로봇을 목적에 맞게 제어하는 실습을 시도합니다.

수년전부터 엑스비 통신으로 1:N 망구조에 관심을 가진 분이라면, 유사한 무선 네트워크 지식을 마이크로비트의 라디오 통신에 적용할 수 있습니다. 이 책 지면의 제한과 목적을 크게 벗어나지 않는 선에서 다양한 조건과 환경에서 활용하는 무선통신 경험을 소개하려고 노력했습니다.

6.1 마이크로비트 무선기능 살펴보기

마이크로비트는 교육용 마이크로컨트롤러이고, 소형 컴퓨터입니다. 내장된 Nordic 칩을 통해 블루투스(bluetooth)와 라디오(radio) 통신을 기본적으로 모두 지원합니다. 두 가지 무선통신 기능은 파이썬(python) 언어로 'radio' 및 'bluetooth' 모듈을 통해 프로그래밍할 수 있습니다. MakeCode 블록코딩으로는 BLE통신을 사용할 수 있지만, 마이크로파이썬 코딩으로는 BLE통신을 사용할 수 없습니다.

MicroPython 언어를 사용하는 경우에는 Bluetooth 모듈에 필요한 메모리가 충분하지 않아서 Bluetooth 기능이 비활성화되고, 따라서 Python 편집기에서 생성된 .hex 파일에는 DFU 코드가 존재하지 않습니다. 따라서 'import bluetooth' 기능이 동작하지 않는 제한이 존재합니다.

텐덤로봇은 마이크로비트의 내장 무선기능을 사용해서, 로봇의 접근성 제어성을 확장할 수 있습니다. 블루투스 통신과 라디오 통신 방식을 비교하면 표 6.1과 같습니다.

표 6.1 마이크로비트의 블루투스 통신과 라디오 통신 비교

	블루투스 통신	라디오 통신
통신 목적	1:1 통신방식으로 스마트폰, 태블릿, 노트북 등과 같은 기기와 연결하여 데이터를 주고받는 데 사용	1:N 통신방식으로 다른 마이크로비트 기기들과 직접적으로 통신하기 위해 사용. 여러 마이크로비트가 데이터를 교환하거나 협력하여 작업을 수행할 때 유용
통신 범위	일반적으로 실내 환경에서 10미터에서 30미터 사이의 범위를 가지며, 장애물이 있을 경우 이 범위가 줄어들 수 있음.	비슷하거나 약간 더 넓은 범위를 제공할 수 있지만, 설정에 따라 달라질 수 있습니다. 마이크로비트의 안테나 설계와 전송 파워 설정에 따라 최대 수십 미터까지 도달할 수 있음.
데이터 전송속도	블루투스 5.1의 이론적 최대 속도는 초당 수 Mbps에 달할 수 있음.	마이크로비트 라디오는 간단한 메시지와 명령어 전송에 최적화되어 있으며, 큰 데이터 양을 처리하기에는 제한적일 수 있음.
데이터 처리량	더 큰 데이터 패킷을 처리할 수 있어 센서 데이터 로깅, 이미지 전송 등의 용도로 적합	작은 크기의 데이터 패킷을 전송하는 데 적합하며, 단순한 텍스트 메시지나 작은 데이터 세트 전송에 주로 사용
설정 복잡성	페어링 과정이 필요하며, 보안과 연결성을 고려해야함.	마이크로비트 간에는 보안 설정이나 복잡한 페어링 과정 없이 간단하게 통신할 수 있음.

블루투스 통신은 외부 기기와의 통신에 더 적합하며, 데이터 전송 속도와 범위에서 뛰어난 성능을 제공합니다. 반면, 라디오 통신은 마이크로비트 간의 간단한 데이터 교환에 적합하며 설정이 훨씬 간단합니다.

(1) 블루투스(bluetooth) 통신

마이크로비트는 BLE(bluetooth low energy) 통신을 사용해서 간단한 메시지를 전송하거

나, 센서 데이터를 스마트폰 앱으로 전송하여 가공할 수 있습니다. 예를 들어, 온도 센서의 데이터를 주기적으로 스마트폰에 전송하여 모니터링할 수 있습니다. 그리고 원격제어나 인터랙티브 프로젝트 용도로 사용될 수 있습니다.

마이크로비트 v2의 경우, 블루투스 마스터(Central) 기능을 소프트웨어적으로 활성화하여 활용할 수 있는 가능성은 있지만, 기본 제공되는 라이브러리와 예제들은 주로 슬레이브(Peripheral) 기능의 사용에 초점을 맞추고 있습니다.

표 6.2 마이크로비트 v1/v2 버전별 블루투스 통신기능 차이점 비교

	마이크로비트 v1	마이크로비트 v2
칩셋	Nordic nRF51822	Nordic nRF52833
블루투스	블루투스 4.0 Low Energy (BLE)	블루투스 5.1 Low Energy (BLE)
통신기능	초기 마이크로비트 v1은 주로 슬레이브 역할로 사용됩니다. 마스터 기능을 제공하지 않으므로, 다른 장치를 제어하거나 연결을 초기화하는 역할을 수행할 수 없습니다.	마이크로비트 v2는 향상된 블루투스 5.1 기능을 통해 더 높은 유연성과 성능을 제공합니다. 이론적으로는 마스터 및 슬레이브 기능 모두 지원할 수 있습니다. 대부분의 표준 사용에서는 여전히 슬레이브 역할에 초점을 맞춥니다.

따라서, 만약 마이크로비트 v2를 사용하여 마스터 역할을 구현하고 싶다면, 추가적인 프로그래밍과 라이브러리 탐색이 필요할 수 있습니다. 이는 개발자의 경험과 프로젝트의 요구에 따라 다를 수 있으며, 특정 기능의 구현 가능 여부를 확인하기 위해서는 최신 개발 도구와 문서를 참조하는 것이 좋습니다.

간단한 블루투스 통신 연결구조

마이크로비트와 스마트폰 또는 노트북을 1:1 상태로 연결할 수 있습니다. 마이크로비트가 센서 데이터를 수집하고 이를 스마트폰 앱을 통해 실시간으로 모니터링하거나, 스마트폰에서 마이크로비트를 원격 제어하는 용도로 통신기능을 사용할 수 있습니다.

- [1] 마이크로비트 준비: 마이크로비트에 적절한 코드가 업로드되어 있어야 하며, 블루투스 모듈이 활성화되어 있어야 합니다.
- [2] 장치 검색 및 페어링: 스마트폰이나 노트북에서 블루투스를 활성화하고, 사용 가능한 장치 목록에서 마이크로비트를 찾아 연결합니다.
- [3] 통신 시작: 연결이 성공하면, 스마트폰 앱 또는 노트북에서 마이크로비트로 데이터를 보내거나 받을 수 있습니다.

(2) 라디오(radio) 통신

마이크로비트 칩셋에 내장된 2.4GHz 주파수 대역의 라디오 기능은 범용의 라디오칩 NRF24L01+와는 차이가 있지만, 교육 및 학습 용도로 사용하기에 충분합니다.

마이크로비트의 라디오 기능은 또 다른 마이크로비트 장치와 직접적인 무선 통신을 가능

하게 합니다. 구체적인 용도를 살펴보면, 여러 마이크로비트 간의 데이터 교환, 그룹 활동 및 협력 프로젝트, 멀티 플레이어 게임, 신호 전송 및 수신을 통한 간단한 원격 제어 시스템 구축 등입니다. 특히, 2대 이상의 텐덤로봇을 함께 무선으로 제어하는 용도로 적합합니다.

마이크로비트의 버전이 v1에서 v2로 변경되면서 라디오 통신기능 역시 기능적인 개선이 이루어졌습니다. v1은 내장된 nRF51822 칩셋의 2.4 GHz ISM 주파수 대역을 사용하는 라디오 기능을 통해 다른 마이크로비트와 통신할 수 있습니다. 이 칩셋은 기본적인 라디오 기능을 제공하며, 간단한 메시지 전송 및 수신에 적합하게 설계되어 있습니다. 반면에, v2는 업그레이드된 nRF52833 칩셋을 사용해서, 보다 강력하고 효율적인 성능을 발휘합니다. v2의 라디오 통신은 더 나은 신호 범위와 안정성을 제공하며, 보다 복잡한 통신 요구 사항을 지원할 수 있습니다.

결론적으로, 마이크로비트 v1에서 v2로의 업그레이드는 라디오 통신 기능뿐만 아니라 전체적인 하드웨어 성능의 상당한 향상을 가져왔습니다.

마이크로비트의 라디오통신을 시작하려면 기초 설정들을 이해해야할 필요가 있습니다. 라디오 통신을 설정할 때 사용되는 몇 가지 핵심 개념에는 ID, 그룹, 채널이 있습니다. 이러한 개념들은 마이크로비트 간의 통신을 조직하고 관리하는데 중요하며, 각각 다른 목적과 기능을 가집니다.

[1] 채널(Channel)

마이크로비트 v1은 0~83 사이의 채널(84개)을 사용할 수 있고, v2는 0~100 사이에서 채널(101개)을 설정해서 사용할 수 있습니다.

채널은 라디오 주파수의 특정 부분을 의미하며, 마이크로비트가 통신할 때 사용하는 '주파수 대역'을 설정합니다. 채널을 설정함으로써, 주변에 다른 무선 기기나 라디오 소스가 있더라도 서로 간섭받지 않고 통신할 수 있습니다. 채널 번호를 다르게 설정하면, 각기 다른 그룹의 마이크로비트가 독립적으로 통신할 수 있습니다.

예시: 마이크로비트 A와 B가 채널 7을 사용하여 통신하고, 마이크로비트 C와 D가 채널 12를 사용한다면, A와 B는 서로 통신할 수 있지만, C와 D와는 통신할 수 없습니다.

[2] 그룹(Group)

마이크로비트에서 라디오 통신을 설정할 때 사용할 수 있는 그룹 번호의 범위는 0부터 255까지입니다. 총 256개의 다른 그룹 ID를 설정할 수 있으며, 이를 통해 동일한 채널을 사용하면서도 서로 다른 그룹의 마이크로비트 간에 통신을 구분할 수 있습니다.

그룹은 같은 채널을 사용하는 마이크로비트 중에서도 특정한 하위 집합을 형성하게 해줍니다. 그룹 ID를 설정함으로써, 같은 채널을 공유하더라도 서로 다른 그룹에 속한 마이크로비트끼리는 통신하지 않습니다. 이는 교실과 같이 여러 팀이 동시에 작업할 때 유용합니다.

예시: 모든 마이크로비트가 채널 1을 사용하지만, 팀 1은 그룹 101을, 팀 2는 그룹 102를 사용합니다. 이 설정으로 팀 1의 마이크로비트들만 서로 통신할 수 있으며, 팀 2의 마이크로비트들은 서로 통신할 수 있지만, 팀 1과는 통신할 수 없습니다.

마이크로비트 A, B, C가 모두 그룹 101을 사용한다면, 세 마이크로비트는 서로 데이터를 공유하고 교환할 수 있습니다. 이 설정은 마이크로비트들이 같은 '통신 그룹'에 속해 있기 때문에 가능합니다.

```
from microbit import *
import radio

radio.on()
radio.config(channel=1, group=101, power=7)

# 마이크로비트 A에서 메시지 보내기
if button_a.was_pressed():
    radio.send("Hello from A")

# 마이크로비트 B와 C에서 메시지 수신
incoming = radio.receive()
if incoming:
    display.scroll(incoming)
```

라디오 설정: 모든 마이크로비트가 같은 채널과 그룹 설정을 사용하면, 이들은 동일한 무선 통신 네트워크 내에 있게 됩니다. 이 설정은 마이크로비트들이 서로 신호를 수신할 수 있는 조건을 만듭니다.

메시지 전송 및 수신: 한 마이크로비트에서 radio.send() 함수를 사용해 메시지를 보낸다면, 같은 채널과 그룹 설정을 공유하는 다른 모든 마이크로비트들이 이 메시지를 radio.receive() 함수를 통해 수신할 수 있습니다.

[3] 주소 ID(Network ID)

각 마이크로비트에는 고유의 식별자(ID)가 있어, 라디오 통신에서 개별적으로 특정 마이크로비트를 식별할 수 있습니다. 이 주소 ID는 메시지가 특정 마이크로비트로 직접 전송될 때 사용됩니다. 주소 ID를 설정하면, 그룹 내에서도 특정 마이크로비트에게만 메시지를 보낼 수 있습니다.

마이크로비트에서는 "ID"라는 개념이 별도로 정의되어 있지 않으며, "주소(address)"라는 개념으로 사용자들이 특정 마이크로비트를 구분할 수 있게 해주는 값입니다. 이 주소는 라디오 통신에서 개별 마이크로비트를 구별하고, 특정 마이크로비트로 메시지를 보내고자 할 때 사용됩니다. 라디오 통신에서 마이크로비트는 기본적으로 모든 마이크로비트가 메시지를 수신할 수 있도록 브로드캐스트 모드로 설정됩니다. 하지만 특정 마이크로비트에 메시지를 보내고 싶다면, 각 마이크로비트에 고유한 주소를 할당하고, 해당 주소로 메시지를 보낼 수 있습니다.

```
from microbit import *
import radio

# 라디오를 활성화
radio.on()

# 라디오 설정 - 주소 설정
```



```
# 주소는 0에서 255 사이의 값
radio.config(address=123)

# 메시지 보내기
radio.send('Hello, Microbit!')

# 끊임없이 메시지 수신 대기
while True:
    incoming = radio.receive()
    if incoming:
        display.scroll(incoming)
```

주소 범위: 주소 값은 0부터 255 사이의 값으로 설정할 수 있습니다. 각 마이크로비트에 고유한 주소를 할당하면, 그 주소를 가진 마이크로비트만 특정 메시지를 수신할 수 있습니다.

동일한 주소: 동일한 주소를 가진 마이크로비트들은 같은 메시지를 수신할 수 있습니다. 이를 통해 그룹 내 메시지 공유가 가능합니다.

브로드캐스트: 주소를 설정하지 않거나 모든 마이크로비트가 동일한 주소를 가지고 있을 경우, 메시지는 모든 마이크로비트에게 전송됩니다 (브로드캐스트).

마이크로비트에서 라디오를 사용할 때, 각 파라미터에 기본적으로 설정된 값들이 있으며, 이를 사용자가 필요에 따라 변경할 수 있습니다.

```
# 기본 설정으로 라디오 구성
radio.config()
```

(참고: radio.config() 기본 설정값: channel=7, group=0, power=6 (범위는 0~7), data rate=1Mbps(250kbps, 1Mbps, 2Mbps 중 선택), address=0)

(Notice)-----

I2C 바이트(byte) 통신과 Radio 패킷(packet)통신 차이

I2C 통신은 유선 기반의 데이터 전송 방식으로, 주로 짧은 거리에서 소형 장치 간의 데이터를 주고받는 데 사용됩니다. 이 방식은 마스터-슬레이브 구조를 기반으로 하는데, 한 장치(마스터)가 통신의 주도권을 쥐고 다른 장치(슬레이브)와 데이터를 교환합니다. 동기성 전송방식으로 바이트 단위의 데이터 전송 정확성이 보장됩니다.

라디오/블루투스 통신은 무선 방식을 사용하여 장거리에서도 데이터를 전송할 수 있습니다. 이 방식은 패킷 통신을 기반으로 하며, 각 패킷은 헤더, 데이터 페이로드, 그리고 선택적 트레일러로 구성됩니다. 비동기성 방식으로 정확한 시간 조정없이 데이터를 교환합니다. 또한 다중접속으로 동시에 여러 장치와 데이터를 교환합니다.

6.2 라디오(Radio) 통신 시작하기

(1) 라디오통신으로 두 대의 마이크로비트 채팅하기

라디오통신을 사용해서 마이크로비트 사이를 무선으로 채팅하는 실습을 소개합니다. 예제에서는 두 대의 마이크로비트가 서로 메시지를 주고받을 수 있도록 설정됩니다. 사용자는 버튼_A와 버튼_B를 사용하여 메시지를 입력하고 보낼 수 있습니다. 서로 주고받는 메시지는 보안에는 취약합니다. 보안을 강화하려면 다양한 추가 조치들이 사용될 수 있지만 생략합니다.

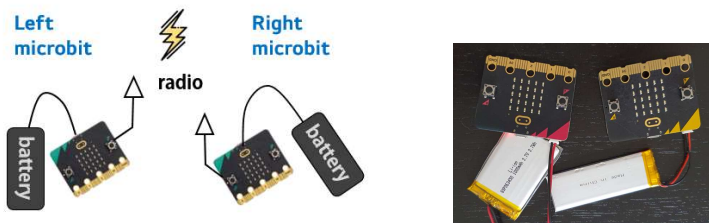


그림 6.1 두 대의 마이크로비트 라디오 통신 1:1 채팅 개요도

먼저, 두 개의 마이크로비트에서 사용할 기본 설정을 정의합니다. 라디오를 활성화하고 채널을 설정합니다. 메시지 전송방식은 버튼_A를 누를 때마다 문자를 선택하고, 버튼_B를 누를 때 선택한 문자를 전송합니다. 여기서는 간단한 예로 알파벳 몇 개만 사용합니다. 두 대의 마이크로비트는 양방향 송수신할 수 있으며, 각 마이크로비트는 메시지를 보내고 받는 두 기능을 모두 수행할 수 있습니다. 양방향 통신을 위해 두 대의 마이크로비트에 아래 코드를 각각 플래싱하고 동작시켜 보세요. 전원공급은 리포배터리로 각각 공급할 수 있지만, 준비된 배터리가 없다면 USB케이블을 연결해서 전원을 공급하십시오.

예제6.1 "마이크로비트를 사용한 라디오 알파벳 전송 및 수신"

```
# ch6_01_radio_alphabet_transmitter.py

from microbit import *
import radio

# Radio setup
radio.on()
radio.config(channel=7)

# Initialize alphabet
alphabet = ['A', 'B', 'C', 'D', 'E']
index = 0

while True:
    # Select character
    if button_a.was_pressed():
        index = (index + 1) % len(alphabet)
        display.show(alphabet[index])

    # Send selected character
    elif button_b.was_pressed():
        radio.send(alphabet[index])
        display.clear()
        sleep(500) # Wait briefly after transmission
```

```
# Receive message
incoming = radio.receive()
if incoming:
    display.scroll(incoming)
```

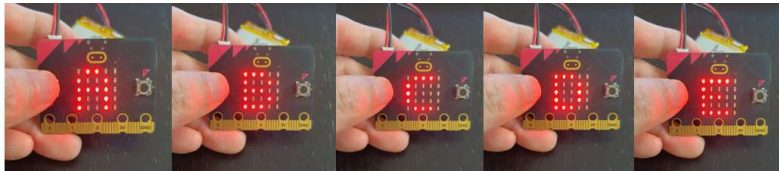


그림6.2 마이크로비트 버튼_A를 클릭하면 출력되는 알파벳 문자들

채팅 사용방법

- 버튼_A (문자 선택): 송신용 마이크로비트 버튼_A를 누를 때마다 알파벳 배열에서 다음 문자를 선택하고 마이크로비트의 LED 디스플레이에 해당 문자를 보여줍니다.
- 버튼_B (문자 전송): 버튼_B를 눌러 현재 선택된 문자를 다른 마이크로비트로 전송합니다. 전송 후 디스플레이는 잠시 깨끗이 청소되고 다시 문자 선택이 가능합니다.
- 메시지 수신: 수신용 마이크로비트가 계속해서 라디오 채널을 청취하고, 메시지가 수신되면 그 메시지를 디스플레이에 스크롤하여 보여줍니다.

두 개의 마이크로비트는 송신 기능과 수신 기능을 모두 수행합니다. 따라서 양방향 채팅을 위한 코드실행에서, 왼쪽과 오른쪽 마이크로비트가 동시에 송신하려고 한다면 어떤 문제가 생길까요? 두 대의 마이크로비트가 동시에 송신할 때 발생할 수 있는 주요 문제는 데이터 충돌입니다. 이는 양쪽 마이크로비트가 동시에 라디오 신호를 전송하면 서로의 신호가 겹쳐서 제대로된 수신이 어려워지는 현상입니다. 이로 인해 메시지가 손상되거나 전혀 도착하지 않을 수 있습니다.

이런 문제를 해결하기 위한 몇가지 방법중 ‘랜덤 백오프(Random Backoff)’ 방식의 코드를 소개합니다. 이 방법 이외에도 타임 슬롯 (Time Slotting) 방식, 메시지 확인 응답 (Acknowledgment), CSMA (Carrier Sense Multiple Access) 기법 등을 사용할 수 있습니다.

예제6.2 "무작위 지연을 사용한 마이크로비트 라디오 통신"

```
# ch6_02_radio_with_random_delay.py

from microbit import *
import radio
import random

radio.on()
radio.config(channel=7)
```

```

alphabet = ['A', 'B', 'C', 'D', 'E']
index = 0

def attempt_to_send(message):
    # Set a random delay time
    delay = random.randint(100, 500)
    sleep(delay)
    radio.send(message)

while True:
    if button_a.was_pressed():
        index = (index + 1) % len(alphabet)
        display.show(alphabet[index])

    elif button_b.was_pressed():
        attempt_to_send(alphabet[index])
        display.clear()
        sleep(500)

    incoming = radio.receive()
    if incoming:
        display.scroll(incoming)

```

random.randint(100, 500)를 사용하여 전송 전에 무작위 대기 시간을 설정하여 네트워크 충돌을 줄입니다. 전송 전에 무작위 대기 시간을 추가하여 충돌을 줄이는 효과를 만듭니다. 버튼_A를 통해 문자를 선택하고, 버튼_B를 통해 문자가 송신됩니다. 수신된 문자는 LED 디스플레이에서 스크롤 표시됩니다.

(2) 다양한 데이터 송수신하기

마이크로비트의 무선기능을 사용해서 다양한 형태의 데이터를 송수신할 수 있습니다. 아래에는 파이썬의 문자열, 리스트, 튜플, 사전 데이터를 송수신하는 사례입니다.

표6.3 파이썬 데이터 유형별 송수신 방법

	송신기 코드	수신기 코드
무선설정 형식	<pre> #ch6_03_sender.py from microbit import * import radio radio.on() radio.config(channel=7) message = "Hello" radio.send(message) </pre>	<pre> #ch6_03_receiver.py from microbit import * import radio radio.on() radio.config(channel=7) while True: incoming = radio.receive() if incoming: display.scroll(incoming) </pre>
문자열	<pre> # 문자열 보내기 message = "Hello, Microbit!" radio.send(message) </pre>	<pre> # 문자열 받기 while True: incoming = radio.receive() if incoming: display.scroll(incoming) </pre>
리스트	<pre> # 리스트 보내기 fruits = ["apple", "banana", "cherry"] radio.send(str(fruits)) </pre>	<pre> # 리스트/튜플/사전 받기 while True: incoming = radio.receive() if incoming: </pre>

튜플	# 튜플 보내기 colors = ("red", "green", "blue") radio.send(str(colors))	# 문자열을 리스트로 변환 incoming_list = eval(incoming) print(incoming_list)
사전	# 사전 보내기 data = {"name": "Alice", "age": 12} radio.send(str(data))	

(3) 개별통신과 다중통신(브로드캐스팅)의 조합

라디오 통신에서 그룹을 전환하는 경우, 전송 데이터를 놓치거나 늦게 수신할 수 있습니다. 이런 이유로 정상적인 통신이 어려울 수 있습니다.

아래 예제는 두 개의 라디오 그룹을 번갈아 사용하여 1:1 통신과 다중 통신을 송신기에서 선택해서 사용하는 방법의 코드입니다. 예제에서는 송신기에서 버튼_A를 누르면 1:1 통신으로 '1' 메시지를 전송하고, 버튼_B를 누르면 다중 통신을 사용해서 '99' 메시지를 전송합니다.

예제6.3 "마이크로비트를 사용한 라디오 그룹 통신-송신기용"

```
# ch6_03_group_radio_communication_sender.py

from microbit import *
import radio

radio.on() # Turn on the radio

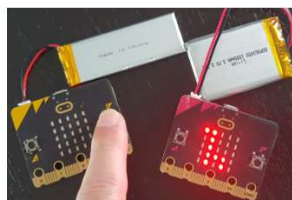
def send_message(group, message):
    radio.config(group=group)
    # Send the message multiple times to increase the probability of
    reception
    for _ in range(5):
        radio.send(message)
        sleep(100)

while True:
    if button_a.was_pressed():
        # Send an individual message
        send_message(1, 'Hello, Micro:bit 1!')
        display.scroll('To 1!')

    elif button_b.was_pressed():
        # Send a message to everyone
        send_message(99, 'Hello, everyone!')
        display.scroll('To all!')
```



Button_A
: HAPPY



Button_B
: GIRAFFE

그림6.3 마이크로비트로 개별통신과 그룹통신 하기

그룹간의 메시지 손실을 최소화하기 위해, send_message 메서드를 사용해서 메시지를 여러 번 전송하는 방법을 사용합니다.

예제6.4 "마이크로비트를 사용한 라디오 그룹 통신-수신기용"

```
# ch6_04_group_radio_communication_receiver.py

from microbit import *
import radio

radio.on() # Turn on the radio

groups = [1, 99] # List of groups to receive
group_index = 0
current_group = groups[group_index]
radio.config(group=current_group)

# Time to stay on each group (milliseconds)
GROUP_SWITCH_INTERVAL = 500
last_switch_time = running_time()

def flush_radio_buffer():
    # Read all received messages to clear the buffer
    while radio.receive() is not None:
        pass

while True:
    incoming = radio.receive()
    if incoming:
        if current_group == 1:
            # Display when individual message is received
            display.show(Image.HAPPY)
            sleep(2000)
            display.clear()
        elif current_group == 99:
            # Display when multi-message is received
            display.show(Image.GIRAFFE)
            sleep(2000)
            display.clear()

    # Switch groups at every GROUP_SWITCH_INTERVAL
    if running_time() - last_switch_time > GROUP_SWITCH_INTERVAL:
        # Flush the buffer before switching groups
        flush_radio_buffer()
        group_index = (group_index + 1) % len(groups)
        current_group = groups[group_index]
        radio.config(group=current_group)
        last_switch_time = running_time()
```

위 코드의 동작방식은 다음과 같습니다.

- 송신기: 버튼_A를 누를 때마다 그룹 1로 설정하여 특정 대상에게만 메시지를 보냅니다. 버튼_B를 누를 때는 그룹 99로 설정하여 네트워크상의 모든 마이크로비트에 메시지를 보냅니다.
- 수신기: 모든 메시지를 받지만, 메시지의 내용에 따라 다르게 반응합니다. '1'이 포함된 메시지를 받으면 '행복' 그림을, 그 외의 메시지를 받으면 '기린' 그림을 표시합니다.

(4) 라디오통신 신호강도와 통신거리

마이크로비트의 라디오 통신은 Nordic Semiconductor의 nRF51822 Bluetooth Smart

칩을 기반으로 하는데, 이 칩은 2.4 GHz ISM (Industrial, Scientific and Medical) 주파수 대역에서 작동합니다. 마이크로비트는 이 칩을 사용하여 간단한 무선 통신을 수행할 수 있습니다.

마이크로비트의 최대 통신 거리는 여러 요인에 따라 달라질 수 있습니다.

[1] 출력 파워: 마이크로비트는 출력 파워를 0에서 7까지 조절할 수 있습니다. 숫자가 높을수록 출력 파워가 강해져 통신 거리가 증가합니다.

[2] 환경 조건: 벽이나 다른 물리적 장애물, 전자기 간섭 등 환경적 요인도 통신 거리에 영향을 미칩니다. 야외와 같이 장애물이 없는 환경에서는 통신 거리가 늘어납니다.

[3] 안테나의 방향성과 위치: 마이크로비트의 안테나 방향과 위치도 신호의 전달 범위에 영향을 줍니다.

신호 강도는 주로 Received Signal Strength Indicator (RSSI) 값으로 측정됩니다. RSSI는 수신된 신호의 강도를 나타내는 지표로, 단위는 dBm입니다. RSSI 값은 보통 음수로 표시되며, 값이 0에 가까울수록 강한 신호를 의미합니다. 예를 들어, -30 dBm은 매우 강한 신호를, -90 dBm은 매우 약한 신호를 나타냅니다.

마이크로비트에서는 `radio.receive_full()` 함수를 사용하여 메시지와 함께 RSSI 값을 얻을 수 있으며, 이 값을 통해 두 장치 사이의 상대적인 거리나 신호의 질을 추정할 수 있습니다.

아래에서 라디오 통신을 이용해 신호 강도를 측정하는 방법에 대한 예제와 상세한 설명을 제공하겠습니다. 이를 통해 두 마이크로비트 간의 신호 강도를 확인하고, 그 값을 디스플레이에 나타낼 수 있습니다.

예제6.5 "마이크로비트를 사용한 라디오 핑 메시지 전송"

```
# ch6_05_radio_ping_sender.py

from microbit import *
import radio

radio.on()
radio.config(power=7) # Set the maximum transmission power

while True:
    radio.send('ping')
    sleep(1000) # Send the message every second
```

예제6.6 "마이크로비트를 사용한 라디오 신호 강도 측정 및 표시"

```
# ch6_06_radio_signal_strength.py

from microbit import *
import radio
```

```

radio.on()

while True:
    incoming = radio.receive_full()
    if incoming:
        message, signal_strength, timestamp = incoming
        display.clear()
        # Display the signal strength (multiply the received signal
        strength by -1 to convert it to a positive number)
        display.scroll(str(-signal_strength))

```

- 송신기 코드: `radio.send('ping')`을 통해 정기적으로 'ping' 메시지를 전송합니다. 이 메시지는 수신기에 의해 인식되며, 수신기는 이 메시지를 받을 때마다 신호 강도를 측정합니다.
- 수신기 코드: `radio.receive_full()` 함수를 사용하여 메시지를 받습니다. 이 함수는 튜플을 반환하는데, 이 튜플에는 메시지의 내용, 신호 강도, 그리고 타임스탬프가 포함됩니다. 신호 강도는 기본적으로 음수 값으로 반환되므로, 이를 양수로 표현하기 위해 `-1`을 곱합니다.

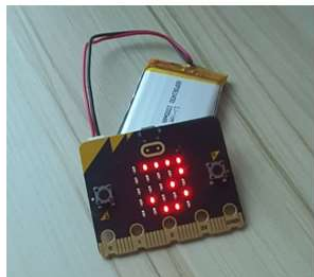


그림6.4 신호강도 측정 값을 스크롤하는 마이크로비트 수신단말기

마이크로비트의 라디오통신 신호강도 측정방법은 송신기 코드와 수신기 코드를 각각 마이크로비트에 플래싱하고, 마이크로비트의 전원을 켜다음 'ping' 메시지가 계속 전송되는지 확인해야 합니다. 이때 신호강도가 마이크로비트의 디스플레이 화면에 표시됩니다. 신호 강도는 신호가 약할수록 큰 값 (양의 값)으로 표시됩니다.

마이크로비트의 라디오 신호 강도

마이크로비트의 라디오 신호 강도는 일반적으로 -42 dBm에서 -128 dBm 사이에서 측정됩니다. 이 값들은 수신된 신호의 강도를 나타내며, 신호가 강할수록 숫자는 0에 가까워지고, 신호가 약할수록 더 큰 음수 값을 가집니다.

- -42 dBm에서 -70 dBm: 매우 강한 신호. 이 범위에서는 안정적인 통신이 가능합니다.
- -70 dBm에서 -85 dBm: 중간 강도의 신호. 통신이 가능하지만, 신호 간섭이나 장애

물이 있을 때 통신 품질이 떨어질 수 있습니다.

- -85 dBm에서 -100 dBm: 약한 신호. 통신이 불안정해질 수 있으며, 신호 손실 가능성이 큼니다.
- -100 dBm 이하: 매우 약한 신호. 이 범위에서는 통신이 거의 불가능할 수 있습니다.

6.3 텐덤로봇 라디오 통신으로 원격제어하기

(1) 버튼으로 텐덤로봇 제어하기

텐덤로봇의 기본동작을 원격제어하는 라디오통신 실습입니다. 로봇의 기본 구성은 아래 그림 6.4와 같습니다. 실습의 요지는 노트북에 연결된 마이크로비트에 파이썬 예제 코드를 플래싱한 다음, 제어용 마이크로비트에서 직접 로봇을 제어하기 위한 제어 신호를 생성하고 전송하는 방식입니다.

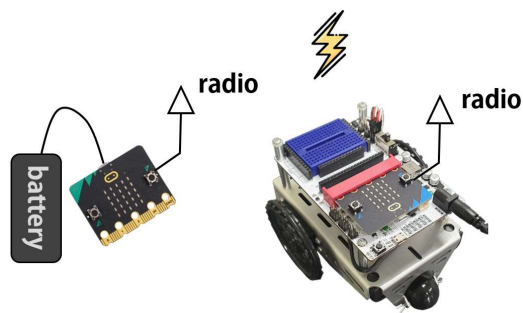


그림 6.4 텐덤로봇 라디오통신으로 원격제어하는 개념도

준비물은 라디오통신을 위해 텐덤로봇에 장착된 마이크로비트 이외에 노트북에 연결할 마이크로비트가 추가로 필요합니다. 로봇을 원격제어하기 위해 마이크로비트에 내장된 가속도 센서 데이터를 활용할 수 있습니다. 아래 그림은 마이크로비트 가속도 센서의 축 방향 개념을 나타낸 그림입니다. 마이크로비트의 3축 x-축, y-축, z-축 방향을 그림으로 표시합니다.

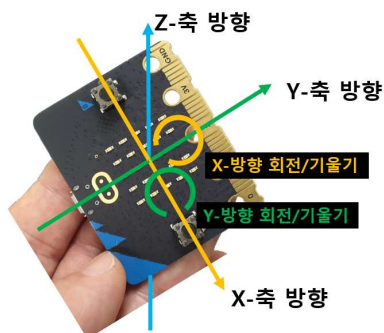


그림 6.5 마이크로비트 가속도센서의 x,y,z 3축 방향

마이크로비트의 가속도 센서는 기본적으로 기기의 선형 가속도, 즉 직선 운동을 측정합니다. 하지만 기기를 기울일 때 중력 가속도의 변화를 측정함으로써 기기의 회전 또는 기울기를 간접적으로 파악할 수 있습니다.

마이크로비트를 x축 방향으로 기울이면 `accelerometer.get_x()`의 값이 변화합니다. 비슷하게 y축, z축 방향으로 기울이면 각각 `accelerometer.get_y()`, `accelerometer.get_z()` 값이 변화합니다. 다만, 회전 운동(특히 각속도)은 자이로스코프로 측정하는 것이 일반적입니다.

가속도 센서 기초 사용법은 이 책의 4장 4절 내용을 참고할 수 있습니다. 3축 방향으로 마이크로비트를 움직이면서 기울기 값을 표시하는 코드 예제는 다음과 같습니다.

```
from microbit import *
while True:
    x = accelerometer.get_x() # x축 방향의 가속도 값 읽기
    y = accelerometer.get_y() # y축 방향의 가속도 값 읽기
    z = accelerometer.get_z() # z축 방향의 가속도 값 읽기

    # 읽어온 값을 스크롤로 표시
    display.scroll("X: " + str(x) + " Y: " + str(y) + " Z: " + str(z))
    sleep(1000) # 1초 동안 대기
```

텐덤로봇에 장착된 마이크로비트 프로그래밍

아래 예제6.7과 6.8은 텐덤로봇에 플래싱할 파이썬 코드와 노트북에 연결할 마이크로비트의 파이썬 코드입니다. 텐덤로봇은 아두이노 라이브러리와 파이썬 라이브러리를 설치한 다음, 아래 예제6.7을 플래싱합니다.

기본동작은 전진 / 좌회전 / 우회전 / 후진 그리고 정지동작을 구분하는 제어신호를 수신 받아서 로봇동작에 반영합니다. 각각의 로봇 동작마다 대응하는 마이크로비트 화살표 모양의 디스플레이 화면이 방향에 따라 각각 만들어집니다. 그리고 정지동작에서는 사각형 모양이 만들어집니다.

예제6.7 "마이크로비트-아두이노 텐덤로봇 수신기"

```
# ch6_07_tandem_radio_controlled_abot.py

import radio
from microbit_abot import *

# Initialize the radio
radio.on()
radio.config(channel=7)

# Attach servos to pins
abot_left = bot(13)
abot_right = bot(12)

bot(13, 12).servo_attachpins()

while True:
    incoming = radio.receive()
    if incoming:
        if incoming == 'forward':
```

```

        abot_left.servo_speed(100)
        abot_right.servo_speed(-100)
        display.show(Image.ARROW_S)

    elif incoming == 'backward':
        abot_left.servo_speed(-100)
        abot_right.servo_speed(100)
        display.show(Image.ARROW_N)

    elif incoming == 'left':
        abot_left.servo_speed(0)
        abot_right.servo_speed(-100)
        display.show(Image.ARROW_E)

    elif incoming == 'right':
        abot_left.servo_speed(100)
        abot_right.servo_speed(0)
        display.show(Image.ARROW_W)

    elif incoming == 'stop':
        abot_left.servo_speed(0)
        abot_right.servo_speed(0)
        display.show(Image.SQUARE)

    # Small delay to allow servos to move
    sleep(100)

```

제어용 마이크로비트 프로그래밍

마이크로비트는 라디오통신으로 로봇을 제어하기 위한 용도입니다. 좌회전과 우회전 신호는 버튼_A와 버튼_B를 사용합니다. 그리고 전진과 후진은 마이크로비트를 전후로 기울여서 마이크로비트의 가속도 값(y축-가속도) 변화를 만들고, 이 값을 로봇으로 전송해서 로봇의 속도변화에 반영합니다.

참고로 노트북에 연결된 마이크로비트는 아래 코드를 마이크로비트로 플래싱한 다음에는 마이크로비트에 배터리 전원만 공급할 수 있다면, 노트북과 분리해서 사용할 수 있습니다.



그림6.6 버튼제어 회전과 전진/후진 가속도센서 제어 (노트북 USB케이블로 전원공급)

예제6.8 "마이크로비트를 사용한 로봇 제어 송신기"

```

# ch6_08_micro_controlled_sender.py
# Remote_controller

from microbit import *

```

```

import radio

# Initialize the radio
radio.on()
radio.config(channel=7)

while True:
    # If both buttons are pressed
    if button_a.is_pressed() and button_b.is_pressed():
        radio.send('stop')

    # If only button A is pressed
    elif button_a.is_pressed():
        radio.send('right')

    # If only button B is pressed
    elif button_b.is_pressed():
        radio.send('left')

    else:
        # Use the accelerometer to determine forward/backward
        movement
        x, y, z = accelerometer.get_values()
        if y < -200:
            radio.send('backward')
        elif y > 200:
            radio.send('forward')
        else:
            radio.send('stop')

    sleep(100)

```

(2) 가속도센서로 텐덤로봇 제어하기

조금 전 로봇을 제어하는 마이크로비트 신호는 좌/우 회전은 마이크로비트의 버튼_A/B를 사용했습니다. 지금은 로봇의 좌/우 회전동작 역시 가속도 센서의 x축 방향 값을 사용해서 버튼의 동작없이 로봇을 제어하는 코드를 소개합니다.

실습을 위한 하드웨어 구성은 이전의 그림과 동일하게 구성합니다. 아래 코드를 노트북에 연결된 마이크로비트에 각각 플래싱하고 로봇을 동작하면 됩니다. 텐덤로봇에 플래싱할 코드는 이전 실습의 코드와 동일하게 적용하면 됩니다.



그림6.7 가속도센서로 전진/후진 그리고 회전 동작 만들기

예제6.9 "마이크로비트를 사용한 제스처 기반 로봇 제어 송신기"

```
# ch6_09_micro_gesture_controlled_sender.py
# Remote_controller

from microbit import *
import radio

# Initialize the radio
radio.on()
radio.config(channel=7)

while True:
    x, y, z = accelerometer.get_values()

    if y < -200:
        radio.send('backward')
    elif y > 200:
        radio.send('forward')
    elif x < -200:
        radio.send('right')
    elif x > 200:
        radio.send('left')
    else:
        radio.send('stop')

    sleep(100)
```

(3) 노트북 키보드 신호로 텐덤로봇 제어하기

한 가지 더 실습해보려는 예제는 마이크로비트에서 직접 입력신호를 만드는 것이 아니라, 마이크로비트는 단순히 라디오 통신을 위한 중계용으로만 사용하고 로봇의 동작을 제어하기 위한 입력신호는 노트북의 키보드 입력 정보를 사용해서 데이터를 전송하는 예제입니다. 코드에서 부여한 키보드 문자는 w(전진) a(좌회전) d(우회전) s(후진) x(정지) 용도입니다.

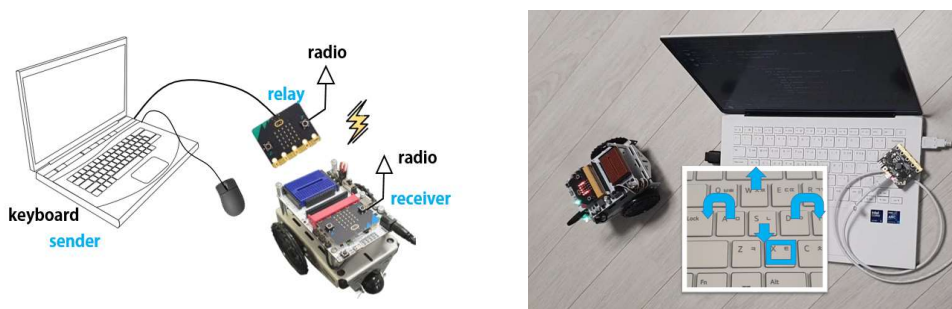


그림 6.6 키보드 문자로 텐덤로봇 진행방향 제어하기

현재 MicroPython을 실행하는 마이크로비트는 키보드 입력을 직접 수신하는 기능이 없습니다. 그래서 실습 목적을 달성하려면 다음 방법을 사용할 수 있습니다.

키보드 입력을 처리하는 코드는 컴퓨터에서 실행되고, 시리얼 통신을 통해 마이크로비트

로 전송되어야 합니다. 그렇지만, 실습에서 추천하는 파이썬 온라인 에디터는 시리얼통신을 지원하지 않습니다. 따라서 파이썬 공식 홈페이지에서 지원하는 에디터를 추가로 설치하고 파이썬 코드를 실행해야 합니다.

우선 별도의 온라인 에디터 설치에 대하여 간단히 소개합니다.

[1] 파이썬 공식 홈페이지(<https://www.python.org/>)에 연결해서 공식 편집기 소프트웨어를 download 하고 설치합니다.

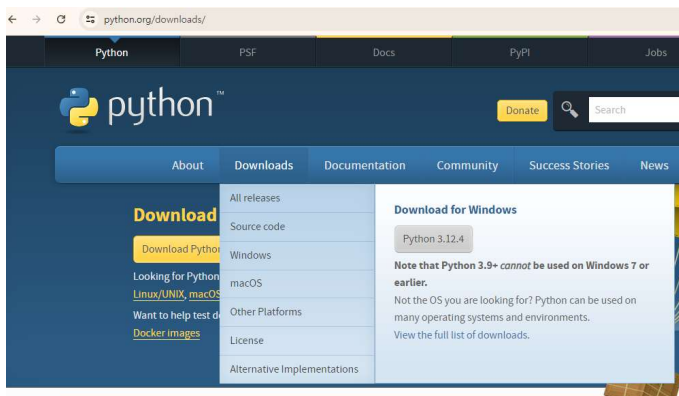


그림 6.7 파이썬 공식 홈페이지의 다운로드 페이지

[2] 설치한 프로그램을 열고, 아래 터미널 창에서 'pyserial' 모듈을 설치합니다.

모듈을 설치하는 명령은 'pip install pyserial'을 입력하고 엔터를 누르면 됩니다. 모듈 설치가 완료되면, 키보드 입력을 수신받고 USB 통신 방식(Uart통신)으로 시리얼데이터를 마이크로비트로 전송하는 코드를 실행합니다.

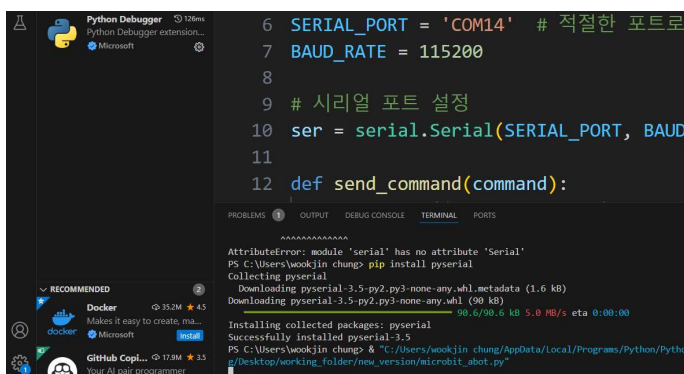


그림 6.8 파이썬 프로그램의 'pyserial' 모듈설치 화면

실습에 필요한 파이썬 코드는 모두 3가지 종류입니다. 하나는 노트북 편집기에서 키보드 입력 처리할 파이썬 코드이고, 또 다른 하나는 노트북과 연결된 제어용 마이크로비트 파이썬 코드이며, 나머지 하나는 텐덤로봇에서 실행할 마이크로비트 파이썬 코드입니다.

세 번째 로봇 동작 코드는 예제6.7 "마이크로비트-아두이노 텐덤로봇 라디오 통신 제어" (ch6_07_tandem_radio_controlled_abot.py)를 사용하기 바랍니다. 아래에서는 첫 번째 노트북에서 실행하는 코드와 두 번째 마이크로비트에서 실행하는 코드를 소개합니다.

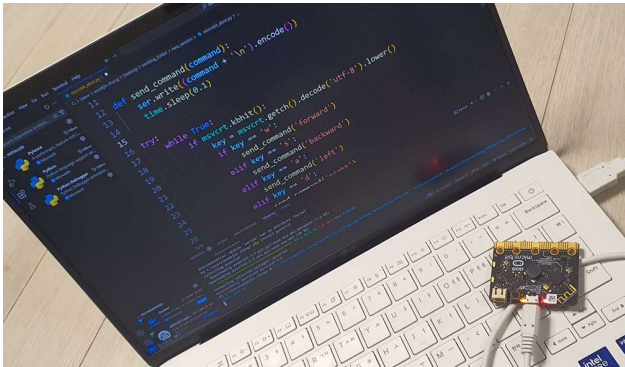


그림 6.9 Uart 시리얼통신하는 마이크로비트

첫 번째: 노트북에서 키보드 입력신호를 시리얼통신으로 처리할 코드
아래 예제6.10은 노트북에서 실행되는 코드입니다.

예제6.10 "PC 키보드를 사용한 마이크로비트 시리얼 제어"

```
# ch6_10_pc_to_micro_serial_control.py
# notebook_keyboard_serial

import serial
import time
import msvcrt

# The serial port to which the micro:bit is connected
SERIAL_PORT = 'COM14' # Change to the appropriate port
BAUD_RATE = 115200

# Configure the serial port
ser = serial.Serial(SERIAL_PORT, BAUD_RATE)

def send_command(command):
    ser.write((command + '\n').encode())
    time.sleep(0.1)

try:
    while True:
        if msvcrt.kbhit():
            key = msvcrt.getch().decode('utf-8').lower()
            if key == 'w':
                send_command('forward')
            elif key == 's':
                send_command('backward')
            elif key == 'a':
                send_command('left')
            elif key == 'd':
                send_command('right')
            elif key == 'x':
                send_command('stop')
finally:
    ser.close()
```

두 번째: 노트북에 연결된 마이크로비트 라디오통신 중계

노트북에 연결된 마이크로비트가 데이터를 중계하는 파이썬 코드입니다. 마이크로비트에 플래싱하는 예제6.11 코드는 노트북에서 Uart 시리얼통신으로 키보드 데이터를 수신하고 다시 라디오 통신을 사용해서 로봇으로 데이터를 전송하는 중계코드입니다.

예제6.11 "PC 데이터를 로봇으로 전송하는 마이크로비트 라디오 중계"

```
# ch6_11_micro_uart_radio_relay.py
# Remote_controller

from microbit import *
import radio

# Initialize the radio
radio.on()
radio.config(channel=7)
uart.init(baudrate=115200)

def clean_command(command):
    # Filter out only letters and numbers from the byte string
    return ''.join(chr(b) for b in command if 48 <= b <= 57 or 65 <=
b <= 90 or 97 <= b <= 122)

while True:
    if uart.any():
        command = uart.read()
        command = clean_command(command)
        if command in ['forward', 'backward', 'left', 'right', 'stop']:
            radio.send(command)
        sleep(100)
```

(4) 플레이스테이션 무선 컨트롤러로 텐덤로봇 제어하기

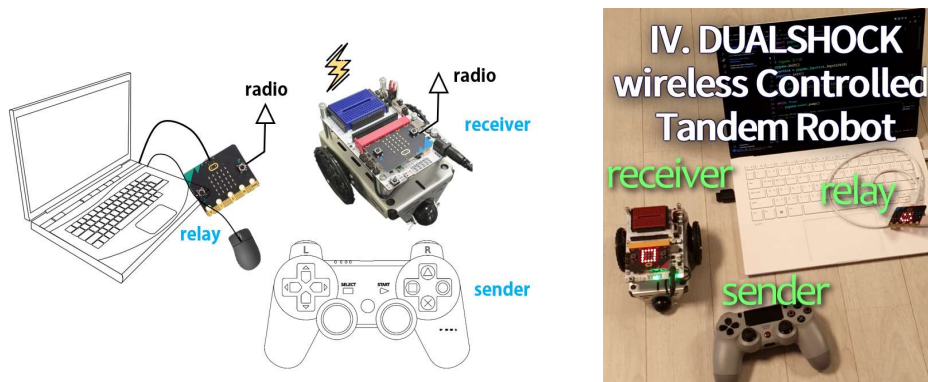


그림 6.10 무선컨트롤러로 텐덤로봇 제어하는 개념도

플레이스테이션 무선 컨트롤러와 마이크로비트의 직접적인 통신 연결은 기술적으로 불가능합니다. 무선 컨트롤러의 블루투스 통신과 마이크로비트의 블루투스 통신은 사용하는 프로토콜과 기술이 서로 다르기 때문입니다. 그리고 무선 컨트롤러는 마이크로비트의 라디오 통신 프로토콜을 인식하지 않습니다. 그림에서 추천하는 방법은 이런 문제를 우회하는 한가

지 수단입니다.

우회 수단의 개념은 '(sender)무선컨트롤러'-'(relay)노트북과 연결된 마이크로비트'-'(receiver)로봇에 연결된 마이크로비트'로 역할을 정의하면 데이터 흐름이 완성됩니다. 다음에서 각 단계별 작업을 소개합니다.

실습 준비물

- 텐덤로봇 1대 (소프트웨어 라이브러리 포함)
- 컴퓨터 1대 또는 노트북 1대 (블루투스 기능이 없는 경우 dongle이 필요함)
- 컴퓨터 연결용 마이크로비트 1개
- 플레이스테이션의 무선 컨트롤러 1대

(Notice) -----

플레이스테이션과 마이크로비트의 블루투스 통신 프로토콜 차이점

플레이스테이션 무선 컨트롤러는 블루투스 HID (Human Interface Device) 프로토콜을 사용합니다. 블루투스 HID는 키보드, 마우스, 게임 컨트롤러와 같은 장치에 사용되며, 높은 전력 소비와 빠른 데이터 전송, 지속적이고 안정적인 연결이 특징입니다.

반면에 마이크로비트는 BLE (Bluetooth Low Energy)를 지원합니다. 블루투스 BLE는 스마트워치, 피트니스 트래커 등 저전력 소모 장치에 적합하며, 매우 낮은 전력 소비, 느린 데이터 전송, 간헐적 연결 및 전송이 특징입니다.

(sender) 플레이스테이션 컨트롤러 데이터 블루투스로 노트북과 연결

- 노트북의 블루투스 설정에서 플레이스테이션 컨트롤러를 페어링합니다.
- 컨트롤러를 페어링하는 방법은 일반적으로 PS 버튼과 Share 버튼을 동시에 눌러서 페어링 모드로 들어가는 것입니다. 블루투스 설정에서 "Wireless Controller"를 선택합니다. (또는 유선으로 연결해서 데이터를 전송할 수 있습니다.)

(relay-1) 노트북으로 컨트롤러 데이터 중계하기

- 파이썬이 설치되어 있지 않다면, 파이썬 공식 웹사이트 (<https://www.python.org/>)를 방문해서 다운로드 받고 설치하기 바랍니다.
- 마이크로비트를 Python과 통신하기 위한 패키지로 'pygame'과 'pyserial' 라이브러리를 사용할 수 있습니다. pyserial은 마이크로비트와의 시리얼 통신을 가능하게 합니다. 라이브러리 설치의 터미널이나 명령 프롬프트에서 다음 명령어를 실행하면 됩니다.

```
pip install pygame pyserial
```

- 다음에는 아래 코드를 파이썬 프로그램에서 실행하면 됩니다.

예제6.12 "조이스틱을 사용한 마이크로비트 시리얼 제어"

```
#ch6_12_joystick_to_micro_serial.py

import pygame
import serial
import time

# Initialize Pygame
pygame.init()
joystick = pygame.joystick.Joystick(0)
joystick.init()

# Configure serial communication with the micro:bit connected to the laptop
ser = serial.Serial('COM3', 115200) # The COM port may vary depending on
the system
time.sleep(2) # Wait for the serial connection to stabilize

while True:
    pygame.event.pump()

    # Read joystick axis values
    x_axis = joystick.get_axis(0)
    y_axis = joystick.get_axis(1)

    # Determine command based on joystick input
    if y_axis < -0.5:
        command = 'forward'
    elif y_axis > 0.5:
        command = 'backward'
    elif x_axis < -0.5:
        command = 'left'
    elif x_axis > 0.5:
        command = 'right'
    else:
        command = 'stop'

    # Send data to micro:bit
    ser.write((command + '\n').encode())

    # Simple output
    print(f'Sent: {command}')

    # Wait briefly so as not to send data too quickly
    time.sleep(0.1)
```

(relay-2) 마이크로비트 라디오 데이터 중계기

- 마이크로비트를 USB 케이블을 통해 노트북에 연결합니다.
- 마이크로비트에 데이터를 전송하기 위해 USB 시리얼 통신을 사용할 수 있습니다.
- 아래 코드를 마이크로비트에 플래싱합니다.

예제6.13 "마이크로비트를 사용한 시리얼 입력 기반 라디오 중계기"

```
# ch6_13_micro_uart_to_radio_relay.py

from microbit import *
import radio

# Turn on the radio
radio.on()
# Configure the radio channel
radio.config(channel=7)
```

```

while True:
    # Check if there is any data available from the UART
    if uart.any():

        # Read a line from the UART
        command = uart.readline()

        if command:
            # Strip whitespace and decode
            command = command.strip().decode()
            # Send the command via radio
            radio.send(command)
            # Provide feedback on the micro:bit display
            display.show(Image.HAPPY)
            # Small delay
            sleep(100)

```

(receiver) 텐덤로봇 파이썬 코드실행(라이브러리 포함)

- 텐덤로봇에 설치할 파이썬 코드입니다. 물론 마이크로비트와 아두이노의 라이브러리가 모두 설치된 조건에서 플래싱해야 합니다.

예제6.14 "마이크로비트를 사용한 라디오 제어 로봇"

```

# ch6_14_tandem_radio_controlled_abot.py

from microbit import *
import radio
from microbit_abot import *

# Initialize the radio
radio.on()
radio.config(channel=7)

# Attach servos to pins
abot_left = bot(13)
abot_right = bot(12)

bot(13, 12).servo_attachpins()

while True:
    incoming = radio.receive()

    if incoming:
        if incoming == 'forward':
            abot_left.servo_speed(100)
            abot_right.servo_speed(-100)
            display.show(Image.ARROW_S)

        elif incoming == 'backward':
            abot_left.servo_speed(-100)
            abot_right.servo_speed(100)
            display.show(Image.ARROW_N)

        elif incoming == 'left':
            abot_left.servo_speed(0)
            abot_right.servo_speed(-100)
            display.show(Image.ARROW_E)

        elif incoming == 'right':
            abot_left.servo_speed(100)
            abot_right.servo_speed(0)
            display.show(Image.ARROW_W)

        elif incoming == 'stop':

```

```

abot_left.servo_speed(0)
abot_right.servo_speed(0)
display.show(Image.SQUARE)

# Small delay to allow servos to move
sleep(100)

```

(5) 플레이스테이션 무선 컨트롤러로 실감 제어하기

이전 실습보다 텐덤로봇을 더 실감나게 제어하기 위한 방법입니다. 무선 컨트롤러의 좌측/우측 조이스틱 레버를 활용해서 텐덤 로봇의 좌측 바퀴와 우측 바퀴동작을 각각 제어하면 로봇의 속도와 방향 제어를 더 실감나게 만들 수 있습니다. 이런 목적을 달성하기 위한 코드를 소개합니다.

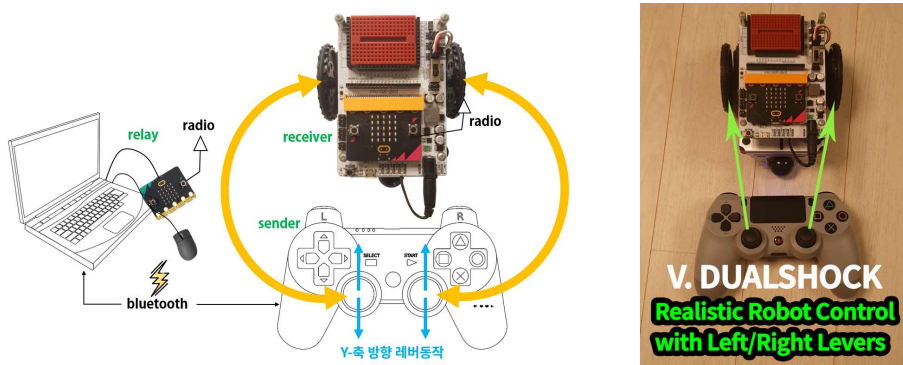


그림 6.11 플레이스테이션 조이스틱으로 텐덤로봇 제어하기

좌측 레버 센서값과 우측 레버 센서 값을 중계 마이크로비트에서는 하나의 데이터 세트로 만들고, 텐덤로봇에 라디오 통신 방식으로 전달하면 됩니다. 여기서 로봇의 바퀴는 하나는 시계방향으로 회전하고 다른 하나의 바퀴는 반시계방향으로 회전해야 로봇은 전진 또는 후진할 수 있습니다. 그리고 무선 컨트롤러의 좌측 레버값은 로봇의 왼쪽 바퀴에 전달되고, 우측 레버값은 로봇의 오른쪽 바퀴에 전달됩니다.

(sender) 플레이스테이션 컨트롤러 데이터 블루투스로 노트북 연결

- 노트북의 블루투스 설정에서 플레이스테이션 컨트롤러를 페어링합니다.
- 컨트롤러를 페어링하는 방법은 일반적으로 PS 버튼과 Share 버튼을 동시에 눌러서 페어링 모드로 들어가는 것입니다. 블루투스 설정에서 "Wireless Controller"를 선택합니다.

(relay-1) 노트북으로 컨트롤러 데이터 중계하기

- 아래 코드를 파이썬 프로그램에서 실행하면 됩니다.

예제6.15 "조이스틱 좌/우 레버를 사용한 마이크로비트 시리얼 제어"

```
# ch6_15_micro_serial_control_with_joystick_levers.py

import pygame
import serial
import time

# Initialize Pygame
pygame.init()
joystick = pygame.joystick.Joystick(0)
joystick.init()

# Configure serial communication
ser = serial.Serial('COM14', 115200)
time.sleep(2) # Wait for the serial connection to stabilize

# Set threshold
threshold = 0.08 # Range considered as neutral

while True:
    pygame.event.pump()

    # Read the y-axis values for the left and right joystick levers
    left_y_axis = joystick.get_axis(1) # Value between -1 and 1
    right_y_axis = joystick.get_axis(3) # Value between -1 and 1

    # Check if the lever values are outside the threshold
    left_speed = int(-left_y_axis * 100) if abs(left_y_axis) > threshold else 0
    right_speed = int(right_y_axis * 100) if abs(right_y_axis) > threshold else 0

    # Format values to two decimals and create a command string
    command = f"{left_speed:.2f},{right_speed:.2f}\n"

    # Send data to the micro:bit
    ser.write(command.encode())

    # Simple output
    print(f"Sent: {command}")

    time.sleep(0.1)
```

(relay-2) 마이크로비트 라디오 데이터 중계기

- 마이크로비트를 USB 케이블을 통해 노트북에 연결합니다.
- 마이크로비트에 데이터를 전송하기 위해 USB 시리얼 통신을 사용할 수 있습니다.
- 아래 코드를 마이크로비트에 플래싱합니다.

예제6.16 "마이크로비트를 사용한 시리얼 입력 기반 라디오 중계기"

```
# ch6_16_micro_uart_to_radio_relay.py

from microbit import *
import radio

# Turn on the radio
radio.on()

# Configure the radio channel and maximum power
radio.config(channel=7, power=7)

while True:
    # Check if there's any data available via UART
    if uart.any():
```

```

command = uart.readline()

# If a command is available, strip and decode it
if command:
    command = command.strip().decode()
    # Send the command via radio
    radio.send(command)
    # Provide some feedback on the micro:bit display
    display.show(Image.HAPPY)
    sleep(100)

```

(receiver) 텐덤로봇 파이썬 코드실행(라이브러리 포함)

- 텐덤로봇에 설치할 파이썬 코드입니다. 물론 마이크로비트와 아두이노의 라이브러리가 모두 설치된 조건에서 플래싱해야 합니다.

예제6.17 "마이크로비트를 사용한 라디오 제어 로봇"

```

# ch6_17_tandem_radio_controlled_abot.py

from microbit import *
import radio
from microbit_abot import *

# Turn on the radio
radio.on()
# Configure the radio channel and max power
radio.config(channel=7, power=7)

# Attach servos to pins
abot_left = bot(13)
abot_right = bot(12)
bot(13, 12).servo_attachpins()

while True:
    incoming = radio.receive()
    if incoming:
        try:
            # Split the received data by comma
            parts = incoming.strip().split(',')
            if len(parts) == 2:
                # Convert the split data to floats
                left_speed, right_speed = map(float, parts)
                abot_left.servo_speed(int(left_speed * 100))
                abot_right.servo_speed(int(right_speed * 100))
                display.show(Image.HEART)
            else:
                raise ValueError("Incorrect number of elements")
        except ValueError as e:
            print("Error:", e)
            display.show(Image.SAD)

        sleep(100)
    else:
        # Actions to perform when there is no incoming data
        abot_left.servo_speed(0)
        abot_right.servo_speed(0)
        display.show(Image.SAD)

        sleep(100)

```

제 III 부 텐덤로봇 네트워크/스마트 제어

제 7 장 텐덤로봇 스마트제어

7.1 간단한 데이터 통합 텐덤로봇

- (1) 단일축 기반 로봇제어
- (2) 다축 통합 로봇제어

7.2 초음파센서 스캔으로 장애물 회피하기

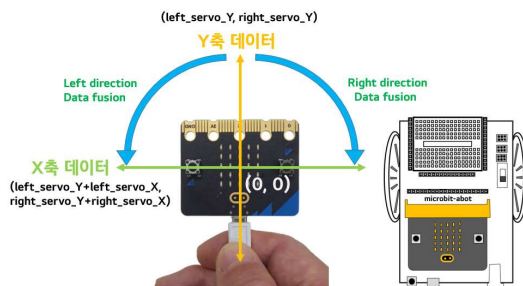
- (1) 조건별 탐색 스캔으로 장애물 회피하기
- (2) 전범위 탐색 스캔으로 장애물 회피하기
- (3) 오픈루프 동적제어 방법으로 장애물 회피하기
- (4) PID 페루프 동적제어 방법으로 장애물 회피하기

7.3 팔로워-로봇 리더 따라가기

- (1) 하드웨어 성능 차이를 극복하는 제자리회전 보정
- (2) 로봇에 장착된 초음파센서 하드웨어 정렬하기
- (3) 순차 제어로 따라가는 로봇 만들기
- (4) 조향과 거리유지 데이터 합산으로 전방로봇 따라가기

7.4 두 대 이상의 로봇 제어하기

- (1) 서로 다른 라디오 성능을 극복하는 방법
- (2) 마스터-슬레이브 로봇 군무 만들기
- (3) 분산제어로 로봇 군무 만들기



코딩의 궁극적인 목적은 “데이터를 얼마나 효율적으로 목적에 부합되도록 사용하는가”라고 표현할 수 있습니다. 데이터를 처리하는 속도와 자원 사용을 최소화하는 것, 목적에 부합하는 정확한 데이터를 사용하는 것, 그리고 코드가 얼마나 쉽게 수정되고 재사용될 수 있는지에 대한 관점을 유지하는 것입니다.

센서 통합 기술은 독자에게 로봇 기술, 프로그래밍, 센서 기능 및 인공지능의 기본 원리를 소개하는데 목적이 있습니다. 텐덤로봇은 비교적 간단한 센서 배열을 사용하며, 실제 자동차나 고급 로봇에서 볼 수 있는 복잡한 통합 시스템보다는 더 기초적인 수준의 통합을 경험할 수 있습니다.

여러 대의 로봇을 통합 제어하는 기술은 중앙 제어와 분산 제어 방식을 상황에 맞게 선택하여 효율적인 협력 시스템을 구축할 수 있습니다. 이를 통해 여러 대의 로봇이 정밀하게 협력하는 환경을 만들어낼 수 있습니다.

7.1 간단한 데이터 통합 텐덤로봇

데이터 통합은 여러 소스에서 얻은 데이터를 결합하여 더 정확하고 신뢰성 있는 정보를 생성하는 과정입니다. 데이터 통합의 주요 목표는 다음과 같습니다:

- 신뢰성 향상: 여러 센서 데이터를 결합하여 단일 센서의 오류나 노이즈를 줄입니다.
- 정확도 향상: 통합된 데이터를 통해 더 정확한 결정을 내릴 수 있습니다.
- 다양한 정보 제공: 여러 데이터 소스를 결합하여 더 풍부한 정보를 제공합니다.

마이크로비트의 x-축과 y-축 가속도 센서 값을 로봇제어에 사용하는 다음 두가지 사례 '단일축 기반 로봇제어'와 '다축 융합 로봇제어'를 통해 데이터 통합의 특성을 비교하겠습니다. 두 가지 실습 차이는 데이터 처리 방식과 제어의 복잡성에 있습니다.

결과적으로 첫 번째 사례는 단일 축 데이터를 독립적으로 사용하여 구분된 동작을 수행하는 반면, 두 번째 사례는 두 개의 축 데이터를 통합하여 더 복잡하고 정밀한 움직임을 제어합니다. 데이터 통합 관점에서 두 번째 방식이 더 발전된 접근 방식이고 다양한 방향성 제어와 에러 감소 등의 장점을 제공합니다.

(1) 단일축 기반 로봇제어

이 책의 앞부분에서 소개한 목차의 6.3 세 번째 예제 '마이크로비트간 라디오통신으로 텐덤로봇 제어하기' 코드는 단일 축 기반으로 로봇을 제어하는 사례입니다. 아래 코드의 특징은 다음과 같이 요약할 수 있습니다.

- ① 단일축 기반 제어: 이 코드는 x축과 y축의 가속도 값을 별도로 사용하여 로봇의 움직임을 제어합니다.
- ② 구분된 동작: 각 축의 변화에 따라 특정 동작(전진, 후진, 좌회전, 우회전)을 수행합니다.
- ③ 직관적 제어: 특정 축의 가속도 변화만으로 동작을 결정하므로, 제어가 단순하고 명확합니다.

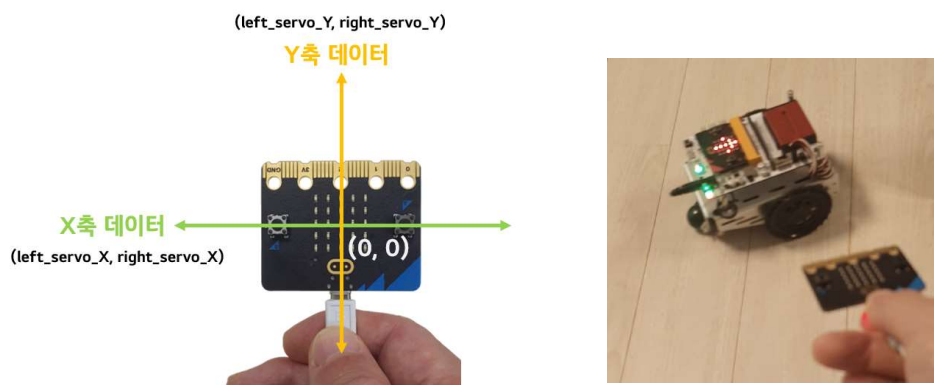


그림 7.1 x-축과 y-축 데이터가 분리된 형태의 데이터 처리방식

이 실습은 라디오통신으로 별도의 마이크로비트와 텐덤로봇을 사용합니다. 제어용 마이크로비트는 sender 코드를 플래싱한 다음 별도의 전원으로 독립해서 사용할 수 있습니다.

(sender) 마이크로비트 x-축(좌/우 회전) y-축(전진/후진) 가속도 데이터

단일 축 기반 로봇제어는 앞 장의 예제6.9 "마이크로비트를 사용한 제스처 기반 로봇 제어 송신기"를 코드를 사용합니다.

텐덤로봇 제어용 마이크로비트에 플래싱할 코드입니다. 원격제어용 마이크로비트가 라디오 통신을 시작하고, y-축 가속도가 음의 값을 가질 때와 양의 값을 가질 때를 구분해서 로봇의 동작을 각각 후진과 전진으로 실행합니다. x-축 가속도의 경우 음의 값을 가지면 로봇이 오른쪽으로 회전하고, 양의 값을 가지면 왼쪽으로 회전합니다. 모든 로봇동작은 임계값 이상의 가속도 값을 가질 때만 실행되기 때문에, x-y 어떤 방향으로든 임계값 이상의 기울임이 없는 경우 로봇동작은 멈춥니다.

예제6.9 "마이크로비트를 사용한 제스처 기반 로봇 제어 송신기"

```
#ch6_09_micro_gesture_controlled_sender.py
# Remote_controller

from microbit import *
import radio

# Initialize the radio
radio.on()
radio.config(channel=7)

while True:
    x, y, z = accelerometer.get_values()

    if y < -200:
        radio.send('backward')
    elif y > 200:
        radio.send('forward')
    elif x < -200:
        radio.send('right')
    elif x > 200:
        radio.send('left')
    else:
        radio.send('stop')

    sleep(100)
```

(receiver) 텐덤로봇 파이썬 코드실행(라이브러리 포함)

마이크로비트 송신기 데이터를 텐덤로봇이 수신하는 코드입니다. 제시된 예제 역시 6장에서 실습해본 코드입니다.

예제6.7 "마이크로비트-아두이노 텐덤로봇 수신기"

```
#ch6_07_tandem_radio_controlled_abot.py

import radio
from microbit_abot import *

# Initialize the radio
radio.on()
radio.config(channel=7)
```

```

# Attach servos to pins
abot_left = bot(13)
abot_right = bot(12)
bot(13, 12).servo_attachpins()

while True:
    incoming = radio.receive()
    if incoming:
        if incoming == 'forward':
            abot_left.servo_speed(100)
            abot_right.servo_speed(-100)
            display.show(Image.ARROW_S)

        elif incoming == 'backward':
            abot_left.servo_speed(-100)
            abot_right.servo_speed(100)
            display.show(Image.ARROW_N)

        elif incoming == 'left':
            abot_left.servo_speed(0)
            abot_right.servo_speed(-100)
            display.show(Image.ARROW_E)

        elif incoming == 'right':
            abot_left.servo_speed(100)
            abot_right.servo_speed(0)
            display.show(Image.ARROW_W)

        elif incoming == 'stop':
            abot_left.servo_speed(0)
            abot_right.servo_speed(0)
            display.show(Image.SQUARE)

    # Small delay to allow servos to move
    sleep(100)

```

(2) 다축 통합 로봇제어

데이터 통합의 사례를 보여주기 위해 x-y축 데이터를 함께 사용하는 예제 코드를 소개합니다. 다축통합 코드의 특징은 다음과 같습니다.

- ① 다축 기반 제어: 이 코드는 x축과 y축의 가속도 값을 모두 사용하여 로봇의 움직임을 제어합니다.
- ② 통합된 동작: 두 축의 가속도 값을 합산하여 로봇의 움직임을 더 세밀하게 제어합니다.
- ③ 복잡한 제어: 두 축의 변화가 동시에 반영되므로, 다양한 방향과 속도로 움직일 수 있습니다.

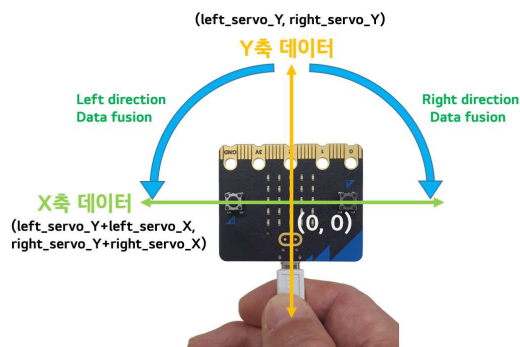


그림 7.2 X-Y축 데이터를 비례변환하고 양쪽 바퀴로 전달하기

(sender) 마이크로비트 x-y가속도 데이터 통합

텐덤로봇을 마이크로비트 가속도센서 x-y축 데이터 값으로 통합 제어하는 예제입니다. 로봇의 왼 쪽 바퀴와 오른 쪽 바퀴속도를 x-y축 데이터에 모두 동시에 반영해서 각각의 바퀴속도를 결정하는 하나의 예시입니다. 이 방식은 로봇의 두 개 바퀴속도를 제어용 마이크로비트로 더 섬세하게 제어할 수 있는 점이 특징입니다.

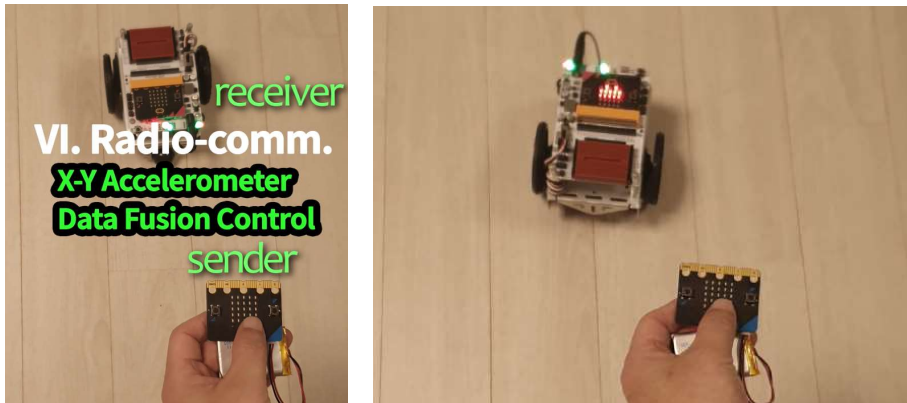


그림7.3 x-y가속도 값으로 로봇 속도제어하기

예제7.1 “마이크로비트 x-y축 데이터통합 라디오 송신기”

```
# ch7_01_xy_axis_data_fusion_radio_sender.py

from microbit import *
import radio

# Radio setup
radio.on()
radio.config(channel=7, power=7)

def map_value(value, in_min, in_max, out_min, out_max):
    return (value - in_min) * (out_max - out_min) // (in_max - in_min)
    + out_min

while True:
    y_accel = accelerometer.get_y()
    x_accel = accelerometer.get_x()

    # Calculate (a_y, b_y) based on y-axis acceleration
    a_y = map_value(y_accel, -1024, 1024, -100, 100)
    b_y = -a_y # Opposite value of a_y

    if -100 < y_accel < 100:
        a_y = 0
        b_y = 0

    # Calculate (a_x, b_x) based on x-axis acceleration
    a_x = map_value(x_accel, -1024, 1024, 100, -100)
    b_x = a_x # Same value as a_x

    if -100 < x_accel < 100:
        a_x = 0
        b_x = 0

    # The range of a and b values is between -200 and +200
```

```

a = (a_y + a_x) * 2
b = (b_y + b_x) * 2

# Limit a and b values between -200 and +200
a = max(min(a, 200), -200)
b = max(min(b, 200), -200)

# Assign (a, b) values to left_speed and right_speed
left_speed = a
right_speed = b

# Send (left_speed, right_speed) via radio
radio.send("{}{}".format(left_speed, right_speed))

# Print values for debugging
print("left_speed: {}, right_speed: {}".format(left_speed, right_speed))

sleep(100)

```

이 코드는 다음과 같은 방식으로 작동합니다:

- ① accelerometer.get_y()와 accelerometer.get_x()를 사용하여 y축과 x축의 가속도 값을 읽어옵니다.
- ② y축 가속도 값에 따라 서보모터 속도 값(a_y, b_y)을 계산하고, x축 가속도 값에 따라 서보모터 속도 값(a_x, b_x)을 계산합니다.
- ③ 각각의 축에 대한 속도 값을 산술적으로 합산하고, 그 평균을 계산하여 최종 (a, b) 값을 구합니다.
- ④ (a, b) 값을 left_speed와 right_speed에 할당합니다.
- ⑤ 라디오를 통해 left_speed와 right_speed 값을 전송합니다.
- ⑥ 현재 값을 출력하여 디버깅에 사용합니다.

이렇게 하면 송신부가 (a, b) 값을 left_speed와 right_speed로 변환하여 라디오를 통해 수신부로 전송할 수 있습니다.

(receiver) 텐덤로봇 파이썬 코드실행(라이브러리 포함)

텐덤로봇에 설치할 파이썬 코드입니다. 물론 마이크로비트와 아두이노의 라이브러리가 모두 설치된 조건에서 플래싱해야 합니다.

예제7.2 “라디오 수신으로 텐덤로봇 동작하기”

```

# ch7_02_dual_servo_radio_receiver.py

from microbit import *
import radio
from microbit_abot import *

radio.on()
radio.config(channel=7, power=7)

abot_left = bot(13)
abot_right = bot(12)
bot(13, 12).servo_attachpins()

```

```

while True:
    incoming = radio.receive()
    if incoming:
        try:
            # Split the received data by commas
            parts = incoming.strip().split(',')
            if len(parts) == 2:
                # Convert the split data to floating point
                left_speed, right_speed = map(float, parts)
                abot_left.servo_speed(int(left_speed))
                abot_right.servo_speed(int(right_speed))
                display.show(Image.HEART)

            else:
                raise ValueError("Incorrect number of elements")

        except ValueError as e:
            print("Error:", e)
            display.show(Image.SAD)

        sleep(100)

    else:
        # Action to perform when there is no data
        abot_left.servo_speed(0)
        abot_right.servo_speed(0)
        display.show(Image.SAD)

        sleep(100)

```

텐덤로봇은 마이크로비트 라디오 통신으로 로봇동작을 제어하기 위해 단계별로 코드가 실행됩니다.

맨 먼저 필요한 모듈을 임포트하고, 라디오를 설정합니다. 서보모터 핀설정과 동작을 준비하고, 무한루프에서 수신된 데이터를 처리합니다. 데이터 처리방법은 다음과 같습니다.

`incoming.strip().split(',')`: 받은 메시지에서 불필요한 공백을 제거하고, 쉼표(.)로 나눕니다. 예를 들어, "50,60" 같은 메시지를 받으면 ['50', '60']로 분리됩니다.

`len(parts) == 2`: 받은 메시지가 두 개의 속도로 나뉘었는지 확인합니다.

`map(float, parts)`: 두 값을 실수(float)로 변환합니다. 이 값은 서보 모터의 속도를 나타냅니다.

`bot(13).servo_speed(int(left_speed))`: 왼쪽 서보 모터의 속도를 설정합니다.

`bot(12).servo_speed(int(right_speed))`: 오른쪽 서보 모터의 속도를 설정합니다.

`display.show(Image.HEART)`: 정상적으로 속도가 설정되면 마이크로비트 화면에 하트 모양을 표시합니다.

7.2 초음파센서 스캔으로 장애물 회피하기

로봇이 전방 장애물을 초음파 센서 스캔으로 탐색하고 회피하면서 주행하는 실습은 로봇의 기본적인 움직임 제어부터 센서를 이용한 환경 인식, 그리고 이를 바탕으로 한 자율적인 행동까지 로봇 공학의 핵심 개념을 체험할 수 있는 중요한 기회입니다. 초보자들이 로봇의

기본 구조와 동작 원리를 이해하고, 실제로 동작하는 로봇을 제작하며 학습의 기초를 다지는 데 큰 도움이 될 것입니다. 또한, 이러한 경험은 향후 더 복잡한 로봇 시스템을 설계하고 구현하는 데 필요한 자신감과 기초 지식을 제공할 것입니다.

소개할 내용들은 아래 사진 그림7.4와 같은 환경에서 전방 로봇을 장애물로 인식하고 회피하는 로봇동작을 4 가지 프로그래밍 방법으로 살펴볼 것입니다.

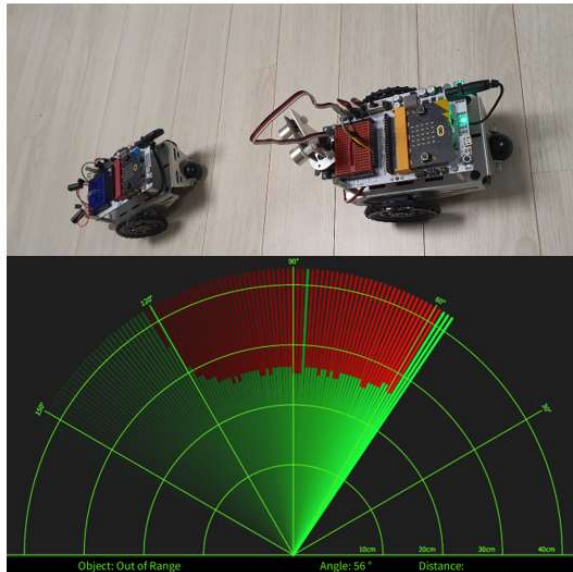


그림7.4 사진 속의 전방 로봇을 초음파 스캔하는 레이더 화면

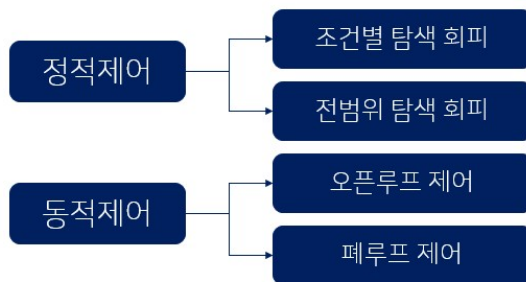


그림7.5 전방 로봇을 회피하는 4가지 프로그래밍 방법

전방 장애물을 회피하는 접근방법은 그림7.5처럼 정적제어와 동적제어로 구분할 수 있습니다. 정적제어는 센서로 전방 장애물을 탐색하는 동작과 로봇이 주행하는 동작을 분리해서 순차적으로 처리하는 방식이고, 동적제어는 탐색동작과 주행동작 두 가지를 동시에 처리하는 방식입니다.

'정적제어 방식'은 코드구조가 단순하고, 초보자가 쉽게 이해하고 구현할 수 있습니다. 그리고 정지상태에서 센서가 장애물을 탐지하므로 정밀하게 감지할 수 있습니다. 다만, 주행

도중에 로봇이 반복적으로 멈추어야 하므로 속도가 느려질 수 있습니다.

반면에 '동적제어 방식'은 실시간으로 수집하는 센서데이터로 장애물에 대응합니다. 이 방식은 멀티태스킹이나 센서 데이터의 주기적인 업데이트가 필요합니다. 이 방식은 주행중에도 장애물을 회피할 수 있어서, 환경변화에 실시간 대응이 가능합니다. 다만, 구현이 복잡하고, 센서 데이터의 안정성과 로봇동작 타이밍을 조율해야 하거나, 성능 제한이 초래될 수 있습니다.

초음파 센서로 장애물을 회피하는 로봇 실습을 위해서는 표준형 서보모터로 센서를 스캔하기 위한 초음파 센서키트가 추가로 필요합니다. 또한, 실습을 성공적으로 진행하려면, 먼저 기초 구성에 해당하는 서보모터와 초음파 센서의 동작원리를 이해해야 합니다. 그 다음 센서 데이터를 사용해서 로봇이 스스로 판단하고 서보모터를 움직이는 프로그래밍 방법을 터득해야 합니다. 프로그래밍 방법은 다양한 조건별 제어로직과 예상하지 못한 문제를 해결하는 방법 그리고 논리적 해결에 적합한 공학개념을 배우는 것입니다.

(1) 조건별 탐색 스캔으로 장애물 회피하기

먼저 가장 간단한 형태의 로봇동작 원리를 적용해서 장애물을 회피하면서 주행하는 실습입니다. 코드의 동작흐름은 먼저 전진하는 모드에서 전방의 장애물을 초음파센서로 감지합니다. 일정한 거리(아래 실습은 20cm) 이내로 장애물이 감지되면, 좌우 방향 45도 각도로 센서를 회전시켜 전방의 장애물을 탐색하고 더 장애물이 없는 방향으로 로봇을 회전합니다. 그 다음 로봇은 다시 전진하며 장애물 탐색을 계속합니다.

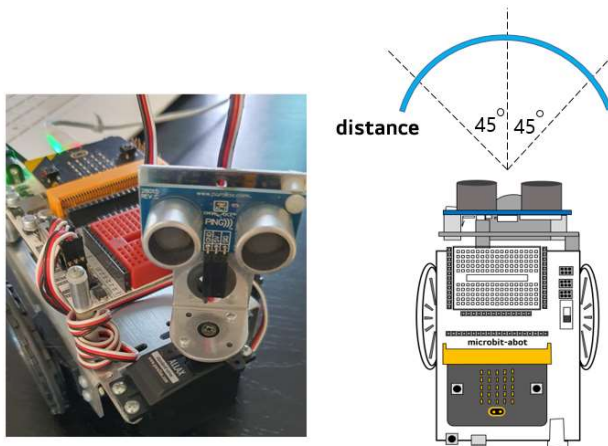


그림7.6 초음파 스캔으로 장애물 탐색하고 회피하기

예제7.3 “조건별 스캔으로 장애물 회피하면서 주행하는 로봇”

```
# ch7_03_abot_obstacle_avoid.py  
  
from microbit import *  
from microbit_abot import *  
from ultrasonic import *
```

```

# Initialize the wheel servos on pins 13 (left) and 12 (right)
abot = bot(13, 12)
abot.servo_attachpins()

# Initialize the ultrasonic sensor on pin 9
ultrasonic_sensor = Ultrasonic(9)

# Initialize the servo for scanning on pin 10
scan_servo = bot(10)

# Movement functions
def move_forward():
    abot.servo_speed(80, -80) # Move forward
    display.show("F")

def stop_movement():
    abot.servo_speed(0, 0) # Stop movement
    display.show("-")

def turn_right():
    abot.servo_speed(100, 100) # Turn right
    display.show("R")
    sleep(300)
    abot.servo_speed(0, 0)

def turn_left():
    abot.servo_speed(-100, -100) # Turn left
    display.show("L")
    sleep(300)
    abot.servo_speed(0, 0)

# Main loop
while True:
    # Move forward continuously
    move_forward()

    # Set the scanning servo to the center position (90 degrees)
    scan_servo.servo_angle(90)
    sleep(100)

    # Measure distance to the object ahead
    distance = ultrasonic_sensor.distance('cm') or 0
    #print("distance is {} cm".format(distance))

    if 0 < distance < 20:
        # Obstacle detected within 20 cm
        stop_movement()

        # Scan to the left
        scan_servo.servo_angle(45)
        sleep(100)
        distance_left = ultrasonic_sensor.distance('cm') or 0

        # Scan to the right
        scan_servo.servo_angle(135)
        sleep(100)
        distance_right = ultrasonic_sensor.distance('cm') or 0

        # Decide the direction based on the distance readings
        if distance_left > distance_right:
            turn_left()
        else:
            turn_right()
    else:
        # No obstacle detected; continue moving forward
        pass

    # Return the scanning servo to the center position
    scan_servo.servo_angle(90)
    sleep(100)

```


이 코드는 로봇이 초음파 센서와 서보모터를 활용하여 전방의 장애물을 회피하면서 전진하도록 설계되었습니다. 메인 함수는 다음과 같이 동작합니다.

로봇은 기본적으로 `move_forward` 함수를 호출하여 전진합니다.

전방의 거리를 측정하기 위해 스캔 서보모터를 90도(정면)로 설정하고, 초음파 센서로 거리를 측정합니다.

만약 전방 20cm 이내에 장애물이 감지되면, 로봇은 `stop_movement` 함수를 호출하여 정지합니다.

이후, 좌우를 탐색하기 위해 서보모터를 각각 45도(좌측), 135도(우측)로 회전시켜 거리를 측정합니다.

좌우 거리 중 더 먼 방향으로 회전하기 위해 `turn_left` 또는 `turn_right` 함수를 호출합니다.

회전 후에는 다시 서보모터를 90도(정면)으로 복귀시킵니다.

(Notice) -----

텐덤로봇 연속형 서보와 표준형 서보를 함께 사용시 유의사항

연속형 서보 함수를 동작하기 위해 `servo_attachpins()` 함수를 실행하면, 바퀴동작에 필요한 2개의 핀을 제외한 나머지 핀들의 서보 준비상태를 모두 자동으로 해제합니다. 이 작업으로 서보 `detach` 함수를 실행하지 않아서 발생할 동작오류를 최소화합니다.

표준형 서보 함수 `servo_angle()`은 서보 `attach` 함수를 내장하고 있어서, 별도의 코드실행이 필요 없지만, 반드시 `servo_attachpins()` 함수 다음 순서에서 실행해야 합니다.

(2) 전범위 탐색 스캔으로 장애물 회피하기

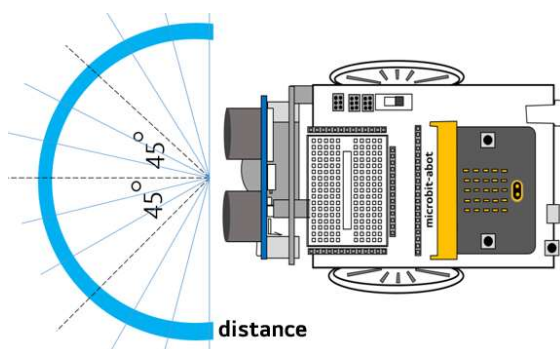


그림7.7 초음파센서 전방 전체영역 탐색하고 장애물 회피하기

앞에서 소개한 장애물 회피방식은 로봇동작과 스캔조건을 먼저 정의하고, 조건별로 로봇 동작을 매칭시키는 방식이었습니다. 지금부터 소개할 로봇의 장애물 회피 논리는 초음파센

서로 전방의 모든 범위를 탐색하고, 가장 장애물이 없는 먼 거리 방향으로 로봇이 나아가도록 하는 조금 더 유연한 제어 방식입니다.

동작원리는 초음파센서 스캔으로 장애물을 찾고, 장애물이 없는 로봇 진행방향이 결정되면 그 방향으로 로봇을 제자리 회전시키는 방법이 사용됩니다. 이 방법을 사용하려면 초음파센서 스캔 각도별 측정거리를 사전형식 데이터로 수집하고 사용해야 합니다.

이 코드는 노이즈로 판단되는 데이터를 처리하는 방법이 필요할 수 있습니다. 마지막으로 로봇이 실제 원하는 각도로 회전하는 데 필요한 보상 계수 값(현재 값은 5)을 조절해야 할 수 있습니다.

```
def turn(angle):
    # Positive angle: turn right, Negative angle: turn left
    turn_time = abs(angle) * 5 # Adjust this factor based on your robot's turning speed
    if angle > 0:
        abot.servo_speed(100, 100) # Turn right
        display.show("R")
    else:
        abot.servo_speed(-100, -100) # Turn left
        display.show("L")
    sleep(turn_time)
    abot.servo_speed(0, 0)
```

아래 소개하는 예제7.3은 0~180도 사이를 스캔하고, 가장 먼 거리를 찾아서 로봇을 회전시키는 방법입니다.

예제7.4 “초음파센서 스캔으로 가장 먼거리를 탐색하고 장애물 회피하기”

```
# ch7_04_abot_obstacle_avoid_maxdistance.py

from microbit import *
from microbit_abot import *
from ultrasonic import *

# Initialize the wheel servos on pins 13 (left) and 12 (right)
abot = bot(13, 12)
abot.servo_attachpins()

# Initialize the ultrasonic sensor on pin 9
ultrasonic_sensor = Ultrasonic(9)

# Initialize the servo for scanning on pin 10
scan_servo = bot(10)

# Movement functions
def move_forward():
    abot.servo_speed(80, -80) # Move forward
    display.show("F")

def stop_movement():
    abot.servo_speed(0, 0) # Stop movement
    display.show("-")

def turn(angle):
    # Positive angle: turn right, Negative angle: turn left
    turn_time = abs(angle) * 5 # Adjust turn angle duration
    if angle > 0:
        abot.servo_speed(100, 100) # Turn right
        display.show("R")
    else:
        abot.servo_speed(-100, -100) # Turn left
```

```

        display.show("L")
        sleep(turn_time)
        abot.servo_speed(0, 0)

# Main loop
while True:
    # Move forward continuously
    move_forward()

    # Set the scanning servo to the center position (90 degrees)
    scan_servo.servo_angle(90)
    sleep(100)

    # Measure distance to the object ahead
    distance = ultrasonic_sensor.distance('cm') or 0
    # print("distance is {} cm".format(distance))

    if 0 < distance < 30:
        # Obstacle detected within 30 cm
        stop_movement()

        # Scan surroundings
        distance_data = {}
        for angle in range(0, 181, 15): # Scan every 15 degrees
            scan_servo.servo_angle(angle)
            sleep(50)
            distance = ultrasonic_sensor.distance('cm') or 0
            distance_data[angle] = distance
            sleep(50)

        # Find angle with maximum distance manually
        max_angle = None
        max_distance = -1
        for angle, dist in distance_data.items():
            if dist > max_distance:
                max_distance = dist
                max_angle = angle

        # Handle case where no maximum is found
        if max_angle is not None:
            # Calculate angle to turn (relative to current heading at 90 degrees)
            angle_to_turn = max_angle - 90 # Ensure this is a regular minus sign

            # Turn robot towards the angle with no obstacle
            turn(angle_to_turn)

        # Reset scanning servo to center
        scan_servo.servo_angle(90)
        sleep(100)
    else:
        # No obstacle detected, continue moving forward
        pass

```

메인 루프 동작을 설명하면 다음과 같습니다.

로봇은 기본적으로 `move_forward` 함수를 호출하여 전진합니다.

전방의 거리를 확인하기 위해 스캔 서보모터를 90도(정면)로 설정하고, 초음파 센서로 거리를 측정합니다.

만약 전방 30cm 이내에 장애물이 감지되면:

`stop_movement` 함수를 호출하여 로봇을 정지시킵니다.

서보모터를 사용하여 0도부터 180도까지 15도 간격으로 주변을 스캔하고, 각도와 거리를 `distance_data` 딕셔너리에 저장합니다.

스캔 결과 중 가장 먼 거리가 측정된 각도를 찾아 `max_angle`에 저장합니다.

현재 로봇의 방향(90도)에서 max_angle을 빼서 회전해야 할 각도 angle_to_turn을 계산합니다.

turn(angle_to_turn) 함수를 호출하여 로봇을 해당 방향으로 회전시킵니다.

회전 후 스캔 서보모터를 90도로 복귀시킵니다.

장애물이 없으면 계속 전진합니다.

(3) 오픈루프 동적제어 방법으로 장애물 회피하기

지금 소개할 로봇 제어 방법은 로봇이 움직이면서 초음파 센서와 서보모터를 사용하여 동적으로 전방의 장애물을 회피하도록 설계하는 것입니다. 로봇은 장애물을 감지하면 속도를 줄이고, 서보모터를 사용하여 좌우를 빠르게 스캔하며, 바퀴의 속도를 조절하여 방향을 변경합니다.

동적제어 방법으로 로봇 전방의 장애물을 회피하는 상황은 아래 그림7.8과 같은 주변에 하나의 장애물만 존재하는 가장 간단한 조건에 대하여 시도합니다. 장애물 개수가 2개 이상으로 늘어나고, 장애물과의 거리도 다양하게 분포하면 더 복잡한 회피논리가 필요할 수 있습니다.

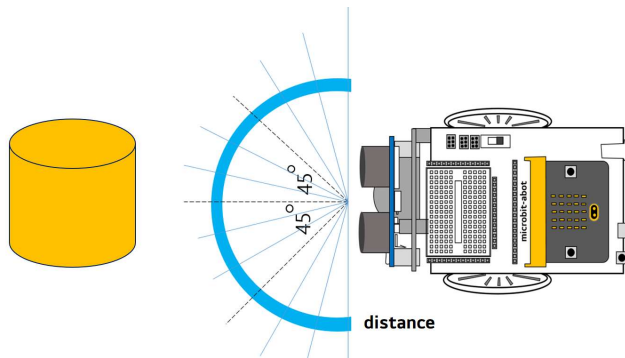


그림7.8 로봇 전방 장애물을 회피하는 실습조건 예시

표7.1 정적 제어시스템과 동적 제어시스템의 차이 비교

	정적 제어시스템	동적 제어시스템
시간적변화	시간적 변화 없음, 입력에 즉각 반응 후 상태 유지	시간에 따른 변화 고려, 점진적으로 목표에 도달
응답특성	즉각적 반응 후 안정화, 변동 없음	과도 상태 후 점진적 안정화, 목표 도달 전 지속적 조정
제어특성	단순한 설계 및 구현, 피드백 없음	복잡한 설계, 피드백 사용, 지속적 조정 필요
오차허용	오차 보정 불가능, 외부 요인에 민감	오차를 지속적으로 보정, 더 높은 정확성

표7.1의 내용에 따르면, 동적제어 시스템은 피드백을 반드시 사용하는 것으로 이해될 수 있지만 대부분의 경우 피드백을 사용한다는 의미로 이해하기 바랍니다. 동적시스템도 오픈

루프 방식의 제어를 사용할 수 있으며, 발사체 제어 시스템에서 발사체 동작을 제어할 때 피드백없이 일정한 시간에 따른 제어 명령을 실행할 수 있습니다.

동적제어를 오픈루프 방식으로 실행하는 실습 세부 주제는 전방 장애물을 찾으면 로봇의 속도를 조절하고, 속도 조절의 크기는 장애물과의 거리에 따라 다르게 반응합니다. 초음파 센서의 스캔 범위는 60~120도 사이를 15도 간격으로 회전하면서 거리를 탐색합니다. 각도 별로 탐색된 거리는 사전에 저장하고, 스캔시간은 짧게 설정해서 로봇 주행에 영향을 최대한 주지 않도록 합니다.

로봇의 동작 흐름은 정상속도로 전진하는 동안 일정한 거리 이내에서 장애물을 감지하면 먼저 속도를 줄이고, 로봇이 움직이면서 좌우로 빠르게 스캔하고 양쪽 바퀴 속도를 조절해서 방향을 변경합니다. 장애물을 안전하게 회피하면, 다시 정상속도로 주행하고 이 과정을 반복합니다.

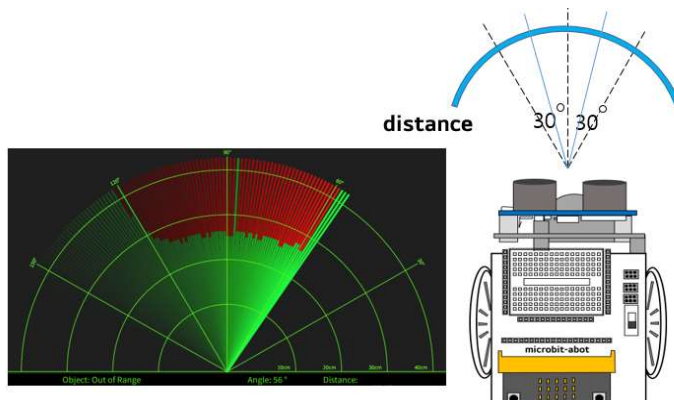


그림7.9 전방 로봇을 탐색하는 로봇 개념도

로봇이 동적으로 움직이기 위해 전방 50cm 이내에서 장애물이 감지되는 경우 장애물을 회피하는 예제7.5의 if문 코드입니다.

```
# As an obstacle approaches, the speed decreases
move_forward(40, -40)
sleep(50)

# Scan left and right while moving
angles = [60, 75, 90, 105, 120]
distances = {}
for angle in angles:
    scan_servo.servo_angle(angle)
    sleep(20)
    distance = ultrasonic_sensor.distance('cm') or 0
    distances[angle] = distance
    sleep(20)

# Average distance left and right
left_avg_distance = (distances.get(60, 0) + distances.get(75, 0)) / 2
right_avg_distance = (distances.get(105, 0) + distances.get(120, 0)) / 2

# The distance difference (positive if farther to the right)
error = right_avg_distance - left_avg_distance
```

```

# Gain setting for speed control
steering_gain = 2.0 # Adjustable
speed_adjustment = - steering_gain * error

# Speed control value limit
speed_adjustment = min(max(speed_adjustment, -20), 20)

# Wheel speed control
base_speed = 30
speed_left = base_speed - speed_adjustment
speed_right = -(base_speed + speed_adjustment)

# Moving forward at a controlled speed
move_forward(speed_left, speed_right)
else:
    # No obstacles, move forward at normal speed
    move_forward(80, -80)

sleep(100)

```

전방 장애물과의 거리가 50cm 보다 작으면, 장애물이 있다고 판단하고 전진 속도를 줄입니다. 그 다음 초음파센서 스캔으로 60도, 75도, 90도, 105도, 120도 방향의 거리측정 데이터를 distances 라는 사전에 저장합니다.

좌우 평균 거리를 계산하고 회피 방향을 결정하기 위한 코드를 실행합니다.

왼 쪽(60°, 75°)과 오른 쪽(105°, 120°) 측정거리를 각각 평균 내어 left_avg_distance와 right_avg_distance에 저장합니다. 이때 distances.get(각도, 0)를 사용해 만약 해당 각도에 대한 측정값이 없으면 기본값 0을 사용합니다.

error는 오른 쪽 평균 거리에서 왼 쪽 평균 거리를 뺀 값을 사용합니다.

로봇이 진행하는 동안 방향을 조절하려면, 좌측과 우측 바퀴의 회전속도 차이를 적용해야 합니다. 이때 좌측과 우측 센서 오차 비율을 크게 적용하면, 로봇의 회전을 더 크게 만들 수 있습니다.

steering_gain 은 error 에 따른 로봇의 회전 크기를 만들고, 부호는 장애물을 회피하는 목적으로 사용합니다. 장애물 방향으로 전진하려면 지금과 반대부호를 사용할 수 있습니다.

base_speed 는 회피모드 기본 속도이며, 로봇이 전방의 장애물을 처음 감지했을 때 줄어드는 속도 크기에 영향을 줍니다.

(Notice) -----

파이썬 딕셔너리의 get() 함수

- 키가 있으면: 해당 키에 연결된 값을 반환합니다.

- 키가 없으면: 기본 값을 반환하여, 에러 없이 안전하게 값을 얻을 수 있게 해줍니다.

```

data = {'a': 1, 'b': 2}
print(data.get('a', 0)) # 결과: 1
print(data.get('c', 0)) # 결과: 0 ('c' 키가 없으므로 기본값 0을 반환)

```

예제7.5 “초음파센서 스캔과 동적제어 방법으로 장애물 회피하기”

ch7_05_obstacle_avoidance_ultrasonic_scan.py
--

```

from microbit import *
from microbit_abot import *
from ultrasonic import *

# Initialize the wheel servos on pins 13 (left) and 12 (right)
abot = bot(13, 12)
abot.servo_attachpins()
# Initialize the ultrasonic sensor on pin 9
ultrasonic_sensor = Ultrasonic(9)
# Initialize the servo for scanning on pin 10
scan_servo = bot(10)

# Movement function
def move_forward(speed_left, speed_right):
    abot.servo_speed(speed_left, speed_right)

# Main loop
while True:
    # Measure distance to the object ahead
    scan_servo.servo_angle(90)
    sleep(50)
    distance_center = ultrasonic_sensor.distance('cm') or 0

    if 0 < distance_center < 50:
        # As an obstacle approaches, the speed decreases
        move_forward(40, -40)
        sleep(50)

        # Scan left and right while moving
        angles = [60, 75, 90, 105, 120]
        distances = {}
        for angle in angles:
            scan_servo.servo_angle(angle)
            sleep(20)
            distance = ultrasonic_sensor.distance('cm') or 0
            distances[angle] = distance
            sleep(20)

        # Average distance left and right
        left_avg_distance = (distances.get(60, 0) + distances.get(75, 0)) / 2
        right_avg_distance = (distances.get(105, 0) + distances.get(120, 0)) / 2

        # The distance difference (positive if farther to the right)
        error = right_avg_distance - left_avg_distance

        # Gain setting for speed control
        steering_gain = 2.0 # Adjustable
        speed_adjustment = - steering_gain * error

        # Speed control value limit
        speed_adjustment = min(max(speed_adjustment, -20), 20)

        # Wheel speed control
        base_speed = 30
        speed_left = base_speed - speed_adjustment
        speed_right = -(base_speed + speed_adjustment)

        # Moving forward at a controlled speed
        move_forward(speed_left, speed_right)
    else:
        # No obstacles, move forward at normal speed
        move_forward(80, -80)

    sleep(100)

```

이 코드를 실행하면, 로봇은 지속적으로 주변을 스캔하고 바퀴 속도를 조절하면서 부드럽게 방향을 변경합니다.

(4) PID 페루프 동적제어 방법으로 장애물 회피하기

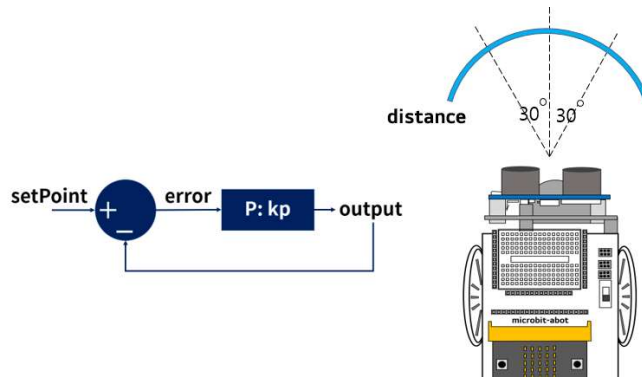


그림7.10 초음파센서로 전방 장애물을 회피하는 PID 제어

앞에서 제시한 오픈루프 제어방식과 다르게 페루프 제어방식으로 장애물을 회피하는 실습입니다. 피드백 제어(페루프 제어)는 로봇의 주행상태를 지속적으로 모니터링하기 때문에, 예상치 못한 상황에서의 오류를 최소화하고 안정적인 동작을 유지할 수 있어서, 신뢰성과 안정성을 향상시킬 수 있습니다. 특히, 환경에 대한 정보를 바탕으로 동작을 조정할 수 있어서 다양한 상황에 적응할 수 있으며, 머신 러닝 알고리즘을 적용하는 방법으로 성능을 향상시킬 수 있습니다.

장애물 회피와 같은 동적인 환경에서의 로봇 제어는 페루프 제어 방식이 필수적입니다. 센서로부터 얻은 피드백을 활용하여 로봇의 움직임을 실시간으로 조정함으로써, 로봇은 예기치 않은 장애물이나 환경 변화에 적응하고 안전하게 목표 지점으로 이동할 수 있습니다. 따라서, 로봇이 장애물을 감지하고 회피하며, 장애물이 없는 방향으로 진행시키기 위해서는 페루프 제어 방식을 사용하는 것이 더 안정적이고 신뢰할 수 있습니다.

그림7.10의 PID 피드백제어는 5장 3절 “텐덤로봇 자율주행하기”의 초음파 센서 편에서 설명하였습니다.

예제7.6은 피드백제어 개념이 적용된 장애물 회피 코드입니다. 페루프 알고리즘으로 채택된 PID제어 방식에서, PID제어는 비례제어만 사용하고 적분제어와 미분제어는 복잡함을 피하기 위해 생략합니다.

예제7.6 “PID제어를 이용해서 로봇이 장애물 회피하기”

```
# ch7_06_pid_control_obstacle_avoidance.py

from microbit import *
from microbit_abot import *
from ultrasonic import *

# Initialize the wheel servos on pins 13 (left) and 12 (right)
abot = bot(13, 12)
abot.servo_attachpins()
```



```

# Initialize the ultrasonic sensor on pin 9
ultrasonic_sensor = Ultrasonic(9)
# Initialize the servo for scanning on pin 10
scan_servo = bot(10)

# PID controller settings
Kp = 1.0 # Proportional gain
Ki = 0.0 # Integral gain (not used)
Kd = 0.0 # Differential gain (not used)

MAX_DISTANCE = 100 # <-- Maximum distance to clamp at 100 cm

# Movement functions
def move(speed_left, speed_right):
    abot.servo_speed(speed_left, speed_right)
def move_forward(speed_left, speed_right):
    abot.servo_speed(speed_left, speed_right)
def get_stable_distance(threshold=5, max_attempts=20):
    """
    Continuously measure with the ultrasonic sensor, and if 3 consecutive
    readings
    are similar (difference from the previous reading <= threshold),
    take the average of those 3 readings as the final result.
    threshold : allowable difference (cm) between consecutive readings
    max_attempts : maximum number of measurement attempts
    """
    stable_count = 1
    # First reading
    reading = ultrasonic_sensor.distance('cm') or 0
    # Clamp the distance to MAX_DISTANCE
    if reading > MAX_DISTANCE:
        reading = MAX_DISTANCE

    stable_readings = [reading]
    for _ in range(max_attempts):
        new_reading = ultrasonic_sensor.distance('cm') or 0
        # Clamp the distance to MAX_DISTANCE
        if new_reading > MAX_DISTANCE:
            new_reading = MAX_DISTANCE

        sleep(20) # short interval between measurements
        # If the difference from the previous reading is within the threshold,
        # consider it "similar"
        if abs(new_reading - stable_readings[-1]) <= threshold:
            stable_count += 1
            stable_readings.append(new_reading)
        else:
            # If the value changes significantly, reset
            stable_count = 1
            stable_readings = [new_reading]
        # If 3 consecutive similar values are found, stop
        if stable_count >= 3:
            break
    # If we have at least 3 readings, return the average of the last 3
    # (since they were consecutively similar)
    if len(stable_readings) >= 3:
        return sum(stable_readings[-3:]) / 3
    else:
        # If 3 consecutive similar readings were not obtained,
        # just return the average of what we have
        return sum(stable_readings) / len(stable_readings)
def measure_angle(angle):
    """
    Move the servo to the specified angle, wait 200ms for stabilization,
    then retrieve a stable distance value from the ultrasonic sensor.
    """
    scan_servo.servo_angle(angle)
    sleep(100) # time for the servo to stabilize
    return get_stable_distance(threshold=5, max_attempts=20)

# Main loop

```

```

while True:
    # Base speed
    base_speed = 80

    # Measure distance to the object ahead
    center_distance = measure_angle(90)

    if 0 < center_distance < 40:
        # When an obstacle approaches, reduce speed
        base_speed = 20
        move(base_speed, -base_speed)
        left_distance = measure_angle(45)
        measure_angle(90)
        sleep(50)
        right_distance = measure_angle(135)
        measure_angle(90)

        # Error calculation
        error = right_distance - left_distance
        print(right_distance, left_distance, error)
        P = Kp * error

        # Limit the steering value
        steering = min(max(P, -20), 20)

        # Wheel speed control
        speed_left = base_speed - steering
        speed_right = -(base_speed + steering)

        # Move forward at a controlled speed
        move(speed_left, speed_right)
    else:
        # If there are no obstacles, move forward at normal speed
        move_forward(base_speed, -base_speed)

    sleep(100)

```

이 코드는 피드백 제어를 통해 로봇이 장애물을 회피하면서 전진하는 예제입니다. 이전 예제7.5에서는 단순히 거리 차이에 비례하여 바퀴 속도를 조절했지만, 예제7.6은 PID 제어를 도입하여 피드백 제어를 명시적으로 구현한 점이 특징입니다.

로봇이 장애물을 효과적으로 회피하도록 설계하기 위하여 센서 데이터를 안정적으로 수집하고 필터링한 후, 비례 제어 원리를 활용합니다. 센서 데이터를 어떻게 처리하고, 그 데이터를 기반으로 로봇이 어떻게 움직임을 결정하는지 제어 원리에 대하여 상세히 설명합니다.

[1] 장애물 감지 및 결정

정면(중앙) 측정:

로봇은 먼저 서보를 사용하여 센서를 정면(90°)으로 향하게 한 후, 초음파 센서를 통해 앞쪽 거리를 측정합니다. 이 측정값을 통해 로봇은 경로에 장애물이 있는지 판단합니다. 예를 들어, 정면 거리가 40cm 미만이면 장애물이 있다고 인식합니다.

속도 조절:

장애물이 감지되면 로봇은 기본 속도를 낮춥니다. 속도를 줄이면 센서 데이터를 보다 안정적으로 처리하고, 장애물을 회피할 때 더 신중하게 움직일 수 있습니다.

[2] 좌우 스캔과 조향 결정

좌우 측정:

로봇은 정면 측정 후 서보를 사용하여 센서를 왼쪽(45°)과 오른쪽(135°)으로 이동시켜 각 각 거리를 측정합니다. 각도 이동 후 100ms 정도 기다려 서보가 완전히 자리 잡고 안정화 되도록 합니다.

제어 원리:

좌측과 우측의 거리를 비교하여 어느 쪽이 더 열려 있는지를 판단합니다. 오차 계산은

$$\text{error} = \text{right_distance} - \text{left_distance}$$

양의 오차는 오른쪽이 더 멀리 열려 있으므로 왼쪽에 장애물이 가까이 있다는 의미이며, 로봇은 오른쪽으로 회피하고, 음의 오차는 왼쪽이 더 열려 있다는 의미로, 로봇은 왼쪽으로 회피합니다.

[3] 센서 데이터 안정화와 필터링

안정적인 측정값 확보:

초음파 센서는 환경에 따라 노이즈가 많을 수 있습니다. 따라서 코드는 여러 번 측정을 반복하면서, 3회 연속으로 측정값의 차이가 작을 때(예, 5cm 이하) 그 값을 안정적이라고 판단합니다. 제어 원리는 갑자기 튀는 값(노이즈)을 걸러내어, 신뢰할 수 있는 데이터로 제어 결정을 내리게 합니다.

최대 거리 제한 (클램핑):

만약 센서가 100cm 이상의 거리를 읽는다면, 그 값을 100cm로 제한합니다. 장애물과의 거리가 한쪽으로 매우 열려 있을 때, 센서가 매우 큰 값을 읽어 제어에 과도한 영향을 주는 것을 방지합니다. 즉, 좌우 측정값 차이가 지나치게 커지는 상황을 줄여줍니다.

[4] 비례 제어를 통한 조향 및 바퀴 속도 조절

조향 값 계산:

좌우 측정값의 차이(오차)를 K_p (비례 상수)와 곱하여 조향 값을 계산합니다. 이 값은 너무 크지 않도록 -20에서 20 사이로 제한합니다. 오차가 클수록 로봇은 더 강하게 회전합니다. 즉, 차이가 클수록 더 큰 조향 명령을 내리게 됩니다.

바퀴 속도 조정:

계산된 조향 값에 따라 왼쪽과 오른쪽 바퀴의 속도를 다르게 조절하여 로봇이 원하는 방향으로 회전하도록 만듭니다. 바퀴 속도의 차이를 통해 로봇이 회전하게 되는 데, 이를 차동제어(Differential Steering)라고 합니다.

기본 속도(base_speed)에서 제어 입력을 더하거나 빼서 좌우 바퀴의 속도를 조절합니다. 이를 통해 로봇이 에러에 따라 좌우로 방향을 조절하면서 장애물을 회피합니다.

$$\text{speed_left} = \text{base_speed} - \text{steering}$$

$$\text{speed_right} = -(\text{base_speed} + \text{steering})$$

7.3 팔로워-로봇 리더 따라가기

여러 대의 로봇을 제어하는 방법 중 하나인 리더-팔로워 패턴에 관한 것입니다. 하나의 로봇(또는 소수의 로봇) 리더가 군집 전체 동작을 주도하고, 나머지 팔로워 로봇들이 리더를 따라 이동합니다. 이런 패턴은 크게 하나의 리더와 여러 팔로워가 병렬로 배치되는 방식, 또는 모든 팔로워를 주종방식으로 꼬리에 꼬리를 연결하는 배치 방식이 예상됩니다.

전방 로봇을 회피하는 동작이 아니라 전방 로봇을 따라가는 동작을 구현하려면, 팔로워 로봇의 제어 성능이 더 정교해져야 합니다. 전방 로봇을 따라가는 로봇을 만들고 싶다면 아래 제자리회전 보정과 센서 정렬 작업을 먼저 실행하기 바랍니다.

(1) 하드웨어 성능 차이를 극복하는 제자리회전 보정

로봇의 제자리회전 보정은 좌측 바퀴와 우측 바퀴 회전속도가 동일해야만 정확도를 유지하기 용이합니다. 제자리회전 보정은 'abot_rotation_calibrate.py' 모듈을 추가합니다.

제자리회전 보정 모듈은 'microbit_abot.py' 모듈과 부모-자식관계 클래스로 정의하고, 앞서 소개했던 직선주행 보정 모듈 'abot_straight_calibrate.py' 과는 독립적 관계입니다.

로봇의 제자리회전은 회전속도와 회전시간 설정에 따라 동작합니다. 회전속도가 크면 빨리 회전하고, 회전시간이 길면 많은 각도를 회전합니다. 따라서 정확한 회전은 ①로봇의 회전속도를 미세 조정하거나 ②로봇의 회전시간을 미세 조정하는 방법으로 보정할 수 있습니다.

아래 소개하는 제자리회전 실습은 버튼_A와 버튼_B를 사용해서 로봇의 회전속도를 미세 조정합니다. 이를 위해 로봇 보정은 servo_speed(70, 70) 회전속도와 2200ms 회전시간을 기본 값으로 시작합니다.

로봇의 제자리회전 정확도가 시계방향과 반시계방향에서 서로 차이를 보일 수 있지만 오차가 크지는 않습니다. 시계방향 보정과 반시계방향 보정을 각각 실행하면 더 정확한 로봇 회전동작을 만들 수 있습니다. 다만, 빨리 제자리회전 보정을 마치고 싶다면 시계방향 보정을 반시계방향 보정에 동일하게 적용해도 됩니다.

아래 예제7.7은 제자리회전 보정 방법입니다. 시계방향 보정은 반시계방향 보정에도 동일한 원리로 적용되는 점을 기억하고 다음 설명을 참고하기 바랍니다.

예제7.7 “로봇 제자리 회전 시계방향-반시계방향 보정하기”

```
# ch7_07_rotation_calibration_cw_ccw.py
from microbit_abot import *
from abot_rotation_calibrate import *

def main():
    abot = rotationCalibratedBot(13, 12)
    abot.servo_attachpins()
```

```

abot.servo_speed(0, 0)
sleep(500)

display.scroll("Ready")
sleep(500)

while True:
    # Clockwise calibration using the start_calibration function.
    # Here, we set the calibration speed (e.g., 70) and the base rotation time
    (2200 ms).
    abot.start_rotation(speed=70, direction=True, base_rotation_time=2200)
    sleep(1000)

    # Counter-Clockwise calibration.
    #abot.start_rotation(speed=-70, direction=False, base_rotation_time=2200)
    sleep(1000)

main()

```

로봇이 제자리에서 회전할 때 양쪽 서보의 속도 차이로 인해 발생할 수 있는 오차를 보정하는 과정의 예제입니다. 코드의 주요 흐름 중에서 보정원리를 중심으로 설명합니다.

로봇이 시계방향(CW) 제자리회전하는 경우

abot.start_rotation(speed=70, direction=True, base_rotation_time=2200) 함수를 사용합니다.

로봇이 반시계방향(CCW) 제자리회전하는 경우

abot.start_rotation(speed=-70, direction=False, base_rotation_time=2200) 함수를 사용합니다.

로봇의 시계방향 회전과 반시계방향 회전에서 회전시간 base_rotation_time은 모두 동일하게 사용해야 하지만, 속도 speed는 사용자가 입력하는 서로 다른 값을 사용할 수 있습니다.

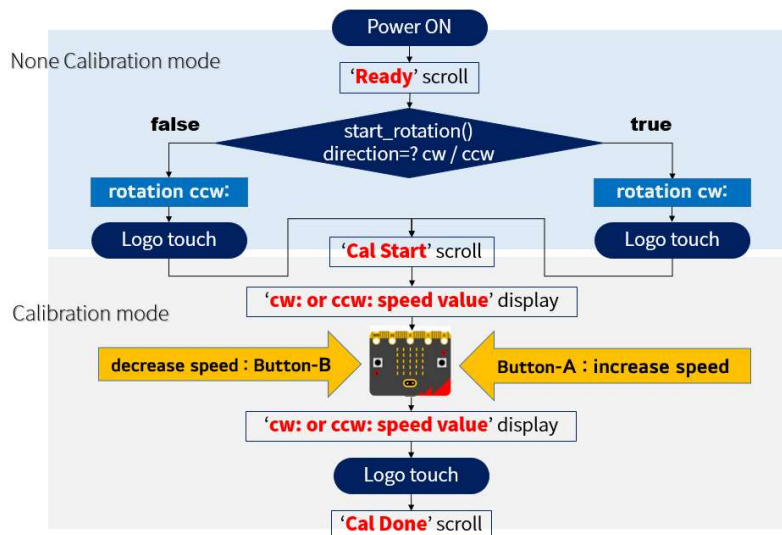


그림7.11 로봇의 제자리회전을 위한 실습 순서도

또한, 회전시간 변수를 기본 값(2200)으로 사용하는 경우 위 함수를 아래와 같이 간단히 사용할 수 있습니다.

```
abot.start_rotation(speed=70, direction=True)
abot.start_rotation(speed=-70, direction=False)
```

예제를 실행하고 마이크로비트 로고를 터치하면 rotationCalibratedBot가 실행되고 보정이 시작됩니다. 로봇의 제자리회전 보정 순서는 다음과 같습니다.

로봇 회전 보정에서 버튼_A와 버튼_B를 사용해서 한 바퀴 회전에 정확히 맞춰지도록 조절합니다. 버튼_A는 로봇이 회전하는 속도를 '1'씩 증가시키고, 버튼_B는 회전하는 속도를 '1'씩 감소시킵니다. 그리고 두 개의 버튼을 모두 누르면, 현재 보정을 진행 중인 회전방향의 서보모터 속도 값 '70'으로 초기화합니다.

아래에서 회전동작 보정작업 진행순서를 설명합니다.

① 로봇에 AA배터리 전원을 준비하고, 보정을 준비합니다.

② 예제5.16 “모듈 방식으로 텐덤로봇 직선주행 보정하기”코드로 직선주행 보정을 먼저 실행합니다. 만약 직선주행 보정을 시도할 만큼 시간 여유가 부족하면 생략해도 좋습니다. 직선주행 보정은 회전 보정 정확도를 높이기 위한 사전 동작입니다.

③ 실습 예제7.7을 마이크로비트에 플래싱합니다. 시계방향 회전동작 보정 예제를 실행하면 로봇이 약 한 바퀴 제자리회전을 합니다. 이때 세 번째 매개변수 base_rotation_time를 사용하지 않고, abot.start_rotation(70, True)함수를 사용할 수 있습니다. base_rotation_time은 사용자 설정 값으로 기본 값과 다르게 설정할 수 있습니다.

한 바퀴 제자리회전은 센서 없이 아래 그림의 바닥면 직선을 이용해서 로봇이 직선과 평행한지 눈으로 확인하면서 진행합니다.

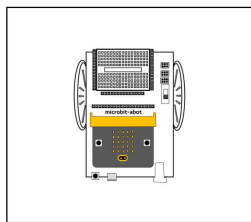


그림7.12 바닥면 직선을 기준으로 제자리회전 검사방법

④ 마이크로비트 로고를 터치하면 ‘Cal Start’가 디스플레이에 스크롤되고 현재의 회전 방향 속도 값이 표시됩니다. 앞선 제자리 회전 동작에서 한 바퀴에 모자라면 버튼_A(회전속도 증가)를 누르고 한 바퀴를 넘으면 버튼_B(회전속도 감소)를 누릅니다. 버튼_A와 버튼_B를 동시에 누르면 바퀴 속도가 기본 값으로 초기화됩니다.

전체 보정작업을 반복해서 정확한 한 바퀴 제자리회전이 되도록 속도 값을 선택합니다.

⑤ 로봇의 한 바퀴 회전이 정확하게 완성되면, 로고를 누르면 rotation_speeds.txt 파일에 현재 보정 값을 저장합니다. 파일에 저장된 값은 전원을 켜고 끄는 작업 이후에도 계속 로봇의 제자리회전 동작에서 자동으로 사용됩니다. 다만, 새로운 코드를 플래싱하면 마이크로

비트에 저장해둔 파일도 함께 지워지므로 재사용할 수 없습니다.

사용자 로봇의 보정작업을 진행해서 로봇 보정 값들을 이미 알고 있다면, 로봇의 제자리 회전 속도와 시간을 아래 외부함수로 입력하고 재사용할 수 있습니다.

```
abot.set_calibrated_speed(cw, ccw, base_rotation_time=None)
abot.set_calibrated_speed(70, -70, 2200)
abot.set_calibrated_speed(70, -70)
```

⑥ 반시계방향(is_cw=False)으로 로봇이 회전하는 보정도 전체과정을 반복하면 됩니다.

제자리회전 보정이 완료되면, 로봇을 원하는 각도만큼 회전하기 위해 아래의 함수를 사용합니다.

```
abot.rotate_degrees(90)    # 90도 시계 방향 회전
abot.rotate_degrees(-90)   # 90도 반시계 방향 회전
```

회전 보정을 완료한 로봇을 사용해서 사용자가 원하는 각도로 로봇을 회전시키는 것은 간단히 실행됩니다. 아래 예제7.8 코드를 사용하면, 로봇의 회전 각도를 매개변수로 사용하는 함수로 로봇 회전을 제어합니다.

예제7.8 “회전 보정을 완료한 로봇이 임의의 각도로 회전하기”

```
# ch7_08_rotation_calibrated_robot_arbitrary_rotation.py

from microbit_abot import *
from abot_rotation_calibrate import *

def main():
    abot = rotationCalibratedBot(13, 12)
    abot.servo_attachpins()
    abot.servo_speed(0, 0)
    sleep(500)

    abot.set_calibrated_speed(73, -71, 2200)
    display.scroll("Ready")
    sleep(500)

    while True:
        # Rotate clockwise by 90 degrees
        print("Rotating clockwise by 90 degrees")
        abot.rotate_degrees(90)
        sleep(1000) # Wait for 1 second

        # Rotate counter-clockwise by 90 degrees
        print("Rotating counter-clockwise by 90 degrees")
        abot.rotate_degrees(-90)
        sleep(1000) # Wait for 1 second

if __name__ == "__main__":
    main()
```

예제를 실행하면 로봇이 시계 방향으로 90도 회전하고 1초 대기한 후, 반시계 방향으로 90도 회전하고 다시 1초 대기하는 동작을 반복합니다. 예제에서 소개된 보정 값들은 사용자 로봇 값과 다를 수 있습니다.

```
if __name__ == "__main__":
```

파이썬 스크립트의 표준 구조를 따르기 위해 main() 함수를 호출할 때 if __name__ == "__main__": 조건을 사용합니다. 이는 모듈이 직접 실행될 때만 main() 함수가 호출되도록 보장합니다.

(2) 로봇에 장착된 초음파센서 하드웨어 정렬하기

여기서는 로봇에 장착된 초음파센서를 90도 방향으로 정렬하는 것이 목표입니다. 동시에 초음파센서로 전방 로봇을 추적하는 로봇을 만들기 위한 선행 작업입니다.

로봇의 초음파센서가 90도 각도를 가르키는 코드(servo_angle(90))일 때, 실제로 초음파센서가 로봇 정면을 바라보고 있는지는 코드 작성의 정확도를 유지하고 관리하는 데 중요합니다. 아래 그림7.15는 하드웨어 정렬 작업하기 전에는 약간 기울어진 방향으로 센서가 전방을 바라보지만, 아래 그림처럼 하드웨어 정렬 작업 이후에는 센서가 거의 90도 정면 방향을 로봇이 바라봅니다.

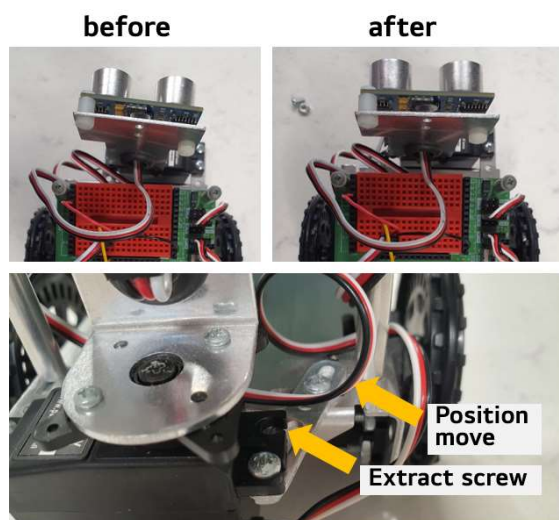


그림7.13 로봇의 초음파센서 90도 방향 정렬방법

위의 초음파센서 하드웨어 정렬방법은 아래 예제7.9를 실행하고 위의 그림7.15를 참고해서 센서의 하드웨어 정렬작업을 완료하기 바랍니다. 초음파센서를 지지하는 두 개의 스크루 중에서 하나를 제거하면 센서가 바라보는 방향을 미세 수정할 수 있습니다.

예제7.9 “로봇의 초음파센서 90도 방향으로 하드웨어 정렬하기”

```
# ch7_09_Ultrasonic_sensor_90_align.py  
  
from microbit import *  
from microbit_abot import *  
  
abot = bot(10)  
abot.servo_angle(90)
```



```
sleep(1000)
```

(3) 순차 제어로 따라가는 로봇 만들기

지금부터 소개할 간단한 사례는 전방의 리더 로봇을 초음파센서 스캔으로 따라가면서 이동하는 주종방식의 팔로워 로봇을 구현하는 것입니다. 이런 시도는 앞 절에서 소개한 “(4) PID 페루프 동적제어 방법으로 장애물 회피하기” 실습의 또 다른 접근입니다.

‘전방 로봇을 회피하는 동작’이나 반대로 ‘전방 로봇을 따라가는 동작’은 초음파 센서로 전방의 로봇을 스캔하고 로봇을 식별하는 과정은 동일하지만, 이후 단계에서 회피 동작은 일회성 판단과 동작으로 종료되지만 따라가는 동작은 계속 판단을 반복하고 해당 판단에 반응해야 하는 차이가 있습니다.

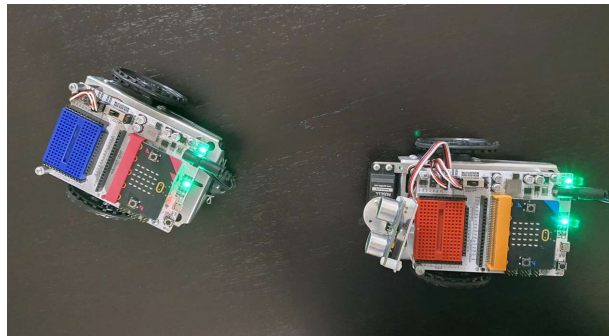


그림 7.14 전방 로봇을 따라가는 초음파 센서 팔로워 로봇 모습

리더-팔로워 패턴에서, 전방의 리더 로봇을 초음파센서 스캔으로 따라가면서 이동하는 주종방식의 팔로워 로봇을 구현하는 방법에 대하여 5가지 제어방식과 개요를 간단히 소개합니다.

[1] 순차 제어 (Sequential Control)

제어 단계가 순차적으로 진행됩니다. 먼저 거리 오차에 대한 PID 제어로 목표 거리 유지에 집중한 후, 그 결과를 바탕으로 조향 제어를 적용합니다. 거리 오차가 크면 우선 안전 거리를 확보하는 것이 중요하다는 전제에서 출발합니다.

장점은 단순한 구조로 구현 및 디버깅이 용이하고 큰 거리 오차 상황에서 우선적으로 대응할 수 있습니다.

단점은 두 제어가 완전히 독립적으로 반응하지 않아, 조향 반응이 후속 처리되어 늦게 나타날 수 있습니다.

[2] 병렬 제어 및 가중치 합산 (Parallel Control with Weighted Summation)

거리 제어와 조향 제어를 동시에 계산한 후, 각각의 출력을 미리 정한 가중치로 합산해 최종 제어 신호를 만듭니다. 두 제어 목표(거리와 각도)를 동시에 고려함으로써, 둘 사이의

상호 작용을 반영하려는 접근입니다.

장점은 동시에 두 가지 오차를 보상할 수 있어, 복합적인 상황에 빠르게 대응 가능합니다. 그리고 거리와 조향에 대한 비중을 상황에 맞게 조정할 수 있습니다.

단점은 적절한 가중치 설정이 어렵고, 두 제어 신호가 상호 간섭하여 예측하지 못한 동작이 발생할 수 있습니다.

[3] 계층형(캐스케이드) 제어 (Hierarchical or Cascade Control)

외부 루프가 전반적인 이동(속도)을 결정하고, 내부 루프가 빠른 조향 반응을 담당합니다. 거리 제어와 조향 제어의 시간 상수가 다르다는 점을 이용하여, 느린(안정적인) 외부 루프와 빠른(세밀한) 내부 루프로 분리합니다.

장점은 각 루프에 최적화된 튜닝이 가능해, 안정성과 응답 속도를 동시에 잡을 수 있습니다. 동시에 제어 목표가 명확하게 계층화되어 혼합 신호에 대한 해석이 용이합니다.

단점은 두 루프 간의 인터페이스 설계가 복잡하여, 외부 루프의 결과가 내부 루프에 적절히 반영되도록 신중한 튜닝이 필요합니다.

[4] 상황에 따른 적응형 제어 (Adaptive or Switching Control)

로봇의 현재 상태나 오차 크기에 따라 제어 전략(거리 우선 vs. 조향 우선)을 전환합니다. 환경이나 목표와의 거리에 따라 서로 다른 제어 모드를 적용함으로써, 변화하는 상황에 유연하게 대응할 수 있도록 설계합니다.

장점은 상황에 맞는 제어 우선순위를 동적으로 조정할 수 있어, 복합 환경에서도 유연하게 작동합니다.

단점은 전환 조건과 임계값 설정이 까다로워, 전환 시의 부드러운 인터페이스 보장이 어려울 수 있습니다.

[5] 디커플링 (Decoupled Control)

전진/후진(거리 제어)과 회전(조향 제어)을 별도의 독립 채널로 분리한 뒤, 마지막에 두 신호를 합산하는 방식입니다. 거리와 조향의 제어 신호를 최대한 서로 독립적으로 처리하여 상호 간섭을 최소화하는 접근입니다.

장점은 각 제어 요소에 대해 독립적으로 튜닝할 수 있어, 세밀한 제어가 가능합니다. 동시에 두 제어의 물리적 특성이 다를 때 각각에 맞는 최적 제어가 가능합니다.

단점은 실제 시스템에서는 완벽한 분리가 어려워, 최종 합산 단계에서 미세 조정이 필요합니다.

소개한 5가지 제어방식 이외에도 더 많은 창의적 수단이 존재하지만, 이 절에서는 5가지 제어방식에 기초해서 소개할 예정입니다. 또 다른 접근 방식은 독자 스스로 인공지능의 도움을 받아서 코드를 작성하고 로봇 제어성능 차이를 비교해보기 바랍니다.

리더-팔로워 패턴으로 로봇 따라가기를 완성하려면, 리더 로봇과 팔로워 로봇 코딩이 기본적으로 필요합니다.

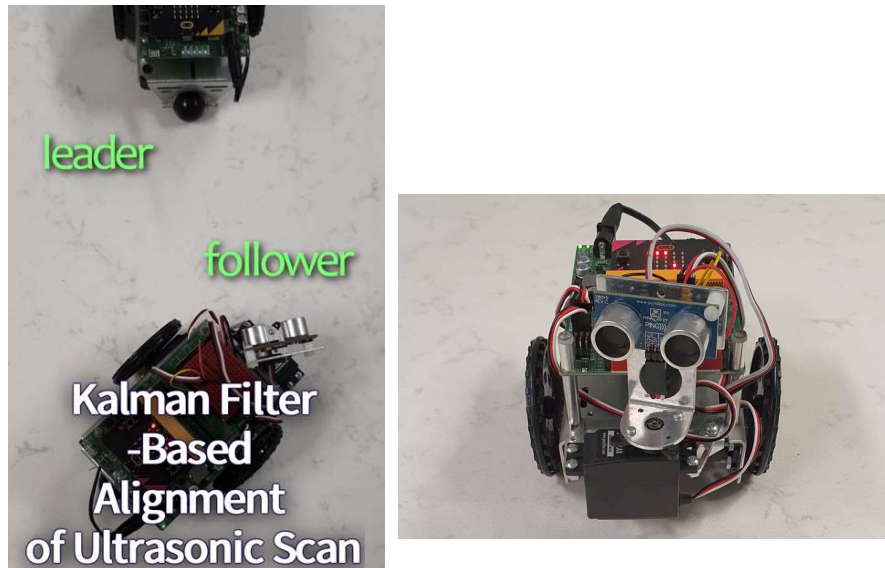


그림7.15 칼만필터로 초음파 스캔 데이터 기반의 로봇 만들기

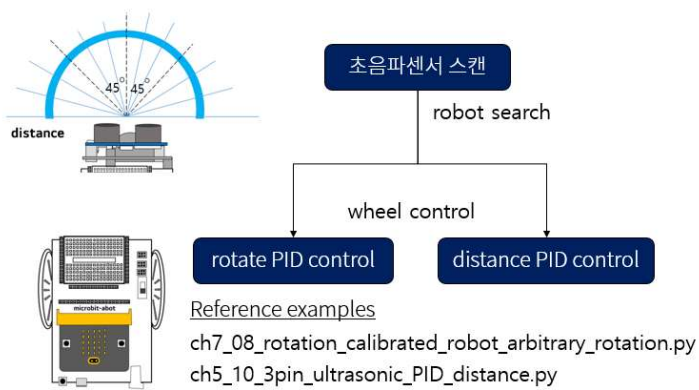


그림7.16 초음파센서 스캔 방식의 팔로워 로봇 PID제어 개념도

리더 로봇은 x-y축 가속도 데이터로 로봇을 제어하는 예제7.1과 예제7.2 코드를 사용하여 리더 로봇의 다양한 동작을 표현할 수 있습니다. 이 방식은 리더 로봇 코딩과 원격 제어용 마이크로비트 코딩으로 구현되어 있습니다.

팔로워 로봇은 초음파 센서 스캔 방식으로 칼만필터를 적용해서 로봇을 감지하고, PID제어방법으로 속도제어와 방향제어를 모두 적용해서 전방의 로봇을 부드럽게 따라갑니다. 여기서는 팔로워 로봇 코딩에 대해서만 구체적으로 설명합니다.

전방 로봇을 따라가는 로봇을 구현하려면 초음파센서 스캔으로 전방 로봇을 안정적으로 찾아야 하고, 찾은 다음 로봇 진행방향을 전방 로봇으로 향하게 자세 정렬하는 동작과 전방 로봇과의 거리를 일정하게 유지하기 위한 전진 또는 후진동작을 만들고 연결하면 됩니다.

먼저 초음파센서 스캔으로 전방 로봇을 안정적으로 찾기 위해 칼만 필터(Kalman Filter)를 사용합니다. 칼만 필터는 센서 데이터에 포함된 노이즈를 줄여 실제 상태(예: 물체의 위치, 각도 등)를 추정하는 알고리즘입니다. 센서 데이터가 불안정하거나 노이즈가 큰 상황에서, 칼만 필터를 사용하면 보다 부드러운 추정치를 얻을 수 있습니다. 다만, 칼만 필터 자체는 '제어 알고리즘'이 아니라 '상태 추정 알고리즘'입니다.

(Notice) -----

칼만 필터와 PID 제어의 관점 차이

칼만 필터(Kalman Filter)는 노이즈가 포함된 센서 데이터로부터 시스템의 상태(예: 위치, 속도 등)를 최적 추정하기 위한 알고리즘으로, 제어 알고리즘(Control Algorithm)과는 목적과 역할이 근본적으로 다릅니다.

- 칼만필터(상태 추정 알고리즘): 실제 시스템의 내부 상태를 정확하게 알 수 없는 상황에서, 노이즈가 섞인 관측 값(예를 들어, 초음파 센서로 측정한 거리 데이터)과 시스템의 동적 모델(예: 로봇의 운동 모델)을 결합하여 현재 상태를 "추정"합니다.

- PID제어(제어 알고리즘): 추정된 상태나 목표 상태를 바탕으로 시스템이 원하는 동작을 할 수 있도록 외부에서 명령(제어 입력)을 생성합니다. 제어 알고리즘은 상태 추정기가 제공한 정보를 활용하지만, 그 자체가 상태를 추정하지 않습니다.

초음파 센서 스캔으로 전방 로봇을 찾기 위한 칼만 필터는 “어디에 있는가?” 또는 “어떤 상태인가?”를 추정합니다. 초음파 센서로부터 얻은 불완전하고 노이즈가 많은 데이터를 바탕으로 로봇의 위치나 속도, 방향 등 상태에 대한 확률적 예측을 제공합니다.

칼만 필터로 추정하는 전방 로봇에 대한 데이터를 기초로 제어 알고리즘은 “어떻게 해야 원하는 목표로 갈 수 있는가?”를 결정합니다. 예를 들어, 추정된 상태와 목표 상태 사이의 오차를 줄이기 위해 어떤 입력(예: 모터 속도, 방향 조정 등)을 실행할지를 결정합니다.

요약하면, 초음파 센서 스캔 데이터를 통해 전방의 로봇 위치를 추정하는 작업에서 칼만 필터는 해당 위치와 관련된 불확실한 정보를 정제하여, 추후 제어 알고리즘이 보다 정확한 상태 정보를 바탕으로 각도 제어의 안정성과 응답성을 동시에 확보하여 로봇을 올바르게 제어할 수 있는 최적의 방법으로 선택합니다.

전방 로봇의 각도와 거리가 확인되면, 제자리회전으로 자세를 정렬하고, 거리제어로 전방 로봇과의 거리를 일정하게 유지하기 위해 PID제어를 실행합니다. 자세제어와 거리제어는 순차적 제어를 사용합니다. 이런 목적의 로봇 설계 개념을 아래 표7.2에서 요약합니다.

표7.2 초음파센서로 따라가는 로봇의 설계 조건들

탐색제어	- 물체가 없는 경우 20~160도 범위 탐색 - 탐색 각도 간격은 2 도 - 물체 인식거리는 50cm - 탐색 최대거리는 1m 로 제한 - 스캔 동안 디스플레이에 "C" 출력 - 칼만 필터를 통한 측정 노이즈 보정 및 각도 추정 : 초기 추정 값은 90°(STEERING_TARGET)을 기준 : 초기 오차 분산(15), 측정 노이즈(R=15), 프로세스 노이즈(Q=0.2)를 설정
자세제어	- 로봇의 정렬 및 회전제어는 PID제어에서 PD(비례-미분)제어를 사용하고, 목표는 전방 90°(STEERING_TARGET) 정렬 - 계산된 회전각이 임계치($\pm 8^\circ$) 이상일 때만 실제 회전 명령 - 조향제어 동안 디스플레이에 "R" 출력
거리제어	- 전방 90°(STEERING_TARGET) 거리측정 데이터로 PID제어에서 PD(비례-미분)제어를 사용해서 거리유지 - 물체와의 일정한 거리는 20cm - 거리제어 동안 디스플레이에 "D" 출력

따라가는 로봇을 만들기 위해 두 가지 단계로 나누어 실습들을 소개합니다. 단계1은 초음파센서로 칼만 필터를 사용해서 전방 로봇 위치를 탐색하고, 탐색된 로봇 방향으로 제자리 회전 자세 제어하는 코드입니다.

이어서 단계2는 단계1의 코드에 자세제어와 거리제어를 순차제어 방식으로 통합해서 전방 로봇의 방향으로 따라가는 코드를 소개합니다.

(주의할 점) 로봇의 초음파 센서 시선방향에 집안의 가구나 벽면, 여러 가지 물건들이 놓여 있으면 전방의 로봇을 식별하는 데 어려움이 생길 수 있다는 점을 기억하기 바랍니다.

단계1 : 초음파센서 스캔으로 로봇탐색과 자세정렬 하기

아래 코드에서 직선주행 오프셋과 제자리회전 보정 값은 사용자 환경에 맞추어야 합니다.

예제7.10 “칼만 필터로 자세 정렬하는 초음파센서 스캔 로봇”

```
# ch7_10_Kalman_filter_Steering_Control.py

from microbit import *
from microbit_abot import *          # Provides basic robot control functions
from ultrasonic import *             # Provides ultrasonic sensor functions
from abot_calibrated import *        # Provides calibrated in-place rotation and servo control functions

# =====
# 1. Constants and Initial Setup
# =====
MIN_ANGLE = 20                       # Lower bound for servo scanning (°)
MAX_ANGLE = 160                      # Upper bound for servo scanning (°)
STEP = 2                             # Increment (°) per iteration for continuous scan
MAX_SENSOR_DISTANCE = 100            # Maximum measurable distance (cm) for
```

```

ultrasonic sensor
OBJECT_THRESHOLD = 50          # Distance threshold (cm) to consider an object
detected
STEERING_TARGET = 90           # Target forward direction (°)
STEERING_KP = 0.3              # Proportional constant for steering control
STEERING_KD = 0.4              # Derivative constant for steering control

# Kalman filter parameters for estimating the object's angle:
kalman_estimate = STEERING_TARGET # Initial estimate (start at forward direction)
kalman_error = 15                # Initial error variance (tuning parameter)
R = 15                           # Measurement noise variance (tuning parameter)
Q = 0.2                           # Process noise variance (tuning parameter)

# =====
# 2. Create Instances for Robot, Sensor, and Servo
# =====
# Create a robot instance for calibrated rotation and straight motion
abot = CalibratedMotionBot(13, 12)
abot.servo_attachpins()          # Attach control pins
abot.set_left_speed_offset(-5)
abot.set_calibrated_speed(65, -66, 2200)
abot.servo_speed_calibrated(0, 0) # Start with the robot stopped

# Setup the ultrasonic sensor servo on pin 10
scan = bot(10)
ultrasonic_sensor = Ultrasonic(9)

display.scroll("Ready")
sleep(500)

# =====
# 3. Steering Control Function (PD Control)
# =====
prev_error = 0 # Initialize the previous error for derivative calculation
def steering_control(detection_angle):
    """
    [Steering Control - PD Control]
    Rotates the robot based on the error between the steering target (90°)
    and the estimated object angle. Both proportional and derivative terms are used.
    """
    global prev_error
    # Optionally, show "R" on the display to indicate a rotation command is being
    executed.
    display.show("R")
    error = STEERING_TARGET - detection_angle
    derivative = error - prev_error
    prev_error = error
    rotation_angle = STEERING_KP * error + STEERING_KD * derivative

    # Execute rotation if the signal is significant to avoid jitter.
    if abs(rotation_angle) > 8:
        abot.rotate_degrees(rotation_angle)
        sleep(100)

# =====
# 4. Simple 1D Kalman Filter Function
# =====
def kalman_update(measurement, prev_estimate, prev_error, R, Q):
    """
    Performs a basic Kalman filter update.

    Parameters:
        measurement - current measurement (servo angle at which object is detected)
        prev_estimate - previous estimate of the object angle
        prev_error - previous error variance
        R - measurement noise variance
        Q - process noise variance

    Returns:
        (new_estimate, new_error) - updated estimate and error variance.
    """
    # Compute the Kalman gain.

```

```

kalman_gain = prev_error / (prev_error + R)
# Update the estimate with the new measurement.
new_estimate = prev_estimate + kalman_gain * (measurement - prev_estimate)
# Update the error variance.
new_error = (1 - kalman_gain) * prev_error + Q
return new_estimate, new_error

# =====
# 5. Continuous Scanning Main Loop
# =====
# In continuous scanning, the servo sweeps from MIN_ANGLE to MAX_ANGLE and back.
angle = MIN_ANGLE
scanning_direction = STEP # Positive means increasing angle; negative reverses the
scan.

while True:
    # Set the servo to the current angle
    scan.servo_angle(angle)
    sleep(15)

    # Read the ultrasonic sensor's current distance measurement
    sensor_reading = ultrasonic_sensor.distance('cm')
    if sensor_reading is None:
        measured_distance = MAX_SENSOR_DISTANCE
    else:
        measured_distance = int(sensor_reading)
        if measured_distance > MAX_SENSOR_DISTANCE:
            measured_distance = MAX_SENSOR_DISTANCE

    # Display "C" to indicate continuous scanning mode
    display.show("C")

    # If the measured distance is below the threshold, update the Kalman filter with
    the current angle.
    if measured_distance <= OBJECT_THRESHOLD:
        # Using the current servo angle as the measurement for the object's position.
        kalman_estimate, kalman_error = kalman_update(angle, kalman_estimate,
kalman_error, R, Q)
        # For debugging purposes, print the details to the serial console.
        print("Detected at", angle, "° with distance", measured_distance, "cm; Kalman
estimate =", kalman_estimate)

        # Use the filtered angle to update the robot's orientation.
        steering_control(kalman_estimate)
    else:
        # Optionally, you could choose to decay the Kalman filter if no detection is
        available.
        pass

    # Update the servo angle continuously:
    angle += scanning_direction
    if angle >= MAX_ANGLE:
        angle = MAX_ANGLE
        scanning_direction = -STEP # Reverse direction at the upper bound.
    elif angle <= MIN_ANGLE:
        angle = MIN_ANGLE
        scanning_direction = STEP # Reverse direction at the lower bound.

    sleep(10) # Short pause to yield processing time.

```

코드에서는 칼만 필터를 통해 센서의 불확실성을 보정하여 정확한 객체 방향을 추정하고, PD 제어를 통해 그 방향으로 로봇이 회전하도록 제어합니다. 이 구조 덕분에 노이즈가 많은 초음파 센서 환경에서도 안정적으로 목표를 탐색하고 정렬할 수 있습니다.

로봇의 위치 탐색 (칼만 필터)

초음파 센서와 서보 모터를 이용하여 전방의 객체(예, 다른 로봇)를 감지하고, 센서 노이즈나 측정 오차를 줄이기 위해 칼만 필터를 사용하여 객체의 정확한 각도(위치)를 추정합니다.

- 측정 노이즈(R): 센서 측정의 불확실성을 나타내며, 실제 값과의 차이를 줄여주기 위한 보정값입니다.
- 프로세스 노이즈(Q): 시스템 동작 자체의 오차를 반영하는 값입니다.
- 오차 분산 (kalman_error): 추정치에 대한 불확실성의 정도를 나타냅니다.

코드에서 초기 추정치는 로봇이 전방(90°)을 바라보고 있다고 가정하고, 센서의 불확실성을 감안하여 오차 값을 설정합니다. 측정 시점은 서보 모터가 MIN_ANGLE (20°)부터 MAX_ANGLE (160°)까지 일정한 간격으로 회전하며 각도에 따른 초음파 거리 측정을 실행합니다.

```
sensor_reading = ultrasonic_sensor.distance('cm')
```

만약 측정된 거리가 OBJECT_THRESHOLD (50cm) 이하이면, 객체가 감지된 것으로 보고 현재 서보 모터의 각도(angle)를 측정값으로 사용합니다.

칼만 필터 업데이트 함수 (kalman_update) 원리

[1] 칼만 이득 (kalman_gain) 계산:

현재 오차 분산과 측정 노이즈를 바탕으로 측정값에 얼마나 “믿음을” 둘지 결정합니다.

[2] 새로운 추정치 (new_estimate) 계산:

이전 추정치에 측정값과의 차이에 칼만 이득을 곱한 값을 더해 보정된 추정치를 구합니다.

[3] 오차 분산 (new_error) 업데이트:

추정치에 불확실성을 갱신합니다. 시간에 따른 오차의 변화와 시스템의 노이즈를 반영하여 값을 조절합니다.

탐색된 로봇으로 제자리 회전하는 제어 (PD 제어)

칼만 필터를 통해 추정된 객체의 위치(각도)를 이용하여 로봇이 해당 방향으로 제자리 회전하도록 합니다. 제어의 목표(STEERING_TARGET)는 전방을 나타내는 90°로 설정됩니다.

PD (비례-미분) 제어는 비례(P, Proportional) 항에서 현재 오차(목표 각도와 추정 각도의 차이)에 비례하여 회전각을 결정하고, 미분(D, Derivative) 항에서 이전 오차와의 변화량(즉, 오차의 변화 속도)을 반영하여 회전 명령에 관성을 부여하거나 과도한 변화로 인한 흔들림을 줄입니다.

PD제어 함수의 원리

[1] 오차 계산:

목표각(90°)과 칼만 필터를 통해 얻은 추정각(detection_angle) 사이의 차이를 계산합니다.

[2] 미분 항 계산:

이전 오차(prev_error)와 현재 오차의 차이를 계산하여 변화량을 구합니다.

[3] 회전각 명령 결정:

비례 상수(STEERING_KP)와 미분 상수(STEERING_KD)를 적용하여 최종 회전각(rotation_angle)을 결정합니다. 만약 이 회전각이 8° 이상이면 회전을 실행하여 로봇이 대상 객체 방향으로 정렬되도록 합니다.

단계2 : 초음파센서 탐색으로 자세제어와 거리제어 통합하기

앞서 소개한 단계1의 코드에 거리유지 제어코드를 추가해서 통합한 예제입니다. 아래 코드로 전방의 로봇을 찾아서 일정한 거리(여기서는 20cm)로 접근하는 로봇을 경험하기 바랍니다.

예제7.11 “순차제어로 따라가는 초음파센서 로봇 만들기”

```
# ch7_11_sequential_controlled_follower.py

from microbit import *
from microbit_abot import *          # Provides basic robot control functions
from ultrasonic import *             # Provides ultrasonic sensor functions
from abot_calibrated import *        # Provides calibrated in-place rotation & servo control functions

# =====
# 1. Constants and Initial Setup
# =====
MIN_ANGLE = 20                       # Minimum servo angle (°)
MAX_ANGLE = 160                      # Maximum servo angle (°)
STEP = 2                             # Servo angle increment (°) per iteration
MAX_SENSOR_DISTANCE = 100            # Maximum measurable distance (cm) for the ultrasonic sensor
OBJECT_THRESHOLD = 50                 # Threshold distance (cm) to consider an object detected
STEERING_TARGET = 90                 # Target forward direction (°)
STEERING_KP = 0.3                    # Steering PD: proportional gain
STEERING_KD = 0.4                    # Steering PD: derivative gain

# Kalman filter parameters (for angle estimation)
kalman_estimate = STEERING_TARGET    # Initial estimate (starts at 90°)
kalman_error = 15                     # Initial error variance (tuning parameter)
R = 15                                # Measurement noise variance (tuning parameter)
Q = 0.2                               # Process noise variance (tuning parameter)

# Distance control parameters
TARGET_DISTANCE = 20                  # Desired front distance (cm)
DISTANCE_KP = -10                     # Distance PD: proportional gain (negative for forward drive)
# For improved sensitivity, consider adjusting the derivative term for distance control.
DISTANCE_KD = -2                      # Distance PD: derivative gain

# =====
# 2. Create Instances for Robot, Sensor, and Servo
# =====
abot = CalibratedMotionBot(13, 12)    # Attach servo control pins
```

```

abot.set_left_speed_offset(-5)
abot.set_calibrated_speed(65, -66, 2200)
abot.servo_speed_calibrated(0, 0) # Initially stopped

# Create instances for the servo motor and ultrasonic sensor
scan = bot(10)
ultrasonic_sensor = Ultrasonic(9)

display.scroll("Ready")
sleep(500)

# =====
# 3. Steering Control Function (PD Control)
# =====
prev_error = 0 # Previous error for derivative calculation
def steering_control(detection_angle):
    """
    Steering (PD Control):
    Rotates the robot in place based on the error between the target
    forward direction (90°) and the estimated object's angle (from the Kalman filter).
    Uses both proportional and derivative terms.
    """
    global prev_error
    display.show("R") # Indicate rotation phase
    error = STEERING_TARGET - detection_angle
    derivative = error - prev_error
    prev_error = error
    rotation_angle = STEERING_KP * error + STEERING_KD * derivative
    # Avoid jitter: only apply if control signal is significant
    if abs(rotation_angle) > 8:
        abot.rotate_degrees(rotation_angle)
        sleep(100)

# =====
# 4. Simple 1D Kalman Filter Function
# =====
def kalman_update(measurement, prev_estimate, prev_err, R, Q):
    """
    Basic Kalman filter update:
    - measurement: current servo angle measurement
    - prev_estimate: previous angle estimate
    - prev_err: previous error variance
    - R: measurement noise variance
    - Q: process noise variance
    Returns: (new_estimate, new_error)
    """
    kalman_gain = prev_err / (prev_err + R)
    new_estimate = prev_estimate + kalman_gain * (measurement - prev_estimate)
    new_error = (1 - kalman_gain) * prev_err + Q
    return new_estimate, new_error

# =====
# 5. Distance Control Function (Using PD Control)
# =====
prev_distance_error = 0
def distance_control():
    """
    Distance Control (PD):
    Fixes the ultrasonic sensor at 90° and continuously measures the distance to the
    front object.
    A PD controller (using both proportional and derivative terms) maintains
    TARGET_DISTANCE (20cm).
    If the measured distance exceeds OBJECT_THRESHOLD (e.g., 70cm), the object is
    considered lost and control stops.
    """
    # Directly show a display indicator for distance control
    display.show("D")
    # Fix the servo at 90° (forward)
    scan.servo_angle(90)
    sleep(50)

    global prev_distance_error

```

```

prev_distance_error = 0

while True:
    dist = ultrasonic_sensor.distance('cm')
    if dist is None:
        measured = MAX_SENSOR_DISTANCE
    else:
        measured = int(dist)
        if measured > MAX_SENSOR_DISTANCE:
            measured = MAX_SENSOR_DISTANCE

    if measured > OBJECT_THRESHOLD: # Object lost
        abot.servo_speed_calibrated(0, 0)
        break

    # Calculate the error and derivative (for PD control)
    error = TARGET_DISTANCE - measured
    derivative = error - prev_distance_error
    prev_distance_error = error

    speed_adjust = DISTANCE_KP * error + DISTANCE_KD * derivative
    # Limit the control signal
    if speed_adjust > 200:
        speed_adjust = 200
    elif speed_adjust < -200:
        speed_adjust = -200

    # Drive both motors with opposite signs for forward motion
    abot.servo_speed_calibrated(int(speed_adjust), int(-speed_adjust))
    sleep(30)

# =====
# 6. Main Loop (Continuous Scanning)
# =====
angle = MIN_ANGLE
scanning_direction = STEP # Positive: increasing; negative: decreasing

while True:
    # Set the servo to the current angle and scan
    scan.servo_angle(angle)
    sleep(15)

    # Measure distance using ultrasonic sensor
    sensor_reading = ultrasonic_sensor.distance('cm')
    if sensor_reading is None:
        measured_distance = MAX_SENSOR_DISTANCE
    else:
        measured_distance = int(sensor_reading)
        if measured_distance > MAX_SENSOR_DISTANCE:
            measured_distance = MAX_SENSOR_DISTANCE

    display.show("C") # Indicate scanning mode

    # If an object is detected (i.e., measured distance is below threshold)
    if measured_distance <= OBJECT_THRESHOLD:
        # Update the Kalman filter using the current servo angle
        kalman_estimate, kalman_error = kalman_update(angle, kalman_estimate,
        kalman_error, R, Q)
        print("Detected at", angle, "° with distance", measured_distance, "cm; Kalman
        estimate =", kalman_estimate)

        # Apply steering control using the Kalman estimate
        steering_control(kalman_estimate)

        # When the estimated angle is very close to forward (within ±2° of 90°),
        # assume the front robot is aligned and switch to distance control mode.
        if abs(kalman_estimate - STEERING_TARGET) < 2:
            distance_control()
            # When distance control ends (e.g., object is lost), resume scanning.
        else:
            # Continue scanning if no object is detected.
            pass

```

```

# Update the servo angle for the next scan iteration.
angle += scanning_direction
if angle >= MAX_ANGLE:
    angle = MAX_ANGLE
    scanning_direction = -STEP # Reverse direction at the upper bound.
elif angle <= MIN_ANGLE:
    angle = MIN_ANGLE
    scanning_direction = STEP # Reverse direction at the lower bound.

sleep(10) # Short delay for processing time.

```

거리제어 (전방 로봇과의 20cm 간격 유지)

TARGET_DISTANCE를 20cm로 설정하여, 로봇이 전방의 다른 로봇과 일정 거리(20cm)를 유지합니다. 거리유지는 칼만 필터와 PD 제어를 통해 로봇이 대상의 방향으로 정렬된 후, 초음파 센서로 읽은 거리를 기준으로 거리제어 함수가 호출되어 속도를 조절합니다.

거리 오차는 매 스캔 시 초음파 센서를 통해 측정된 거리를 읽어, 목표 거리와 비교합니다.

(4) 조향과 거리유지 데이터 합산으로 전방로봇 따라가기

실시간 제어용 로봇 만들기를 위해 캐스케이드와 디커플링 제어 방법을 사용할 수 있습니다. 다만, 현재의 마이크로비트와 아두이노 그리고 초음파 센서 스캔주기가 로봇동작을 실시간으로 충분히 반응하도록 하는 데 한계가 있습니다.

여기서는 더 간단한 방법으로 로봇의 조향제어와 거리유지제어 데이터를 합산하여 전방 로봇을 따라가는 예제를 소개합니다.

제어의 원리는 스캔 중에도 “방향 제어”와 “거리 제어” 신호를 동시에 계산해 양쪽 바퀴 속도로 합산(혹은 합·차 연산)하는 방식입니다. 아래 예제는 이런 목적을 달성하기 위한 가장 기초적인 코드이므로, 부드러운 로봇동작을 위해 새로운 아이디어 코드가 더 포함될 수 있습니다. 다만 아래 예제에서는 기본 코드의 명확한 전달을 위해 추가 코드들은 생략합니다.

예제7.12 “동적 윈도우 스캔과 PD 제어를 이용한 초음파 추종 로봇”

```

# ch7_12_ultrasonic_dynamic_window_pd_follower.py

from microbit import *
from microbit_abot import *
from ultrasonic import *
from abot_calibrated import *

# =====
# 1. 설정 상수
# =====
MIN_ANGLE = 20
MAX_ANGLE = 160
STEP = 2
MAX_DIST = 100
OBJ_THRESH = 50

```

```

STEER_TARGET    = 90
STEER_KP, STEER_KD = 0.3, 0.4

TARGET_DIST     = 20
DIST_KP, DIST_KD = -10, -2

WINDOW          = 45 # 동적 윈도우 반폭 (°)
LOST_LIMIT      = 3  # 연속 감지 실패 후 스캔 모드 진입

# 칼만 필터 파라미터
kalman_est = STEER_TARGET
kalman_err = 15
R, Q = 15, 0.2

# =====
# 2. 유틸리티
# =====
def kalman_update(meas, est, err, R, Q):
    K = err / (err + R)
    new_est = est + K * (meas - est)
    new_err = (1 - K) * err + Q
    return new_est, new_err
def clamp(x, lo, hi):
    return lo if x < lo else hi if x > hi else x

# =====
# 3. 인스턴스 초기화
# =====
abot = CalibratedMotionBot(13, 12)
abot.servo_attachpins()
abot.set_left_speed_offset(-5)
abot.set_calibrated_speed(65, -66, 2200)
abot.servo_speed_calibrated(0, 0)

scan_servo = bot(10)
ultra      = Ultrasonic(9)
display.scroll("Ready")
sleep(500)

# =====
# 4. 상태 변수
# =====
scan_min, scan_max = MIN_ANGLE, MAX_ANGLE
angle, direction = scan_min, STEP
lost_count = 0
last_left = 0
last_right = 0
last_angle = STEER_TARGET
prev_steering_e = 0
prev_dist_e = 0

# =====
# 5. 메인 루프
# =====
while True:
    # — 서보 위치: 스캔 모드일 때만 sweep, 아니면 마지막 검출 각도 고정
    if lost_count >= LOST_LIMIT:
        scan_servo.servo_angle(angle)
    else:
        scan_servo.servo_angle(int(last_angle))
    sleep(15)
    # — 거리 측정
    d = ultra.distance('cm')
    meas = MAX_DIST if d is None else min(int(d), MAX_DIST)
    if meas <= OBJ_THRESH:
        # — 객체 검출!
        lost_count = 0

        # 1) 칼만 필터로 각도 추정
        kalman_est, kalman_err = kalman_update(angle, kalman_est, kalman_err, R, Q)
        last_angle = kalman_est

```

```

# 2) 방향 PD
err_s = STEER_TARGET - kalman_est
der_s = err_s - prev_steering_e
prev_steering_e = err_s
steering_c = STEER_KP * err_s + STEER_KD * der_s

# 3) 거리 PD
err_d = TARGET_DIST - meas
der_d = err_d - prev_dist_e
prev_dist_e = err_d
dist_c = DIST_KP * err_d + DIST_KD * der_d

# 4) 휠 속도 조합
MAX_CTRL = 70
left_spd = clamp(int((dist_c + steering_c) * (MAX_CTRL/100)), -MAX_CTRL,
MAX_CTRL)
right_spd = clamp(int((-dist_c + steering_c) * (MAX_CTRL/100)), -MAX_CTRL,
MAX_CTRL)
last_left, last_right = left_spd, right_spd
abot.servo_speed_calibrated(left_spd, right_spd)
display.show("M") # Moving

# 5) 동적 윈도우 갱신
scan_min = max(MIN_ANGLE, int(kalman_est) - WINDOW)
scan_max = min(MAX_ANGLE, int(kalman_est) + WINDOW)
angle = clamp(angle, scan_min, scan_max)
else:
# — 감지 실패
lost_count += 1
if lost_count < LOST_LIMIT:
# *히스테리시스 구간*: 마지막 속도 유지, 서버 각도 고정
abot.servo_speed_calibrated(last_left, last_right)
display.show("H") # Hold
continue
else:
# *진짜 스캔 모드* 진입
abot.servo_speed_calibrated(0, 0)
display.show("C") # Scanning

# — 스캔 모드일 때만 각도 sweep
if lost_count >= LOST_LIMIT:
angle += direction
if angle >= scan_max:
angle = scan_max
direction = -STEP
elif angle <= scan_min:
angle = scan_min
direction = STEP
sleep(20)

```

앞 절의 ‘순차제어로 따라가는 로봇 만들기’에서는 적용하던 방식과 다르게 초음파센서 신호를 사용해서 로봇의 조향제어와 거리유지제어를 실행합니다.

여기서는 steering_control() → rotate_degrees() → distance_control() 흐름을 통합하여, 스캔 중에도 매 사이클마다 “회전(steer)” 신호와 “전/후진(dist)” 신호를 동시에 계산·조합합니다. 로봇에 적용된 두가지 원리를 아래에서 설명합니다.

[로봇 바퀴제어 원리]

전진/후진을 위한 dist_cmd 는 좌/우 휠에 (+d, -d) 형태로,
회전을 위한 steer_cmd 는 (+w, +w) 형태로 각각 분산시켜,
최종 휠 명령은 left = d + w, right = -d + w 로 합산합니다.

위의 방식으로 로봇 양쪽바퀴 속도를 적용하면, 스캔 주기($\approx 30\sim 40$ ms)마다 즉시 차/회전 제어가 적용되어, 캐스케이드 모드 대비 훨씬 빠르고 부드러운 추종 동작이 가능합니다.

휠 속도조합은 PD 제어 출력 범위를 'MAX_CTRL = 70'을 기준으로 제어합니다. 현재 로봇의 속도 제어기준은 ± 70 입니다. 로봇의 동작속도를 더 느리게 또는 더 빠르게 조절하고 싶다면 MAX_CTRL 값을 50~100 사이 값으로 변경해보세요.

MAX_CTRL 값에 따라 최종 로봇속도를 비율대로 스케일링하고 더 큰 값을 잘라내는 방법입니다.

[로봇 스캔 윈도우 동적제어]

로봇 스캔을 주기적으로 동일하게 반복하지 않고, 마지막으로 검출된 로봇 방향(kalman_est)을 중심으로 $\pm 45^\circ$ (총 90°)의 동적 윈도우로 좁혀 연속 추적하도록 바꾸는 것이 목표입니다.

- 윈도우 크기 WINDOW = 45°
- 검출 시마다 scan_min/scan_max를 $\max(\text{MIN_ANGLE}, \text{est}-\text{WINDOW})$ 부터 $\min(\text{MAX_ANGLE}, \text{est}+\text{WINDOW})$ 으로 갱신
- 검출이 사라지면(거리 임계치 초과 시) 다시 전체 범위로 리셋

[객체 분실 횡수 기능]

로봇이 초음파센서로 전방의 로봇을 인식하고 추적하는 기능을 수행합니다. 로봇이 스캔하고 움직이는 환경에서 센서 데이터의 부정확 또는 잡음 등으로 전방의 물체를 인식하지 못하는 순간이 자주 발생할 수 있습니다. 로봇이 객체를 놓치면 “C 모드(스캔)”으로 진입하는 데, “C 모드(스캔)”진입을 연속으로 N회 이상 객체를 놓쳤을 때만 하도록 하는 기능을 추가합니다.

- LOST_LIMIT = 3 이면, 3회 연속 감지 실패 전까지는
- 서보를 마지막으로 검출된 각도(last_angle)에 고정하고
- 마지막으로 계산된 바퀴 속도(last_left, last_right)를 유지합니다.
- 3회 연속 놓친 뒤에야 비로소 스캔 윈도우(scan_min...scan_max)를 따라 “C 모드”로 천천히 스윙하게 됩니다.

전방로봇을 따라가는 로봇을 만들기 위해, 이상의 기능들이 추가된 로봇을 실행하지만 로봇의 실제 제어 성능에 있어서 더 부드럽게 출발하고 정지하거나, 더 유연하게 전방의 로봇을 곡선으로 따라가는 기능들을 구현하려면 많은 아이디어와 하드웨어 개선이 필요할 수 있습니다. 지금의 코드를 즐기길 바랍니다.

7.4 두 대 이상의 로봇 제어하기

병렬패턴 제어 방식으로 두 대의 로봇 또는 그 이상의 로봇에 대하여 간단한 군무 동작을 만들려고 합니다. 여러 대의 로봇이 동일한 타이밍에 움직이려면, 명령을 전송할 때 발생하는 지연 시간을 고려해야 합니다. 로봇 간에 지연이 생기지 않도록 통신 타이밍과 실행 타이밍을 맞추는 것이 중요합니다. 아울러 두 대의 로봇이 동일한 동작을 동시에 실행하려면 동기화를 유지해야 합니다.

여러 대의 로봇으로 군무동작을 만들려고 하면, 각각의 로봇 동작이 동일한 크기로 움직여야 합니다. 이런 목적을 달성하려면 앞에서 언급한 5.5절의 직선주행 보정과 제자리회전 보정을 실행해야 할 수 있습니다. 로봇의 서보 모터는 고유한 특성을 가지고 있어서, 같은 명령을 보내더라도 각 로봇이 조금씩 다르게 움직일 수 있습니다. 이런 이유로 사전에 모터 속도를 교정하는 것이 중요합니다.

이전에서 소개한 보정 모듈을 활용하여 각 로봇의 동작을 보정하면 보다 일관된 동작을 만들 수 있습니다. 로봇의 실제 동작과 관련된 차이는 배터리 잔류량에도 영향을 받는다는 점을 기억하세요.

이상 소개한 몇 가지 유의사항에 기초해서 두 대의 로봇이 동일한 패턴으로 동작하기 위한 기초지식과 실습을 소개합니다.

먼저 소개할 내용은 마이크로비트의 버전 v1과 v2는 라디오 통신 성능에서 금방 차이를 느낄 정도로 통신속도 차이가 큼니다. 이러한 하드웨어 통신 성능차이에도 불구하고 소프트웨어 수단으로 로봇이 동일한 동작을 하도록 코딩하는 방법들입니다. 이어서 두 대 이상의 로봇에 대하여 간단한 군무 동작을 만드는 실습들을 차례로 소개합니다.

(1) 서로 다른 라디오 성능을 극복하는 방법

마이크로비트의 버전이 v1과 v2로 다른 경우 데이터 송수신 속도는 크게 차이가 납니다. 두 가지 버전을 동시에 사용하면서 데이터 송수신 속도차이를 개선하거나 최소화할 수 있는 방법을 소개합니다.

- 타이밍 제어

v1과 v2 간의 하드웨어 차이를 완화하기 위해 v1이 처리할 수 있는 속도에 맞춰 전체 프로그램의 타이밍을 제어할 수 있습니다. 대표적인 방법은 대기 시간 (Delay Time)을 추가하는 것입니다.

```
import radio

# 마이크로비트 버전 감지
is_v1 = True # 실제로는 v1, v2 감지 로직이 필요

radio.config(group=1)
radio.on()

while True:
    message = radio.receive()
```



```

if message:
    if is_v1:
        process_message(message)
    else:
        process_message(message)
        sleep(50) # v2에서 추가 지연

```

- 피드백을 활용한 동기화

피드백 기반 동기화는 v1과 v2 사이의 통신 상태나 처리 상태를 실시간으로 모니터링하고, 이를 기반으로 두 장치가 동기화되도록 하는 방식입니다. 이는 주로 확인 메시지 (acknowledgement)와 같은 형태로 구현됩니다. 예를 들어, v2에서 명령을 빠르게 처리한 뒤에, v1 으로부터 명령이 성공적으로 수행되었는지 확인하는 ack 신호를 받아야 다음 명령을 실행하도록 하는 방식입니다.

```

import radio

radio.config(group=1)
radio.on()

def send_with_ack(data):
    while True:
        radio.send(data)
        received_ack = radio.receive()
        if received_ack == "ack":
            break

while True:
    send_with_ack("move")
    sleep(100) # 명령 간 타이밍 조절

```

- 타임스탬프 기반 동기화

타임스탬프를 기반으로 하는 동기화는 v1과 v2 모두가 특정 시간에 같은 동작을 수행하도록 하는 방식입니다. 각 로봇이 라디오로 명령을 받으면, 명령에 포함된 타임스탬프 정보를 활용해 특정 시점에 동작을 수행하도록 코드를 작성할 수 있습니다. 예를 들어, radio.send()로 데이터를 전송할 때 현재 시간을 포함시켜서, 수신한 로봇들이 동일한 시간에 동작을 실행하도록 할 수 있습니다. 이를 통해 두 버전이 다르게 동작하는 것을 방지할 수 있습니다.

```

import radio

radio.config(group=1)
radio.on()

while True:
    timestamp = time.ticks_ms() + 100 # 100ms 후에 동작하도록 설정
    radio.send(f"move,{timestamp}")
    sleep(1000) # 주기적 전송

```

위 코드는 송신부 코드이고, 수신부 코드는 아래와 같습니다.

```

import radio
import time

radio.config(group=1)
radio.on()

while True:
    message = radio.receive()
    if message:

```

```

action, timestamp = message.split(',')
current_time = time.ticks_ms()
delay = int(timestamp) - current_time
if delay > 0:
    time.sleep_ms(delay)
process_message(action)

```

- 버전 감지 및 동작 차별화

마이크로비트의 버전(v1, v2)을 코드에서 자동으로 감지하여 버전에 맞는 별도의 명령 처리를 적용할 수 있습니다. 예를 들어, v1은 성능이 낮으므로 더 간단한 명령을 수행하고, v2는 더 복잡한 명령을 처리하도록 설정할 수 있습니다. 이를 통해 동작의 불균형을 줄일 수 있습니다.

```

import os

def is_v2():
    return 'v2' in os.uname().machine

if is_v2():
    # v2에서 실행할 동작
    perform_advanced_tasks()
else:
    # v1에서 실행할 동작
    perform_basic_tasks()

```

- 메시지 큐 (Message Queue) 활용

메시지 큐 시스템을 도입하여, v1과 v2가 동시에 동작하기 위해 순차적으로 메시지를 처리할 수 있도록 할 수 있습니다. 이 방식은 명령을 순서대로 처리하도록 함으로써 v1의 느린 처리 속도도 대응할 수 있게 해줍니다.

(2) 마스터-슬레이브 로봇 군무 만들기

병렬패턴 제어 방식으로 두 대의 로봇에 대하여 간단한 군무 동작을 만들려고 합니다. 여러 대의 로봇이 군무동작을 만들기 위해서는 동일한 크기의 동작을 실행할 목적으로 직진 보정과 제자리회전 보정을 먼저 적용합니다.

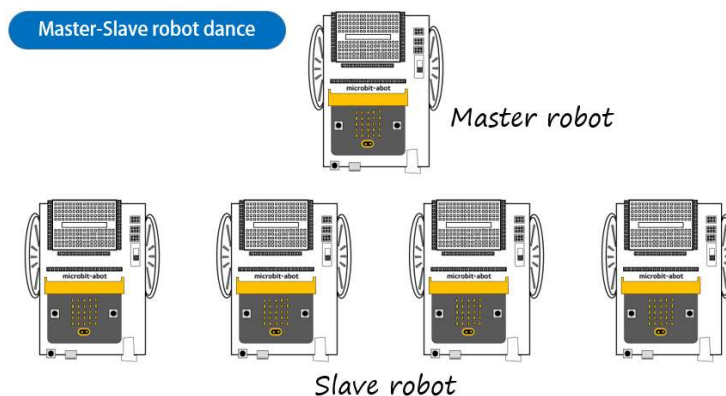


그림7.17 군무동작을 위한 마스터-슬레이브 로봇 정렬 패턴 예시

표7.3 군무 동작 제어를 위한 두 가지 접근방법

	중앙 집중 방식	분산 제어 방식(SWARM ROBOTICS)
구조형태	마스터 로봇 존재	모든 로봇이 대등
제어방식	마스터-슬레이브 방식	분산제어 또는 Peer to Peer
동기화 방식	MOVE:idx 메시지 → 상태 동기화	토큰 패싱, 가십, 타임스탬프 등 합의 기반
장점	- 단순 구현 - 명확한 제어 흐름 - 일관된 타이밍 보장	- 중앙 장애점 없음(내결함성) - 확장성 우수 - 유연성 높음
단점	- 마스터 장애 시 전체 중단 - 확장성 제한 - 네트워크 병목 가능	- 합의 알고리즘 복잡 - 구현 디버깅 난이도 상승
확장성	슬레이브 로봇을 추가하기 쉬움	로봇이 많아질수록 메시지 관리가 복잡해질 수 있음

로봇 군무 제어 방식을 선택한 다음 두 대 이상 로봇이 동시에 동일한 동작을 실행하려면 동기화를 유지해야 합니다.

로봇들이 동기화를 유지하는 방법으로 주로 시간 동기화(Time Synchronization) 또는 상태 동기화(State Synchronization)를 선택합니다.

이외에도 합의 기반 동기화(Consensus-Based Synchronization), 이벤트 기반 동기화(Event-Driven Synchronization), 장벽 동기화(Barrier Synchronization), 하이브리드 동기화(Hybrid Synchronization) 방법들이 사용됩니다.

요약하면, 중앙집중형/분산형 구조를 먼저 결정하고, 그 위에서 시간 동기화, 상태 동기화 중 필요에 따라 고르거나, 합의 알고리즘, 이벤트 기반, 장벽 동기화, 하이브리드 같은 다른 기법들을 보강하여 설계합니다. 시스템 규모, 네트워크 품질, 외부 인프라 유무, 구현 복잡도를 고려해 가장 적합한 동기화 수단을 선택하면 됩니다.

(Notice) -----

‘시간 동기화’ vs ‘상태 동기화’

시간 동기화(time synchronization)는 공통의 시계(타임스탬프)를 맞춰 놓고 일정 시각에 동작을 실행하는 방식입니다.

상태 동기화는 특정 이벤트(여기서는 MOVE:idx 메시지)를 통해 시스템의 상태 전환을 관리하는 방식입니다.

표7.3에서 소개한 두 가지 방식을 다시 비유적으로 설명하면, ‘마스터-슬레이브 방식’이 한 명의 지휘자가 여러 명의 연주자들을 지휘하는 오케스트라와 같고, ‘분산제어 방식’은 친구들끼리 모여서 동시에 게임을 시작하거나 멈추는 상황과 유사합니다.

‘중앙집중(마스터-슬레이브) 방식’은 지휘자(마스터 로봇)만이 언제 연주를 시작하고 멈출지 결정하고, 연주자들(슬레이브 로봇)은 지휘자의 지시에 따라 연주합니다.

반면에, ‘분산제어 방식’은 각 친구들(로봇) 누구든 게임을 시작하거나 멈추자고 제안할 수 있고, 제안한 친구가 "게임 시작!"이라고 외치면, 모두 동시에 게임을 시작합니다. 그리고 다른 친구가 "이제 그만하자!"라고 하면, 모두 게임을 멈춥니다.

마스터-슬레이브 방식은 구현이 비교적 쉽고, 동기화 문제가 적은 단순한 구조입니다. 특정 로봇이 마스터 로봇으로 설정되고, 그 로봇이 시스템을 제어해야 하는 경우에 적합합니다.

반면 분산제어 방식은 유연한 제어가 필요할 때 어느 로봇에서나 동작을 시작하고 멈출 수 있어 편리합니다. 이 방식은 모든 로봇이 동등한 역할을 수행하므로 특정 로봇에 의존하지 않고 시스템을 운영할 수 있습니다. 따라서 더 많은 로봇이 참여하거나, 로봇 간의 상호작용이 필요한 경우에 유용합니다.

간단한 마스터-슬레이브 로봇 군무 만들기

마스터-슬레이브 방식의 시작 명령만으로 두 대의 로봇 군무를 만드는 실습입니다. 이 방식에서는 한 대의 로봇을 마스터(Master)로 지정하고, 나머지 로봇을 슬레이브(Slave)로 지정합니다. 마스터 로봇만이 로고 터치에 반응하여 동작을 시작하거나 멈추며, 슬레이브 로봇은 마스터로부터 명령을 받아 동일한 동작을 수행합니다.

이 방식은 마스터 로봇이 명령하는 ‘시작’과 ‘정지’만으로 로봇의 군무를 실행합니다. 마스터 로봇의 마이크로비트 로고를 손으로 터치해서 군무를 시작해보세요. 이 실습은 두 대 이상의 텐덤로봇이 준비되어야 하지만, 아래 예제7.13은 슬레이브 로봇동작과 관계없이 한 대의 로봇만 가지고 있어도 ‘로봇 춤추기’를 실행할 수 있습니다.

명심하세요! 아래 예제7.13은 로봇에 전원을 공급하고 플래싱하면, 마이크로비트 디스플레이 화면에서 ‘Ready’가 출력됩니다. 그 다음 마이크로비트 로고를 터치해야만 로봇의 군무 시나리오가 실행됩니다.

예제7.13 “마스터 로봇 군무동작 만들기”

```
# ch7_13_master_robot_dance_moves.py

from microbit import *
from microbit_abot import *
from abot_calibrated import *
import radio
import music

# Configure radio communication
radio.on()
radio.config(group=1)
```

```

# Initialize the robot
abot = CalibratedMotionBot(13, 12)
abot.servo_attachpins()
abot.set_left_speed_offset(-5)
abot.set_calibrated_speed(65, -66, 2200)
abot.servo_speed_calibrated(0, 0)
dancing = False
prev_logo_state = False

# Define the dance move sequence
dance_moves = [
    {'action': 'forward', 'duration': 300},
    {'action': 'pause', 'duration': 200},
    {'action': 'rotate', 'angle': 90},
    {'action': 'pause', 'duration': 200},
    {'action': 'spin360', 'duration': 200}, # 360° 회전
    {'action': 'pause', 'duration': 200},
    {'action': 'wiggle', 'duration': 200}, # 좌우 흔들기
    {'action': 'pause', 'duration': 200},
    {'action': 'zigzag', 'duration': 400}, # 지그재그
    {'action': 'pause', 'duration': 200},
    {'action': 'flash', 'duration': 200}, # LED 플래시
    {'action': 'melody', 'duration': 200}, # 간단 멜로디
]

def dance():
    global dancing
    try:
        music.play(music.DADADADUM, wait=False)
        for move in dance_moves:
            if not dancing:
                break
            action = move['action']
            print("Executing action:", action)
            if action == 'forward':
                abot.servo_speed_calibrated(50, -50)
                sleep(move['duration'])
                abot.servo_speed_calibrated(0, 0)
            elif action == 'backward':
                abot.servo_speed_calibrated(-50, 50)
                sleep(move['duration'])
                abot.servo_speed_calibrated(0, 0)
            elif action == 'rotate':
                abot.rotate_degrees(move['angle'])
            elif action == 'spin360':
                abot.rotate_degrees(360)
            elif action == 'wiggle':
                for _ in range(3):
                    abot.rotate_degrees(30)
                    sleep(100)
                    abot.rotate_degrees(-30)
                    sleep(100)
            elif action == 'zigzag':
                # 앞뒤 전진 → 회전 → 전진 → 반대 회전
                abot.servo_speed_calibrated(40, -40)
                sleep(move['duration'])
                abot.rotate_degrees(45)
                abot.servo_speed_calibrated(40, -40)
                sleep(move['duration'])
                abot.rotate_degrees(-45)
                abot.servo_speed_calibrated(0, 0)
            elif action == 'flash':
                for img in (Image.HEART, Image.HAPPY, Image.SMILE):
                    display.show(img)
                    sleep(150)
                display.clear()
            elif action == 'melody':
                # 간단한 음계 재생
                music.play(['c4:4', 'e4:4', 'g4:4'], wait=True)
            elif action == 'pause':
                sleep(move['duration'])
    
```

```

        # 동작 중간에 스톱 요청이 들어오면 즉시 종료
        if not dancing:
            break
        sleep(50)

        music.stop()

    except Exception as e:
        print("Error during dance:", e)
        abot.servo_speed_calibrated(0, 0)
        display.show(Image.SAD)
        dancing = False

def main():
    global dancing, prev_logo_state
    display.scroll("Ready")

    while True:
        current_logo_state = pin_logo.is_touched()
        if current_logo_state and not prev_logo_state:
            dancing = not dancing
            if dancing:
                radio.send('start')
                display.show(Image.HAPPY)
                dance()
            else:
                radio.send('stop')
                abot.servo_speed_calibrated(0, 0)
                display.show(Image.SAD)
            prev_logo_state = current_logo_state
            sleep(50)

if __name__ == "__main__":
    main()

```

위 코드에서 마스터 로봇은 슬레이브 로봇으로부터의 데이터를 처리하지 않습니다. 로봇의 춤추기 동작은 전진/후진/회전/멈춤 동작 외에 아래 동작들이 정의되고 사용자의 요구에 따라 선택 사용할 수 있습니다.

- 360° 회전(Spin360)
- 좌우 흔들기(Wiggle): $\pm 30^\circ$ 를 반복
- 지그재그(Zigzag): 전진과 회전을 엮은 패턴
- LED 플래시(Flash): 내장 이미지를 빠르게 깜빡이기
- 간단 멜로디(Melody): 짧은 음계 재생

위의 마스터 로봇 동작과 슬레이브 로봇 동작 동기화를 위해 슬레이브 로봇으로부터 ACK신호를 수신하고 처리하면 더 정교한 동기화를 만들 수 있습니다. 아래 코드는 마스터 로봇이 한 대의 슬레이브로부터 'ACK'신호를 수신하고 군무동작을 시작하는 데 사용할 수 있습니다.

```

def wait_for_ack(timeout_ms=2000):
    """timeout_ms 내에 'ACK' 수신을 대기하여 True/False 반환"""
    start = running_time()
    while running_time() - start < timeout_ms:
        msg = radio.receive()
        if msg == 'ACK':
            return True
    return False

def main():

```

```

global dancing, prev_logo_state
display.scroll("Ready")
while True:
    current_logo_state = pin_logo.is_touched()
    if current_logo_state and not prev_logo_state:
        dancing = not dancing
        if dancing:
            # 1) START 명령 전송
            radio.send('start')
            display.show(Image.SQUARE) # 대기 중 표시
            # 2) ACK 수신 대기
            if wait_for_ack():
                display.show(Image.HAPPY)
                dance()
            else:
                # ACK 타임아웃
                display.show(Image.NO)
                dancing = False
        else:
            # STOP 명령 전송
            radio.send('stop')
            abot.servo_speed_calibrated(0, 0)
            display.show(Image.SAD)

    prev_logo_state = current_logo_state
    sleep(50)

```

위의 코드 방식은 단순히 한 번의 wait_for_ack() 호출로 첫 번째로 들어오는 ACK만을 기다릴 뿐이어서, 슬레이브가 2대 이상일 때 모두의 응답을 확인하지 않습니다. 두 대 이상의 모든 슬레이브 로봇으로부터 ACK를 받았는지 확인하는 방법은 아래와 같습니다.

TOTAL_SLAVES = 3 # 사전에 정해진 슬레이브 대수

```

def wait_for_all_acks(timeout_ms=2000):
    start = running_time()
    received = set()
    while running_time() - start < timeout_ms and len(received) < TOTAL_SLAVES:
        msg = radio.receive()
        if msg and msg.startswith("ACK:"):
            slave_id = msg.split(":")[1]
            received.add(slave_id)
    return len(received) == TOTAL_SLAVES

# 사용 예
radio.send('start')
display.show(Image.SQUARE)
if wait_for_all_acks():
    display.show(Image.HAPPY)
    dance()
else:
    display.show(Image.NO)
    dancing = False

```

위 코드는 “사전에 정해진 여러 대의 슬레이브 모두로부터 ACK”을 확인할 수 있습니다.

[1] 슬레이브 식별자(ID) 포함하기

- 슬레이브가 radio.send("ACK:ID") 같이 자신만의 고유 ID를 함께 보내도록 수정

[2] 수신된 ACK 집합으로 관리하기

- 마스터는 received = set() 을 유지하고,
- while 루프 안에서 msg = radio.receive() 하며
- msg.startswith("ACK:") 이면 received.add(msg.split(":")[1])

[3] 예정된 슬레이브 수와 비교

- 슬레이브 총 수를 TOTAL_SLAVES = 3 처럼 미리 정의해 두고
- while running_time() - start < timeout_ms and len(received) < TOTAL_SLAVES: 과 같이
- len(received) == TOTAL_SLAVES 가 되면 “모두 응답”으로 판단

마스터로봇 동작을 위한 차례입니다.

슬레이브 로봇으로부터 ACK동기화 신호를 수신하고 댄스를 시작하는 마스터 로봇 예제는 다음과 같습니다. 두 대이상의 슬레이브 로봇으로 군무를 만들고 싶다면 위의 설명을 참고해서 아래 코드를 적용하기 바랍니다.

예제7.14 “슬레이브 ACK신호로 동기화된 마스터 로봇 군무 만들기”

```
# ch7_14_master_robot_dance_moves_with_ACK_response.py

from microbit import *
from microbit.abot import *
from abot_calibrated import *
import radio
import music

# Configure radio communication
radio.on()
radio.config(group=1)

# Initialize the robot
abot = CalibratedMotionBot(13, 12)
abot.servo_attachpins()
abot.set_left_speed_offset(-5)
abot.set_calibrated_speed(65, -66, 2200)
abot.servo_speed_calibrated(0, 0)
dancing = False
prev_logo_state = False

# Define the dance move sequence
dance_moves = [
    {'action': 'forward', 'duration': 300},
    {'action': 'pause', 'duration': 200},
    {'action': 'rotate', 'angle': 90},
    {'action': 'pause', 'duration': 200},
    {'action': 'spin360', 'duration': 200}, # 360° 회전
    {'action': 'pause', 'duration': 200},
    {'action': 'wiggle', 'duration': 200}, # 좌우 흔들기
    {'action': 'pause', 'duration': 200},
    {'action': 'zigzag', 'duration': 400}, # 지그재그
    {'action': 'pause', 'duration': 200},
    {'action': 'flash', 'duration': 200}, # LED 플래시
    {'action': 'melody', 'duration': 200}, # 간단 멜로디
]

def wait_for_ack(timeout_ms=2000):
    """timeout_ms 내에 'ACK' 수신을 대기하여 True/False 반환"""
    start = running_time()
    while running_time() - start < timeout_ms:
        msg = radio.receive()
        if msg == 'ACK':
            return True
    return False

def dance():
    global dancing
    try:
```



```

music.play(music.DADADADUM, wait=False)
for move in dance_moves:
    if not dancing:
        break
    action = move['action']
    print("Executing action:", action)
    if action == 'forward':
        abot.servo_speed_calibrated(50, -50)
        sleep(move['duration'])
        abot.servo_speed_calibrated(0, 0)
    elif action == 'backward':
        abot.servo_speed_calibrated(-50, 50)
        sleep(move['duration'])
        abot.servo_speed_calibrated(0, 0)
    elif action == 'rotate':
        abot.rotate_degrees(move['angle'])
    elif action == 'spin360':
        abot.rotate_degrees(360)
    elif action == 'wiggle':
        for _ in range(3):
            abot.rotate_degrees(30)
            sleep(100)
            abot.rotate_degrees(-30)
            sleep(100)
    elif action == 'zigzag':
        # 앞뒤 전진 → 회전 → 전진 → 반대 회전
        abot.servo_speed_calibrated(40, -40)
        sleep(move['duration'])
        abot.rotate_degrees(45)
        abot.servo_speed_calibrated(40, -40)
        sleep(move['duration'])
        abot.rotate_degrees(-45)
        abot.servo_speed_calibrated(0, 0)
    elif action == 'flash':
        for img in (Image.HEART, Image.HAPPY, Image.SMILE):
            display.show(img)
            sleep(150)
        display.clear()
    elif action == 'melody':
        # 간단한 음계 재생
        music.play(['c4:4', 'e4:4', 'g4:4'], wait=True)
    elif action == 'pause':
        sleep(move['duration'])
    # 동작 중간에 스톱 요청이 들어오면 즉시 종료
    if not dancing:
        break
    sleep(50)
music.stop()

except Exception as e:
    print("Error during dance:", e)
    abot.servo_speed_calibrated(0, 0)
    display.show(Image.SAD)
    dancing = False

def main():
    global dancing, prev_logo_state
    display.scroll("Ready")

    while True:
        current_logo_state = pin_logo.is_touched()
        if current_logo_state and not prev_logo_state:
            dancing = not dancing
            if dancing:
                # 1) START 명령 전송
                radio.send('start')
                display.show(Image.SQUARE) # 대기 중 표시

                # 2) ACK 수신 대기
                if wait_for_ack():
                    display.show(Image.HAPPY)

```

```

        dance()
    else:
        # ACK 타임아웃
        display.show(Image.NO)
        dancing = False
    else:
        # STOP 명령 전송
        radio.send('stop')
        abot.servo_speed_calibrated(0, 0)
        display.show(Image.SAD)

    prev_logo_state = current_logo_state
    sleep(50)

if __name__ == "__main__":
    main()

```

이제 슬레이브 로봇 동작을 위한 차례입니다.

아래 실습은 예제7.14와 함께 사용되는 슬레이브 로봇의 예제7.15입니다. 마스터 로봇은 1대이지만, 슬레이브 로봇은 2대 이상 많이 플래싱하고 군무에 사용할 수 있습니다.

예제7.15 “슬레이브 로봇 군무 만들기”

```

# ch7_15_slave_robot_dance_moves.py

from microbit import *
from microbit.abot import *
from abot_calibrated import *
import radio
import music

# 1. Radio 설정 (마스터와 같은 그룹)
radio.on()
radio.config(group=1)

# 2. 로봇 초기화
abot = CalibratedMotionBot(13, 12)
abot.servo_attachpins()
abot.set_left_speed_offset(-5)
abot.set_calibrated_speed(65, -66, 2200)
abot.servo_speed_calibrated(0, 0)

# 3. 마스터와 동일한 dance_moves 정의
dance_moves = [
    {'action': 'forward', 'duration': 300},
    {'action': 'pause', 'duration': 200},
    {'action': 'rotate', 'angle': 90},
    {'action': 'pause', 'duration': 200},
    {'action': 'spin360'},
    {'action': 'pause', 'duration': 200},
    {'action': 'wiggle'},
    {'action': 'pause', 'duration': 200},
    {'action': 'zigzag', 'duration': 400},
    {'action': 'pause', 'duration': 200},
    {'action': 'flash'},
    {'action': 'melody'},
]

# 4. Dance 함수 (중간에 stop 감지)
def dance():
    music.play(music.DADADADUM, wait=False)

    for move in dance_moves:

```

```

# 수행 도중 stop 메시지 수신 체크
msg = radio.receive()
if msg == 'stop':
    break

action = move['action']
# print는 디버깅용입니다
print("Slave action:", action)

if action == 'forward':
    abot.servo_speed_calibrated(50, -50)
    sleep(move['duration'])
    abot.servo_speed_calibrated(0, 0)

elif action == 'backward':
    abot.servo_speed_calibrated(-50, 50)
    sleep(move['duration'])
    abot.servo_speed_calibrated(0, 0)

elif action == 'rotate':
    abot.rotate_degrees(move['angle'])

elif action == 'spin360':
    abot.rotate_degrees(360)

elif action == 'wiggle':
    for _ in range(3):
        abot.rotate_degrees(30)
        sleep(100)
        abot.rotate_degrees(-30)
        sleep(100)

elif action == 'zigzag':
    abot.servo_speed_calibrated(40, -40)
    sleep(move['duration'])
    abot.rotate_degrees(45)
    abot.servo_speed_calibrated(40, -40)
    sleep(move['duration'])
    abot.rotate_degrees(-45)
    abot.servo_speed_calibrated(0, 0)

elif action == 'flash':
    for img in (Image.HEART, Image.HAPPY, Image.SMILE):
        display.show(img)
        sleep(150)
    display.clear()

elif action == 'melody':
    music.play(['c4:4', 'e4:4', 'g4:4'], wait=True)

elif action == 'pause':
    sleep(move['duration'])

# 반복 중에도 stop 수신 체크
msg = radio.receive()
if msg == 'stop':
    break

music.stop()
# 혹시 남아 있을 모터 출력을 확실히 0으로
abot.servo_speed_calibrated(0, 0)

# 5. 메인 루프: start/stop 신호 수신 처리
def main():
    """
    Main loop that listens for radio messages to start or stop dancing.
    """
    display.scroll("Ready") # Indicate readiness

    while True:
        msg = radio.receive()
        if msg == 'start':

```

```

        # 준비 완료 ACK
        radio.send('ACK')
        dance()
    elif msg == 'stop':
        abot.servo_speed_calibrated(0, 0)
        sleep(50)

if __name__ == "__main__":
    main()

```

라디오 설정은 마스터와 슬레이브 로봇 간의 통신을 위해 라디오를 켜고 그룹 번호를 설정합니다. 그 다음 로봇을 초기화하고 로봇 객체를 생성한 다음 서보모터 핀을 활성화합니다.

코드의 주요 내용은 군무동작 시퀀스를 `dance_moves` 리스트로 정의합니다.

그리고 `dance()` 함수를 호출해서 `dance_moves`에 정의된 동작을 순서대로 실행하는데, 각 동작은 `action`에 따라 처리됩니다. 동작 중에도 `dancing` 상태를 확인하여 중간에 멈출 수 있습니다.

마스터 로봇의 경우 메인루프는 로고 터치를 감지하고 상태 변경할 수 있습니다. 만약 로고 터치를 감지하면 `dancing` 상태를 토글합니다.

`dancing` 상태에 따라 'start' 또는 'stop' 메시지를 라디오로 브로드캐스트합니다. `dancing`이 True이면 `dance()` 함수를 호출하여 군무를 시작합니다. 이때 마스터 로봇은 슬레이브로부터의 메시지를 처리하지 않습니다.

반면에 슬레이브 로봇의 경우 로고 터치 기능이 없으며, 라디오 메시지 수신 및 처리 기능을 수행합니다. `radio.receive()`로 메시지를 수신합니다.

start 메시지를 받자마자 `radio.send('ACK')`으로 마스터에게 준비 완료 신호 전달합니다.

마스터와 동일한 동작 시퀀스(`dance_moves`)를 `dance()`에서 반복하고, 각 동작 전·후에 `radio.receive()`를 호출해 stop이 수신되면 즉시 루프 탈출합니다.

`dance()`가 끝나면 (정상 완료든 중도 중단이든) 모터 속도를 0으로 설정하여 확실히 정지합니다.

여기서 필요에 따라 슬레이브별 ID를 붙이고 싶다면, `radio.send('ACK:01')`처럼 ID를 포함시키고 마스터에서 파싱하도록 확장하면 됩니다.

만약 두 대의 로봇이 마스터-슬레이브 방식으로 군무를 수행할 때, 동작 시퀀스를 일치시키기 위해 `dance_moves` 리스트를 동일하게 사용하지만, 리스트의 길이가 길어질수록 각 로봇의 동작 진행 타이밍에 차이가 발생할 수 있습니다.

타이밍 차이가 발생하는 이유는 [1] 각 동작 사이에 발생하는 미세한 지연 시간이 동작이 길어질수록 누적되어 전체 타이밍에 영향을 미칩니다. [2] 통신 지연으로 인해 마스터에서 슬레이브로 명령을 전달하는 데 걸리는 시간이 일정하지 않을 수 있습니다. [3] 각 로봇의

처리 속도나 성능 차이로 인해 동작 시간(프로세싱 시간)이 조금씩 달라질 수 있습니다.

각 동작별 실행명령으로 마스터-슬레이브 로봇 군무 만들기

군무를 시작하는 순간에만 동기화를 확인하고, 마스터 로봇 시작명령에 따라 모든 슬레이브 로봇들이 전체 약속된 dance_moves 리스트를 각자 실행하는 방법은 동작이 길어질수록 서로 차이나는 동작을 만들 수 있습니다.

이런 문제를 개선하기 위해 군무 시작뿐 아니라 각 개별 동작마다 모든 슬레이브와 동기화를 거친 후 실행하도록 수정한 마스터 코드 예제7.16을 소개합니다.

읽지마세요! 아래 군무 동작을 각각의 로봇에 플레이싱할 때, 코드 속에서 실제 연결될 슬레이브 로봇숫자를 군무 환경에 맞게 맞추어야 합니다.

예제7.16 “각 동작마다 동기화된 마스터 로봇 군무 만들기”

```
# ch7_16_master_robot_dance_moves_with_per_move_sync.py

from microbit import *
from microbit.abot import *
from abot.calibrated import *
import radio
import music

# 1. 설정: 슬레이브 대수
TOTAL_SLAVES = 2 # ← 실제 연결된 슬레이브 로봇 수로 맞춰주세요

# 2. Radio 설정
radio.on()
radio.config(group=1)

# 3. 로봇 초기화
abot = CalibratedMotionBot(13, 12)
abot.servo_attachpins()
abot.set_left_speed_offset(-5)
abot.set_calibrated_speed(65, -66, 2200)
abot.servo_speed_calibrated(0, 0)

dancing = False
prev_logo_state = False

# 4. 동작 시퀀스 정의 (마스터·슬레이브 공통)
dance_moves = [
    {'action': 'forward', 'duration': 300},
    {'action': 'pause', 'duration': 200},
    {'action': 'rotate', 'angle': 90},
    {'action': 'pause', 'duration': 200},
    {'action': 'spin360'},
    {'action': 'pause', 'duration': 200},
    {'action': 'wiggle'},
    {'action': 'pause', 'duration': 200},
    {'action': 'zigzag', 'duration': 400},
    {'action': 'pause', 'duration': 200},
    {'action': 'flash'},
    {'action': 'melody'},
]

# 5. ACK 동기화 함수
def wait_for_all_acks(timeout_ms=2000):
    """timeout_ms 내에 TOTAL_SLAVES 만큼 'ACK' 수신을 대기"""
    start = running_time()
    ack_count = 0
    while running_time() - start < timeout_ms and ack_count < TOTAL_SLAVES:
```

```

    msg = radio.receive()
    if msg == 'ACK':
        ack_count += 1
    return ack_count >= TOTAL_SLAVES

# 6. 동기화된 dance() 함수
def dance():
    global dancing

    # 배경음악 비동기 재생
    music.play(music.DADADADUM, wait=False)

    for idx, move in enumerate(dance_moves):
        if not dancing:
            break

        # 6-1) 마스터에서 각 move 브로드캐스트
        radio.send('MOVE:{}'.format(idx))
        display.show(Image.SQUARE) # 동기화 대기 표시
        if not wait for all acks():
            # 타임아웃 또는 일부 슬레이브 응답 누락 시 전체 중단
            display.show(Image.NO)
            dancing = False
            break

        # 6-2) 마스터 본체도 같은 동작 수행
        action = move['action']
        print("Master exec:", action)

        if action == 'forward':
            abot.servo_speed_calibrated(50, -50)
            sleep(move['duration'])
            abot.servo_speed_calibrated(0, 0)

        elif action == 'pause':
            sleep(move['duration'])

        elif action == 'rotate':
            abot.rotate_degrees(move['angle'])

        elif action == 'spin360':
            abot.rotate_degrees(360)

        elif action == 'wiggle':
            for _ in range(3):
                abot.rotate_degrees(30); sleep(100)
                abot.rotate_degrees(-30); sleep(100)

        elif action == 'zigzag':
            abot.servo_speed_calibrated(40, -40)
            sleep(move['duration'])
            abot.rotate_degrees(45)
            abot.servo_speed_calibrated(40, -40)
            sleep(move['duration'])
            abot.rotate_degrees(-45)
            abot.servo_speed_calibrated(0, 0)

        elif action == 'flash':
            for img in (Image.HEART, Image.HAPPY, Image.SMILE):
                display.show(img); sleep(150)
            display.clear()

        elif action == 'melody':
            music.play(['c4:4','e4:4','g4:4'], wait=True)

    # 다음 동작 전 잠깐 대기
    sleep(50)

    music.stop()
    abot.servo_speed_calibrated(0, 0)

# 7. 메인 루프: start/stop 신호 처리

```

```

def main():
    global dancing, prev_logo_state
    display.scroll("Ready")

    while True:
        current = pin_logo.is_touched()
        if current and not prev_logo_state:
            dancing = not dancing

            if dancing:
                # 1) 시작 명령
                radio.send('start')
                display.show(Image.SQUARE)
                # 2) 모든 슬레이브 준비 ACK 대기
                if wait_for_all_acks():
                    display.show(Image.HAPPY)
                    dance()
                else:
                    display.show(Image.NO)
                    dancing = False

            else:
                # 중지 명령
                radio.send('stop')
                abot.servo_speed_calibrated(0, 0)
                display.show(Image.SAD)

        prev_logo_state = current
        sleep(50)

if __name__ == "__main__":
    main()

```

마스터 로봇 군무는 다음 내용을 담고 있습니다.

TOTAL_SLAVES 상수로 사전 예정된 슬레이브 대수를 정의합니다.

wait_for_all_acks() 함수로 각 브로드캐스트(start, MOVE:idx)마다 모든 슬레이브의 ACK 수신을 확인하고,

dance() 안에서 각 move 실행 전 MOVE:{idx} 메시지를 보내고 동기화 대기 → 모든 ACK 수신 시에만 해당 동작 실행합니다.

이 구조를 사용하면 네트워크 지연이나 각 로봇의 타이밍 차이에도 불구하고, 모든 로봇이 동일한 순서·타이밍으로 움직일 수 있습니다.

앞서 소개한 마스터 코드와 함께 로봇군무를 만들기 위해, 각 동작마다 마스터와 슬레이브가 완전 동기화된 상태로 움직일 수 있도록 설계된 슬레이브용 MicroPython 코드 예제를 소개합니다.

start 수신 시 곧바로 ACK 응답하고, 이후 마스터가 보내는 MOVE:{idx} 메시지별로 ACK를 보내고 해당 동작만 수행하고, stop 수신 시 즉시 정지하는 방식입니다.

예제7.17 “각 동작마다 동기화된 슬레이브 로봇 군무 만들기”

```

# ch7_17_slave_robot_dance_moves_with_per_move_sync.py

from microbit import *
from microbit_abot import *

```

```

from abot_calibrated import *
import radio
import music

# 1. Radio 설정 (마스터와 동일한 그룹)
radio.on()
radio.config(group=1)

# 2. 로봇 초기화
abot = CalibratedMotionBot(13, 12)
abot.servo_attachpins()
abot.set_left_speed_offset(-5)
abot.set_calibrated_speed(65, -66, 2200)
abot.servo_speed_calibrated(0, 0)

# 3. 마스터와 동일한 dance_moves 정의
dance_moves = [
    {'action': 'forward', 'duration': 300},
    {'action': 'pause', 'duration': 200},
    {'action': 'rotate', 'angle': 90},
    {'action': 'pause', 'duration': 200},
    {'action': 'spin360'},
    {'action': 'pause', 'duration': 200},
    {'action': 'wiggle'},
    {'action': 'pause', 'duration': 200},
    {'action': 'zigzag', 'duration': 400},
    {'action': 'pause', 'duration': 200},
    {'action': 'flash'},
    {'action': 'melody'},
]

# 4. 개별 동작 처리 함수
def perform_move(move):
    action = move['action']

    if action == 'forward':
        abot.servo_speed_calibrated(50, -50)
        sleep(move['duration'])
        abot.servo_speed_calibrated(0, 0)

    elif action == 'pause':
        sleep(move['duration'])

    elif action == 'rotate':
        abot.rotate_degrees(move['angle'])

    elif action == 'spin360':
        abot.rotate_degrees(360)

    elif action == 'wiggle':
        for _ in range(3):
            abot.rotate_degrees(30)
            sleep(100)
            abot.rotate_degrees(-30)
            sleep(100)

    elif action == 'zigzag':
        abot.servo_speed_calibrated(40, -40)
        sleep(move['duration'])
        abot.rotate_degrees(45)
        abot.servo_speed_calibrated(40, -40)
        sleep(move['duration'])
        abot.rotate_degrees(-45)
        abot.servo_speed_calibrated(0, 0)

    elif action == 'flash':
        for img in (Image.HEART, Image.HAPPY, Image.SMILE):
            display.show(img)
            sleep(150)
        display.clear()

    elif action == 'melody':

```



```

music.play(['c4:4', 'e4:4', 'g4:4'], wait=True)

# 5. 메인 루프: 마스터 명령 수신 → ACK 응답 → MOVE별 수행
while True:
    msg = radio.receive()
    if msg is None:
        sleep(10)
        continue

    # 5-1) 전체 시작 동기화
    if msg == 'start':
        radio.send('ACK')

    # 5-2) 즉시 정지
    elif msg == 'stop':
        abot.servo_speed_calibrated(0, 0)
        music.stop()

    # 5-3) 개별 동작 동기화
    elif msg.startswith('MOVE:'):
        # 1) ACK 보내고
        radio.send('ACK')
        # 2) 해당 동작 수행
        idx = int(msg.split(':')[1])
        perform_move(dance_moves[idx])

    sleep(10)

```

코드의 동작 흐름은

- ① 마스터가 start를 브로드캐스트 → 모든 슬레이브가 ACK
- ② 마스터가 MOVE:0, MOVE:1, ... 순으로 브로드캐스트
 - 슬레이브는 매번 ACK를 보내고
 - 해당 인덱스의 동작만 perform_move()로 실행
- ③ 중간에 stop 메시지가 오면 즉시 모든 모터 정지

이제 마스터와 슬레이브가 각 단계마다 완전히 동기화된 군무를 실행할 수 있습니다.

(3) 분산제어로 로봇 군무 만들기

로봇 군무 구현에서 분산제어 방식은 더욱 정확한 동기화, 유연한 제어, 높은 안정성을 제공하므로 마스터-슬레이브 방식보다 우수하며, 복잡한 군무 동작을 구현하는 데 필수적인 방법입니다.

분산 제어(Peer-to-Peer) 방식으로, 어느 로봇이든 버튼을 누르면 “준비 완료” 메시지를 브로드캐스트하고, 모든 로봇이 서로의 준비 메시지를 받으면 동시에 간단한 군무(LED 점멸)를 시작하는 예제입니다. 중앙 마스터 없이 각 기기가 동일한 권한을 가지는 구조로 군무를 실행하는 제어 방식을 따라해 보세요.

아래 예제7.18은 로봇군무를 시도하기 위한 간단한 실습모형입니다. 코드를 실행해도 로

봇이 물리적인 움직임을 만들지 않고, 마이크로비트 디스플레이 화면에서 LED출력으로 군무를 대신합니다.

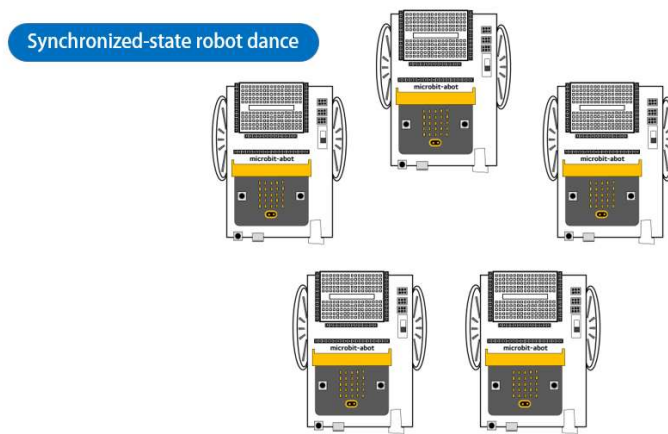


그림7.18 군무동작을 위한 상태동기화 메시지 방식의 로봇 정렬 패턴 예시

아래 예제7.18은 각 로봇마다 고유ID를 설정해야 합니다. 고유ID는 0~255 사이 값 중에서 하나를 선택해서 적용해야 합니다. 모든 로봇은 서로 다른 고유ID를 가지도록 설정해주세요. 분산제어 방식은 합의(Consensus) 방식에 따른 군무동작입니다.

임의의 로봇 버튼_A를 누르고 5초 이내에 다른 모든 로봇의 버튼_A도 누르면 합의가 성립되고, 디스플레이 화면에서 ‘행복’을 3번 점멸 표시합니다.

여러 대의 로봇 중에서 임의의 로봇이 버튼_A를 누르면서 전체 로봇과의 군무 시작을 요청하고 나머지 로봇들이 5초 이내에 버튼_A를 눌러 호응할 수 있습니다.

예제7.18 “분산제어 군무LED 만들기”

```
# ch7_18_peer_to_peer_dance_LEDs.py

from microbit import *
import radio

# 1. 설정
radio.on()
radio.config(group=1)

# 각 로봇마다 고유 ID를 지정 (0,1,2,... 등)
MY_ID = 0          # ← 기기별로 바꿔주세요
TOTAL_NODES = 3    # 네트워크에 참여한 로봇 대수

# 준비 메시지 수신 집합
ready_set = set()

# 2. 메시지 브로드캐스트 함수
def broadcast_ready():
    radio.send('READY:{}'.format(MY_ID))

# 3. 합의(Consensus) 확인 함수
def wait_for_all_ready(timeout=5000):
```

```

"""
timeout(ms) 동안 READY:<ID> 메시지를 수집해서
서로 TOTAL_NODES 만큼 모였으면 True 반환
"""
start = running_time()
ready_set.clear()
# 자기 자신도 포함
ready_set.add(str(MY_ID))
while running_time() - start < timeout:
    msg = radio.receive()
    if msg and msg.startswith('READY:'):
        peer_id = msg.split(':')
        ready_set.add(peer_id)
    if len(ready_set) >= TOTAL_NODES:
        return True
return False

# 4. 분산 군무 동작 예시
def do_dance():
    # LED 3번 깜빡이기
    for _ in range(3):
        display.show(Image.HEART)
        sleep(300)
        display.clear()
        sleep(300)

# 5. 메인 루프
while True:
    # A 버튼: 준비 신호 발신
    if button_a.was_pressed():
        display.show('R')          # Ready 알림
        broadcast_ready()

        if wait_for_all_ready():
            # 모두 준비되면 동시에 군무 시작
            display.show(Image.HAPPY)
            do_dance()
        else:
            # 타임아웃 시 실패 표시
            display.show(Image.NO)

    # 종료 후 초기화
    ready_set.clear()
    sleep(500)
    display.clear()

    # B 버튼: 전체 정지(예시)
    if button_b.was_pressed():
        display.show(Image.SAD)
        # 필요 시 모터 정지 로직 삽입
        sleep(500)
        display.clear()

    sleep(100)

```

실습 코드의 동작원리는 다음과 같습니다.

- 버튼_A를 누르면 자신의 MY_ID를 담은 READY:ID 메시지를 브로드캐스트합니다.
- wait_for_all_ready()는 타임아웃(5초) 동안 받은 모든 READY: 메시지를 집합에 모아, 집합 크기가 TOTAL_NODES에 도달하면 True 반환
- 모든 로봇이 서로의 메시지를 수집해 **동시 합의(consensus)**가 되면 do_dance()를 실행합니다.
- 버튼_B로 언제든지 취소(정지) 상태를 만들 수 있습니다.

이 코드를 각 로봇에 동일하게 올리되, MY_ID만 서로 다르게 설정하면, 중앙 제어 없이도 분산 제어로 간단한 동기화 군무를 실습할 수 있습니다.

분산제어 환경에서 어느 로봇이든 버튼_A를 누르면 사전에 약속된 동작(Command)을 브로드캐스트하고, 다른 모든 로봇이 그 명령을 받아 동일한 동작을 수행하는 시나리오를 만들어봅시다. 그리고 다른 어떤 로봇의 버튼_B로 언제든 모든 로봇의 취소(정지) 상태를 만들 수 있습니다.

우리는 단순히 로봇 군무를 시작하거나 멈추는 것에 그치지 않고, 사전 예정된 군무 시퀀스 세부동작들 중에서 버튼_B로 특정 동작을 임의의 로봇에서 선택하고, 이어서 로봇 버튼_A로 군무동작을 시작하고 멈추는 토글 스위치 기능을 부여할 수 있습니다.

버튼_B로 미리 정의해둔 여러 “군무 시퀀스”(choreography)를 순환 선택하고, 버튼_A로 선택된 시퀀스를 모든 로봇이 동기화된 상태에서 시작/중지하는 분산 제어 코드를 소개합니다.

예제7.19 “분산제어로 세부동작 제어하는 군무 만들기”

```
# ch7_19_distributed_dance_sequence.py

from microbit import *
from microbit_abot import *
from abot_calibrated import *
import radio
import music

# 1. Radio 설정 (같은 그룹)
radio.on()
radio.config(group=1)

# 2. 로봇 초기화
abot = CalibratedMotionBot(13, 12)
abot.servo_attachpins()
abot.set_left_speed_offset(-5)
abot.set_calibrated_speed(65, -66, 2200)
abot.servo_speed_calibrated(0, 0)

# 3. 미리 정의된 군무 시퀀스들
sequence_names = ['Classic', 'Quick', 'SpinOnly']

# (1) Classic: 원래의 다채로운 동작
classic_moves = [
    {'action': 'forward', 'duration': 300},
    {'action': 'pause', 'duration': 200},
    {'action': 'rotate', 'angle': 90},
    {'action': 'pause', 'duration': 200},
    {'action': 'spin360'},
    {'action': 'pause', 'duration': 200},
    {'action': 'wiggle'},
    {'action': 'pause', 'duration': 200},
    {'action': 'zigzag', 'duration': 400},
    {'action': 'pause', 'duration': 200},
    {'action': 'flash'},
    {'action': 'melody'},
]

# (2) Quick: 빠른 전진 ↔ 후진 패턴
quick_moves = [
```

```

        {'action': 'forward', 'duration': 200},
        {'action': 'pause', 'duration': 100},
        {'action': 'backward', 'duration': 200},
        {'action': 'pause', 'duration': 100},
    ]

    # (3) SpinOnly: 360° 회전만 반복
    spin_only_moves = [
        {'action': 'spin360'},
        {'action': 'pause', 'duration': 300},
    ]

    dance_sequences = [classic_moves, quick_moves, spin_only_moves]

    # 초기 선택
    seq_index = 0
    dance_moves = dance_sequences[seq_index]

    # 4. 개별 동작 처리 함수
    # (중간에 TOGGLE/SELECT 수신 시에도 반응)
    def perform_move(move):
        global dancing, seq_index, dance_moves

        # 토글 신호 우선 처리
        msg = radio.receive()
        if msg == 'TOGGLE':
            dancing = not dancing
            return False
        # 시퀀스 변경 신호 처리
        elif msg and msg.startswith('SELECT:'):
            idx = int(msg.split(':')[1])
            if 0 <= idx < len(dance_sequences):
                seq_index = idx
                dance_moves = dance_sequences[idx]
            return False

        # 실제 동작
        action = move['action']
        if action == 'forward':
            abot.servo_speed_calibrated(50, -50)
            sleep(move['duration'])
            abot.servo_speed_calibrated(0, 0)

        elif action == 'backward':
            abot.servo_speed_calibrated(-50, 50)
            sleep(move['duration'])
            abot.servo_speed_calibrated(0, 0)

        elif action == 'pause':
            sleep(move['duration'])

        elif action == 'rotate':
            abot.rotate_degrees(move['angle'])

        elif action == 'spin360':
            abot.rotate_degrees(360)

        elif action == 'wiggle':
            for _ in range(3):
                abot.rotate_degrees(30); sleep(100)
                abot.rotate_degrees(-30); sleep(100)

        elif action == 'zigzag':
            abot.servo_speed_calibrated(40, -40)
            sleep(move['duration'])
            abot.rotate_degrees(45)
            abot.servo_speed_calibrated(40, -40)
            sleep(move['duration'])
            abot.rotate_degrees(-45)
            abot.servo_speed_calibrated(0, 0)

        elif action == 'flash':

```

```

        for img in (Image.HEART, Image.HAPPY, Image.SMILE):
            display.show(img); sleep(150)
        display.clear()

    elif action == 'melody':
        music.play(['c4:4','e4:4','g4:4'], wait=True)

# 마무리 토글/선택 신호 체크
msg = radio.receive()
if msg in ('TOGGLE',) or (msg and msg.startswith('SELECT:')):
    if msg == 'TOGGLE':
        dancing = not dancing
    else:
        idx = int(msg.split(':')[1])
        if 0 <= idx < len(dance_sequences):
            seq_index = idx
            dance_moves = dance_sequences[idx]
        return False
    return True

# 5. dance(): 선택된 dance_moves 전체 반복
def dance():
    global dancing
    music.play(music.DADADADUM, wait=False)
    for move in dance_moves:
        if not dancing:
            break
        if not perform_move(move):
            break
    music.stop()
    abot.servo_speed_calibrated(0, 0)

# 6. 메인 루프: 버튼 A 토글 / 버튼 B 시퀀스 선택
dancing = False
while True:
    # (1) 버튼 A: dancing 토글 & 방송
    if button_a.was_pressed():
        dancing = not dancing
        radio.send('TOGGLE')
        display.show(Image.HAPPY if dancing else Image.SAD)
        sleep(200)
        display.clear()
        # dancing 상태일 때 바로 dance() 실행
        if dancing:
            dance()

    # (2) 버튼 B: seq_index 순환 & 방송
    if button_b.was_pressed() and not dancing:
        seq_index = (seq_index + 1) % len(dance_sequences)
        dance_moves = dance_sequences[seq_index]
        # 화면에 시퀀스 이름 표시
        display.scroll(sequence_names[seq_index])
        radio.send('SELECT:{}'.format(seq_index))
        sleep(200)
        display.clear()

    # (3) 네트워크 토글/선택 메시지 수신
    msg = radio.receive()
    if msg == 'TOGGLE':
        dancing = not dancing
        display.show(Image.HAPPY if dancing else Image.SAD)
        sleep(200)
        display.clear()
        if dancing:
            dance()
    elif msg and msg.startswith('SELECT:'):
        idx = int(msg.split(':')[1])
        if 0 <= idx < len(dance_sequences):
            seq_index = idx
            dance_moves = dance_sequences[idx]
            display.scroll(sequence_names[idx])

```

```
sleep(200)
display.clear()

sleep(50)
```

코드의 특징은 다음과 같습니다.

[1] 분산 제어(P2P): 중앙 마스터 없이 어느 로봇에서든 버튼 A/B를 누르면

- TOGGLE으로 군무 시작/중지
- SELECT:{idx}로 시퀀스 변경하면 각각이 브로드캐스트되어, 모든 로봇이 동일한 상태를 유지합니다. 모든 로봇 디스플레이에 변경된 군무 시나리오가 표시될 때까지 시간이 소요됩니다.

[2] 버튼_B 유지: 사용자가 사전에 정의된 세 가지 시퀀스(Classic, Quick, SpinOnly) 중 하나를 선택할 수 있습니다.

[3] 실시간 반응: perform_move() 내부에서도 TOGGLE/SELECT 메시지를 상시 체크하여, 중간에 멈추거나 시퀀스를 바꿔 적용할 수 있습니다.

이제 버튼 B로 군무 스타일을 골라두고, 버튼_A로 네트워크 전체를 동기화된 동작 시작·중지할 수 있습니다!

버튼_B: Classic → Quick → SpinOnly → ... 순으로 시퀀스 변경 → 화면에 이름 표시 → SELECT:{idx} 메시지 브로드캐스트

버튼_A: TOGGLE 메시지 브로드캐스트 → 모든 로봇이 dancing 상태 토글 → 선택된 시퀀스를 동기화된 상태에서 실행/중지

현재 코드는 한 번 dance()가 시작되면 각 동작(forward, pause, rotate 등) 안에서 긴 sleep()을 사용하고, 그 사이에 메시지를 받거나 버튼 입력을 처리할 기회가 거의 없습니다.

예를 들어 forward 동작이 400ms 동안 sleep(400) 상태일 때는 라디오 수신도, 버튼 입력도 전혀 처리되지 않습니다.

설령 perform_move() 처음과 끝에서 한 번씩 radio.receive()를 호출하더라도, 중간 긴 구간에서는 업데이트가 불가능합니다.

따라서 “버튼_B로 시퀀스를 금방 바꿔도, 현재 움직임이 끝날 때까지 다음 시퀀스를 반영하기 어려운 점이 현재 코드의 한계입니다.

부록1: MicroAbot.h / MicroAbot.cpp & microbit_abot.ino

(1) MicroAbot.h

```
/**
 * @file MicroAbot.h & MicroAbot.cpp
 * @brief Arduino library for Micro:bit (master) to Arduino Uno (slave) I2C robot.
 *
 * This library enables I2C communication between a Micro:bit acting as the master
 * and an Arduino Uno as the slave, facilitating coordinated control of the robot.
 * Use alongside the provided Micro:bit library and example codes.
 *
 * @repository https://github.com/wookjin-chung/Tandem\_Robotics
 * @report_bugs mail@fribot.com
 * @created Dec 18, 2024
 * @version 0.1.0
 * @license MIT License. See LICENSE file for details.
 */
#ifndef MICROABOT_H
#define MICROABOT_H

#include <Arduino.h>
#include <Wire.h>
#include <Servo.h>

enum I2CState {
    I2C_IDLE, RECEIVING, REQUESTED, I2C_ERROR
};

enum RCTimeFSMState {
    RCTIME_IDLE,
    RCTIME_START,
    RCTIME_READ,
    RCTIME_COMPLETE
};

enum class IRState {
    Idle,
    SingleSignal,
    TwoSignals,
    ProcessComplete
};

enum class UltrasonicSensorType {
    ThreePin,
    FourPin
};

enum class UltrasonicState {
    Idle,
    TriggerPulse,
    WaitForEchoStart,
    WaitForEchoEnd,
    Done
};

class MicroAbot {
public:
    MicroAbot();
    ~MicroAbot();

    static void receiveEventWrapper(int howMany);
    static void requestEventWrapper();

    void processData(uint8_t data);
    void servoStandardDriveWrapper(uint8_t data);
    void servoStopWrapper(uint8_t data);
    void servoDriveWrapper(uint8_t data);
    void attachservoPinsWrapper(uint8_t data);
    void servoDetachWrapper(uint8_t data);
    void digitalWriteCommandWrapper(uint8_t data);
    void analogWriteCommandWrapper(uint8_t data);
    void digitalReadCommandWrapper(uint8_t data);
    void toneCommandWrapper(uint8_t data);
    void pulseInCommandWrapper(uint8_t data);
    void pulseOutCommandWrapper(uint8_t data);
    void pulseCountCommandWrapper(uint8_t data);
    void threePinUltrasonicCommandWrapper(uint8_t data);
    void fourPinUltrasonicCommandWrapper(uint8_t data);
    void rcTimeWrapper(uint8_t data);
};
```



```

    void irDetectWrapper(uint8_t data);
    void irDistanceWrapper(uint8_t data);

    void setup();
    void loop();

private:
    void initialize();
    void servoStandardDrive(uint8_t pin, int16_t angleVal);
    void servoStop(uint8_t pin1, uint8_t pin2, int16_t speedL);
    void servoDrive(uint8_t pin1, uint8_t pin2, int16_t speedL, int16_t speedR);
    void attachservoPins(uint8_t svoLeftPin, uint8_t svoRightPin);
    void detachOtherServos(uint8_t newLeftPin, uint8_t newRightPin);
    void servoDetach(uint8_t pin);

    void digitalWriteCommand(uint8_t pin, uint8_t state);
    void analogWriteCommand(uint8_t pin, uint16_t value);
    uint32_t digitalReadCommand(uint8_t pin);
    void toneCommand(uint8_t pin, uint16_t frequency, uint16_t duration);
    uint32_t pulseInCommand(uint8_t pin, uint8_t state);
    void pulseOutCommand(uint8_t pin, uint16_t duration);
    uint32_t pulseCountCommand(uint8_t pin, uint16_t duration);
    uint32_t ping(uint8_t ping_pin);
    uint32_t hr04(uint8_t trig_pin, uint8_t echo_pin);
    uint32_t irDetect(uint8_t irLedPin, uint8_t irReceiverPin, uint16_t frequency);
    uint32_t irDistance(uint8_t irLedPin, uint8_t irReceivePin);

    void receiveEvent(int howMany);
    void requestEvent();
    const char* currentI2CStateToString(I2CState state);

    void handleI2CError();
    void handleMultiByteData(uint8_t* data, uint8_t howmany);
    void handleOneByteCommand(uint8_t ack);
    void handleReadCommand(uint8_t secondByte);
    void handleTwoByteCommand(uint8_t secondByte);
    void handleCommandError(const char* errorMessage);
    void prepareDataFor0x18Return(uint8_t requestedPin);

    bool isalivePin(uint8_t pin);
    bool arePinsValidPair(uint8_t pin1, uint8_t pin2);

    I2CState currentI2CState;
    static volatile MicroAbot* instance;

    volatile bool _sendAcknowledged;
    volatile bool _handshakeReceived;
    volatile bool _commandDataReceived;
    volatile bool _receivedDataUpdated;

    Servo servoLeft, servoRight, servoStandard;
    static const uint8_t SERVO_PINS[][2];

    bool _isServoLeftAttached;
    bool _isServoRightAttached;
    bool _isServoStandardAttached;

    volatile uint8_t _ackByte, _firstByte, _secondByte, _howmanyByte;
    volatile uint8_t _pAvalue, _pBvalue, _sValue;
    volatile int16_t _leftValue, _rightValue;
    volatile uint16_t _uleftValue;
    volatile uint8_t _servoLeftPin, _servoRightPin, _servoAnglePin;
    volatile uint8_t _irLedPin, _irReceiverPin;
    uint8_t _sendData[4], _receivedData[12];

    struct CommandMapping {
        uint8_t command;
        void (MicroAbot::*function)(uint8_t);
    };
    CommandMapping commandMappings[18];
    void initializeCommandMappings();

    struct RequestData {
        uint8_t reqPin;
        uint32_t reqValue;
        RequestData() : reqPin(255), reqValue(0) {}
    };
    RequestData reqData[10];
    uint8_t _pincount;
    volatile bool _dataAccessFlag;

    // rcTime FSM manage

```

```

static const int numRTimeFSMs = 2;
struct RTimeFSM {
    RTimeFSMState state;
    uint8_t pin;
    uint8_t mode;
    uint32_t value;
    uint32_t startTime;
};
RTimeFSM rcTimeFSMs[numRTimeFSMs];
uint8_t _currentFSMIndex = 0;

void startRTimeFSM(uint8_t data, uint8_t mode);
void processRTimeFSM(RTimeFSM &fsm);
void storeRTimeData(uint8_t pin, uint32_t value);
void saveResultToRequestData(uint8_t pin, uint32_t value);

// irDistance processing
struct IRSignal {
    uint8_t irReceivePin;
    uint8_t irLedPin;
};
IRSignal irSignalQueue[2];
int _irSignalCount;
uint8_t _processingSignalCount;

struct IRSensorData {
    uint32_t distance;
    uint8_t irLedPin;
    uint8_t irReceivePin;
};
IRSensorData measIRValue[2];
void addIRSignal(IRSignal newSignal);
void processIRSignal(uint8_t irLedPin, uint8_t irReceivePin);
void processTwoIRSignals(uint8_t irLedPin1, uint8_t irReceivePin1, uint8_t irLedPin2, uint8_t
irReceivePin2);
void measureDistanceForFrequency(IRSensorData& sensor, unsigned int frequency);

IRState currentState;

// ultrasonic distance measure processing
static const int NUM_ULTRASONIC_SENSORS = 2;
struct UltrasonicFSM {
    UltrasonicSensorType sensorType;
    uint8_t trig_pin; // 3pin sensor trigger/echo
    uint8_t echo_pin; // 4pin sensor echo
    UltrasonicState state;
    uint32_t triggerTime;
    uint32_t echoStartTime;
    uint32_t duration;
    bool measurementComplete;
};
UltrasonicFSM ultrasonicFSMs[NUM_ULTRASONIC_SENSORS];
void processUltrasonicFSM(UltrasonicFSM& fsm);
};

#endif // MICROABOT_H

```

(2) MicroAbot.cpp

```

/**
 * @file MicroAbot.h & MicroAbot.cpp
 * @brief Arduino library for Micro:bit (master) to Arduino Uno (slave) I2C robot.
 *
 * This library enables I2C communication between a Micro:bit acting as the master
 * and an Arduino Uno as the slave, facilitating coordinated control of the robot.
 * Use alongside the provided Micro:bit library and example codes.
 *
 * @repository https://github.com/wookjin-chung/Tandem\_Robotics
 * @report_bugs mail@fribot.com
 * @created Dec 18, 2024
 * @version 0.1.0
 * @license MIT License. See LICENSE file for details.
 */
#include "MicroAbot.h"
volatile MicroAbot* MicroAbot::instance = nullptr;

MicroAbot::MicroAbot() : _isServoLeftAttached(true), _isServoRightAttached(true),
_isServoStandardAttached(true), _sendAcknowledged(false), currentI2CState(I2C_IDLE), _commandDataReceived(false),
_receivedDataUpdated(false), _handshakeReceived(false), _dataAccessFlag(false), _ackByte(0), _firstByte(0), _secondByte(0),

```

```

_howmanyByte(0),
_servoLeftPin(255), _servoRightPin(255), _servoAnglePin(255), _pAvalue(27), _pBvalue(33), _sValue(0),
_leftValue(0), _rightValue(0), _uleftValue(0), _pincount(0), _irSignalCount(0), _processingSignalCount(0),
currentState(IRState::Idle)
{
    MicroAbot::instance = this;
    memset(_sendData, 0, sizeof(_sendData));
    memset(_receivedData, 0, sizeof(_receivedData));
    initializeCommandMappings();

    for (int i = 0; i < numRCTimeFSMs; i++) {
        rcTimeFSMs[i].state = RCTIME_IDLE;
    }
    for (int i = 0; i < NUM_ULTRASONIC_SENSORS; ++i) {
        ultrasonicFSMs[i].state = UltrasonicState::Idle;
        ultrasonicFSMs[i].trig_pin = 255;
        ultrasonicFSMs[i].echo_pin = 255;
        ultrasonicFSMs[i].measurementComplete = false;
    }
}

MicroAbot::~MicroAbot() {
    MicroAbot::instance = nullptr;
}

void MicroAbot::receiveEventWrapper(int howMany) {
    if (MicroAbot::instance != nullptr) {
        MicroAbot::instance->receiveEvent(howMany);
    }
}

void MicroAbot::requestEventWrapper() {
    if (MicroAbot::instance != nullptr) {
        MicroAbot::instance->requestEvent();
    }
}

const char* MicroAbot::currentI2CStateToString(I2CState state) {
    switch (state) {
        case I2C_IDLE: return "I2C_IDLE";
        case RECEIVING: return "RECEIVING";
        case REQUESTED: return "REQUESTED";
        case I2C_ERROR: return "I2C_ERROR";
        default: return "Unknown State";
    }
}

// Wrapper function for function pointer mapping
void MicroAbot::initializeCommandMappings() {
    commandMappings[0] = {24, &MicroAbot::servoStandardDriveWrapper};
    commandMappings[1] = {28, &MicroAbot::servoStopWrapper};
    commandMappings[2] = {25, &MicroAbot::servoDriveWrapper};
    commandMappings[3] = {38, &MicroAbot::attachservoPinsWrapper};
    commandMappings[4] = {39, &MicroAbot::servoDetachWrapper};

    commandMappings[5] = {3, &MicroAbot::digitalReadCommandWrapper};
    commandMappings[6] = {1, &MicroAbot::digitalWriteCommandWrapper};
    commandMappings[7] = {2, &MicroAbot::digitalWriteCommandWrapper};
    commandMappings[8] = {32, &MicroAbot::analogWriteCommandWrapper};
    commandMappings[9] = {13, &MicroAbot::toneCommandWrapper};

    commandMappings[10] = {16, &MicroAbot::rcTimeWrapper};
    commandMappings[11] = {31, &MicroAbot::irDetectWrapper};
    commandMappings[12] = {33, &MicroAbot::irDistanceWrapper};

    commandMappings[13] = {10, &MicroAbot::pulseInCommandWrapper};
    commandMappings[14] = {11, &MicroAbot::pulseOutCommandWrapper};
    commandMappings[15] = {12, &MicroAbot::pulseCountCommandWrapper};
    commandMappings[16] = {35, &MicroAbot::threePinUltrasonicCommandWrapper};
    commandMappings[17] = {36, &MicroAbot::fourPinUltrasonicCommandWrapper};
}

void MicroAbot::processData(uint8_t data) {
    for (int i = 0; i < 18; ++i) {
        if (commandMappings[i].command == data && commandMappings[i].function != nullptr) {
            (this->*(commandMappings[i].function))(data);
            //Serial.print("command data is ");
            //Serial.println(data);
            break;
        }
    }
}

```

```

void MicroAbot::servoStandardDriveWrapper(uint8_t data) {
    uint8_t pin = _pAvalue;
    int16_t angleVal = _leftValue;
    servoStandardDrive(pin, angleVal);
}

void MicroAbot::servoStopWrapper(uint8_t data) {
    uint8_t pin1 = _pAvalue;
    uint8_t pin2 = _pBvalue;
    int16_t speedL = _leftValue;
    servoStop(pin1, pin2, speedL);
}

void MicroAbot::servoDriveWrapper(uint8_t data) {
    uint8_t pin1 = _pAvalue;
    uint8_t pin2 = _pBvalue;
    int16_t speedL = _leftValue;
    int16_t speedR = _rightValue;
    servoDrive(pin1, pin2, speedL, speedR);
}

void MicroAbot::attachservoPinsWrapper(uint8_t data) {
    uint8_t svoLeftPin = _pAvalue;
    uint8_t svoRightPin = _pBvalue;
    attachservoPins(svoLeftPin, svoRightPin);
}

void MicroAbot::servoDetachWrapper(uint8_t data) {
    uint8_t pin = _pAvalue;
    servoDetach(pin);
}

void MicroAbot::digitalWriteCommandWrapper(uint8_t data) {
    uint8_t pin = _pAvalue;
    uint8_t state = data;
    digitalWriteCommand(pin, state);
}

void MicroAbot::analogWriteCommandWrapper(uint8_t data) {
    uint8_t pin = _pAvalue;
    uint16_t value = _leftValue;
    analogWriteCommand(pin, value);
}

void MicroAbot::digitalReadCommandWrapper(uint8_t data) {
    uint8_t pin = _pAvalue;
    bool pinFound = false;

    for (int i = 0; i < _pincount; i++) {
        if (reqData[i].reqPin == pin) {
            reqData[i].reqValue = digitalReadCommand(pin);
            pinFound = true;
            break;
        }
    }

    if (!pinFound && _pincount < 10) {
        reqData[_pincount].reqPin = pin;
        reqData[_pincount].reqValue = digitalReadCommand(pin);
        _pincount++;
    }
}

void MicroAbot::toneCommandWrapper(uint8_t data) {
    uint8_t pin = _pAvalue;
    uint16_t frequency = _rightValue;
    uint16_t duration = _leftValue;
    toneCommand(pin, frequency, duration);
}

void MicroAbot::pulseInCommandWrapper(uint8_t data) {
    uint8_t pin = _pAvalue;
    uint8_t state = _sValue;
    bool pinFound = false;

    for (int i = 0; i < _pincount; i++) {
        if (reqData[i].reqPin == pin) {
            reqData[i].reqValue = pulseInCommand(pin, state);
            pinFound = true;
            break;
        }
    }
}

```

```

        if (!pinFound && _pincount < 10) {
            reqData[_pincount].reqPin = pin;
            reqData[_pincount].reqValue = pulseInCommand(pin, state);
            _pincount++;
        }
    }

void MicroAbot::pulseOutCommandWrapper(uint8_t data) {
    uint8_t pin = _pAvalue;
    uint16_t duration = _leftValue;
    pulseOutCommand(pin, duration);
}

void MicroAbot::pulseCountCommandWrapper(uint8_t data) {
    uint8_t pin = _pAvalue;
    uint16_t duration = _leftValue;
    bool pinFound = false;

    for (int i = 0; i < _pincount; i++) {
        if (reqData[i].reqPin == pin) {
            reqData[i].reqValue = pulseCountCommand(pin, duration);
            pinFound = true;
            break;
        }
    }

    if (!pinFound && _pincount < 10) {
        reqData[_pincount].reqPin = pin;
        reqData[_pincount].reqValue = pulseCountCommand(pin, duration);
        _pincount++;
    }
}

void MicroAbot::threePinUltrasonicCommandWrapper(uint8_t data) {
    uint8_t ping_pin = _pAvalue;
    bool fsmFound = false;
    int fsmIndex = -1;

    for (int i = 0; i < NUM_ULTRASONIC_SENSORS; ++i) {
        if (ultrasonicFSMs[i].trig_pin == ping_pin &&
            ultrasonicFSMs[i].echo_pin == ping_pin &&
            ultrasonicFSMs[i].sensorType == UltrasonicSensorType::ThreePin) {
            fsmIndex = i;
            fsmFound = true;
            break;
        }
    }

    if (!fsmFound) {
        for (int i = 0; i < NUM_ULTRASONIC_SENSORS; ++i) {
            if (ultrasonicFSMs[i].state == UltrasonicState::Idle) {
                fsmIndex = i;
                ultrasonicFSMs[i].sensorType = UltrasonicSensorType::ThreePin;
                ultrasonicFSMs[i].trig_pin = ping_pin;
                ultrasonicFSMs[i].echo_pin = ping_pin;
                ultrasonicFSMs[i].measurementComplete = false;
                break;
            }
        }
    }

    if (fsmIndex != -1) {
        ultrasonicFSMs[fsmIndex].state = UltrasonicState::TriggerPulse;
        ultrasonicFSMs[fsmIndex].measurementComplete = false;
    } else {
        Serial.println("초음파 FSM 슬롯이 부족합니다.");
    }
}

void MicroAbot::fourPinUltrasonicCommandWrapper(uint8_t data) {
    uint8_t trig_pin = _pAvalue;
    uint8_t echo_pin = _pBvalue;
    bool fsmFound = false;
    int fsmIndex = -1;

    for (int i = 0; i < NUM_ULTRASONIC_SENSORS; ++i) {
        if (ultrasonicFSMs[i].trig_pin == trig_pin &&
            ultrasonicFSMs[i].echo_pin == echo_pin &&
            ultrasonicFSMs[i].sensorType == UltrasonicSensorType::FourPin) {
            fsmIndex = i;
            fsmFound = true;
            break;
        }
    }
}

```

```

    }

    if (!fsmFound) {
        for (int i = 0; i < NUM_ULTRASONIC_SENSORS; ++i) {
            if (ultrasonicFSMs[i].state == UltrasonicState::Idle) {
                fsmIndex = i;
                ultrasonicFSMs[i].sensorType = UltrasonicSensorType::FourPin;
                ultrasonicFSMs[i].trig_pin = trig_pin;
                ultrasonicFSMs[i].echo_pin = echo_pin;
                ultrasonicFSMs[i].measurementComplete = false;
                break;
            }
        }
    }

    if (fsmIndex != -1) {
        ultrasonicFSMs[fsmIndex].state = UltrasonicState::TriggerPulse;
        ultrasonicFSMs[fsmIndex].measurementComplete = false;
    } else {
        Serial.println("초음파 FSM 슬롯이 부족합니다.");
    }
}

void MicroAbot::rcTimeWrapper(uint8_t data) {
    uint8_t pin = _pAvalue;
    uint8_t s_value = _sValue;
    startRCTimeFSM(pin, s_value);
}

void MicroAbot::irDetectWrapper(uint8_t data) {
    uint8_t irLedPin = _pBvalue;
    uint8_t irReceiverPin = _pAvalue;
    uint16_t frequency = _leftValue;
    bool pinFound = false;

    for (int i = 0; i < _pincount; i++) {
        if (reqData[i].reqPin == irLedPin) {
            reqData[i].reqValue = irDetect(irLedPin, irReceiverPin, frequency);
            pinFound = true;
            break;
        }
    }

    if (!pinFound && _pincount < 10) {
        reqData[_pincount].reqPin = irLedPin;
        reqData[_pincount].reqValue = irDetect(irLedPin, irReceiverPin, frequency);
        _pincount++;
    }
}

void MicroAbot::irDistanceWrapper(uint8_t data) {
    IRSignal newSignal;
    newSignal.irLedPin = _pBvalue;
    newSignal.irReceivePin = _pAvalue;
    addIRSignal(newSignal);
}

void MicroAbot::addIRSignal(IRSignal newSignal) {
    switch (currentState) {
        case IRState::Idle:
            irSignalQueue[_irSignalCount++] = newSignal;
            currentState = IRState::SingleSignal;
            break;

        case IRState::SingleSignal:
            if (irSignalQueue[0].irLedPin == newSignal.irLedPin && irSignalQueue[0].irReceivePin ==
newSignal.irReceivePin) {
                currentState = IRState::ProcessComplete;
            } else {
                irSignalQueue[_irSignalCount++] = newSignal;
                currentState = IRState::TwoSignals;
            }
            break;

        case IRState::TwoSignals:
            if (_irSignalCount >= 2) {
                processTwoIRSignals(irSignalQueue[0].irLedPin, irSignalQueue[0].irReceivePin,
irSignalQueue[1].irLedPin, irSignalQueue[1].irReceivePin);
                _irSignalCount = 0;
                currentState = IRState::Idle;
            }
            break;
    }
}

```

```

        case IRState::ProcessComplete:
            processIRSignal(irSignalQueue[0].irLedPin, irSignalQueue[0].irReceivePin);
            _irSignalCount = 0;
            currentState = IRState::Idle;
            break;

        default:
            currentState = IRState::Idle;
            Serial.println("State unchanged.");
            break;
    }
}

// FSM state management
// rcTime FSM routine
void MicroAbot::startRCTimeFSM(uint8_t pin, uint8_t mode) {
    rcTimeFSMs[_currentFSMIndex].pin = pin;
    rcTimeFSMs[_currentFSMIndex].mode = mode;
    rcTimeFSMs[_currentFSMIndex].state = RCTIME_START;
    _currentFSMIndex = (_currentFSMIndex + 1) % numRCTimeFSMs;
}

void MicroAbot::processRCTimeFSM(RCTimeFSM &fsm) {
    switch (fsm.state) {
        case RCTIME_IDLE:
            break;

        case RCTIME_START:
            if (fsm.mode == 1) {
                pinMode(fsm.pin, OUTPUT);
                digitalWrite(fsm.pin, HIGH);
                delay(1); // charge the capacitor
                pinMode(fsm.pin, INPUT);
                fsm.startTime = micros();
                fsm.state = RCTIME_READ;
            }
            break;

        case RCTIME_READ:
            uint32_t elapsedTime = micros() - fsm.startTime;
            const uint32_t maxElapsedTime = 100000; // max time 100 ms

            if (digitalRead(fsm.pin) == LOW || elapsedTime >= maxElapsedTime) {
                fsm.value = elapsedTime;
                // Set data access flags
                while (_dataAccessFlag);
                _dataAccessFlag = true;
                storeRCTimeData(fsm.pin, fsm.value);
                _dataAccessFlag = false;

                fsm.state = RCTIME_COMPLETE;
            }
            break;

        case RCTIME_COMPLETE:
            fsm.state = RCTIME_IDLE;
            break;
    }
}

void MicroAbot::storeRCTimeData(uint8_t pin, uint32_t value) {
    bool pinFound = false;
    for (int i = 0; i < _pincount; i++) {
        if (reqData[i].reqPin == pin) {
            reqData[i].reqValue = value;
            pinFound = true;
            break;
        }
    }
    if (!pinFound && _pincount < 10) {
        reqData[_pincount].reqPin = pin;
        reqData[_pincount].reqValue = value;
        _pincount++;
    }
}

// irSensor signal state machine
void MicroAbot::processIRSignal(uint8_t irLedPin, uint8_t irReceivePin) {
    IRSensorData sensor = {0, irLedPin, irReceivePin};

    for (unsigned int f = 38000; f <= 42000; f += 500) {

```

```

        measureDistanceForFrequency(sensor, f);
    }
    //Serial.print("pin is: ");
    //Serial.print(sensor.irLedPin);
    //Serial.print("Sensor Distance: ");
    //Serial.println(sensor.distance);
    saveResultToRequestData(sensor.irLedPin, sensor.distance);
}

void MicroAbot::processTwoIRSignals(uint8_t irLedPin1, uint8_t irReceivePin1, uint8_t irLedPin2, uint8_t
irReceivePin2) {
    IRSensorData sensor1 = {0, irLedPin1, irReceivePin1};
    IRSensorData sensor2 = {0, irLedPin2, irReceivePin2};
    //Serial.print(sensor1.irReceivePin);
    //Serial.print(" ");
    //Serial.println(sensor2.irReceivePin);

    for (unsigned int f = 38000; f <= 42000; f += 500) {
        measureDistanceForFrequency(sensor1, f);
        measureDistanceForFrequency(sensor2, f);
    }
    saveResultToRequestData(sensor1.irLedPin, sensor1.distance);
    saveResultToRequestData(sensor2.irLedPin, sensor2.distance);
    //Serial.print("Sensor 1 Distance: ");
    //Serial.println(sensor1.distance);
    //Serial.print("Sensor 2 Distance: ");
    //Serial.println(sensor2.distance);
}

void MicroAbot::saveResultToRequestData(uint8_t pin, uint32_t value) {
    bool pinFound = false;
    for (int i = 0; i < _pincount; i++) {
        if (reqData[i].reqPin == pin) {
            reqData[i].reqValue = value;
            pinFound = true;
            break;
        }
    }
    if (!pinFound && _pincount < 10) {
        reqData[_pincount].reqPin = pin;
        reqData[_pincount].reqValue = value;
        _pincount++;
    }
}

// ultrasonic sensor distance measuring FSM -- nonblocking code (pulseIn)
void MicroAbot::processUltrasonicFSM(UltrasonicFSM& fsm) {
    switch (fsm.state) {
        case UltrasonicState::Idle:
            break;

        case UltrasonicState::TriggerPulse:
            if (fsm.sensorType == UltrasonicSensorType::ThreePin) {
                // 3pin sensor trigger pulse sending
                pinMode(fsm.trig_pin, OUTPUT);
                digitalWrite(fsm.trig_pin, LOW);
                delayMicroseconds(2);
                digitalWrite(fsm.trig_pin, HIGH);
                delayMicroseconds(5);
                digitalWrite(fsm.trig_pin, LOW);

                // setup echo pin
                pinMode(fsm.echo_pin, INPUT);
            } else if (fsm.sensorType == UltrasonicSensorType::FourPin) {
                // 4pin sensor trigger pulse sending
                pinMode(fsm.trig_pin, OUTPUT);
                digitalWrite(fsm.trig_pin, LOW);
                delayMicroseconds(2);
                digitalWrite(fsm.trig_pin, HIGH);
                delayMicroseconds(10);
                digitalWrite(fsm.trig_pin, LOW);

                // setup echo pin
                pinMode(fsm.echo_pin, INPUT);
            }
            fsm.triggerTime = micros();
            fsm.state = UltrasonicState::WaitForEchoStart;
            break;

        case UltrasonicState::WaitForEchoStart:
            if (digitalRead(fsm.echo_pin) == HIGH) {
                fsm.echoStartTime = micros();
            }
    }
}

```



```

        fsm.state = UltrasonicState::WaitForEchoEnd;
    } else if (micros() - fsm.triggerTime > 30000) {
        // 30ms timeout --- approx. 4 meter
        fsm.duration = 0;
        fsm.state = UltrasonicState::Done;
    }
    break;

case UltrasonicState::WaitForEchoEnd:
    if (digitalRead(fsm.echo_pin) == LOW) {
        uint32_t echoEndTime = micros();
        fsm.duration = echoEndTime - fsm.echoStartTime;
        fsm.state = UltrasonicState::Done;
    } else if (micros() - fsm.echoStartTime > 30000) {
        // 30ms timeout --- approx. 4 meter
        fsm.duration = 0;
        fsm.state = UltrasonicState::Done;
    }
    break;

case UltrasonicState::Done:
    fsm.measurementComplete = true;
    fsm.state = UltrasonicState::Idle;

    // save to reqData
    bool pinFound = false;
    for (int i = 0; i < _pincount; ++i) {
        if (reqData[i].reqPin == fsm.trig_pin) {
            reqData[i].reqValue = fsm.duration;
            pinFound = true;
            break;
        }
    }
    if (!pinFound) {
        if (_pincount >= 10) {
            _pincount = 0;
        }
        reqData[_pincount].reqPin = fsm.trig_pin;
        reqData[_pincount].reqValue = fsm.duration;
        _pincount++;
    }

    // Serial.print(" pin = ");
    // Serial.println(fsm.trig_pin);
    // Serial.print(" duration = ");
    // Serial.println(fsm.duration);
    break;
}

}

// abot operation
const uint8_t MicroAbot::SERVO_PINS[][2] = {{13, 12}, {11, 10}, {9, 8}};

bool MicroAbot::isAlivePin(uint8_t pin) {
    return pin != 33;
};

bool MicroAbot::arePinsValidPair(uint8_t pin1, uint8_t pin2) {
    for (const auto &pair : SERVO_PINS) {
        if ((pair[0] == pin1 && pair[1] == pin2) || (pair[0] == pin2 && pair[1] == pin1)) {
            return true;
        }
    }
    return false;
}

void MicroAbot::initialize() {
    this->attachServoPins(255, 255); // servo inactive state
    Serial.println("MicroAbot software ver. @0.1.0");
    Serial.println("MicroAbot initialized .... ");
}

void MicroAbot::setup() {
    Wire.begin(0x5D); // Slave address 0x5D = 93
    Wire.onReceive(MicroAbot::receiveEventWrapper);
    Wire.onRequest(MicroAbot::requestEventWrapper);
    initialize();
}

void MicroAbot::loop() {
    if (this->currentI2CState == I2C_IDLE && _receivedDataUpdated && _commandDataReceived) {
        this->processData(_secondByte);
        _commandDataReceived = false;
    }
}

```

```

        _receivedDataUpdated = false;
    } else if (this->currentI2CState == I2C_ERROR) {
        handleI2CError();
    }

    processRTimeFSM(rcTimeFSMs[0]);
    processRTimeFSM(rcTimeFSMs[1]);
    for (int i = 0; i < NUM_ULTRASONIC_SENSORS; ++i) {
        processUltrasonicFSM(ultrasonicFSMs[i]);
    }
}

// return data prepared for requestEvent
void MicroAbot::prepareDataFor0x18Return(uint8_t requestedPin) {
    bool pinFound = false;

    for (int i = 0; i < _pincount; i++) {
        if (reqData[i].reqPin == requestedPin) {
            uint32_t value = reqData[i].reqValue;
            for (int j = 0; j < 4; j++) {
                _sendData[j] = (value >> (j * 8)) & 0xFF;
            }
            pinFound = true;
            break;
        }
    }

    // If no matching pin is found, prepare to send a default or error value
    if (!pinFound) {
        // define an appropriate default/error response
        for (int j = 0; j < 4; j++) {
            _sendData[j] = 0xFF;
        }
    }
}

// receiveEvent data treatment
void MicroAbot::handleOneByteCommand(uint8_t ack) {
    switch (ack) {
        case 0x00:
            _sendAcknowledged = true;
            break;
        case 0xAA:
            _handshakeReceived = true;
            //Serial.println("0xAA data received");
            break;
        default:
            Serial.print(ack);
            handleCommandError("unknown oneByte data");
            break;
    }
}

void MicroAbot::handleReadCommand(uint8_t secondByte) {
    this->prepareDataFor0x18Return(secondByte);
}

void MicroAbot::handleTwoByteCommand(uint8_t secondByte) {
    if (secondByte > 0x00 && secondByte < 0x63) {
        _commandDataReceived = true;
    } else {
        handleCommandError("unknown 2-byte data");
    }
}

void MicroAbot::handleCommandError(const char* errorMessage) {
    Serial.print(" -- error: ");
    Serial.println(errorMessage);
}

void MicroAbot::handleMultiByteData(uint8_t* data, uint8_t howmany) {
    switch (howmany) {
        case 4: // Handle 4-byte data
            _pAvalue = data[1];
            _pBvalue = data[2];
            _sValue = data[3];
            break;
        case 8: // Handle 8-byte data
            _pAvalue = data[1];
            _pBvalue = data[2];
            _sValue = data[3];
    }
}

```

```

        _uleftValue = ((uint16_t)data[5] << 8) | data[4];
        _leftValue = ((int16_t)data[5] << 8) | data[4];
        break;

    case 12: // Handle 12-byte data
        _pAvalue = data[1];
        _pBvalue = data[2];
        _sValue = data[3];
        _uleftValue = ((uint16_t)data[5] << 8) | data[4];
        _leftValue = ((int16_t)data[5] << 8) | data[4];
        _rightValue = ((int16_t)data[9] << 8) | data[8];
        break;

    default: // Handle other cases or throw an error
        Serial.println("Error: Unexpected data size");
        break;
    }
}

void MicroAbot::handleI2CError() {
    Wire.end();
    delay(100); // Short delay before reinitializing
    Wire.begin(0x5D);
    Serial.println("I2C Error Occurred");
    this->currentI2CState = I2C_IDLE;
}

void MicroAbot::receiveEvent(int howMany) {
    if (instance == nullptr || this->currentI2CState != I2C_IDLE) {
        return; // Exit if instance is null or state is not idle
    }
    this->currentI2CState = RECEIVING;

    // Serial.println("I2C START!");
    if (howMany <= 0) {
        handleI2CError();
        this->currentI2CState = I2C_IDLE;
        return;
    }

    if (howMany == 1) {
        _ackByte = Wire.read();
        handleOneByteCommand(_ackByte);
        // Serial.print("1-byte : ");
        // Serial.print(_ackByte);
        // Serial.print("\n");
    }

    } else if (howMany == 2) {
        _firstByte = Wire.read();
        _secondByte = Wire.read();
        if (_firstByte == 0x18) {
            handleReadCommand(_secondByte);
        } else if (_firstByte == 0x00) {
            handleTwoByteCommand(_secondByte);
        } else {
            // Do nothing or add additional handling if required
        }
        /* Print command byte data
        Serial.print("first-byte : ");
        Serial.print(_firstByte);
        Serial.print("\n");
        Serial.print("command-byte : ");
        Serial.print(_secondByte);
        Serial.println("\n");
        */
    }

    } else if (howMany >= 4 && howMany <= sizeof(_receivedData)) {
        memset(_receivedData, 0, sizeof(_receivedData));
        for (int i = 0; i < howMany; i++) {
            _receivedData[i] = Wire.read();
        }
        handleMultiByteData(_receivedData, howMany);
        _howmanyByte = howMany;
        _receivedDataUpdated = true;
        /* Print the receiveBuffer values
        Serial.print("    Received data : ");
        for (int i = 0; i < howMany; i++) {
            Serial.print(_receivedData[i]);
            Serial.print(" ");
        }
        Serial.println();
        // Serial.flush(); // Ensure all serial data is sent before continuing
        */
    }
}

```

```

    } else {
        handleI2CError(); // Handle cases where howMany is 3 or > sizeof(_receivedData)
    }
    this->currentI2CState = I2C_IDLE;
}

void MicroAbot::requestEvent() {
    if (instance == nullptr || this->currentI2CState != I2C_IDLE) {
        return; // Exit if instance is null or state is not idle
    }
    this->currentI2CState = REQUESTED;

    if (_handshakeReceived) {
        uint8_t handshakeResponse = 0x55;
        Wire.write(&handshakeResponse, 1);
        _handshakeReceived = false;

    } else if (_sendAcknowledged) {
        byte ack = 0x00;
        Wire.write(&ack, sizeof(ack));
        _sendAcknowledged = false;

    } else if (_firstByte == 0x18) {
        Wire.write(_sendData, 4);
        _firstByte = 0x99; // initialize to the default value 0x99

    } else {
        // Default response or error handling
        byte defaultResponse = 0xFF; // Choose an appropriate default response
        Wire.write(&defaultResponse, sizeof(defaultResponse));
    }

    this->currentI2CState = I2C_IDLE;
}

// abot defined functions
void MicroAbot::servoStandardDrive(uint8_t pin, int16_t angleVal) {
    if (pin != _servoAnglePin || !_isServoStandardAttached) {
        servoStandard.detach();
        servoStandard.attach(pin);
        _servoAnglePin = pin;
        _isServoStandardAttached = true;
    }
    if (angleVal >= 0 && angleVal <= 180) {
        servoStandard.write(angleVal);
    } else {
        Serial.println("Error: Invalid input value");
    }
}

void MicroAbot::servoStop(uint8_t pin1, uint8_t pin2, int16_t speedL) {
    if (this->isalivePin(pin2) == true && _howmanyByte == 8) {
        servoLeft.writeMicroseconds(1500);
        servoRight.writeMicroseconds(1500+speedL);
    } else if (this->isalivePin(pin2) == true && _howmanyByte == 4) {
        servoLeft.writeMicroseconds(1500);
        servoRight.writeMicroseconds(1500);
    } else if (_howmanyByte == 4) {
        if (pin1 == _servoLeftPin) {
            servoLeft.writeMicroseconds(1500);
        } else if (pin1 == _servoRightPin) {
            servoRight.writeMicroseconds(1500);
        } else if (pin1 == _servoAnglePin) {
            servoStandard.write(90);
        }
    }
}

void MicroAbot::servoDrive(uint8_t pin1, uint8_t pin2, int16_t speedL, int16_t speedR) {
    if (this->isalivePin(pin2) == true && _howmanyByte == 12) {
        servoLeft.writeMicroseconds(1500+speedL);
        servoRight.writeMicroseconds(1500+speedR);
        //Serial.println(" --> Driving both servos");
    } else if (this->isalivePin(pin2) == true && _howmanyByte == 8) {
        //Serial.println(" --> Driving left servo only");
        servoLeft.writeMicroseconds(1500+speedL);
        servoRight.writeMicroseconds(1500);
    } else if (this->isalivePin(pin2) != true && _howmanyByte == 8) {
        //Serial.println(" --> Driving single servo");
        if (pin1 == _servoLeftPin) {
            servoLeft.writeMicroseconds(1500+speedL);
        }
    }
}

```

```

        } else if (pin1 == _servoRightPin) {
            servoRight.writeMicroseconds(1500+speedL);
        }
    }
}

void MicroAbot::detachOtherServos(uint8_t newLeftPin, uint8_t newRightPin) {
    for (int i = 0; i < 3; i++) {
        uint8_t leftPin = SERVO_PINS[i][0];
        uint8_t rightPin = SERVO_PINS[i][1];
        if (leftPin != newLeftPin && rightPin != newRightPin) {
            if (servoLeft.attached() && _servoLeftPin == leftPin) {
                servoLeft.detach();
                _isServoLeftAttached = false;
            }
            if (servoRight.attached() && _servoRightPin == rightPin) {
                servoRight.detach();
                _isServoRightAttached = false;
            }
        }
    }
}

void MicroAbot::attachservoPins(uint8_t svoLeftPin, uint8_t svoRightPin) {
    if (!arePinsValidPair(svoLeftPin, svoRightPin)) {
        //Serial.println("Error: wrong servo pins");
        return;
    }
    // Detach all other servos except the new pair
    this->detachOtherServos(svoLeftPin, svoRightPin);

    // Attach new servo pins
    if (!servoLeft.attached()) {
        servoLeft.attach(svoLeftPin);
        _servoLeftPin = svoLeftPin; // Update the pin number
        _isServoLeftAttached = true;
    }
    if (!servoRight.attached()) {
        servoRight.attach(svoRightPin);
        _servoRightPin = svoRightPin; // Update the pin number
        _isServoRightAttached = true;
    }
}

void MicroAbot::servoDetach(uint8_t pin) {
    if ((_servoAnglePin != pin && _servoLeftPin != pin && _servoRightPin != pin) ||
    (!_isServoStandardAttached && !_isServoLeftAttached && !_isServoRightAttached)) {
        Serial.println("Error: No servo is engaged on this pin");
        return;
    }
    if (_servoAnglePin == pin && _isServoStandardAttached) {
        servoStandard.detach();
        _isServoStandardAttached = false;
        //Serial.println(" Standard servo detached ... ");
    }
    else if (_servoLeftPin == pin && _isServoLeftAttached) {
        servoLeft.detach();
        _isServoLeftAttached = false;
        //Serial.println(" Left servo detached ... ");
    }
    else if (_servoRightPin == pin && _isServoRightAttached) {
        servoRight.detach();
        _isServoRightAttached = false;
        //Serial.println(" Right servo detached ... ");
    }
}

void MicroAbot::digitalWriteCommand(uint8_t pin, uint8_t state) {
    pinMode(pin, OUTPUT);
    digitalWrite(pin, (state != 2) ? HIGH : LOW);
}

void MicroAbot::analogWriteCommand(uint8_t pin, uint16_t value) {
    if (this->currentI2CState == I2C_IDLE) {
        analogWrite(pin, value);
    }
}

uint32_t MicroAbot::digitalReadCommand(uint8_t pin) {
    pinMode(pin, INPUT);
    return static_cast<uint32_t>(digitalRead(pin));
}

```

```

void MicroAbot::toneCommand(uint8_t pin, uint16_t frequency, uint16_t duration) {
    tone(pin, frequency, duration);
}

uint32_t MicroAbot::pulseInCommand(uint8_t pin, uint8_t state) {
    pinMode(pin, INPUT);
    return pulseIn(pin, state);
}

void MicroAbot::pulseOutCommand(uint8_t pin, uint16_t duration) {
    pinMode(pin, OUTPUT);
    digitalWrite(pin, HIGH);
    delayMicroseconds(duration);
    digitalWrite(pin, LOW);
}

uint32_t MicroAbot::pulseCountCommand(uint8_t pin, uint16_t duration) {
    pinMode(pin, INPUT);
    uint32_t startTime = micros();
    uint32_t endTime = startTime + static_cast<uint32_t>(duration);
    uint32_t count = 0;

    while (micros() < endTime) {
        if (digitalRead(pin) == HIGH) {
            count++;
            while (digitalRead(pin) == HIGH);
        }
    }
    return count;
}

uint32_t MicroAbot::irDetect(uint8_t irLedPin, uint8_t irReceiverPin, uint16_t frequency) {
    _irLedPin = irLedPin;
    _irReceiverPin = irReceiverPin;
    pinMode(_irLedPin, OUTPUT);
    pinMode(_irReceiverPin, INPUT);
    tone(_irLedPin, frequency, 8);

    unsigned long startTime = millis();
    while(millis() - startTime < 1);

    uint8_t _lastReadValue_ir = digitalRead(_irReceiverPin);

    startTime = millis();
    while(millis() - startTime < 1);

    return static_cast<uint32_t>(_lastReadValue_ir);
}

void MicroAbot::measureDistanceForFrequency(IRSensorData& sensor, unsigned int frequency) {
    pinMode(sensor.irLedPin, OUTPUT);
    tone(sensor.irLedPin, frequency, 8);

    unsigned long startTime = micros();
    while (micros() - startTime < 1000);

    pinMode(sensor.irReceivePin, INPUT);
    uint8_t readValue = digitalRead(sensor.irReceivePin);

    startTime = micros();
    while (micros() - startTime < 1000);

    sensor.distance += static_cast<uint32_t>(readValue);
}

```

(3) microbit_abot.ino

```

#include <MicroAbot.h>

MicroAbot abot;
void setup() {
    Serial.begin(9600);
    abot.setup();
}

void loop() {
    abot.loop();
}

```

부록2: microbit_abot.py 라이브러리

```
"""
microbit_abot.py
I2C communication for Micro:bit (master) to Arduino Uno (slave) robot.
Use alongside the provided Arduino library and example codes.
Repo: https://github.com/wookjin-chung/Tandem\_Robotics | MIT License
@fribot.com created Apr 7, 2025
@version 0.1.0
"""
from microbit import *

# Handshake with I2C device at address 93
handshake_successful = False
while not handshake_successful:
    try:
        i2c.write(93, b'\xaa')
        sleep(100)
        if i2c.read(93, 1) == b'\x55':
            print("Handshake successful")
            handshake_successful = True
    except OSError:
        pass

class bot:
    def __init__(self, p=27, q=33):
        super().__init__()
        self.pA, self.pB = p, q

    def botdisable(self):
        pin8.set_pull(pin8.NO_PULL)
        sleep(200)
        reset()

    def send_c(self, c, s=0, d=None, f=None):
        a = bytes([1, self.pA, self.pB, s])
        if d is not None:
            a += int(round(d)).to_bytes(4, 'little')
        if f is not None:
            a += int(round(f)).to_bytes(4, 'little')
        try:
            i2c.write(93, a)
            i2c.write(93, bytes([0, c]))
            c_val = b'\x01'
            while c_val != b'\x0':
                i2c.write(93, b'\x0')
                c_val = i2c.read(93, 1)
        except:
            self.botdisable()

    def read_r(self):
        try:
            req = bytes([0x18, self.pA])
            i2c.write(93, req)
            r = i2c.read(93, 4)
            return int.from_bytes(r, 'little')
        except:
            self.botdisable()

    def write_digital(self, s):
        if s > 1 or s < 0:
            s = 4
        elif s == 0:
            s = 2
        self.send_c(s)

    def write_analog(self, f):
        self.send_c(32, 0, f)

    def read_digital(self):
        self.send_c(3)
        return self.read_r()

    def pulse_out(self, d):
        self.send_c(11, 0, d)

    def pulse_in(self, s):
        self.send_c(10, s)
        return self.read_r()

    def pulse_count(self, d):
```

```
        self.send_c(12, 0, d)
        return self.read_r()

def rc_time(self, s, d=None, f=None):
    self.send_c(16, s, d, f)
    return self.read_r()

def tone(self, f, d):
    self.send_c(13, 0, d, f)

def ir_detect(self, f):
    bot(self.pB, self.pA).send_c(31, 0, f)
    return self.read_r()

def ir_distance(self):
    bot(self.pB, self.pA).send_c(33, 0)
    return self.read_r()

def servo_attachpins(self):
    self.send_c(38)

def servo_angle(self, v=None):
    c = 28 if v is None else 24
    self.send_c(c, 0, v)

def servo_speed(self, vL=None, vR=None):
    c = 28 if vL is None else 25
    self.send_c(c, 0, vL, vR)

def detach(self):
    self.send_c(39, 0)
```


부록3: ultrasonic.py 라이브러리

```
"""
ultrasonic.py
Provides ultrasonic sensor functionality for distance measurements.
Integrated into the Micro:bit-Arduino robot system (has-a relationship).
Repo: https://github.com/wookjin-chung/Tandem\_Robotics | MIT License
@fribot.com created Apr 7, 2025
@version 0.1.0
"""
from microbit_abot import *

class Ultrasonic:
    def __init__(self, p=33, q=None):
        self.pinA = p
        self.pinB = q

        if self.pinB is None:
            self.bot_instance = bot(self.pinA)
        else:
            self.bot_instance = bot(self.pinA, self.pinB)

    def distance(self, u=None):
        try:
            if self.pinB is None:
                # 3-pin ultrasonic sensor like as Ping sensor
                self.bot_instance.send_c(35)
            else:
                # 4-pin ultrasonic sensor like as HR-04
                self.bot_instance.send_c(36)

            d = self.bot_instance.read_r()

            if d is None:
                print("Error: No signal received or invalid reading.")
                return None

            if u == 'us': return d
            elif u == 'in': return d / 148
            else: return d / 58

        except Exception as e:
            print("Ultrasonic error:", e)
            return None
```

부록4: abot_straight_calibrate.py 라이브러리

```
"""
abot_straight_calibrate.py
Calibration routines for straight driving of the Micro:bit-Arduino robot.
Extends microbit_abot.py to provide precise linear movement control.
Repo: https://github.com/wookjin-chung/Tandem\_Robotics | MIT License
@fribot.com created Apr 11, 2025
@version 0.1.0
"""

from microbit_abot import bot
from microbit import *

class calibratedbot(bot):
    def __init__(self, p=27, q=33):
        super().__init__(p, q)
        self.left_base_speed = 0
        self.right_base_speed = 0
        self.left_speed_offset = 0
        self.right_speed_offset = 0
        self.max_offset = 20
        self.min_offset = -20
        self.calibration_animation = [
            Image("90009:09090:00900:09090:90009"),
            Image("00000:09090:00900:09090:00000")
        ]
        self.load_offsets()
        # Store the user-defined offset values to restore later
        self._default_left_offset = self.left_speed_offset
        self._default_right_offset = self.right_speed_offset

    def save_offsets(self):
        try:
            with open('offsets.txt', 'w') as f:
                f.write("{}\n{}\n".format(self.left_speed_offset, self.right_speed_offset))
        except OSError:
            display.scroll("Save Error")

    def load_offsets(self):
        try:
            with open('offsets.txt', 'r') as f:
                lines = f.read().split('\n')
                if len(lines) >= 2:
                    self.left_speed_offset = int(lines[0].strip())
                    self.right_speed_offset = int(lines[1].strip())
                else:
                    display.scroll("Invalid")
        except OSError:
            pass

    def calibrate(self):
        display.scroll("Cal Start")
        animation_index = 0
        prev_a = prev_b = False
        while True:
            left_speed = self.left_base_speed + self.left_speed_offset
            right_speed = self.right_base_speed + self.right_speed_offset
            left_speed = max(min(left_speed, 100), -100)
            right_speed = max(min(right_speed, 100), -100)
            self.servo_speed(left_speed, right_speed)
            a_pressed = button_a.is_pressed()
            b_pressed = button_b.is_pressed()
            if a_pressed and b_pressed and not (prev_a and prev_b):
                self.left_speed_offset = 0
                self.right_speed_offset = 0
                display.show("0")
                sleep(500)
            elif a_pressed and not prev_a:
                self.left_speed_offset = min(max(self.left_speed_offset - 1, self.min_offset), self.max_offset)
                display.show(Image.ARROW_E)
                sleep(200)
            elif b_pressed and not prev_b:
                self.right_speed_offset = min(max(self.right_speed_offset + 1, self.min_offset), self.max_offset)
                display.show(Image.ARROW_W)
                sleep(200)
            elif accelerometer.was_gesture('right'):
                display.scroll("R Off: {}".format(self.right_speed_offset))
                sleep(500)
            elif accelerometer.was_gesture('left'):
                display.scroll("L Off: {}".format(self.left_speed_offset))
                sleep(500)
```

```

        elif pin_logo.is_touched():
            self.servo_speed(0, 0)
            display.scroll("Cal Done")
            self.save_offsets()
            sleep(500)
            break
        else:
            display.show(self.calibration_animation[animation_index])
            animation_index = (animation_index + 1) % len(self.calibration_animation)
            sleep(200)
        sleep(50)
        prev_a, prev_b = a_pressed, b_pressed

def drive_straight(self, left_base_speed, right_base_speed, duration_ms):
    self.left_base_speed = left_base_speed
    self.right_base_speed = right_base_speed
    if abs(self.left_base_speed) < 20 and abs(self.right_base_speed) < 20:
        left_speed = self.left_base_speed
        right_speed = self.right_base_speed
    elif self.left_base_speed < -20 and self.right_base_speed > 20:
        left_speed = self.left_base_speed - self.left_speed_offset
        right_speed = self.right_base_speed - self.right_speed_offset
    else:
        left_speed = self.left_base_speed + self.left_speed_offset
        right_speed = self.right_base_speed + self.right_speed_offset
    left_speed = max(min(left_speed, 100), -100)
    right_speed = max(min(right_speed, 100), -100)
    self.servo_speed(int(left_speed), int(right_speed))
    start_time = running_time()
    while running_time() - start_time < duration_ms:
        if pin_logo.is_touched():
            self.servo_speed(0, 0)
            self.calibrate()
            start_time = running_time()
        sleep(50)
    self.servo_speed(0, 0)

def servo_speed_calibrated(self, left_base_speed, right_base_speed):
    # If both speed values are 0, reset the offset values to 0
    if left_base_speed == 0 and right_base_speed == 0:
        self.left_speed_offset = 0
        self.right_speed_offset = 0
        self.servo_speed(0, 0)
        return
    # If nonzero speed is commanded and offsets are zero, restore the stored offset values
    if self.left_speed_offset == 0 and self.right_speed_offset == 0:
        self.left_speed_offset = self.default_left_offset
        self.right_speed_offset = self.default_right_offset
    self.left_base_speed = left_base_speed
    self.right_base_speed = right_base_speed
    if abs(self.left_base_speed) < 20 and abs(self.right_base_speed) < 20:
        left_speed = self.left_base_speed
        right_speed = self.right_base_speed
    elif self.left_base_speed < -20 and self.right_base_speed > 20:
        left_speed = self.left_base_speed - self.left_speed_offset
        right_speed = self.right_base_speed - self.right_speed_offset
    else:
        left_speed = self.left_base_speed + self.left_speed_offset
        right_speed = self.right_base_speed + self.right_speed_offset
    left_speed = max(min(left_speed, 100), -100)
    right_speed = max(min(right_speed, 100), -100)
    self.servo_speed(int(left_speed), int(right_speed))

def set_left_speed_offset(self, value):
    self.left_speed_offset = min(max(value, self.min_offset), self.max_offset)
    self.save_offsets()

def set_right_speed_offset(self, value):
    self.right_speed_offset = min(max(value, self.min_offset), self.max_offset)
    self.save_offsets()

```

부록5: abot_rotation_calibrate.py 라이브러리

```
"""
abot_rotation_calibrate.py
Calibration routines for in-place rotation of the Micro:bit-Arduino robot.
Derived from abot_straight_calibrate.py for enhanced turning accuracy.
Repo: https://github.com/wookjin-chung/Tandem\_Robotics | MIT License
@fribot.com created Apr 11, 2025
@version 0.1.0
"""

from microbit_abot import bot
from microbit import *

class rotationCalibratedBot(bot):
    def __init__(self, p=27, q=33):
        super().__init__(p, q)
        self.cw_magnitude = 70
        self.ccw_magnitude = -70
        self.base_rotation_time = 2200
        self.load_rotation_speeds()
        self.calibration_animation = [
            Image("90009:09090:00900:09090:90009"),
            Image("00000:09090:00900:09090:00000")
        ]

    def save_rotation_speeds(self, is_cw):
        try:
            cw = self.cw_magnitude
            ccw = self.ccw_magnitude
            base_time = self.base_rotation_time
            try:
                with open('rotation_speeds.txt', 'r') as f:
                    lines = f.read().strip().split('\n')
                    if len(lines) >= 3:
                        cw = int(lines[0].strip())
                        ccw = int(lines[1].strip())
                        base_time = int(lines[2].strip())
            except OSError:
                pass
            if is_cw:
                cw = self.cw_magnitude
            else:
                ccw = self.ccw_magnitude
            with open('rotation_speeds.txt', 'w') as f:
                f.write('{}\n{}\n{}\n'.format(cw, ccw, base_time))
        except OSError:
            display.scroll("Save Error")

    def load_rotation_speeds(self):
        try:
            with open('rotation_speeds.txt', 'r') as f:
                lines = f.read().strip().split('\n')
                if len(lines) >= 3:
                    self.cw_magnitude = int(lines[0].strip())
                    self.ccw_magnitude = int(lines[1].strip())
                    self.base_rotation_time = int(lines[2].strip())
                else:
                    display.scroll("Rot Speeds Invalid")
        except OSError:
            pass

    def set_calibrated_speed(self, cw_speed, ccw_speed, base_rotation_time=None):
        self.cw_magnitude = cw_speed
        self.ccw_magnitude = ccw_speed
        if base_rotation_time is not None:
            self.base_rotation_time = base_rotation_time
        try:
            with open('rotation_speeds.txt', 'w') as f:
                f.write('{}\n{}\n{}\n'.format(self.cw_magnitude, self.ccw_magnitude, self.base_rotation_time))
        except OSError:
            display.scroll("Save Error")

    def start_rotation(self, speed, direction=True, base_rotation_time=None):
        self.load_rotation_speeds()
        try:
            with open('rotation_speeds.txt', 'r') as f:
                lines = f.read().strip().split('\n')
        except OSError:
            lines = []

        if not lines or len(lines) < 3:
```

```

    if base_rotation_time is not None:
        if direction:
            self.set_calibrated_speed(speed, self.ccw_magnitude, base_rotation_time)
        else:
            self.set_calibrated_speed(self.cw_magnitude, speed, base_rotation_time)
        self.base_rotation_time = base_rotation_time
    else:
        if direction:
            self.set_calibrated_speed(speed, self.ccw_magnitude)
        else:
            self.set_calibrated_speed(self.cw_magnitude, speed)

    self.rotate_degrees(360 if direction else -360)
    start_time = running_time()
    while running_time() - start_time < 2000:
        if pin_logo.is_touched():
            self.servo_speed(0, 0)
            self.calibrate_rotation(is_cw=direction, base_rotation_time=None)
            start_time = running_time()
            sleep(50)
    self.servo_speed(0, 0)

def calibrate_rotation(self, is_cw=True, base_rotation_time=None):
    display.scroll("Cal Start")
    animation_index = 0
    value_scrolled = False
    while True:
        if base_rotation_time is not None:
            self.base_rotation_time = base_rotation_time

        if button_a.is_pressed() and button_b.is_pressed():
            if is_cw:
                self.cw_magnitude = 70
                display.scroll("CW Reset")
            else:
                self.ccw_magnitude = -70
                display.scroll("CCW Reset")
            sleep(500)
            value_scrolled = False
            continue

        if button_a.was_pressed():
            if is_cw:
                self.cw_magnitude = min(self.cw_magnitude + 1, 100)
                display.scroll("CW: {}".format(self.cw_magnitude))
            else:
                self.ccw_magnitude = max(self.ccw_magnitude - 1, -100)
                display.scroll("CCW: {}".format(self.ccw_magnitude))
            sleep(200)
            value_scrolled = False
            continue

        if button_b.was_pressed():
            if is_cw:
                self.cw_magnitude = max(self.cw_magnitude - 1, 0)
                display.scroll("CW: {}".format(self.cw_magnitude))
            else:
                self.ccw_magnitude = min(self.ccw_magnitude + 1, 0)
                display.scroll("CCW: {}".format(self.ccw_magnitude))
            sleep(200)
            value_scrolled = False
            continue

        if pin_logo.is_touched():
            self.servo_speed(0, 0)
            display.scroll("Cal Done")
            self.save_rotation_speeds(is_cw)
            sleep(500)
            break

    self.servo_speed(0, 0)
    display.show(self.calibration_animation[animation_index])
    if not value_scrolled:
        if is_cw:
            display.scroll("CW: {}".format(self.cw_magnitude))
        else:
            display.scroll("CCW: {}".format(self.ccw_magnitude))
        value_scrolled = True
    animation_index = (animation_index + 1) % len(self.calibration_animation)
    sleep(200)

def rotate_degrees(self, angle):

```

```
rotation_time = self.base_rotation_time * abs(angle) / 360
speed = self.cw_magnitude if angle > 0 else self.ccw_magnitude
self.servo_speed(speed, speed)
sleep(int(rotation_time))
self.servo_speed(0, 0)
```

부록6: abot_calibrated.py 라이브러리

```
"""
abot_calibrated.py
Performs calibrated robot operations using external correction values.
Requires prior calibration of both straight driving and in-place rotation.
Built as a child of microbit_abot.py for refined motion control.
Repo: https://github.com/wookjin-chung/Tandem\_Robotics | MIT License
@fribot.com created Apr 13, 2025
@version 0.1.0
"""

from microbit_abot import bot
from microbit import *

class CalibratedMotionBot(bot):
    def __init__(self, p=27, q=33):
        super().__init__(p, q)
        # Offsets and limits for straight driving calibration
        self.left_speed_offset = 0
        self.right_speed_offset = 0
        self.min_offset = -20
        self.max_offset = 20

        # Default values for in-place rotation calibration
        self.cw_magnitude = 70 # Clockwise rotation speed
        self.ccw_magnitude = -70 # Counterclockwise rotation speed
        self.base_rotation_time = 2200 # Base time required for a 360° rotation (milliseconds)

        self.load_offsets()
        # Store the user-defined offset values to restore later
        self.default_left_offset = self.left_speed_offset
        self.default_right_offset = self.right_speed_offset

    # Functions for saving and setting calibration offsets (for straight driving)
    def save_offsets(self):
        try:
            with open('offsets.txt', 'w') as f:
                f.write('{}\n{}\n'.format(self.left_speed_offset, self.right_speed_offset))
        except OSError:
            display.scroll("Save Error")

    def load_offsets(self):
        try:
            with open('offsets.txt', 'r') as f:
                lines = f.read().split('\n')
                if len(lines) >= 2:
                    self.left_speed_offset = int(lines[0].strip())
                    self.right_speed_offset = int(lines[1].strip())
                else:
                    display.scroll("Invalid")
        except OSError:
            pass

    def set_left_speed_offset(self, value):
        self.left_speed_offset = min(max(value, self.min_offset), self.max_offset)
        self.save_offsets()

    def set_right_speed_offset(self, value):
        self.right_speed_offset = min(max(value, self.min_offset), self.max_offset)
        self.save_offsets()

    # Function that applies calibration offsets to the servo motor speeds (for straight driving)
    def servo_speed_calibrated(self, left_base_speed, right_base_speed):
        # If both speed values are 0, reset the offset values to 0
        if left_base_speed == 0 and right_base_speed == 0:
            self.left_speed_offset = 0
            self.right_speed_offset = 0
            self.servo_speed(0, 0)
            return

        # If nonzero speed is commanded and offsets are zero, restore the stored offset values
        if self.left_speed_offset == 0 and self.right_speed_offset == 0:
            self.left_speed_offset = self.default_left_offset
            self.right_speed_offset = self.default_right_offset

        self.left_base_speed = left_base_speed
        self.right_base_speed = right_base_speed
        if abs(self.left_base_speed) < 20 and abs(self.right_base_speed) < 20:
            left_speed = self.left_base_speed
            right_speed = self.right_base_speed
        elif self.left_base_speed < -20 and self.right_base_speed > 20:
            left_speed = self.left_base_speed - self.left_speed_offset
            right_speed = self.right_base_speed - self.right_speed_offset
```

```

else:
    left_speed = self.left_base_speed + self.left_speed_offset
    right_speed = self.right_base_speed + self.right_speed_offset
    left_speed = max(min(left_speed, 100), -100)
    right_speed = max(min(right_speed, 100), -100)
    self.servo_speed(int(left_speed), int(right_speed))

# External input function for in-place rotation calibration
def set_calibrated_speed(self, cw_speed, ccw_speed, base_rotation_time=None):
    self.cw_magnitude = cw_speed
    self.ccw_magnitude = ccw_speed
    if base_rotation_time is not None:
        self.base_rotation_time = base_rotation_time

# Function that performs an in-place rotation
def rotate_degrees(self, angle):
    # The rotation time is calculated proportionally to the base time required for a 360° rotation
    rotation_time = self.base_rotation_time * abs(angle) / 360
    # Rotate clockwise for positive angles and counterclockwise for negative angles
    speed = self.cw_magnitude if angle > 0 else self.ccw_magnitude
    self.servo_speed(speed, speed)
    sleep(int(rotation_time))
    self.servo_speed(0, 0)

```


마무리하며

책을 집필하는 여정이 끝난 지금, 이 책을 읽어주신 모든 분들께 깊은 감사의 마음을 전합니다. 이 책은 마이크로비트와 아두이노를 활용한 로봇 제작의 기초부터 실습까지 안내하며, 여러분의 창의적 탐구와 도전을 지원하기 위해 만들어졌습니다. 여러분이 이 책을 통해 한 발 더 성장하기를 바랍니다.

이 책이 여러분의 손에서 새로운 아이디어와 멋진 프로젝트로 이어지길 바랍니다. 여러분의 열정과 창의력이 더 나은 세상을 만들어갈 것이라 믿습니다.

끝으로, 여러분의 공학기술에 대한 여정에 작은 영감을 줄 수 있다면 그것만으로도 큰 보람을 느낍니다. 여러분이 미래의 엔지니어, 발명가, 그리고 문제 해결사가 될 것이라 믿습니다. 감사합니다.

2025년 4월 즈음에